



XCPC Algorithm Template

v8.4 2023.11.02

*3JSU - Another day of Not
understanding the question
(Wida & Hamine & HW)*

基础算法.....	1
头文件.....	1
常用函数.....	1
最大公约数 gcd.....	2
欧几里得算法.....	2
位运算优化.....	2
整数域二分.....	2
旧版（无法处理负数情况）.....	2
新版.....	3
实数域二分.....	3
整数域三分.....	3
实数域三分.....	3
树上问题.....	5
树的直径.....	5
树论大封装（直径+重心+中心）.....	5
点分治 / 树的重心.....	7
最近公共祖先 LCA.....	8
树链剖分解法.....	8
树上倍增解法.....	9
树上启发式合并（DSU on tree）.....	11
图论.....	13
常见概念.....	13
单源最短路径（SSSP 问题）.....	13
（正权稀疏图）动态数组存图+Dijkstra 算法.....	13
（负权图）Bellman-ford 算法.....	14
（负权图）SPFA 算法.....	14
（正权稠密图）邻接矩阵存图+Dijkstra 算法.....	15
多源汇最短路（APSP 问题）.....	16
（稠密图）邻接矩阵+Floyd 算法.....	16
平面图最短路（对偶图）.....	16
最小生成树（MST 问题）.....	17
（稀疏图）Prim 算法.....	17
（稠密图）Kruskal 算法.....	18
缩点（Tarjan 算法）.....	19
（有向图）强连通分量缩点.....	19
（无向图）割边缩点.....	20
（无向图）割点缩点.....	21
染色法判定二分图（dfs 算法）.....	23
链式前向星建图与搜索.....	23
一般图最大匹配（带花树算法）.....	24
二分图最大匹配.....	26
匈牙利算法（KM 算法）解.....	26
HopcroftKarp 算法（HK 算法、基于最大流模型）解.....	27
二分图最大权匹配（二分图完美匹配）.....	28
二分图最大独立点集（Konig 定理）.....	30
最长路（topsort+DP 算法）.....	30
最短路径树（SPT 问题）.....	31
无源汇点的最小割问题 Stoer - Wagner.....	32
欧拉路径/欧拉回路 Hierholzers.....	33

有向图欧拉路径存在判定.....	33
无向图欧拉路径存在判定.....	34
有向图欧拉路径求解（字典序最小）.....	35
无向图欧拉路径求解.....	35
差分约束.....	35
2-Sat.....	36
基础封装.....	36
答案不唯一时不输出.....	37
常见结论.....	37
常见例题.....	38
杂.....	38
Prüfer 序列：凯莱公式.....	39
Prüfer 序列.....	40
单源最短/次短路计数.....	40
判定图中是否存在负环.....	41
输出任意一个三元环.....	42
带权最小环大小与计数.....	42
最小环大小.....	43
本质不同简单环计数.....	44
输出任意一个非二元简单环.....	44
有向图环计数.....	45
输出有向图任意一个环.....	46
网络流.....	47
最大流.....	47
Dinic 解.....	47
预流推进 HLPP.....	48
最小割.....	50
费用流.....	50
数论.....	52
欧拉筛（线性筛）.....	52
防爆模乘.....	52
借助 int128 实现.....	52
借助浮点数实现.....	52
裴蜀定理.....	52
逆元.....	53
费马小定理（借助快速幂）.....	53
扩展欧几里得解.....	53
离线求解：线性递推解.....	53
扩展欧几里得 exgcd.....	53
离散对数 bsgs 与 exbsgs.....	54
欧拉函数.....	56
直接求解单个数的欧拉函数.....	56
求解 1 到 N 所有数的欧拉函数.....	56
使用莫比乌斯反演求解欧拉函数.....	57
组合数.....	57
逆元+卢卡斯定理.....	57
质因数分解.....	57
求解连续数字的正约数集合——倍数法.....	58
试除法判是否是质数.....	59

标准解.....	59
常数优化法.....	59
容斥原理.....	59
二进制枚举解.....	59
dfs 解.....	60
同余方程组、拓展中国剩余定理 excrt.....	60
求解连续数字的按位异或.....	61
高斯消元求解线性方程组.....	61
康拓展开.....	62
正向展开普通解法.....	62
正向展开树状数组解.....	62
逆向还原.....	63
Min25 筛.....	64
矩阵四则运算.....	65
矩阵快速幂.....	66
矩阵加速.....	66
莫比乌斯函数/反演.....	67
整除（数论）分块.....	68
Miller-Rabin 素数测试.....	68
Pollard-Rho 因式分解.....	69
常见结论.....	70
球盒模型（八种）.....	70
麦乐鸡定理.....	71
抽屉原理（鸽巢原理）.....	71
哥德巴赫猜想.....	71
素数密度与分布.....	71
除法、取模运算的本质.....	71
与、或、异或.....	71
欧拉函数常见性质.....	72
组合数学常见性质.....	72
范德蒙德卷积公式.....	73
卡特兰数.....	73
狄利克雷卷积.....	73
斐波那契数列.....	73
杂.....	74
常见例题.....	75
几何.....	77
平面几何必要初始化.....	77
字符串读入浮点数.....	77
预置函数.....	77
点线封装.....	77
叉乘.....	78
点乘.....	78
欧几里得距离公式.....	78
曼哈顿距离公式.....	79
将向量转换为单位向量.....	79
向量旋转.....	79
平面角度与弧度.....	79
弧度角度相互转换.....	79

正弦定理.....	79
余弦定理（已知三角形三边，求角）.....	79
求两向量的夹角.....	80
向量旋转任意角度.....	80
点绕点旋转任意角度.....	80
平面点线相关.....	80
点是否在直线上（三点是否共线）.....	80
点是否在向量（直线）左侧.....	80
两点是否在直线同侧/异侧.....	81
两直线相交交点.....	81
两直线是否平行/垂直/相同.....	81
点到直线的最近距离与最近点.....	81
点是否在线段上.....	82
点到线段的最近距离与最近点.....	82
点在直线上的投影点（垂足）.....	82
线段的中垂线.....	82
两线段是否相交及交点.....	82
平面圆相关（浮点数处理）.....	83
点到圆的最近点.....	83
根据圆心角获取圆上某点.....	84
直线是否与圆相交及交点.....	84
线段是否与圆相交及交点.....	84
两圆是否相交及交点.....	84
两圆相交面积.....	85
三点确定一圆.....	85
求解点到圆的切线数量与切点.....	86
求解两圆的内公、外公切线数量与切点.....	86
平面三角形相关（浮点数处理）.....	87
三角形面积.....	87
三角形外心.....	87
三角形内心.....	87
三角形垂心.....	87
平面直线方程转换.....	87
浮点数计算直线的斜率.....	87
分数精确计算直线的斜率.....	88
两点式转一般式.....	88
一般式转两点式.....	88
抛物线与 x 轴是否相交及交点.....	89
平面多边形.....	89
两向量构成的平面四边形有向面积.....	89
判断四个点能否组成矩形/正方形.....	89
点是否在任意多边形内.....	90
线段是否在任意多边形内部.....	90
任意多边形的面积.....	91
皮克定理.....	91
任意多边形上/内的网格点个数（仅能处理整数）.....	92
二维凸包.....	92
获取二维静态凸包（Andrew 算法）.....	92
二维动态凸包.....	93

点与凸包的位置关系.....	94
闵可夫斯基和.....	94
半平面交.....	95
三维几何必要初始化.....	96
点线面封装.....	96
其他函数.....	96
三维点线面相关.....	97
空间三点是否共线.....	97
四点是否共面.....	97
空间点是否在线段上.....	97
空间两点是否在线段同侧.....	97
两点是否在平面同侧.....	98
空间两直线是否平行/垂直.....	98
两平面是否平行/垂直.....	98
空间两直线是否是同一条.....	98
两平面是否是同一个.....	98
直线是否与平面平行.....	98
空间两线段是否相交.....	98
空间两直线是否相交及交点.....	99
直线与平面是否相交及交点.....	99
两平面是否相交及交线.....	99
点到直线的最近点与最近距离.....	100
点到平面的最近点与最近距离.....	100
空间两直线的最近距离与最近点对.....	100
三维角度与弧度.....	100
空间两直线夹角的 $\cos$ 值.....	100
空间两平面夹角的 $\cos$ 值.....	101
直线与平面夹角的 $\sin$ 值.....	101
空间多边形.....	101
正 N 棱锥体积公式.....	101
四面体体积.....	101
点是否在空间三角形上.....	101
线段是否与空间三角形相交及交点.....	101
空间三角形是否相交.....	102
常用结论.....	102
平面几何结论归档.....	102
立体几何结论归档.....	103
常用例题.....	103
将平面某点旋转任意角度.....	103
平面最近点对 (set 解)	103
平面若干点能构成的最大四边形的面积 (简单版, 暴力枚举)	104
平面若干点能构成的最大四边形的面积 (困难版, 分类讨论+旋转卡壳)	105
线段将多边形切割为几个部分.....	105
平面若干点能否构成凸包 (暴力枚举)	106
凸包上的点能构成的最大三角形 (暴力枚举)	107
凸包上的点能构成的最大四角形的面积 (旋转卡壳)	108
判断一个凸包是否完全在另一个凸包内.....	109
多项式.....	110
多项式封装.....	110

离散傅里叶变换 dft 与其逆变换 idft.....	113
Berlekamp-Massey 算法.....	114
Linear-Recurrence 算法.....	115
快速傅里叶变换 FFT.....	116
快速数论变换 NTT.....	117
拉格朗日插值.....	118
常用结论.....	119
杂.....	119
普通生成函数 / OGF.....	119
指数生成函数 / EGF.....	119
数据结构.....	121
并查集（全功能）.....	121
树状数组.....	121
基础封装.....	121
求解逆序对数量.....	122
二维树状数组.....	123
线段树.....	124
区间加法修改、区间最小值查询、区间合并.....	124
同时需要处理区间加法与乘法修改.....	126
坐标压缩与离散化.....	128
简单版本.....	128
封装.....	128
轻重链剖分/树链剖分.....	129
普通莫队.....	130
带修改的莫队（带时间维度的莫队）.....	131
分数运算类.....	132
主席树（可持久化线段树）.....	133
KD Tree.....	134
ST 表.....	136
基于状压的线性 RMQ 算法.....	137
平衡二叉树.....	138
线性基（高斯消元法）.....	139
珂朵莉树 (OD Tree).....	140
取模类.....	141
大整数类（高精度计算）.....	143
常见结论.....	148
常见例题.....	148
动态规划.....	150
01 背包.....	150
完全背包.....	150
多重背包.....	151
混合背包.....	152
二维费用的背包.....	152
分组背包.....	153
有依赖的背包.....	153
背包问题求方案数.....	154
背包问题求具体方案.....	155
数位 DP.....	155
状压 DP.....	156

常用例题.....	156
串与相关数据结构.....	159
子串与子序列.....	159
字符串模式匹配算法 KMP.....	159
Z 函数（扩展 KMP）.....	159
最长公共子序列 LCS.....	160
小数据解.....	160
大数据解.....	160
字符串哈希.....	161
双哈希封装.....	161
前后缀去重.....	161
马拉车.....	162
字典树 trie.....	163
基础封装.....	163
01 字典树.....	163
后缀数组 SA.....	164
AC 自动机.....	165
回文自动机 PAM（回文树）.....	166
后缀自动机 SAM.....	167
博弈论.....	168
巴什博弈.....	168
扩展巴什博弈.....	168
Nim 博弈.....	168
Nim 游戏具体取法.....	169
Moore' s Nim 游戏（Nim-K 游戏）.....	169
Anti-Nim 游戏（反 Nim 游戏）.....	169
阶梯 - Nim 博弈.....	169
SG 游戏（有向图游戏）.....	169
Anti-SG 游戏（反 SG 游戏）.....	170
Lasker' s-Nim 游戏（Multi-SG 游戏）.....	170
Every-SG 游戏.....	171
威佐夫博弈.....	171
斐波那契博弈.....	171
树上删边游戏.....	172
无向图删边游戏（Fusion Principle 定理）.....	172
常用例题.....	173
逆序对（归并排序解）.....	173
统计区间不同数字的数量（离线查询）.....	173
选数（DFS 解）.....	174
网格路径计数.....	174
德州扑克.....	175
N*M 数独字典序最小方案.....	178
高精度进制转换.....	179
物品装箱.....	180
从前往后装（线段树解）.....	180
选择最优的箱子装（multiset 解）.....	180
浮点数比较.....	181
STL 与库函数.....	182
查找后继 lower_bound、upper_bound.....	182

二分搜索 binary_search.....	182
批量递增赋值函数 iota.....	182
数组去重函数 unique.....	182
位运算函数 __builtin_.....	182
数字转字符串函数.....	183
字符串转数字.....	183
全排列算法 next_permutation、prev_permutation.....	183
字符串转换为数值函数 sto.....	184
数值转换为字符串函数 to_string.....	184
判断非递减 is_sorted.....	184
累加 accumulate.....	185
迭代器 iterator.....	185
容器与成员函数.....	185
元组 tuple.....	185
数组 array.....	185
变长数组 vector.....	185
栈 stack.....	185
队列 queue.....	186
双向队列 deque.....	186
优先队列 priority_queue.....	186
字符串 string.....	186
有序、多重有序集合 set、multiset.....	187
map、multimap.....	187
bitset.....	188
哈希系列 unordered.....	189
对 pair、tuple 定义哈希.....	189
对结构体定义哈希.....	189
对 vector 定义哈希.....	189
程序标准化.....	190
使用 Lambda 函数.....	190
使用构造函数.....	190
杂类.....	192
最长严格/非严格递增子序列 (LIS).....	192
cout 输出流控制.....	192
约瑟夫问题.....	192
日期换算 (基姆拉尔森公式)	193
单调队列.....	193
高精度快速幂.....	193
快读.....	194
int128 输入输出流控制.....	195
对拍班子.....	195
随机数生成与样例构造.....	196
手工哈希.....	197
Python 常用语法.....	197
读入与定义.....	197
格式化输出.....	197
排序.....	197
文件 IO.....	197
自定义结构体.....	197

数据结构.....	198
其他.....	198
OJ 测试.....	198
OJ 支持语言及编译参数测试.....	198
OJ 运算速度测试.....	198

1 基础算法

1.1 头文件

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 template<class... Args> void _(Args... args) {
5     auto _ = [&](auto x) { cout << x << " "; };
6     cout << "---->";
7     int arr[] = {_(args), 0}...;
8     cout << "\n";
9 }
10
11 signed main() {
12     int Task = 1;
13     for (cin >> Task; Task; Task--) {}
14 }
15 int __OI_INIT__ = []() {
16     ios::sync_with_stdio(0), cin.tie(0);
17     cout.tie(0);
18     cout << fixed << setprecision(12);
19     return 0;
20 }();
```

1.2 常用函数

```
1 int mypow(int n, int k, int p = MOD) { // 复杂度是 log N
2     int r = 1;
3     for (; k >>= 1, n = n * n % p) {
4         if (k & 1) r = r * n % p;
5     }
6     return r;
7 }
8 i64 mysqrt(i64 n) { // 针对 sqrt 无法精确计算 11 型
9     i64 ans = sqrt(n);
10    while ((ans + 1) * (ans + 1) <= n) ans++;
11    while (ans * ans > n) ans--;
12    return ans;
13 }
14 int mylcm(int x, int y) {
15     return x / gcd(x, y) * y;
16 }
17 int Log2(int x) { // 计算log2(x), 向下取整
18     return 31 - __builtin_clz(x);
19 }
20 int ceiledLog2(int x) { // 计算log2(x), 但是向上取整
21     return Log2(x) + (__builtin_popcount(x) != 1);
22 }
23 template <class T> T sign(const T &a) {
24     return a == 0 ? 0 : (a < 0 ? -1 : 1);
25 }
26 template <class T> T floor(const T &a, const T &b) { // 注意大数据计算时会丢失精度
27     T A = abs(a), B = abs(b);
28     assert(B != 0);
29     return sign(a) * sign(b) > 0 ? A / B : -(A + B - 1) / B;
30 }
31 template <class T> T ceil(const T &a, const T &b) { // 注意大数据计算时会丢失精度
32     T A = abs(a), B = abs(b);
33     assert(b != 0);
```

```

34     return sign(a) * sign(b) > 0 ? (A + B - 1) / B : -A / B;
35 }

```

1.3 最大公约数 gcd

1.3.1 欧几里得算法

速度不如内置函数！ 以 $\mathcal{O}(\log(a+b))$ 的复杂度求解最大公约数。与内置函数 `__gcd` 功能基本相同（支持 $a, b \leq 0$ ）。

```

1 inline int mygcd(int a, int b) { return b ? gcd(b, a % b) : a; }

```

1.3.2 位运算优化

略快于内置函数，用于卡常。

```

1 LL gcd(LL a, LL b) { // 卡常 gcd!!
2     #define tz __builtin_ctzll
3     if (!a || !b) return a | b;
4     int t = tz(a | b);
5     a >>= tz(a);
6     while (b) {
7         b >>= tz(b);
8         if (a > b) swap(a, b);
9         b -= a;
10    }
11    return a << t;
12    #undef tz
13 }

```

1.4 整数域二分

1.4.1 旧版（无法处理负数情况）

- 在递增序列 a 中查找 $\geq x$ 数中最小的一个（即 x 或 x 的后继）

```

1 while (l < r) {
2     int mid = (l + r) / 2;
3     if (a[mid] >= x) {
4         r = mid;
5     } else {
6         l = mid + 1;
7     }
8 }
9 return a[l];

```

- 在递增序列 a 中查找 $\leq x$ 数中最大的一个（即 x 或 x 的前驱）

```

1 while (l < r) {
2     int mid = (l + r + 1) / 2;
3     if (a[mid] <= x) {
4         l = mid;
5     } else {
6         r = mid - 1;
7     }
8 }
9 return a[l];

```

1.4.2 新版

- x 或 x 的后继

```
1 int l = 0, r = 1E8, ans = r;
2 while (l <= r) {
3     int mid = (l + r) / 2;
4     if (judge(mid)) {
5         r = mid - 1;
6         ans = mid;
7     } else {
8         l = mid + 1;
9     }
10 }
11 return ans;
```

- x 或 x 的前驱

```
1 int l = 0, r = 1E8, ans = 1;
2 while (l <= r) {
3     int mid = (l + r) / 2;
4     if (judge(mid)) {
5         l = mid + 1;
6         ans = mid;
7     } else {
8         r = mid - 1;
9     }
10 }
11 return ans;
```

1.5 实数域二分

目前主流的写法是限制二分次数。

```
1 for (int t = 1; t <= 100; t++) {
2     ld mid = (l + r) / 2;
3     if (judge(mid)) r = mid;
4     else l = mid;
5 }
6 cout << l << endl;
```

1.6 整数域三分

```
1 while (l < r) {
2     int mid = (l + r) / 2;
3     if (check(mid) <= check(mid + 1)) r = mid;
4     else l = mid + 1;
5 }
6 cout << check(l) << endl;
```

1.7 实数域三分

限制次数实现。

```
1  ld l = -1E9, r = 1E9;
2  for (int t = 1; t <= 100; t++) {
3      ld mid1 = (l * 2 + r) / 3;
4      ld mid2 = (l + r * 2) / 3;
5      if (judge(mid1) < judge(mid2)) {
6          r = mid2;
7      } else {
8          l = mid1;
9      }
10 }
11 cout << l << endl;
```

/END/

2 树上问题

2.1 树的直径

```
1 struct Tree {
2     int n;
3     vector<vector<int>> ver;
4     Tree(int n) {
5         this->n = n;
6         ver.resize(n + 1);
7     }
8     void add(int x, int y) {
9         ver[x].push_back(y);
10        ver[y].push_back(x);
11    }
12    int getlen(int root) { // 获取x所在树的直径
13        map<int, int> dep; // map用于优化输入为森林时的深度计算, 亦可用vector
14        function<void(int, int)> dfs = [&](int x, int fa) -> void {
15            for (auto y : ver[x]) {
16                if (y == fa) continue;
17                dep[y] = dep[x] + 1;
18                dfs(y, x);
19            }
20            if (dep[x] > dep[root]) {
21                root = x;
22            }
23        };
24        dfs(root, 0);
25        int st = root; // 记录直径端点
26
27        dep.clear();
28        dfs(root, 0);
29        int ed = root; // 记录直径另一端点
30
31        return dep[root];
32    }
33 };
```

2.2 树论大封装（直径+重心+中心）

```
1 struct Tree {
2     int n;
3     vector<vector<pair<int, int>>> e;
4     vector<int> dep, parent, maxdep, d1, d2, s1, s2, up;
5     Tree(int n) {
6         this->n = n;
7         e.resize(n + 1);
8         dep.resize(n + 1);
9         parent.resize(n + 1);
10        maxdep.resize(n + 1);
11        d1.resize(n + 1);
12        d2.resize(n + 1);
13        s1.resize(n + 1);
14        s2.resize(n + 1);
15        up.resize(n + 1);
16    }
17    void add(int u, int v, int w) {
18        e[u].push_back({w, v});
19        e[v].push_back({w, u});
20    }
```

```

21 void dfs(int u, int fa) {
22     maxdep[u] = dep[u];
23     for (auto [w, v] : e[u]) {
24         if (v == fa) continue;
25         dep[v] = dep[u] + 1;
26         parent[v] = u;
27         dfs(v, u);
28         maxdep[u] = max(maxdep[u], maxdep[v]);
29     }
30 }
31
32 void dfs1(int u, int fa) {
33     for (auto [w, v] : e[u]) {
34         if (v == fa) continue;
35         dfs1(v, u);
36         int x = d1[v] + w;
37         if (x > d1[u]) {
38             d2[u] = d1[u], s2[u] = s1[u];
39             d1[u] = x, s1[u] = v;
40         } else if (x > d2[u]) {
41             d2[u] = x, s2[u] = v;
42         }
43     }
44 }
45 void dfs2(int u, int fa) {
46     for (auto [w, v] : e[u]) {
47         if (v == fa) continue;
48         if (s1[u] == v) {
49             up[v] = max(up[u], d2[u]) + w;
50         } else {
51             up[v] = max(up[u], d1[u]) + w;
52         }
53         dfs2(v, u);
54     }
55 }
56
57 int radius, center, diam;
58 void getCenter() {
59     center = 1; //中心
60     for (int i = 1; i <= n; i++) {
61         if (max(d1[i], up[i]) < max(d1[center], up[center])) {
62             center = i;
63         }
64     }
65     radius = max(d1[center], up[center]); //距离最远点的距离的最小值
66     diam = d1[center] + up[center] + 1; //直径
67 }
68
69 int rem; //删除重心后剩余连通块体积的最小值
70 int cog; //重心
71 vector<bool> vis;
72 void getCog() {
73     vis.resize(n);
74     rem = INT_MAX;
75     cog = 1;
76     dfsCog(1);
77 }
78 int dfsCog(int u) {
79     vis[u] = true;
80     int s = 1, res = 0;
81     for (auto [w, v] : e[u]) {
82         if (vis[v]) continue;
83         int t = dfsCog(v);
84         res = max(res, t);

```

```

85         s += t;
86     }
87     res = max(res, n - s);
88     if (res < rem) {
89         rem = res;
90         cog = u;
91     }
92     return s;
93 }
94 };

```

2.3 点分治 / 树的重心

重心的定义：删除树上的某一个点，会得到若干棵子树；删除某点后，得到的最大子树最小，这个点称为重心。我们假设某个点是重心，记录此时最大子树的最小值，遍历完所有点后取最大值即可。

重心的性质：重心最多可能会有两个，且此时两个重心相邻。

点分治的一般过程是：取重心为新树的根，随后使用 dfs 处理当前这棵树，灵活运用 `child` 和 `pre` 两个数组分别计算通过根节点、不通过根节点的路径信息，根据需要进行答案的更新；再对子树分治，寻找子树的重心，……。时间复杂度降至 $\mathcal{O}(N \log N)$ 。

```

1  int root = 0, MaxTree = 1e18; //分别代表重心下标、最大子树大小
2  vector<int> vis(n + 1), siz(n + 1);
3  auto get = [&](auto self, int x, int fa, int n) -> void { // 获取树的重心
4      siz[x] = 1;
5      int val = 0;
6      for (auto [y, w] : ver[x]) {
7          if (y == fa || vis[y]) continue;
8          self(self, y, x, n);
9          siz[x] += siz[y];
10         val = max(val, siz[y]);
11     }
12     val = max(val, n - siz[x]);
13     if (val < MaxTree) {
14         MaxTree = val;
15         root = x;
16     }
17 };
18
19 auto clac = [&](int x) -> void { // 以 x 为新的根，维护询问
20     set<int> pre = {0}; // 记录到根节点 x 距离为 i 的路径是否存在
21     vector<int> dis(n + 1);
22     for (auto [y, w] : ver[x]) {
23         if (vis[y]) continue;
24         vector<int> child; // 记录 x 的子树节点的深度信息
25         auto dfs = [&](auto self, int x, int fa) -> void {
26             child.push_back(dis[x]);
27             for (auto [y, w] : ver[x]) {
28                 if (y == fa || vis[y]) continue;
29                 dis[y] = dis[x] + w;
30                 self(self, y, x);
31             }
32         };
33         dis[y] = w;
34         dfs(dfs, y, x);
35
36         for (auto it : child) {
37             for (int i = 1; i <= m; i++) { // 根据询问更新值
38                 if (q[i] < it || !pre.count(q[i] - it)) continue;
39                 ans[i] = 1;
40             }

```

```

41     }
42     pre.insert(child.begin(), child.end());
43 }
44 };
45
46 auto dfz = [&](auto self, int x, int fa) -> void { // 点分治
47     vis[x] = 1; // 标记已经被更新过的旧重心, 确保只对子树分治
48     clac(x);
49     for (auto [y, w] : ver[x]) {
50         if (y == fa || vis[y]) continue;
51         MaxTree = 1e18;
52         get(get, y, x, siz[y]);
53         self(self, root, x);
54     }
55 };
56
57 get(get, 1, 0, n);
58 dfz(dfz, root, 0);

```

2.4 最近公共祖先 LCA

2.4.1 树链剖分解法

预处理时间复杂度 $\mathcal{O}(N)$; 单次查询 $\mathcal{O}(\log N)$, 常数较小。

```

1  struct HLD {
2      int n, idx;
3      vector<vector<int>> ver;
4      vector<int> siz, dep;
5      vector<int> top, son, parent;
6
7      HLD(int n) {
8          this->n = n;
9          ver.resize(n + 1);
10         siz.resize(n + 1);
11         dep.resize(n + 1);
12
13         top.resize(n + 1);
14         son.resize(n + 1);
15         parent.resize(n + 1);
16     }
17     void add(int x, int y) { // 建立双向边
18         ver[x].push_back(y);
19         ver[y].push_back(x);
20     }
21     void dfs1(int x) {
22         siz[x] = 1;
23         dep[x] = dep[parent[x]] + 1;
24         for (auto y : ver[x]) {
25             if (y == parent[x]) continue;
26             parent[y] = x;
27             dfs1(y);
28             siz[x] += siz[y];
29             if (siz[y] > siz[son[x]]) {
30                 son[x] = y;
31             }
32         }
33     }
34     void dfs2(int x, int up) {
35         top[x] = up;
36         if (son[x]) dfs2(son[x], up);
37         for (auto y : ver[x]) {

```

```

38         if (y == parent[x] || y == son[x]) continue;
39         dfs2(y, y);
40     }
41 }
42 int lca(int x, int y) {
43     while (top[x] != top[y]) {
44         if (dep[top[x]] > dep[top[y]]) {
45             x = parent[top[x]];
46         } else {
47             y = parent[top[y]];
48         }
49     }
50     return dep[x] < dep[y] ? x : y;
51 }
52 int clac(int x, int y) { // 查询两点间距离
53     return dep[x] + dep[y] - 2 * dep[lca(x, y)];
54 }
55 void work(int root = 1) { // 在此初始化
56     dfs1(root);
57     dfs2(root, root);
58 }
59 };

```

2.4.2 树上倍增解法

预处理时间复杂度 $\mathcal{O}(N \log N)$ ；单次查询 $\mathcal{O}(\log N)$ ，但是常数比树链剖分解法更大。

封装一：基础封装，针对无权图。

```

1 struct Tree {
2     int n;
3     vector<vector<int>> ver, val;
4     vector<int> lg, dep;
5     Tree(int n) {
6         this->n = n;
7         ver.resize(n + 1);
8         val.resize(n + 1, vector<int>(30));
9         lg.resize(n + 1);
10        dep.resize(n + 1);
11        for (int i = 1; i <= n; i++) { //预处理 log
12            lg[i] = lg[i - 1] + (1 << lg[i - 1] == i);
13        }
14    }
15    void add(int x, int y) { // 建立双向边
16        ver[x].push_back(y);
17        ver[y].push_back(x);
18    }
19    void dfs(int x, int fa) {
20        val[x][0] = fa; // 储存 x 的父节点
21        dep[x] = dep[fa] + 1;
22        for (int i = 1; i <= lg[dep[x]]; i++) {
23            val[x][i] = val[val[x][i - 1]][i - 1];
24        }
25        for (auto y : ver[x]) {
26            if (y == fa) continue;
27            dfs(y, x);
28        }
29    }
30    int lca(int x, int y) {
31        if (dep[x] < dep[y]) swap(x, y);
32        while (dep[x] > dep[y]) {
33            x = val[x][lg[dep[x]] - dep[y] - 1];

```

```

34     }
35     if (x == y) return x;
36     for (int k = lg[dep[x]] - 1; k >= 0; k--) {
37         if (val[x][k] == val[y][k]) continue;
38         x = val[x][k];
39         y = val[y][k];
40     }
41     return val[x][0];
42 }
43 int clac(int x, int y) { // 倍增查询两点间距离
44     return dep[x] + dep[y] - 2 * dep[lca(x, y)];
45 }
46 void work(int root = 1) { // 在此初始化
47     dfs(root, 0);
48 }
49 };

```

封装二：扩展封装，针对有权图，支持“倍增查询两点路径上的最大边权”功能。

```

1 struct Tree {
2     int n;
3     vector<vector<int>> val, Max;
4     vector<vector<pair<int, int>>> ver;
5     vector<int> lg, dep;
6     Tree(int n) {
7         this->n = n;
8         ver.resize(n + 1);
9         val.resize(n + 1, vector<int>(30));
10        Max.resize(n + 1, vector<int>(30));
11        lg.resize(n + 1);
12        dep.resize(n + 1);
13        for (int i = 1; i <= n; i++) { //预处理 log
14            lg[i] = lg[i - 1] + (1 << lg[i - 1] == i);
15        }
16    }
17    void add(int x, int y, int w) { // 建立双向边
18        ver[x].push_back({y, w});
19        ver[y].push_back({x, w});
20    }
21    void dfs(int x, int fa) {
22        val[x][0] = fa;
23        dep[x] = dep[fa] + 1;
24        for (int i = 1; i <= lg[dep[x]]; i++) {
25            val[x][i] = val[val[x][i - 1]][i - 1];
26            Max[x][i] = max(Max[x][i - 1], Max[val[x][i - 1]][i - 1]);
27        }
28        for (auto [y, w] : ver[x]) {
29            if (y == fa) continue;
30            Max[y][0] = w;
31            dfs(y, x);
32        }
33    }
34    int lca(int x, int y) {
35        if (dep[x] < dep[y]) swap(x, y);
36        while (dep[x] > dep[y]) {
37            x = val[x][lg[dep[x]] - dep[y] - 1];
38        }
39        if (x == y) return x;
40        for (int k = lg[dep[x]] - 1; k >= 0; k--) {
41            if (val[x][k] == val[y][k]) continue;
42            x = val[x][k];
43            y = val[y][k];

```

```

44     }
45     return val[x][0];
46 }
47 int clac(int x, int y) { // 倍增查询两点间距离
48     return dep[x] + dep[y] - 2 * dep[lca(x, y)];
49 }
50 int query(int x, int y) { // 倍增查询两点路径上的最大边权 (带权图)
51     auto get = [&](int x, int y) -> int {
52         int ans = 0;
53         if (x == y) return ans;
54         for (int i = lg[dep[x]]; i >= 0; i--) {
55             if (dep[val[x][i]] > dep[y]) {
56                 ans = max(ans, Max[x][i]);
57                 x = val[x][i];
58             }
59         }
60         ans = max(ans, Max[x][0]);
61         return ans;
62     };
63     int fa = lca(x, y);
64     return max(get(x, fa), get(y, fa));
65 }
66 void work(int root = 1) { // 在此初始化
67     dfs(root, 0);
68 }
69 };

```

2.5 树上路径交

计算两条路径的交点数量，直接载入任意 LCA 封装即可。

```

1 int intersection(int x, int y, int X, int Y) {
2     vector<int> t = {lca(x, X), lca(x, Y), lca(y, X), lca(y, Y)};
3     sort(t.begin(), t.end());
4     int r = lca(x, y), R = lca(X, Y);
5     if (dep[t[0]] < min(dep[r], dep[R]) || dep[t[2]] < max(dep[r], dep[R])) {
6         return 0;
7     }
8     return 1 + clac(t[2], t[3]);
9 }

```

2.6 树上启发式合并 (DSU on tree)

$\mathcal{O}(N \log N)$ 。

```

1 struct HLD {
2     vector<vector<int>> e;
3     vector<int> siz, son, cnt;
4     vector<LL> ans;
5     LL sum, Max;
6     int hson;
7     HLD(int n) {
8         e.resize(n + 1);
9         siz.resize(n + 1);
10        son.resize(n + 1);
11        ans.resize(n + 1);
12        cnt.resize(n + 1);
13        hson = 0;
14        sum = 0;
15        Max = 0;
16    }

```

```

17 void add(int u, int v) {
18     e[u].push_back(v);
19     e[v].push_back(u);
20 }
21 void dfs1(int u, int fa) {
22     siz[u] = 1;
23     for (auto v : e[u]) {
24         if (v == fa) continue;
25         dfs1(v, u);
26         siz[u] += siz[v];
27         if (siz[v] > siz[son[u]]) son[u] = v;
28     }
29 }
30 void calc(int u, int fa, int val) {
31     cnt[color[u]] += val;
32     if (cnt[color[u]] > Max) {
33         Max = cnt[color[u]];
34         sum = color[u];
35     } else if (cnt[color[u]] == Max) {
36         sum += color[u];
37     }
38     for (auto v : e[u]) {
39         if (v == fa || v == hson) continue;
40         calc(v, u, val);
41     }
42 }
43 void dfs2(int u, int fa, int opt) {
44     for (auto v : e[u]) {
45         if (v == fa || v == son[u]) continue;
46         dfs2(v, u, 0);
47     }
48     if (son[u]) {
49         dfs2(son[u], u, 1);
50         hson = son[u]; //记录重链编号, 计算的时候跳过
51     }
52     calc(u, fa, 1);
53     hson = 0; //消除的时候所有儿子都清除
54     ans[u] = sum;
55     if (!opt) {
56         calc(u, fa, -1);
57         sum = 0;
58         Max = 0;
59     }
60 }
61 };

```

/END/

3 图论

3.1 常见概念

oriented graph: 有向图

bidirectional edges: 双向边

二分图: 一个图能被分为左右两部分, 任何一条边的两个端点都不在同一部分中。

平面图: 若能将无向图 $G = (V, E)$ 画在平面上使得任意两条无重合顶点的边不相交, 则称 G 是平面图。

匹配 (独立边集): 一个边的集合, 这些边没有公共顶点。

割点 (割顶): 将与某点 i 连接的所有边删去后, 原图分成两个以上不相连的子图, 称 i 为图的割点。

点双连通: 在一张连通的无向图中, 对于两个点 u 和 v , 删去任何一个点 (只能删去一个, 且不能删 u 和 v 自己) 它们依旧连通, 则称 u 和 v 边双连通。如果一个图不存在割点, 那么它是一个点双连通图。

割边 (桥): 将某边 e 删去后, 原图分成两个以上不相连的子图, 称 e 为图的割边。

边双连通: 在一张连通的无向图中, 对于两个点 u 和 v , 删去任何一条边 (只能删去一条) 它们依旧连通, 则称 u 和 v 边双连通。一个图如果不存在割边, 则它是一个边双连通图。

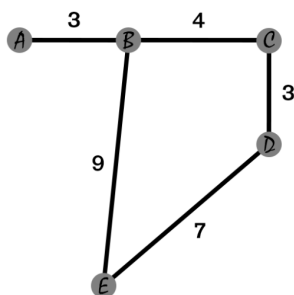
无向正权图上某一点的偏心距: 记为 $ecc(u) = \max \{dist(u, v)\}$, 即以这个点为源, 到其他点的**所有最短路的最大值**。如下图 A 点, $ecc(A)$ 即为 12。

图的直径: 定义为 $d = \max \{ecc(u)\}$, 即**最大的偏心距**, 亦可以简化为图中最远的一对点的距离。

图的中心: 定义为 $arg = \min \{ecc(u)\}$, 即**偏心距最小的点**。如下图, 图的中心即为 B 点。

图的绝对中心: 可以定义在边上的图的中心。

图的半径: 图的半径不同于圆的半径, 其不等于直径的一半 (但对于绝对中心定义上的直径而言是一半)。定义为 $r = \min \{ecc(u)\}$, 即**中心的偏心距**。计算方式: 使用全源最短路, 计算出所有点的偏心距, 再加以计算。



3.2 单源最短路径 (SSSP问题)

3.2.1 (正权稀疏图) 动态数组存图+Dijkstra算法

使用优先队列优化, 以 $\mathcal{O}(M \log N)$ 的复杂度计算。

```
1 vector<int> dis(n + 1, 1E18);
2 auto djikstra = [&](int s = 1) -> void {
3     using PII = pair<int, int>;
4     priority_queue<PII, vector<PII>, greater<PII>> q;
5     q.emplace(0, s);
6     dis[s] = 0;
7     vector<int> vis(n + 1);
8     while (!q.empty()) {
9         int x = q.top().second;
10        q.pop();
11        if (vis[x]) continue;
12        vis[x] = 1;
13        for (auto [y, w] : ver[x]) {
14            if (dis[y] > dis[x] + w) {
15                dis[y] = dis[x] + w;
16                q.emplace(dis[y], y);
17            }
18        }
19    }
20 };
```

3.2.2 (负权图) Bellman ford 算法

使用结构体存边 (该算法无需存图), 以 $\mathcal{O}(NM)$ 的复杂度计算, 注意, 当所求点的路径上存在负环时, 所求点的答案无法得到, 但是会比 INF 小 (因为负环之后到所求点之间的边权会将 `d[end]` 的值更新), 该性质可以用于判断路径上是否存在负环: 在 $N - 1$ 轮后仍无法得到答案 (一般与 $\text{INF}/2$ 进行比较) 的点, 到达其的路径上存在负环。

下方代码例题: 求解从 1 到 n 号节点的、最多经过 k 条边的最短距离。

```
1 const int N = 550, M = 1e5 + 7;
2 int n, m, k;
3 struct node { int x, y, w; } ver[M];
4 int d[N], backup[N];
5
6 void bf() {
7     memset(d, 0x3f, sizeof d); d[1] = 0;
8     for (int i = 1; i <= k; ++ i) {
9         memcpy(backup, d, sizeof d);
10        for (int j = 1; j <= m; ++ j) {
11            int x = ver[j].x, y = ver[j].y, w = ver[j].w;
12            d[y] = min(d[y], backup[x] + w);
13        }
14    }
15 }
16 int main() {
17     cin >> n >> m >> k;
18     for (int i = 1; i <= m; ++ i) {
19         int x, y, w; cin >> x >> y >> w;
20         ver[i] = {x, y, w};
21     }
22     bf();
23     for (int i = 1; i <= n; ++ i) {
24         if (d[i] > INF / 2) cout << "N" << endl;
25         else cout << d[i] << endl;
```

```

26     }
27 }

```

3.2.3 (负权图) SPFA 算法

以 $O(KM)$ 的复杂度计算，其中 K 虽然为常数，但是可以通过特殊的构造退化接近 N ，需要注意被卡。

```

1  const int N = 1e5 + 7, M = 1e6 + 7;
2  int n, m;
3  int ver[M], ne[M], h[N], edge[M], tot;
4  int d[N], v[N];
5
6  void add(int x, int y, int w) {
7      ver[++ tot] = y, ne[tot] = h[x], h[x] = tot;
8      edge[tot] = w;
9  }
10 void spfa() {
11     ms(d, 0x3f); d[1] = 0;
12     queue<int> q; q.push(1);
13     v[1] = 1;
14     while(!q.empty()) {
15         int x = q.front(); q.pop(); v[x] = 0;
16         for (int i = h[x]; i; i = ne[i]) {
17             int y = ver[i];
18             if(d[y] > d[x] + edge[i]) {
19                 d[y] = d[x] + edge[i];
20                 if(v[y] == 0) q.push(y), v[y] = 1;
21             }
22         }
23     }
24 }
25 int main() {
26     cin >> n >> m;
27     for (int i = 1; i <= m; ++ i) {
28         int x, y, w; cin >> x >> y >> w;
29         add(x, y, w);
30     }
31     spfa();
32     for (int i = 1; i <= n; ++ i) {
33         if (d[i] == INF) cout << "N" << endl;
34         else cout << d[i] << endl;
35     }
36 }

```

3.2.4 (正权稠密图) 邻接矩阵存图+Dijkstra算法

很少使用，以 $O(N^2)$ 的复杂度计算。

```

1  const int N = 3010;
2  int n, m, a[N][N];
3  int d[N], v[N];
4
5  void dji() {
6      ms(d, 0x3f); d[1] = 0;
7      for (int i = 1; i <= n; ++ i) {
8          int x = 0;
9          for (int j = 1; j <= n; ++ j) {
10             if(v[j]) continue;
11             if(x == 0 || d[x] > d[j]) x = j;
12         }

```

```

13     v[x] = 1;
14     for (int j = 1; j <= n; ++j) d[j] = min(d[j], d[x] + a[x][j]);
15 }
16 }
17 int main() {
18     cin >> n >> m;
19     ms(a, 0x3f);
20     for (int i = 1; i <= m; ++i) {
21         int x, y, w; cin >> x >> y >> w;
22         a[x][y] = min(a[x][y], w); //注意需要考虑重边问题
23         a[y][x] = min(a[y][x], w); //无向图建双向边
24     }
25     dji();
26     for (int i = 1; i <= n; ++i) {
27         if (d[i] == INF) cout << "N" << endl;
28         else cout << d[n] << endl;
29     }
30 }

```

3.3 多源汇最短路 (APSP问题)

使用邻接矩阵存图，可以处理负权边，以 $\mathcal{O}(N^3)$ 的复杂度计算。注意，这里建立的是单向边，计算双向边需要额外加边。

```

1  const int N = 210;
2  int n, m, d[N][N];
3
4  void floyd() {
5      for (int k = 1; k <= n; k++)
6          for (int i = 1; i <= n; i++)
7              for (int j = 1; j <= n; j++)
8                  d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
9  }
10 int main() {
11     cin >> n >> m;
12     for (int i = 1; i <= n; i++)
13         for (int j = 1; j <= n; j++)
14             if (i == j) d[i][j] = 0;
15             else d[i][j] = INF;
16     while (m--) {
17         int x, y, w; cin >> x >> y >> w;
18         d[x][y] = min(d[x][y], w);
19     }
20     floyd();
21     for (int i = 1; i <= n; ++i) {
22         for (int j = 1; j <= n; ++j) {
23             if (d[i][j] > INF / 2) cout << "N" << endl;
24             else cout << d[i][j] << endl;
25         }
26     }
27 }

```

3.4 平面图最短路 (对偶图)

对于矩阵图，建立对偶图的过程如下（注释部分为建立原图），其中数据的给出顺序依次为：各 $n(n+1)$ 个数字分别代表从左向右、从上向下、从右向左、从下向上的边。

```

1  for (int i = 1; i <= n + 1; i++) {
2      for (int j = 1, w; j <= n; j++) {
3          cin >> w;

```

```

4     int pre = Hash(i - 1, j), now = Hash(i, j);
5     if (i == 1) {
6         add(s, now, w);
7     } else if (i == n + 1) {
8         add(pre, t, w);
9     } else {
10        add(pre, now, w);
11    }
12    // flow.add(Hash(i, j), Hash(i, j + 1), w);
13 }
14 }
15 for (int i = 1; i <= n; i++) {
16     for (int j = 1, w; j <= n + 1; j++) {
17         cin >> w;
18         int now = Hash(i, j), net = Hash(i, j - 1);
19         if (j == 1) {
20             add(now, t, w);
21         } else if (j == n + 1) {
22             add(s, net, w);
23         } else {
24             add(now, net, w);
25         }
26         // flow.add(Hash(i, j), Hash(i + 1, j), w);
27     }
28 }
29 for (int i = 1; i <= n + 1; i++) {
30     for (int j = 1, w; j <= n; j++) {
31         cin >> w;
32         int now = Hash(i, j), net = Hash(i - 1, j);
33         if (i == 1) {
34             add(now, s, w);
35         } else if (i == n + 1) {
36             add(t, net, w);
37         } else {
38             add(now, net, w);
39         }
40         // flow.add(Hash(i, j), Hash(i, j - 1), w);
41     }
42 }
43 for (int i = 1; i <= n; i++) {
44     for (int j = 1, w; j <= n + 1; j++) {
45         cin >> w;
46         int pre = Hash(i, j - 1), now = Hash(i, j);
47         if (j == 1) {
48             add(t, now, w);
49         } else if (j == n + 1) {
50             add(pre, s, w);
51         } else {
52             add(pre, now, w);
53         }
54         // flow.add(Hash(i, j), Hash(i - 1, j), w);
55     }
56 }

```

3.5 最小生成树 (MST问题)

3.5.1 (稀疏图) Prim算法

使用邻接矩阵存图，以 $\mathcal{O}(N^2 + M)$ 的复杂度计算，思想与 `dijkstra` 基本一致。

```

1  const int N = 550, INF = 0x3f3f3f3f;
2  int n, m, g[N][N];

```

```

3 int d[N], v[N];
4 int prim() {
5     ms(d, 0x3f); //这里的d表示到“最小生成树集合”的距离
6     int ans = 0;
7     for (int i = 0; i < n; ++ i) { //遍历 n 轮
8         int t = -1;
9         for (int j = 1; j <= n; ++ j)
10             if (v[j] == 0 && (t == -1 || d[j] < d[t])) //如果这个点不在集合内且当前距离
集合最近
11                 t = j;
12         v[t] = 1; //将t加入“最小生成树集合”
13         if (i && d[t] == INF) return INF; //如果发现不连通, 直接返回
14         if (i) ans += d[t];
15         for (int j = 1; j <= n; ++ j) d[j] = min(d[j], g[t][j]); //用t更新其他点到集合
的距离
16     }
17     return ans;
18 }
19 int main() {
20     ms(g, 0x3f); cin >> n >> m;
21     while (m -- ) {
22         int x, y, w; cin >> x >> y >> w;
23         g[x][y] = g[y][x] = min(g[x][y], w);
24     }
25     int t = prim();
26     if (t == INF) cout << "impossible" << endl;
27     else cout << t << endl;
28 } //22.03.19已测试

```

3.5.2 (稠密图) Kruskal算法

平均时间复杂度为 $\mathcal{O}(M \log M)$, 简化了并查集。

```

1 struct DSU {
2     vector<int> fa;
3     DSU(int n) : fa(n + 1) {
4         iota(fa.begin(), fa.end(), 0);
5     }
6     int get(int x) {
7         while (x != fa[x]) {
8             x = fa[x] = fa[fa[x]];
9         }
10        return x;
11    }
12    bool merge(int x, int y) { // 设x是y的祖先
13        x = get(x), y = get(y);
14        if (x == y) return false;
15        fa[y] = x;
16        return true;
17    }
18    bool same(int x, int y) {
19        return get(x) == get(y);
20    }
21 };
22 struct Tree {
23     using TII = tuple<int, int, int>;
24     int n;
25     priority_queue<TII, vector<TII>, greater<TII>> ver;
26
27     Tree(int n) {
28         this->n = n;
29     }

```

```

30     void add(int x, int y, int w) {
31         ver.emplace(w, x, y); // 注意顺序
32     }
33     int kruskal() {
34         DSU dsu(n);
35         int ans = 0, cnt = 0;
36         while (ver.size()) {
37             auto [w, x, y] = ver.top();
38             ver.pop();
39             if (dsu.same(x, y)) continue;
40             dsu.merge(x, y);
41             ans += w;
42             cnt++;
43         }
44         assert(cnt < n - 1); // 输入有误, 建树失败
45         return ans;
46     }
47 };

```

3.6 缩点 (Tarjan 算法)

3.6.1 (有向图) 强连通分量缩点

强连通分量缩点后的图称为 SCC。以 $\mathcal{O}(N + M)$ 的复杂度完成上述全部操作。

性质：缩点后的图拥有拓扑序 $color_{cnt}, color_{cnt-1}, \dots, 1$ ，可以不需再另跑一遍 topsort；缩点后的图是一张有向无环图（DAG、拓扑图）。

```

1  struct SCC {
2      int n, now, cnt;
3      vector<vector<int>> ver;
4      vector<int> dfn, low, col, S;
5
6      SCC(int n) : n(n), ver(n + 1), low(n + 1) {
7          dfn.resize(n + 1, -1);
8          col.resize(n + 1, -1);
9          now = cnt = 0;
10     }
11     void add(int x, int y) {
12         ver[x].push_back(y);
13     }
14     void tarjan(int x) {
15         dfn[x] = low[x] = now++;
16         S.push_back(x);
17         for (auto y : ver[x]) {
18             if (dfn[y] == -1) {
19                 tarjan(y);
20                 low[x] = min(low[x], low[y]);
21             } else if (col[y] == -1) {
22                 low[x] = min(low[x], dfn[y]);
23             }
24         }
25         if (dfn[x] == low[x]) {
26             int pre;
27             cnt++;
28             do {
29                 pre = S.back();
30                 col[pre] = cnt;
31                 S.pop_back();
32             } while (pre != x);
33         }
34     }

```

```

35     auto work() { // [cnt 新图的顶点数量]
36         for (int i = 1; i <= n; i++) { // 避免图不连通
37             if (dfn[i] == -1) {
38                 tarjan(i);
39             }
40         }
41
42         vector<int> siz(cnt + 1); // siz 每个 scc 中点的数量
43         vector<vector<int>> adj(cnt + 1);
44         for (int i = 1; i <= n; i++) {
45             siz[col[i]]++;
46             for (auto j : ver[i]) {
47                 int x = col[i], y = col[j];
48                 if (x != y) {
49                     adj[x].push_back(y);
50                 }
51             }
52         }
53         return {cnt, adj, col, siz};
54     }
55 };

```

3.6.2 (无向图) 割边缩点

割边缩点后的图称为边双连通图 (E-DCC)，该模板可以在 $\mathcal{O}(N + M)$ 复杂度内求解图中全部割边、划分边双（颜色相同的点位于同一个边双连通分量中）。

性质补充：对于一个边双，删去任意边后依旧联通；对于边双中的任意两点，一定存在两条不相交的路径连接这两个点（路径上可以有公共点，但是没有公共边）。

```

1  struct EDCC {
2      int n, now, cnt;
3      vector<vector<int>> ver;
4      vector<int> dfn, low, col, S;
5      set<array<int, 2>> bridge;
6
7      EDCC(int n) : n(n), ver(n + 1), low(n + 1) {
8          dfn.resize(n + 1, -1);
9          col.resize(n + 1, -1);
10         now = cnt = 0;
11     }
12     void add(int x, int y) { // 和 scc 相比多了一条连边
13         ver[x].push_back(y);
14         ver[y].push_back(x);
15     }
16     void tarjan(int x, int fa) { // 和 scc 相比多了一个 fa
17         dfn[x] = low[x] = now++;
18         S.push_back(x);
19         for (auto y : ver[x]) {
20             if (y == fa) continue;
21             if (dfn[y] == -1) {
22                 bridge.insert({x, y}); // 储存割边
23                 tarjan(y, x);
24                 low[x] = min(low[x], low[y]);
25             } else if (col[y] == -1 && dfn[y] < dfn[x]) {
26                 bridge.insert({x, y});
27                 low[x] = min(low[x], dfn[y]);
28             }
29         }
30         if (dfn[x] == low[x]) {
31             int pre;
32             cnt++;

```

```

33         do {
34             pre = S.back();
35             col[pre] = cnt;
36             S.pop_back();
37         } while (pre != x);
38     }
39 }
40 auto work() { // [cnt 新图的顶点数量, bridge 全部割边]
41     for (int i = 1; i <= n; i++) { // 避免图不连通
42         if (dfn[i] == -1) {
43             tarjan(i, 0);
44         }
45     }
46
47     vector<int> siz(cnt + 1); // siz 每个边双中点的数量
48     vector<vector<int>> adj(cnt + 1); // adj 新图
49     for (int i = 1; i <= n; i++) {
50         siz[col[i]]++;
51         for (auto j : ver[i]) {
52             int x = col[i], y = col[j];
53             if (x != y) {
54                 adj[x].push_back(y);
55             }
56         }
57     }
58     return tuple{cnt, adj, col, siz};
59 }
60 };

```

3.6.3 (无向图) 割点缩点

割点缩点后的图称为点双连通图 (V-DCC)，该模板可以在 $\mathcal{O}(N + M)$ 复杂度内求解图中全部割点、划分点双（颜色相同的点位于同一个点双连通分量中）。

性质补充：每一个割点至少属于两个点双。

```

1 struct V_DCC {
2     int n;
3     vector<vector<int>> ver, col;
4     vector<int> dfn, low, S;
5     int now, cnt;
6     vector<bool> point; // 记录是否为割点
7
8     V_DCC(int n) : n(n) {
9         ver.resize(n + 1);
10        dfn.resize(n + 1);
11        low.resize(n + 1);
12        col.resize(2 * n + 1);
13        point.resize(n + 1);
14        S.clear();
15        cnt = now = 0;
16    }
17    void add(int x, int y) {
18        if (x == y) return; // 手动去除重边
19        ver[x].push_back(y);
20        ver[y].push_back(x);
21    }
22    void tarjan(int x, int root) {
23        low[x] = dfn[x] = now++;
24        S.push_back(x);
25        if (x == root && !ver[x].size()) { // 特判孤立点
26            ++cnt;

```

```

27         col[cnt].push_back(x);
28         return;
29     }
30
31     int flag = 0;
32     for (auto y : ver[x]) {
33         if (!dfn[y]) {
34             tarjan(y, root);
35             low[x] = min(low[x], low[y]);
36             if (dfn[x] <= low[y]) {
37                 flag++;
38                 if (x != root || flag > 1) {
39                     point[x] = true; // 标记为割点
40                 }
41                 int pre = 0;
42                 cnt++;
43                 do {
44                     pre = S.back();
45                     col[cnt].push_back(pre);
46                     S.pop_back();
47                 } while (pre != y);
48                 col[cnt].push_back(x);
49             }
50         } else {
51             low[x] = min(low[x], dfn[y]);
52         }
53     }
54 }
55 pair<int, vector<vector<int>>> rebuild() { // [新图的顶点数量, 新图]
56     work();
57     vector<vector<int>> adj(cnt + 1);
58     for (int i = 1; i <= cnt; i++) {
59         if (!col[i].size()) { // 注意, 孤立点也是 V-DCC
60             continue;
61         }
62         for (auto j : col[i]) {
63             if (point[j]) { // 如果 j 是割点
64                 adj[i].push_back(point[j]);
65                 adj[point[j]].push_back(i);
66             }
67         }
68     }
69     return {cnt, adj};
70 }
71 void work() {
72     for (int i = 1; i <= n; ++i) { // 避免图不连通
73         if (!dfn[i]) {
74             tarjan(i, i);
75         }
76     }
77 }
78 };

```

3.7 染色法判定二分图 (dfs算法)

判断一张图能否被二分染色。

```

1 vector<int> vis(n + 1);
2 auto dfs = [&](auto self, int x, int type) -> void {
3     vis[x] = type;
4     for (auto y : ver[x]) {
5         if (vis[y] == type) {

```

```

6         cout << "NO\n";
7         exit(0);
8     }
9     if (vis[y]) continue;
10    self(self, y, 3 - type);
11 }
12 };
13 for (int i = 1; i <= n; ++i) {
14     if (vis[i]) {
15         dfs(dfs, i, 1);
16     }
17 }
18 cout << "Yes\n";

```

3.8 链式前向星建图与搜索

很少使用这种建图法。dfs：标准复杂度为 $\mathcal{O}(N + M)$ 。节点子节点的数量包含它自己（至少为 1），深度从 0 开始（根节点深度为 0）。bfs：深度从 1 开始（根节点深度为 1）。topsort：有向无环图（包括非联通）才拥有完整的拓扑序列（故该算法也可用于判断图中是否存在环）。每次找到入度为 0 的点并将其放入待查找队列。

```

1 namespace Graph {
2     const int N = 1e5 + 7;
3     const int M = 1e6 + 7;
4     int tot, h[N], ver[M], ne[M];
5     int deg[N], vis[M];
6
7     void clear(int n) {
8         tot = 0; //多组样例清空
9         for (int i = 1; i <= n; ++i) {
10             h[i] = 0;
11             deg[i] = vis[i] = 0;
12         }
13     }
14     void add(int x, int y) {
15         ver[++tot] = y, ne[tot] = h[x], h[x] = tot;
16         ++deg[y];
17     }
18     void dfs(int x) {
19         a.push_back(x); // DFS序
20         siz[x] = vis[x] = 1;
21         for (int i = h[x]; i; i = ne[i]) {
22             int y = ver[i];
23             if (vis[y]) continue;
24             dis[y] = dis[x] + 1;
25             dfs(y);
26             siz[x] += siz[y];
27         }
28         a.push_back(x);
29     }
30     void bfs(int s) {
31         queue<int> q;
32         q.push(s);
33         dis[s] = 1;
34         while (!q.empty()) {
35             int x = q.front();
36             q.pop();
37             for (int i = h[x]; i; i = ne[i]) {
38                 int y = ver[i];
39                 if (dis[y]) continue;
40                 d[y] = d[x] + 1;
41                 q.push(y);

```

```

42     }
43 }
44 }
45 bool topsort() {
46     queue<int> q;
47     vector<int> ans;
48     for (int i = 1; i <= n; ++i)
49         if (deg[i] == 0) q.push(i);
50     while (!q.empty()) {
51         int x = q.front();
52         q.pop();
53         ans.push_back(x);
54         for (int i = h[x]; i; i = ne[i]) {
55             int y = ver[i];
56             --deg[y];
57             if (deg[y] == 0) q.push(y);
58         }
59     }
60     return ans.size() == n; //判断是否存在拓扑排序
61 }
62 } // namespace Graph

```

3.9 一般图最大匹配（带花树算法）

与二分图匹配的差别在于图中可能存在奇环，时间复杂度与边的数量无关，为 $\mathcal{O}(N^3)$ 。下方模板编号从 0 开始，例题为 [UOJ #79. 一般图最大匹配](#)。

```

1  struct Graph {
2      int n;
3      vector<vector<int>> e;
4      Graph(int n) : n(n), e(n) {}
5      void add(int u, int v) {
6          e[u].push_back(v);
7          e[v].push_back(u);
8      }
9      pair<int, vector<int>> work() {
10         vector<int> match(n, -1), vis(n), link(n), f(n), dep(n);
11         auto find = [&](int u) {
12             while (f[u] != u) u = f[u] = f[f[u]];
13             return u;
14         };
15         auto lca = [&](int u, int v) {
16             u = find(u), v = find(v);
17             while (u != v) {
18                 if (dep[u] < dep[v]) swap(u, v);
19                 u = find(link[match[u]]);
20             }
21             return u;
22         };
23         queue<int> q;
24         auto blossom = [&](int u, int v, int p) {
25             while (find(u) != p) {
26                 link[u] = v;
27                 v = match[u];
28                 if (vis[v] == 0) {
29                     vis[v] = 1;
30                     q.push(v);
31                 }
32                 f[u] = f[v] = p;
33                 u = link[v];
34             }
35         };

```

```

36     auto augment = [&](int u) {
37         while (!q.empty()) q.pop();
38         iota(f.begin(), f.end(), 0);
39         fill(vis.begin(), vis.end(), -1);
40         q.push(u);
41         vis[u] = 1;
42         dep[u] = 0;
43         while (!q.empty()) {
44             int u = q.front();
45             q.pop();
46             for (auto v : e[u]) {
47                 if (vis[v] == -1) {
48                     vis[v] = 0;
49                     link[v] = u;
50                     dep[v] = dep[u] + 1;
51                     if (match[v] == -1) {
52                         for (int x = v, y = u, temp; y != -1;
53                             x = temp, y = x == -1 ? -1 : link[x]) {
54                             temp = match[y];
55                             match[x] = y;
56                             match[y] = x;
57                         }
58                         return;
59                     }
60                     vis[match[v]] = 1;
61                     dep[match[v]] = dep[u] + 2;
62                     q.push(match[v]);
63                 } else if (vis[v] == 1 && find(v) != find(u)) {
64                     int p = lca(u, v);
65                     blossom(u, v, p);
66                     blossom(v, u, p);
67                 }
68             }
69         }
70     };
71     auto greedy = [&]() {
72         for (int u = 0; u < n; ++u) {
73             if (match[u] != -1) continue;
74             for (auto v : e[u]) {
75                 if (match[v] == -1) {
76                     match[u] = v;
77                     match[v] = u;
78                     break;
79                 }
80             }
81         }
82     };
83     greedy();
84     for (int u = 0; u < n; u++) {
85         if (match[u] == -1) {
86             augment(u);
87         }
88     }
89     int ans = 0;
90     for (int u = 0; u < n; u++) {
91         if (match[u] != -1) {
92             ans++;
93         }
94     }
95     return {ans / 2, match};
96 }
97 };
98
99 signed main() {

```

```

100     int n, m;
101     cin >> n >> m;
102
103     Graph graph(n);
104     for (int i = 1; i <= m; i++) {
105         int x, y;
106         cin >> x >> y;
107         graph.add(x - 1, y - 1);
108     }
109     auto [ans, match] = graph.work();
110     cout << ans << endl;
111     for (auto it : match) {
112         cout << it + 1 << " ";
113     }
114 }

```

3.10 一般图最大权匹配（带权带花树算法）

下方模板编号从 1 开始，复杂度为 $\mathcal{O}(N^3)$ 。

```

1  namespace Graph {
2      const int N = 403 * 2; //两倍点数
3      typedef int T; //权值大小
4      const T inf = numeric_limits<int>::max() >> 1;
5      struct Q { int u, v; T w; } e[N][N];
6      T lab[N];
7      int n, m = 0, id, h, t, lk[N], sl[N], st[N], f[N], b[N][N], s[N], ed[N], q[N];
8      vector<int> p[N];
9      #define dvd(x) (lab[x.u] + lab[x.v] - e[x.u][x.v].w * 2)
10     #define FOR(i, b) for (int i = 1; i <= (int)(b); i++)
11     #define ALL(x) (x).begin(), (x).end()
12     #define ms(x, i) memset(x + 1, i, m * sizeof x[0])
13     void upd(int u, int v) {
14         if (!sl[v] || dvd(e[u][v]) < dvd(e[sl[v]][v])) {
15             sl[v] = u;
16         }
17     }
18     void ss(int v) {
19         sl[v] = 0;
20         FOR(u, n) {
21             if (e[u][v].w > 0 && st[u] != v && !s[st[u]]) {
22                 upd(u, v);
23             }
24         }
25     }
26     void ins(int u) {
27         if (u <= n) { q[++t] = u; }
28         else {
29             for (int v : p[u]) ins(v);
30         }
31     }
32     void mdf(int u, int w) {
33         st[u] = w;
34         if (u > n) {
35             for (int v : p[u]) mdf(v, w);
36         }
37     }
38     int gr(int u, int v) {
39         v = find(ALL(p[u]), v) - p[u].begin();
40         if (v & 1) {
41             reverse(1 + ALL(p[u]));
42             return (int)p[u].size() - v;

```

```

43     }
44     return v;
45 }
46 void stm(int u, int v) {
47     lk[u] = e[u][v].v;
48     if (u <= n) return;
49     Q w = e[u][v];
50     int x = b[u][w.u], y = gr(u, x);
51     for (int i = 0; i < y; i++) {
52         stm(p[u][i], p[u][i ^ 1]);
53     }
54     stm(x, v);
55     rotate(p[u].begin(), y + ALL(p[u]));
56 }
57 void aug(int u, int v) {
58     int w = st[lk[u]];
59     stm(u, v);
60     if (!w) return;
61     stm(w, st[f[w]]), aug(st[f[w]], w);
62 }
63 int lca(int u, int v) {
64     for (++id; u | v; swap(u, v)) {
65         if (!u) continue;
66         if (ed[u] == id) return u;
67         ed[u] = id;
68         if (u = st[lk[u]]) u = st[f[u]];
69     }
70     return 0;
71 }
72 void add(int u, int a, int v) {
73     int x = n + 1, i, j;
74     while (x <= m && st[x]) ++x;
75     if (x > m) ++m;
76     lab[x] = s[x] = st[x] = 0;
77     lk[x] = lk[a];
78     p[x].clear();
79     p[x].push_back(a);
80     for (i = u; i != a; i = st[f[j]]) {
81         p[x].push_back(i);
82         p[x].push_back(j = st[lk[i]]);
83         ins(j);
84     }
85     reverse(1 + ALL(p[x]));
86     for (i = v; i != a; i = st[f[j]]) { // 复制, 只需改循环
87         p[x].push_back(i);
88         p[x].push_back(j = st[lk[i]]);
89         ins(j);
90     }
91     mdf(x, x);
92     FOR(i, m) {
93         e[x][i].w = e[i][x].w = 0;
94     }
95     memset(b[x] + 1, 0, n * sizeof b[0][0]);
96     for (int u : p[x]) {
97         FOR(v, m) {
98             if (!e[x][v].w || dvd(e[u][v]) < dvd(e[x][v])) {
99                 e[x][v] = e[u][v], e[v][x] = e[v][u];
100             }
101         }
102         FOR(v, n) {
103             if (b[u][v]) { b[x][v] = u; }
104         }
105     }
106     ss(x);

```

```

107     }
108     void ex(int u) {
109         for (int x : p[u]) mdf(x, x);
110         int a = b[u][e[u][f[u]].u], r = gr(u, a);
111         for (int i = 0; i < r; i += 2) {
112             int x = p[u][i], y = p[u][i + 1];
113             f[x] = e[y][x].u;
114             s[x] = 1;
115             s[y] = sl[x] = 0;
116             ss(y), ins(y);
117         }
118         s[a] = 1, f[a] = f[u];
119         for (int i = r + 1; i < p[u].size(); i++) {
120             s[p[u][i]] = -1;
121             ss(p[u][i]);
122         }
123         st[u] = 0;
124     }
125     bool on(const Q &e) {
126         int u = st[e.u], v = st[e.v];
127         if (s[v] == -1) {
128             f[v] = e.u, s[v] = 1;
129             int a = st[lk[v]];
130             sl[v] = sl[a] = s[a] = 0;
131             ins(a);
132         } else if (!s[v]) {
133             int a = lca(u, v);
134             if (!a) {
135                 return aug(u, v), aug(v, u), 1;
136             } else {
137                 add(u, a, v);
138             }
139         }
140         return 0;
141     }
142     bool bfs() {
143         ms(s, -1), ms(sl, 0);
144         h = 1, t = 0;
145         FOR(i, m) {
146             if (st[i] == i && !lk[i]) {
147                 f[i] = s[i] = 0;
148                 ins(i);
149             }
150         }
151         if (h > t) return 0;
152         while (1) {
153             while (h <= t) {
154                 int u = q[h++];
155                 if (s[st[u]] == 1) continue;
156                 FOR(v, n) {
157                     if (e[u][v].w > 0 && st[u] != st[v]) {
158                         if (dvd(e[u][v])) upd(u, st[v]);
159                         else if (on(e[u][v])) return 1;
160                     }
161                 }
162             }
163             T x = inf;
164             for (int i = n + 1; i <= m; i++) {
165                 if (st[i] == i && s[i] == 1) {
166                     x = min(x, lab[i] >> 1);
167                 }
168             }
169             FOR(i, m) {
170                 if (st[i] == i && sl[i] && s[i] != 1) {

```

```

171         x = min(x, dvd(e[sl[i]][i]) >> s[i] + 1);
172     }
173 }
174 FOR(i, n) {
175     if (~s[st[i]]) {
176         if ((lab[i] += (s[st[i]] * 2 - 1) * x) <= 0) return 0;
177     }
178 }
179 for (int i = n + 1; i <= m; i++) {
180     if (st[i] == i && ~s[st[i]]) {
181         lab[i] += (2 - s[st[i]] * 4) * x;
182     }
183 }
184 h = 1, t = 0;
185 FOR(i, m) {
186     if (st[i] == i && sl[i] && st[sl[i]] != i && !dvd(e[sl[i]][i]) &&
on(e[sl[i]][i])) {
187         return 1;
188     }
189 }
190 for (int i = n + 1; i <= m; i++) {
191     if (st[i] == i && s[i] == 1 && !lab[i]) ex(i);
192 }
193 }
194 return 0;
195 }
196 template<typename TT> i64 work(int N, const vector<tuple<int, int, TT>> &edges)
{
197     ms(ed, 0), ms(lk, 0);
198     n = m = N; id = 0;
199     iota(st + 1, st + n + 1, 1);
200     T wm = 0; i64 r = 0;
201     FOR(i, n) FOR(j, n) {
202         e[i][j] = {i, j, 0};
203     }
204     for (auto [u, v, w] : edges) {
205         wm = max(wm, e[v][u].w = e[u][v].w = max(e[u][v].w, (T)w));
206     }
207     FOR(i, n) { p[i].clear(); }
208     FOR(i, n) FOR(j, n) {
209         b[i][j] = i * (i == j);
210     }
211     fill_n(lab + 1, n, wm);
212     while (bfs()) {};
213     FOR(i, n) if (lk[i]) {
214         r += e[i][lk[i]].w;
215     }
216     return r / 2;
217 }
218 auto match() {
219     vector<array<int, 2>> ans;
220     FOR(i, n) if (lk[i]) {
221         ans.push_back({i, lk[i]});
222     }
223     return ans;
224 }
225 } // namespace Graph
226 using Graph::work, Graph::match;
227
228 signed main() {
229     int n, m;
230     cin >> n >> m;
231     vector<tuple<int, int, i64>> ver(m);
232     for (auto &[u, v, w] : ver) {

```

```

233     cin >> u >> v >> w;
234 }
235 cout << work(n, ver) << "\n";
236 auto ans = match();
237 }

```

3.11 二分图最大匹配

定义：找到边的数量最多的那个匹配。

一般我们规定，左半部包含 n_1 个点（编号 $1 - n_1$ ），右半部包含 n_2 个点（编号 $1 - n_2$ ），保证任意一条边的两个端点都不可能在同一部分中。

3.11.1 匈牙利算法（KM算法）解

$\mathcal{O}(NM)$ 。

```

1  signed main() {
2      int n1, n2, m;
3      cin >> n1 >> n2 >> m;
4
5      vector<vector<int>> ver(n1 + 1);
6      for (int i = 1; i <= m; ++i) {
7          int x, y;
8          cin >> x >> y;
9          ver[x].push_back(y); //只需要建立单向边
10     }
11
12     int ans = 0;
13     vector<int> match(n2 + 1);
14     for (int i = 1; i <= n1; ++i) {
15         vector<int> vis(n2 + 1);
16         auto dfs = [&](auto self, int x) -> bool {
17             for (auto y : ver[x]) {
18                 if (vis[y]) continue;
19                 vis[y] = 1;
20                 if (!match[y] || self(self, match[y])) {
21                     match[y] = x;
22                     return true;
23                 }
24             }
25             return false;
26         };
27         if (dfs(dfs, i)) {
28             ans++;
29         }
30     }
31     cout << ans << endl;
32 }

```

3.11.2 HopcroftKarp算法（HK算法、基于最大流模型）解

该算法基于网络流中的最大流模型，但是会比直接使用 dinic 算法更快，因为常数更小，最坏时间复杂度为 $\mathcal{O}(\sqrt{NM})$ ，但实际运行复杂度还要比这一数字小上 10 倍。

```

1  struct HopcroftKarp {
2      vector<vector<int>> g;
3      vector<int> pa, pb, vis;
4      int n, m, dfn, res;
5

```

```

6 HopcroftKarp(int _n, int _m) : n(_n + 1), m(_m + 1) {
7     assert(0 <= n && 0 <= m);
8     pa.assign(n, -1);
9     pb.assign(m, -1);
10    vis.resize(n);
11    g.resize(n);
12    res = 0;
13    dfn = 0;
14 }
15 void add(int x, int y) {
16     assert(0 <= x && x < n && 0 <= y && y < m);
17     g[x].push_back(y);
18 }
19 bool dfs(int v) {
20     vis[v] = dfn;
21     for (int u : g[v]) {
22         if (pb[u] == -1) {
23             pb[u] = v;
24             pa[v] = u;
25             return true;
26         }
27     }
28     for (int u : g[v]) {
29         if (vis[pb[u]] != dfn && dfs(pb[u])) {
30             pa[v] = u;
31             pb[u] = v;
32             return true;
33         }
34     }
35     return false;
36 }
37 int work() {
38     while (1) {
39         dfn++;
40         int cnt = 0;
41         for (int i = 0; i < n; i++) {
42             if (pa[i] == -1 && dfs(i)) {
43                 cnt++;
44             }
45         }
46         if (cnt == 0) break;
47         res += cnt;
48     }
49     return res;
50 }
51 };
52 signed main() {
53     int n1, n2, m;
54     cin >> n1 >> n2 >> m;
55     HopcroftKarp flow(n1, n2);
56     while (m--) {
57         int x, y;
58         cin >> x >> y;
59         flow.add(x, y);
60     }
61     cout << flow.work() << endl;
62 }

```

3.12 二分图最大权匹配（二分图完美匹配）

定义：找到边权和最大的那个匹配。

一般我们规定，左半部包含 n_1 个点（编号 $1 - n_1$ ），右半部包含 n_2 个点（编号 $1 - n_2$ ）。

使用匈牙利算法（KM算法）解，时间复杂度为 $\mathcal{O}(N^3)$ 。下方模板用于求解最大权值、且可以输出其中一种可行方案，例题为 [UOJ #80. 二分图最大权匹配](#)。

```
1 struct MaxCostMatch {
2     vector<int> ansl, ansr, pre;
3     vector<int> lx, ly;
4     vector<vector<int>> ver;
5     int n;
6
7     MaxCostMatch(int n) : n(n) {
8         ver.resize(n + 1, vector<int>(n + 1));
9         ansl.resize(n + 1, -1);
10        ansr.resize(n + 1, -1);
11        lx.resize(n + 1);
12        ly.resize(n + 1, -1E18);
13        pre.resize(n + 1);
14    }
15    void add(int x, int y, int w) {
16        ver[x][y] = w;
17    }
18    void bfs(int x) {
19        vector<bool> visl(n + 1), visr(n + 1);
20        vector<int> slack(n + 1, 1E18);
21        queue<int> q;
22        function<bool(int)> check = [&](int x) {
23            visr[x] = 1;
24            if (~ansr[x]) {
25                q.push(ansr[x]);
26                visl[ansr[x]] = 1;
27                return false;
28            }
29            while (~x) {
30                ansr[x] = pre[x];
31                swap(x, ansl[pre[x]]);
32            }
33            return true;
34        };
35        q.push(x);
36        visl[x] = 1;
37        while (1) {
38            while (!q.empty()) {
39                int x = q.front();
40                q.pop();
41                for (int y = 1; y <= n; ++y) {
42                    if (visr[y]) continue;
43                    int del = lx[x] + ly[y] - ver[x][y];
44                    if (del < slack[y]) {
45                        pre[y] = x;
46                        slack[y] = del;
47                        if (!slack[y] && check(y)) return;
48                    }
49                }
50            }
51            int val = 1E18;
52            for (int i = 1; i <= n; ++i) {
53                if (!visr[i]) {
```

```

54         val = min(val, slack[i]);
55     }
56 }
57 for (int i = 1; i <= n; ++i) {
58     if (visl[i]) lx[i] -= val;
59     if (visr[i]) {
60         ly[i] += val;
61     } else {
62         slack[i] -= val;
63     }
64 }
65 for (int i = 1; i <= n; ++i) {
66     if (!visr[i] && !slack[i] && check(i)) {
67         return;
68     }
69 }
70 }
71 }
72 int work() {
73     for (int i = 1; i <= n; ++i) {
74         for (int j = 1; j <= n; ++j) {
75             ly[i] = max(ly[i], ver[j][i]);
76         }
77     }
78     for (int i = 1; i <= n; ++i) bfs(i);
79     int res = 0;
80     for (int i = 1; i <= n; ++i) {
81         res += ver[i][ansl[i]];
82     }
83     return res;
84 }
85 void getMatch(int x, int y) { // 获取方案 (0代表无匹配)
86     for (int i = 1; i <= x; ++i) {
87         cout << (ver[i][ansl[i]] ? ansl[i] : 0) << " ";
88     }
89     cout << endl;
90     for (int i = 1; i <= y; ++i) {
91         cout << (ver[i][ansr[i]] ? ansr[i] : 0) << " ";
92     }
93     cout << endl;
94 }
95 };
96
97 signed main() {
98     int n1, n2, m;
99     cin >> n1 >> n2 >> m;
100
101     MaxCostMatch match(max(n1, n2));
102     for (int i = 1; i <= m; i++) {
103         int x, y, w;
104         cin >> x >> y >> w;
105         match.add(x, y, w);
106     }
107     cout << match.work() << '\n';
108     match.getMatch(n1, n2);
109 }

```

3.13 二分图最大独立点集 (Konig 定理)

给出一张二分图，要求选择一些点使得它们两两没有边直接连接。最小点覆盖等价于最大匹配数，转换为最小割模板，答案即为总点数减去最大流得到的值。

```
1 | cout << n - flow.work(s, t) << endl;
```

3.14 最长路 (topsort+DP算法)

计算一张 DAG 中的最长路径，在执行前可能需要使用 tarjan 重构一张正确的 DAG，复杂度 $\mathcal{O}(N + M)$ 。

```
1 | namespace LP { // Longest_Path, 最长路封装 (Topsort)
2 |     vector<PII> ver[N];
3 |     int deg[N];
4 |     int d[N];
5 |
6 |     void clear(int n) {
7 |         FOR(i, 1, n) {
8 |             ver[i].clear();
9 |             deg[i] = 0;
10 |         }
11 |     }
12 |     void add(int x, int y, int w) {
13 |         ver[x].pb({y, w});
14 |         ++deg[y];
15 |     }
16 |     void topsort(int n, int s) {
17 |         queue<int> q;
18 |         FOR(i, 1, n) {
19 |             if (deg[i] == 0) q.push(i);
20 |         }
21 |         fill(d + 1, d + 1 + n, -INFF);
22 |         d[s] = 0;
23 |         while (!q.empty()) {
24 |             int x = q.front();
25 |             q.pop();
26 |             for (auto [y, w] : ver[x]) {
27 |                 d[y] = max(d[y], d[x] + w);
28 |                 --deg[y];
29 |                 if (deg[y] == 0) q.push(y);
30 |             }
31 |         }
32 |     }
33 |     void solve(int n, int s) {
34 |         topsort(n, s);
35 |     }
36 | } // namespace LP
37 | int main() {
38 |     int n, m;
39 |     cin >> n >> m;
40 |     FOR(i, 1, n) {
41 |         int x, y, w;
42 |         cin >> x >> y >> w;
43 |         LP::add(x, y, w);
44 |     }
45 |     int start, end;
46 |     cin >> start >> end; //输入源汇
47 |     LP::solve(n, start);
48 |     cout << LP::d[end] << endl;
49 | }
```

```

50 LP::clear(n); //清空
51 }

```

3.15 最短路径树 (SPT问题)

定义：在一张无向带权联通图中，有这样一棵**生成树**：满足从根节点到任意点的路径都为原图中根到任意点的最短路径。

性质：记根节点 $Root$ 到某一结点 x 的最短距离 $dis_{Root,x}$ ，在 SPT 上这两点之间的距离为 $len_{Root,x}$ ——则两者长度相等。

该算法与最小生成树无关，基于最短路 Dijkstra 算法完成（但多了个等于号）。下方代码实现的功能为：读入图后，输出以 1 为根的 SPT 所使用的各条边的编号、边权和。

```

1  map<pair<int, int>, int> id;
2  namespace G {
3      vector<pair<int, int> > ver[N];
4      map<pair<int, int>, int> edge;
5      int v[N], d[N], pre[N], vis[N];
6      int ans = 0;
7
8      void add(int x, int y, int w) {
9          ver[x].push_back({y, w});
10         edge[{x, y}] = edge[{y, x}] = w;
11     }
12     void djikstra(int s) { // ! 注意, 该 djikstra 并非原版, 多加了一个等于号
13         priority_queue<PII, vector<PII>, greater<PII> > q; q.push({0, s});
14         memset(d, 0x3f, sizeof d); d[s] = 0;
15         while (!q.empty()) {
16             int x = q.top().second; q.pop();
17             if (v[x]) continue; v[x] = 1;
18             for (auto [y, w] : ver[x]) {
19                 if (d[y] >= d[x] + w) { // ! 注意, SPT 这里修改为>=号
20                     d[y] = d[x] + w;
21                     pre[y] = x; // 记录前驱结点
22                     q.push({d[y], y});
23                 }
24             }
25         }
26     }
27     void dfs(int x) {
28         vis[x] = 1;
29         for (auto [y, w] : ver[x]) {
30             if (vis[y]) continue;
31             if (pre[y] == x) {
32                 cout << id[{x, y}] << " "; // 输出SPT所使用的边编号
33                 ans += edge[{x, y}];
34                 dfs(y);
35             }
36         }
37     }
38     void solve(int n) {
39         djikstra(1); // 以 1 为根
40         dfs(1); // 以 1 为根
41         cout << endl << ans; // 输出SPT的边权和
42     }
43 }
44 bool Solve() {
45     int n, m; cin >> n >> m;
46     for (int i = 1; i <= m; ++ i) {
47         int x, y, w; cin >> x >> y >> w;
48         G::add(x, y, w), G::add(y, x, w);

```

```

49     id[{x, y}] = id[{y, x}] = i;
50 }
51 G::solve(n);
52 return 0;
53 }

```

3.16 无源汇点的最小割问题 Stoer-Wagner

也称为全局最小割。定义补充（与《网络流》中的定义不同）：

割：是一个边集，去掉其中所有边能使一张网络流图不再连通（即分成两个子图）。

通过递归的方式来解决无向正权图上的全局最小割问题，算法复杂度 $\mathcal{O}(VE + V^2 \log V)$ ，一般可近似看作 $\mathcal{O}(V^3)$ 。

```

1  signed main() {
2      int n, m;
3      cin >> n >> m;
4
5      DSU dsu(n); // 这里引入DSU判断图是否联通，如题目有保证，则不需要此步骤
6      vector<vector<int>> edge(n + 1, vector<int>(n + 1));
7      for (int i = 1; i <= m; i++) {
8          int x, y, w;
9          cin >> x >> y >> w;
10         dsu.merge(x, y);
11         edge[x][y] += w;
12         edge[y][x] += w;
13     }
14
15     if (dsu.Poi(1) != n || m < n - 1) { // 图不联通
16         cout << 0 << endl;
17         return 0;
18     }
19
20     int MinCut = INF, S = 1, T = 1; // 虚拟源汇点
21     vector<int> bin(n + 1);
22     auto contract = [&]() -> int { // 求解S到T的最小割，定义为 cut of phase
23         vector<int> dis(n + 1), vis(n + 1);
24         int Min = 0;
25         for (int i = 1; i <= n; i++) {
26             int k = -1, maxc = -1;
27             for (int j = 1; j <= n; j++) {
28                 if (!bin[j] && !vis[j] && dis[j] > maxc) {
29                     k = j;
30                     maxc = dis[j];
31                 }
32             }
33             if (k == -1) return Min;
34             S = T, T = k, Min = maxc;
35             vis[k] = 1;
36             for (int j = 1; j <= n; j++) {
37                 if (!bin[j] && !vis[j]) {
38                     dis[j] += edge[k][j];
39                 }
40             }
41         }
42         return Min;
43     };
44     for (int i = 1; i < n; i++) { // 这里取不到等号
45         int val = contract();
46         bin[T] = 1;
47         MinCut = min(MinCut, val);

```

```

48     if (!MinCut) {
49         cout << 0 << endl;
50         return 0;
51     }
52     for (int j = 1; j <= n; j++) {
53         if (!bin[j]) {
54             edge[S][j] += edge[j][T];
55             edge[j][S] += edge[j][T];
56         }
57     }
58 }
59 cout << MinCut << endl;
60 }

```

3.17 欧拉路径/欧拉回路 Hierholzers

欧拉路径：一笔画完图中全部边，画的顺序就是一个可行解；当起点终点相同时称欧拉回路。

3.17.1 有向图欧拉路径存在判定

有向图欧拉路径存在：¹ 恰有一个点出度比入度多 1（为起点）；² 恰有一个点入度比出度多 1（为终点）；³ 恰有 $N - 2$ 个点入度均等于出度。如果是欧拉回路，则上方起点与终点的条件不存在，全部点均要满足最后一个条件。

```

1  signed main() {
2      int n, m;
3      cin >> n >> m;
4
5      DSU dsu(n + 1); // 如果保证连通，则不需要 DSU
6      vector<unordered_multiset<int>> ver(n + 1); // 如果对于字典序有要求，则不能使用
unordered
7      vector<int> degI(n + 1), degO(n + 1);
8      for (int i = 1; i <= m; i++) {
9          int x, y;
10         cin >> x >> y;
11         ver[x].insert(y);
12         degI[y]++;
13         degO[x]++;
14         dsu.merge(x, y); // 直接当无向图
15     }
16     int s = 1, t = 1, cnt = 0;
17     for (int i = 1; i <= n; i++) {
18         if (degI[i] == degO[i]) {
19             cnt++;
20         } else if (degI[i] + 1 == degO[i]) {
21             s = i;
22         } else if (degI[i] == degO[i] + 1) {
23             t = i;
24         }
25     }
26     if (dsu.size(1) != n || (cnt != n - 2 && cnt != n)) {
27         cout << "No\n";
28     } else {
29         cout << "Yes\n";
30     }
31 }

```

3.17.2 无向图欧拉路径存在判定

无向图欧拉路径存在：¹ 恰有两个点度数为奇数（为起点与终点）；² 恰有 $N - 2$ 个点度数为偶数。

```
1 signed main() {
2     int n, m;
3     cin >> n >> m;
4
5     DSU dsu(n + 1); // 如果保证连通, 则不需要 DSU
6     vector<unordered_multiset<int>> ver(n + 1); // 如果对于字典序有要求, 则不能使用
    unordered
7     vector<int> deg(n + 1);
8     for (int i = 1; i <= m; i++) {
9         int x, y;
10        cin >> x >> y;
11        ver[x].insert(y);
12        ver[y].insert(x);
13        deg[y]++;
14        deg[x]++;
15        dsu.merge(x, y); // 直接当无向图
16    }
17    int s = -1, t = -1, cnt = 0;
18    for (int i = 1; i <= n; i++) {
19        if (deg[i] % 2 == 0) {
20            cnt++;
21        } else if (s == -1) {
22            s = i;
23        } else {
24            t = i;
25        }
26    }
27    if (dsu.size(1) != n || (cnt != n - 2 && cnt != n)) {
28        cout << "No\n";
29    } else {
30        cout << "Yes\n";
31    }
32 }
```

3.17.3 有向图欧拉路径求解（字典序最小）

```
1 vector<int> ans;
2 auto dfs = [&](auto self, int x) -> void {
3     while (ver[x].size()) {
4         int net = *ver[x].begin();
5         ver[x].erase(ver[x].begin());
6         self(self, net);
7     }
8     ans.push_back(x);
9 };
10 dfs(dfs, s);
11 reverse(ans.begin(), ans.end());
12 for (auto it : ans) {
13     cout << it << " ";
14 }
```

3.17.4 无向图欧拉路径求解

```
1 auto dfs = [&](auto self, int x) -> void {
2     while (ver[x].size()) {
3         int net = *ver[x].begin();
4         ver[x].erase(ver[x].find(net));
5         ver[net].erase(ver[net].find(x));
6         cout << x << " " << net << endl;
7         self(self, net);
8     }
9 };
10 dfs(dfs, s);
```

3.18 差分约束

Provided by Hamine。给出一组包含 m 个不等式，有 n 个未知数的形如：

$$\begin{cases} x_{c_1} - x_{c'_1} \leq y_1 \\ x_{c_2} - x_{c'_2} \leq y_2 \\ \dots \\ x_{c_m} - x_{c'_m} \leq y_m \end{cases}$$

的不等式组，求任意一组满足这个不等式组的解。若无解，输出 "NO"。参考

```
1 const int N = 5e3 + 10;
2 struct edge{
3     LL u, v, w;
4 }e[N];
5 LL n, m, d[N];
6 void bellman_ford(){
7     memset(d, 0x3f, sizeof d);
8     d[1] = 0;
9     for (int i = 1; i < n; i++)
10         for (int j = 0; j < m; j++)
11             d[e[j].v] = min(d[e[j].v], d[e[j].u] + e[j].w);
12     for (int i = 0; i < m; i++)
13         if (d[e[i].v] > d[e[i].u] + e[i].w){
14             cout << "NO\n";
15             return;
16         }
17     for (int i = 1; i <= n; i++)
18         cout << d[i] << " \n"[i == n];
19 }
20 int main(){
21     ios::sync_with_stdio(false); cin.tie(0);
22     cin >> n >> m;
23     for (int i = 0; i < m; i++){
24         LL u, v, w;
25         cin >> v >> u >> w;
26         e[i] = {u, v, w};
27     }
28     bellman_ford();
29     return 0;
30 }
```

3.19 2-Sat

3.19.1 基础封装

基于 tarjan 缩点，时间复杂度为 $\mathcal{O}(N + M)$ 。注意下标从 0 开始，答案输出为字典序最小的一个可行解。

```
1 struct TwoSat {
2     int n;
3     vector<vector<int>> e;
4     vector<bool> ans;
5     TwoSat(int n) : n(n), e(2 * n), ans(n) {}
6     void add(int u, bool f, int v, bool g) {
7         e[2 * u + !f].push_back(2 * v + g);
8         e[2 * v + !g].push_back(2 * u + f);
9     }
10    bool work() {
11        vector<int> id(2 * n, -1), dfn(2 * n, -1), low(2 * n, -1);
12        vector<int> stk;
13        int now = 0, cnt = 0;
14        auto tarjan = [&](auto self, int u) -> void {
15            stk.push_back(u);
16            dfn[u] = low[u] = now++;
17            for (auto v : e[u]) {
18                if (dfn[v] == -1) {
19                    self(self, v);
20                    low[u] = min(low[u], low[v]);
21                } else if (id[v] == -1) {
22                    low[u] = min(low[u], dfn[v]);
23                }
24            }
25            if (dfn[u] == low[u]) {
26                int v;
27                do {
28                    v = stk.back();
29                    stk.pop_back();
30                    id[v] = cnt;
31                } while (v != u);
32                ++cnt;
33            }
34        };
35        for (int i = 0; i < 2 * n; ++i) {
36            if (dfn[i] == -1) {
37                tarjan(tarjan, i);
38            }
39        }
40        for (int i = 0; i < n; ++i) {
41            if (id[2 * i] == id[2 * i + 1]) return false;
42            ans[i] = id[2 * i] > id[2 * i + 1];
43        }
44        return true;
45    }
46    vector<bool> answer() {
47        return ans;
48    }
49 };
```

3.19.2 答案不唯一时不输出

在运行后针对每一个点进行一次 dfs，时间复杂度为 $\mathcal{O}(N^2)$ ，当且仅当答案唯一时才输出，否则输出 ? 替代。

```
1 // 结构体中增加
2 int check(int x, int y) {
3     vector<int> vis(2 * n);
4     auto dfs = [&](auto self, int x) -> void {
5         vis[x] = 1;
6         for (auto y : e[x]) {
7             if (vis[y]) continue;
8             self(self, y);
9         }
10    };
11    dfs(dfs, x);
12    return vis[y];
13 }
14 // 主函数中增加
15 for (int i = 0; i < n; i++) {
16     if (sat.check(2 * i, 2 * i + 1)) {
17         cout << 1 << " ";
18     } else if (sat.check(2 * i + 1, 2 * i)) {
19         cout << 0 << " ";
20     } else {
21         cout << "?" << " ";
22     }
23 }
```

3.20 常见结论

1. 要在有向图上求一个最大点集，使得任意两个点 (i, j) 之间至少存在一条路径（可以从 i 到 j ，也可以反过来，这两种有一个就行），即求解最长路；
2. 要求出连通图上的任意一棵生成树，只需要跑一遍 bfs；
3. 给出一棵树，要求添加尽可能多的边，使得其是二分图：对树进行二分染色，显然，相同颜色的点之间连边不会破坏二分图的性质，故可添加的最多的边数即为 $cnt_{\text{Black}} * cnt_{\text{White}} - (n - 1)$ ；
4. 当一棵树可以被黑白染色时，所有染黑节点的度之和等于所有染白节点的度之和；
5. 在竞赛图中，入度小的点，必定能到达出度小（入度大）的点 [See](#)。
6. 在竞赛图中，将所有点按入度从小到大排序，随后依次遍历，若对于某一点 i 满足前 i 个点的入度之和恰好等于 $\left\lfloor \frac{n \cdot (n + 1)}{2} \right\rfloor$ ，那么对于上一次满足这一条件的点 p ， $p + 1$ 到 i 点构成一个新的强连通分量 [See](#)。

举例说明，设满足上方条件的点为 p_1, p_2 ($p_1 + 1 < p_2$)，那么点 1 到 p_1 构成一个强连通分量、点 $p_1 + 1$ 到 p_2 构成一个强连通分量。

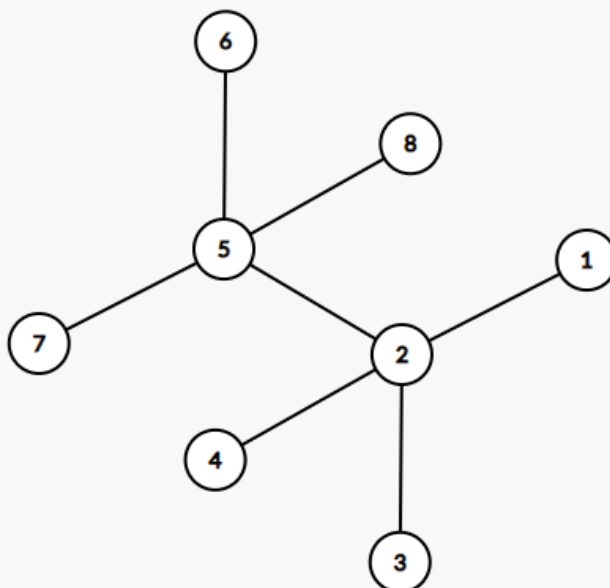
7. 选择图中最少数量的边删除，使得图不连通，即求最小割；如果是删除点，那么拆点后求最小割 [See](#)。
8. 如果一张图是平面图，那么其边数一定小于等于 $3n - 6$ [See](#)。
9. 若一张有向完全图存在环，则一定存在三元环。
10. 有向图判是否存在环直接用 topsort；无向图判是否存在环直接用 dsu，也可以使用 topsort，条件变为 `deg[i] <= 1` 时入队。

3.21 常见例题

3.21.1 杂

题意：给出一棵节点数为 $2n$ 的树，要求将点分割为 n 个点对，使得点对的点之间的距离和最大。

可以转化为边上问题：对于每一条边，其被利用的次数即为 $\min\{\text{其左边的点的数量}, \text{其右边的点的数量}\}$ ，使用树形 dp 计算一遍即可。如下图样例，答案为 10。



```
1 vector<int> val(n + 1, 1);
2 int ans = 0;
3 function<void(int, int)> dfs = [&](int x, int fa) {
4     for (auto y : ver[x]) {
5         if (y == fa) continue;
6         dfs(y, x);
7         val[x] += val[y];
8         ans += min(val[y], k - val[y]);
9     }
10 };
11 dfs(1, 0);
12 cout << ans << endl;
```

题意：以哪些点为起点可以无限的在有向图上绕

概括一下这些点可以发现，一类是环上的点，另一类是可以到达环的点。建反图跑一遍 topsort 板子，根据容斥，未被移除的点都是答案 [See](#)。

题意：添加最少的边，使得有向图变成一个 SCC

将原图的 SCC 缩点，统计缩点后的新图上入度为 0 和出度为 0 的点的数量 cnt_{in} 、 cnt_{out} ，答案即为 $\max(cnt_{in}, cnt_{out})$ 。过程大致是先将一个出度为 0 的点和一个入度为 0 的点相连，剩下的点随便连 [See](#)。

题意：添加最少的边，使得无向图变成一个 E-DCC

将原图的 E-DCC 缩点，统计缩点后的新图上入度为 1 的点（叶子结点）的数量 cnt ，答案即为 $\lceil \frac{cnt}{2} \rceil$ 。过程大致是每次找两个叶子结点（但是还有一些条件限制）相连，若最后余下一个点随便连 [See](#)

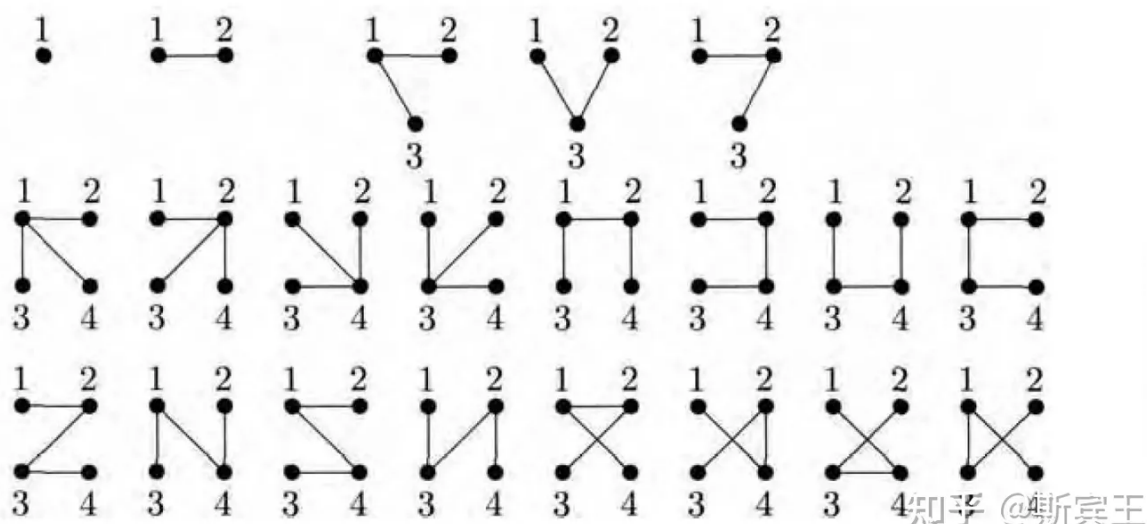
题意：在树上找到一个最大的连通块，使得这个联通内点权和边权之和最大，输出这个值，数据中存在负数的情况。

使用 dfs 即可解决。

```
1 LL n, point[N];
2 LL ver[N], head[N], nex[N], tot; bool v[N];
3 map<pair<LL, LL>, LL> edge;
4 // void add(LL x, LL y) {}
5 void dfs(LL x) {
6     for (LL i = head[x]; i; i = nex[i]) {
7         LL y = ver[i];
8         if (v[y]) continue;
9         v[y] = true; dfs(y); v[y] = false;
10    }
11    for (LL i = head[x]; i; i = nex[i]) {
12        LL y = ver[i];
13        if (v[y]) continue;
14        point[x] += max(point[y] + edge[{x, y}], 0LL);
15    }
16 }
17 void Solve() {
18     cin >> n;
19     FOR(i, 1, n) cin >> point[i];
20     FOR(i, 2, n) {
21         LL x, y, w; cin >> x >> y >> w;
22         edge[{x, y}] = edge[{y, x}] = w;
23         add(x, y), add(y, x);
24     }
25     v[1] = true; dfs(1); LL ans = -MAX18;
26     FOR(i, 1, n) ans = max(ans, point[i]);
27     cout << ans << endl;
28 }
```

3.21.2 Prüfer 序列：凯莱公式

题意：给定 n 个顶点，可以构建出多少棵标记树？



$n \leq 4$ 时的样例如上，通项公式为 n^{n-2} 。

3.21.3 Prüfer 序列

一个 n 个点 m 条边的带标号无向图有 k 个连通块。我们希望添加 $k - 1$ 条边使得整个图连通，求方案数量 [See](#)。

设 s_i 表示每个连通块的数量，通项公式为 $n^{k-2} \cdot \prod_{i=1}^k s_i$ ，当 $k < 2$ 时答案为 1。

3.21.4 单源最短/次短路计数

```

1  const int N = 2e5 + 7, M = 1e6 + 7;
2  int n, m, s, e; int d[N][2], v[N][2]; // 0 代表最短路, 1 代表次短路
3  Z num[N][2];
4
5  void Clear() {
6      for (int i = 1; i <= n; ++ i) h[i] = edge[i] = 0;
7      tot = 0;
8      for (int i = 1; i <= n; ++ i) num[i][0] = num[i][1] = v[i][0] = v[i][1] = 0;
9      for (int i = 1; i <= n; ++ i) d[i][0] = d[i][1] = INF;
10 }
11
12 int ver[M], ne[M], h[N], edge[M], tot;
13 void add(int x, int y, int w) {
14     ver[++ tot] = y, ne[tot] = h[x], h[x] = tot;
15     edge[tot] = w;
16 }
17
18 void dji() {
19     priority_queue<PIII, vector<PIII>, greater<PIII> > q; q.push({0, s, 0});
20     num[s][0] = 1; d[s][0] = 0;
21     while (!q.empty()) {
22         auto [dis, x, type] = q.top(); q.pop();
23         if (v[x][type]) continue; v[x][type] = 1;
24         for (int i = h[x]; i; i = ne[i]) {
25             int y = ver[i], w = dis + edge[i];
26             if (d[y][0] > w) {
27                 d[y][1] = d[y][0], num[y][1] = num[y][0];
28                 // 如果找到新的最短路, 将原有的最短路数据转化为次短路
29                 q.push({d[y][1], y, 1});
30                 d[y][0] = w, num[y][0] = num[x][type];
            }
        }
    }
}

```

```

31         q.push({d[y][0], y, 0});
32     }
33     else if (d[y][0] == w) num[y][0] += num[x][type];
34     else if (d[y][1] > w) {
35         d[y][1] = w, num[y][1] = num[x][type];
36         q.push({d[y][1], y, 1});
37     }
38     else if (d[y][1] == w) num[y][1] += num[x][type];
39 }
40 }
41 }
42 void Solve() {
43     cin >> n >> m >> s >> e;
44     Clear(); //多组样例务必完全清空
45     for (int i = 1; i <= m; ++i) {
46         int x, y, w; cin >> x >> y; w = 1;
47         add(x, y, w), add(y, x, w);
48     }
49     dji();
50     Z ans = num[e][0];
51     if (d[e][1] == d[e][0] + 1) {
52         ans += num[e][1]; // 只有在次短路满足条件时才计算 (距离恰好比最短路大1)
53     }
54     cout << ans.val() << endl;
55 }

```

3.21.5 判定图中是否存在负环

使用 SPFA，复杂度为 $\mathcal{O}(KM)$ ，其中常数 K 相较裸的 SPFA 更高。

```

1  const int N = 1e5 + 7, M = 1e6 + 7;
2  int n, m;
3  int ver[M], ne[M], h[N], edge[M], tot;
4  int d[N], v[N], num[N];
5
6  void add(int x, int y, int w) {
7      ver[++tot] = y, ne[tot] = h[x], h[x] = tot;
8      edge[tot] = w;
9  }
10 bool spfa() {
11     queue<int> q;
12     for (int i = 1; i <= n; ++i) q.push(i), v[i] = 1; //全部入队
13     while(!q.empty()) {
14         int x = q.front(); q.pop();
15         v[x] = 0;
16         for (int i = h[x]; i; i = ne[i]) {
17             int y = ver[i];
18             if(d[y] > d[x] + edge[i]) {
19                 num[y] = num[x] + 1;
20                 if (num[y] >= n) return true;
21                 d[y] = d[x] + edge[i];
22                 if(!v[y]) q.push(y), v[y] = 1;
23             }
24         }
25     }
26     return false;
27 }
28 int main() {
29     cin >> n >> m;
30     for (int i = 1; i <= m; ++i) {
31         int x, y, w; cin >> x >> y >> w;
32         add(x, y, w);

```

```

33     }
34     if(spfa() == true) cout << "Yes" << endl;
35     else cout << "No" << endl;
36 }

```

3.21.6 输出任意一个三元环

原题：给出一张有向完全图，输出任意一个三元环上的全部元素 [See](#)。使用 dfs，复杂度 $\mathcal{O}(N + M)$ ，可以扩展到非完全图和无向图。

```

1  int n;
2  cin >> n;
3  vector<vector<int>> a(n + 1, vector<int>(n + 1));
4  for (int i = 1; i <= n; ++i) {
5      for (int j = 1; j <= n; ++j) {
6          char x;
7          cin >> x;
8          if (x == '1') a[i][j] = 1;
9      }
10 }
11
12 vector<int> vis(n + 1);
13 function<void(int, int)> dfs = [&](int x, int fa) {
14     vis[x] = 1;
15     for (int y = 1; y <= n; ++y) {
16         if (a[x][y] == 0) continue;
17         if (a[y][fa] == 1) {
18             cout << fa << " " << x << " " << y;
19             exit(0);
20         }
21         if (!vis[y]) dfs(y, x); // 这一步的if判断很关键
22     }
23 };
24 for (int i = 1; i <= n; ++i) {
25     if (!vis[i]) dfs(i, -1);
26 }
27 cout << -1;

```

3.21.7 带权最小环大小与计数

原题：给出一张有向带权图，求解图上最小环的长度、有多少个这样的最小环 [See](#)。使用 floyd，复杂度为 $\mathcal{O}(N^3)$ ，可以扩展到无向图。

```

1  LL Min = 1e18, ans = 0;
2  for (int k = 1; k <= n; k++) {
3      for (int i = 1; i <= n; i++) {
4          for (int j = 1; j <= n; j++) {
5              if (dis[i][j] > dis[i][k] + dis[k][j]) {
6                  dis[i][j] = dis[i][k] + dis[k][j];
7                  cnt[i][j] = cnt[i][k] * cnt[k][j] % mod;
8              } else if (dis[i][j] == dis[i][k] + dis[k][j]) {
9                  cnt[i][j] = (cnt[i][j] + cnt[i][k] * cnt[k][j] % mod) % mod;
10             }
11         }
12     }
13     for (int i = 1; i < k; i++) {
14         if (a[k][i]) {
15             if (a[k][i] + dis[i][k] < Min) {
16                 Min = a[k][i] + dis[i][k];
17                 ans = cnt[i][k];
18             } else if (a[k][i] + dis[i][k] == Min) {

```

```

19         ans = (ans + cnt[i][k]) % mod;
20     }
21 }
22 }
23 }

```

3.21.8 最小环大小

原题：给出一张无向图，求解图上最小环的长度、有多少个这样的最小环 [See](#)。使用 floyd，可以扩展到有向图。

```

1  int floyd(int n) {
2      for (int i = 1; i <= n; ++ i) {
3          for (int j = 1; j <= n; ++ j) {
4              val[i][j] = dis[i][j]; // 记录最初的边权值
5          }
6      }
7      int ans = 0x3f3f3f3f;
8      for (int k = 1; k <= n; ++ k) {
9          for (int i = 1; i < k; ++ i) { // 注意这里是没有等于号的
10             for (int j = 1; j < i; ++ j) {
11                 ans = min(ans, dis[i][j] + val[i][k] + val[k][j]);
12             }
13         }
14         for (int i = 1; i <= n; ++ i) { // 往下是标准的floyd
15             for (int j = 1; j <= n; ++ j) {
16                 dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
17             }
18         }
19     }
20     return ans;
21 }

```

使用 bfs，复杂度为 $\mathcal{O}(N^2)$ 。

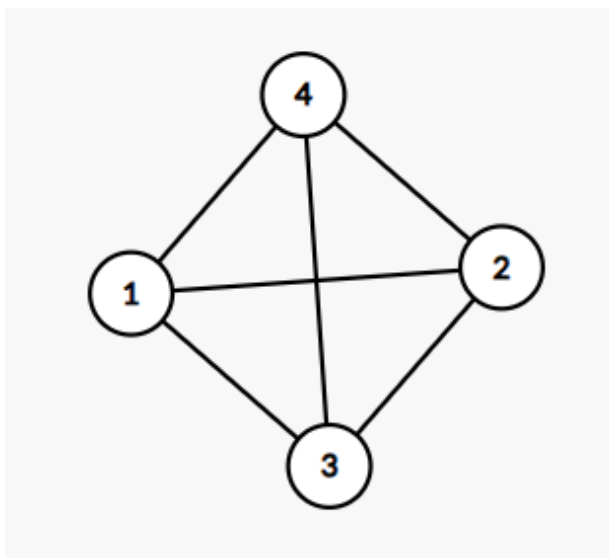
```

1  auto bfs = [&] (int s) {
2      queue<int> q; q.push(s);
3      dis[s] = 0;
4      fa[s] = -1;
5      while (q.size()) {
6          auto x = q.front(); q.pop();
7          for (auto y : ver[x]) {
8              if (y == fa[x]) continue;
9              if (dis[y] == -1) {
10                 dis[y] = dis[x] + 1;
11                 fa[y] = x;
12                 q.push(y);
13             }
14             else ans = min(ans, dis[x] + dis[y] + 1);
15         }
16     }
17 };
18 for (int i = 1; i <= n; ++ i) {
19     fill(dis + 1, dis + 1 + n, -1);
20     bfs(i);
21 }
22 cout << ans;

```

3.21.9 本质不同简单环计数

原题：给出一张无向图，输出简单环的数量 [See](#)。注意这里环套环需要分别多次统计，下图答案应当为 7。使用状压 dp，复杂度为 $\mathcal{O}(M \cdot 2^N)$ ，可以扩展到无向图。



```
1  int n, m;
2  cin >> n >> m;
3  vector<vector<int>>> G(n);
4  for (int i = 0; i < m; i++) {
5      int u, v;
6      cin >> u >> v;
7      u--, v--;
8      G[u].push_back(v);
9      G[v].push_back(u);
10 }
11 vector<vector<LL>>> dp(1 << n, vector<LL>(n));
12 for (int i = 0; i < n; i++) dp[1 << i][i] = 1;
13 LL ans = 0;
14 for (int st = 1; st < (1 << n); st++) {
15     for (int u = 0; u < n; u++) {
16         if (!dp[st][u]) continue;
17         int start = st & -st;
18         for (auto v : G[u]) {
19             if ((1 << v) < start) continue;
20             if ((1 << v) & st) {
21                 if ((1 << v) == start) {
22                     ans += dp[st][u];
23                 }
24             } else {
25                 dp[st | (1 << v)][v] += dp[st][u];
26             }
27         }
28     }
29 }
30 cout << (ans - m) / 2 << "\n";
```

3.21.10 输出任意一个非二元简单环

原题：给出一张无向图，不含自环与重边，输出任意一个简单环的大小以及其上面的全部元素 [See](#)。注意输出的环的大小是随机的，**不等价于最小环**。

由于不含重边与自环，所以环的大小至少为 3，使用 dfs 处理出 dfs 序，复杂度为 $\mathcal{O}(N + M)$ ，可以扩展到无向图；如果有向图中二元环也允许计入答案，则需要删除下方标注行。

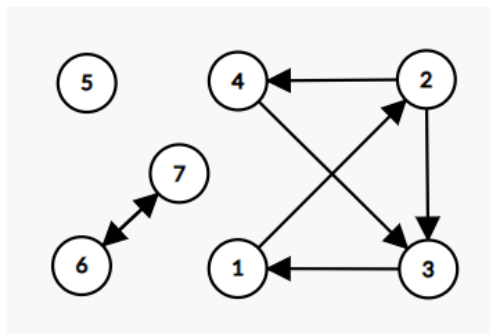
```

1  vector<int> dis(n + 1, -1), fa(n + 1);
2  auto dfs = [&](auto self, int x) -> void {
3      for (auto y : ver[x]) {
4          if (y == fa[x]) continue; // 二元环需删去该行
5          if (dis[y] == -1) {
6              dis[y] = dis[x] + 1;
7              fa[y] = x;
8              self(self, y);
9          } else if (dis[y] < dis[x]) {
10             cout << dis[x] - dis[y] + 1 << endl;
11             int pre = x;
12             cout << pre << " ";
13             while (pre != y) {
14                 pre = fa[pre];
15                 cout << pre << " ";
16             }
17             cout << endl;
18             exit(0);
19         }
20     }
21 };
22 for (int i = 1; i <= n; i++) {
23     if (dis[i] == -1) {
24         dis[i] = 0;
25         dfs(dfs, i);
26     }
27 }

```

3.21.11 有向图环计数

原题：给出一张有向图，输出环的数量。注意这里环套环仅需要计算一次，数据包括二元环和自环，下图例应当输出 3 个环。使用 dfs 染色法，复杂度为 $O(N + M)$ 。



```

1  int ans = 0;
2  vector<int> vis(n + 1);
3  auto dfs = [&](auto self, int x) -> void {
4      vis[x] = 1;
5      for (auto y : ver[x]) {
6          if (vis[y] == 0) {
7              self(self, y);
8          } else if (vis[y] == 1) {
9              ans++;
10         }
11     }
12     vis[x] = 2;
13 };
14 for (int i = 1; i <= n; i++) {
15     if (!vis[i]) {
16         dfs(dfs, i);
17     }
18 }

```

```

18 }
19 cout << ans << endl;

```

3.21.12 输出有向图任意一个环

原题：给出一张有向图，输出任意一个环，数据包括二元环和自环。使用 dfs 染色法。

```

1  vector<int> dis(n + 1), vis(n + 1), fa(n + 1);
2  auto dfs = [&](auto self, int x) -> void {
3      vis[x] = 1;
4      for (auto y : ver[x]) {
5          if (vis[y] == 0) {
6              dis[y] = dis[x] + 1;
7              fa[y] = x;
8              self(self, y);
9          } else if (vis[y] == 1) {
10             cout << dis[x] - dis[y] + 1 << endl;
11             int pre = x;
12             cout << pre << " ";
13             while (pre != y) {
14                 pre = fa[pre];
15                 cout << pre << " ";
16             }
17             cout << endl;
18             exit(0);
19         }
20     }
21     vis[x] = 2;
22 };
23 for (int i = 1; i <= n; i++) {
24     if (!vis[i]) {
25         dfs(dfs, i);
26     }
27 }

```

3.21.13 判定带环图是否是平面图

原题：给定一个环以一些额外边，对于每一条额外边判定其位于环外还是环内，使得任意两条无重合顶点的额外边都不相交（即这张图构成平面图）[See1](#), [See2](#)。

使用 2-sat。考虑全部边都位于环内，那么“一条边完全包含另一条边”、“两条边完全没有交集”这两种情况都不会相交，可以直接跳过这两种情况的讨论。

```

1  signed main() {
2      int n, m;
3      cin >> n >> m;
4      vector<pair<int, int>> in(m);
5      for (int i = 0, x, y; i < m; i++) {
6          cin >> x >> y;
7          in[i] = minmax(x, y);
8      }
9      TwoSat sat(m);
10     for (int i = 0; i < m; i++) {
11         auto [s, e] = in[i];
12         for (int j = i + 1; j < m; j++) {
13             auto [S, E] = in[j];
14             if (s < S && S < e && e < E || S < s && s < E && E < e) {
15                 sat.add(i, 0, j, 0);
16                 sat.add(i, 1, j, 1);
17             }
18         }
19     }

```

```
19     }
20     if (!sat.work()) {
21         cout << "Impossible\n";
22         return 0;
23     }
24     auto ans = sat.answer();
25     for (auto it : ans) {
26         cout << (it ? "out" : "in") << " ";
27     }
28 }
```

/END/

4 网络流

4.1 最大流

4.1.1 Dinic 解

使用 Dinic 算法, 理论最坏复杂度为 $\mathcal{O}(N^2M)$, 例题范围: $N = 1200$, $m = 5 \times 10^3$ 。一般步骤: BFS 建立分层图, 无回溯 DFS 寻找所有可行的增广路径。封装: 求从点 S 到点 T 的最大流。

```
1  template<typename T> struct Flow_ {
2      const int n;
3      const T inf = numeric_limits<T>::max();
4      struct Edge {
5          int to;
6          T w;
7          Edge(int to, T w) : to(to), w(w) {}
8      };
9      vector<Edge> ver;
10     vector<vector<int>> h;
11     vector<int> cur, d;
12
13     Flow_(int n) : n(n + 1), h(n + 1) {}
14     void add(int u, int v, T c) {
15         h[u].push_back(ver.size());
16         ver.emplace_back(v, c);
17         h[v].push_back(ver.size());
18         ver.emplace_back(u, 0);
19     }
20     bool bfs(int s, int t) {
21         d.assign(n, -1);
22         d[s] = 0;
23         queue<int> q;
24         q.push(s);
25         while (!q.empty()) {
26             auto x = q.front();
27             q.pop();
28             for (auto it : h[x]) {
29                 auto [y, w] = ver[it];
30                 if (w && d[y] == -1) {
31                     d[y] = d[x] + 1;
32                     if (y == t) return true;
33                     q.push(y);
34                 }
35             }
36         }
37         return false;
38     }
39     T dfs(int u, int t, T f) {
40         if (u == t) return f;
41         auto r = f;
42         for (int &i = cur[u]; i < h[u].size(); i++) {
43             auto j = h[u][i];
44             auto &[v, c] = ver[j];
45             auto &[u, rc] = ver[j ^ 1];
46             if (c && d[v] == d[u] + 1) {
47                 auto a = dfs(v, t, std::min(r, c));
48                 c -= a;
49                 rc += a;
50                 r -= a;
51                 if (!r) return f;
52             }
53         }
```

```

53     }
54     return f - r;
55 }
56 T work(int s, int t) {
57     T ans = 0;
58     while (bfs(s, t)) {
59         cur.assign(n, 0);
60         ans += dfs(s, t, inf);
61     }
62     return ans;
63 }
64 };
65 using Flow = Flow_<int>;

```

4.1.2 预流推进 HLPP

理论最坏复杂度为 $\mathcal{O}(N^2\sqrt{M})$ ，例题范围： $N = 1200$, $m = 1.2 \times 10^5$ 。

```

1  template <typename T> struct PushRelabel {
2      const int inf = 0x3f3f3f3f;
3      const T INF = 0x3f3f3f3f3f3f3f3f;
4      struct Edge {
5          int to, cap, flow, anti;
6          Edge(int v = 0, int w = 0, int id = 0) : to(v), cap(w), flow(0), anti(id)
7      };
8      vector<vector<Edge>> e;
9      vector<vector<int>> gap;
10     vector<T> ex; // 超额流
11     vector<bool> ingap;
12     vector<int> h;
13     int n, gobalcnt, maxH = 0;
14     T maxflow = 0;
15
16     PushRelabel(int n) : n(n), e(n + 1), ex(n + 1), gap(n + 1) {}
17     void addedge(int u, int v, int w) {
18         e[u].push_back({v, w, (int)e[v].size()});
19         e[v].push_back({u, 0, (int)e[u].size() - 1});
20     }
21     void PushEdge(int u, Edge &edge) {
22         int v = edge.to, d = min(ex[u], 1LL * edge.cap - edge.flow);
23         ex[u] -= d;
24         ex[v] += d;
25         edge.flow += d;
26         e[v][edge.anti].flow -= d;
27         if (h[v] != inf && d > 0 && ex[v] == d && !ingap[v]) {
28             ++gobalcnt;
29             gap[h[v]].push_back(v);
30             ingap[v] = 1;
31         }
32     }
33     void PushPoint(int u) {
34         for (auto k = e[u].begin(); k != e[u].end(); k++) {
35             if (h[k->to] + 1 == h[u] && k->cap > k->flow) {
36                 PushEdge(u, *k);
37                 if (!ex[u]) break;
38             }
39         }
40         if (!ex[u]) return;
41         if (gap[h[u]].empty()) {
42             for (int i = h[u] + 1; i <= min(maxH, n); i++) {
43                 for (auto v : gap[i]) {

```

```

44         ingap[v] = 0;
45     }
46     gap[i].clear();
47 }
48 }
49 h[u] = inf;
50 for (auto [to, cap, flow, anti] : e[u]) {
51     if (cap > flow) {
52         h[u] = min(h[u], h[to] + 1);
53     }
54 }
55 if (h[u] >= n) return;
56 maxH = max(maxH, h[u]);
57 if (!ingap[u]) {
58     gap[h[u]].push_back(u);
59     ingap[u] = 1;
60 }
61 }
62 void init(int t, bool f = 1) {
63     ingap.assign(n + 1, 0);
64     for (int i = 1; i <= maxH; i++) {
65         gap[i].clear();
66     }
67     gobalcnt = 0, maxH = 0;
68     queue<int> q;
69     h.assign(n + 1, inf);
70     h[t] = 0, q.push(t);
71     while (q.size()) {
72         int u = q.front();
73         q.pop(), maxH = h[u];
74         for (auto &[v, cap, flow, anti] : e[u]) {
75             if (h[v] == inf && e[v][anti].cap > e[v][anti].flow) {
76                 h[v] = h[u] + 1;
77                 q.push(v);
78                 if (f) {
79                     gap[h[v]].push_back(v);
80                     ingap[v] = 1;
81                 }
82             }
83         }
84     }
85 }
86 T work(int s, int t) {
87     init(t, 0);
88     if (h[s] == inf) return maxflow;
89     h[s] = n;
90     ex[s] = INF;
91     ex[t] = -INF;
92     for (auto k = e[s].begin(); k != e[s].end(); k++) {
93         PushEdge(s, *k);
94     }
95     while (maxH > 0) {
96         if (gap[maxH].empty()) {
97             maxH--;
98             continue;
99         }
100         int u = gap[maxH].back();
101         gap[maxH].pop_back();
102         ingap[u] = 0;
103         PushPoint(u);
104         if (gobalcnt >= 10 * n) {
105             init(t);
106         }
107     }

```

```

108     ex[s] -= INF;
109     ex[t] += INF;
110     return maxflow = ex[t];
111 }
112 };

```

4.2 最小割

基础模型：构筑二分图，左半部 n 个点代表盈利项目，右半部 m 个点代表材料成本，收益为盈利之和减去成本之和，求最大收益。

建图：建立源点 S 向左半部连边，建立汇点 T 向右半部连边，如果某个项目需要某个材料，则新增一条容量 $+\infty$ 的跨部边。

割边：放弃某个项目则断开 S 至该项目的边，购买某个原料则断开该原料至 T 的边，最终的图一定不存在从 S 到 T 的路径，此时我们得到二分图的一个 $S - T$ 割。此时最小割即为求解最大流，边权之和减去最大流即为最大收益。

```

1  signed main() {
2      int n, m;
3      cin >> n >> m;
4
5      int S = n + m + 1, T = n + m + 2;
6      Flow flow(T);
7      for (int i = 1; i <= n; i++) {
8          int w;
9          cin >> w;
10         flow.add(S, i, w);
11     }
12
13     int sum = 0;
14     for (int i = 1; i <= m; i++) {
15         int x, y, w;
16         cin >> x >> y >> w;
17         flow.add(x, n + i, 1E18);
18         flow.add(y, n + i, 1E18);
19         flow.add(n + i, T, w);
20         sum += w;
21     }
22     cout << sum - flow.work(S, T) << endl;
23 }

```

4.3 费用流

给定一个带费用的网络，规定 (u, v) 间的费用为 $f(u, v) \times w(u, v)$ ，求解该网络中总花费最小的最大流称之为**最小费用最大流**。总时间复杂度为 $\mathcal{O}(NMf)$ ，其中 f 代表最大流。

```

1  struct MinCostFlow {
2      using LL = long long;
3      using PII = pair<LL, int>;
4      const LL INF = numeric_limits<LL>::max();
5      struct Edge {
6          int v, c, f;
7          Edge(int v, int c, int f) : v(v), c(c), f(f) {}
8      };
9      const int n;
10     vector<Edge> e;
11     vector<vector<int>> g;
12     vector<LL> h, dis;
13     vector<int> pre;

```

```

14
15 MinCostFlow(int n) : n(n), g(n) {}
16 void add(int u, int v, int c, int f) { // c 流量, f 费用
17     // if (f < 0) {
18     //     g[u].push_back(e.size());
19     //     e.emplace_back(v, 0, f);
20     //     g[v].push_back(e.size());
21     //     e.emplace_back(u, c, -f);
22     // } else {
23     g[u].push_back(e.size());
24     e.emplace_back(v, c, f);
25     g[v].push_back(e.size());
26     e.emplace_back(u, 0, -f);
27     // }
28 }
29 bool dijkstra(int s, int t) {
30     dis.assign(n, INF);
31     pre.assign(n, -1);
32     priority_queue<PII, vector<PII>, greater<PII>> que;
33     dis[s] = 0;
34     que.emplace(0, s);
35     while (!que.empty()) {
36         auto [d, u] = que.top();
37         que.pop();
38         if (dis[u] < d) continue;
39         for (int i : g[u]) {
40             auto [v, c, f] = e[i];
41             if (c > 0 && dis[v] > d + h[u] - h[v] + f) {
42                 dis[v] = d + h[u] - h[v] + f;
43                 pre[v] = i;
44                 que.emplace(dis[v], v);
45             }
46         }
47     }
48     return dis[t] != INF;
49 }
50 pair<int, LL> flow(int s, int t) {
51     int flow = 0;
52     LL cost = 0;
53     h.assign(n, 0);
54     while (dijkstra(s, t)) {
55         for (int i = 0; i < n; ++i) h[i] += dis[i];
56         int aug = numeric_limits<int>::max();
57         for (int i = t; i != s; i = e[pre[i] ^ 1].v) aug = min(aug,
e[pre[i]].c);
58         for (int i = t; i != s; i = e[pre[i] ^ 1].v) {
59             e[pre[i]].c -= aug;
60             e[pre[i] ^ 1].c += aug;
61         }
62         flow += aug;
63         cost += LL(aug) * h[t];
64     }
65     return {flow, cost};
66 }
67 };

```

/END/

5 数论

5.1 常见数列

5.1.1 调和级数

满足调和级数 $\mathcal{O}\left(\frac{N}{1} + \frac{N}{2} + \frac{N}{3} + \cdots + \frac{N}{N}\right)$, 可以用 $\approx N \ln N$ 来拟合, 但是会略小, 误差量级在 10% 左右。本地可以在 500ms 内完成 10^8 量级的预处理计算。

N 的量级	1	2	3	4	5	6	7	8	9
累加和	27	482	7'069	93'668	1'166'750	13'970'034	162'725'364	1'857'511'568	20'877'697'634

下方示例为求解 1 到 N 中各个数字的因数值。

```
1 const int N = 1E5;
2 vector<vector<int>> dic(N + 1);
3 for (int i = 1; i <= N; i++) {
4     for (int j = i; j <= N; j += i) {
5         dic[j].push_back(i);
6     }
7 }
```

5.1.2 素数密度与分布

N 的量级	1	2	3	4	5	6	7	8	9
素数数量	4	25	168	1'229	9'592	78'498	664'579	5'761'455	50'847'534

除此之外, 对于任意两个相邻的素数 $p_1, p_2 \leq 10^9$, 有 $|p_1 - p_2| < 300$ 成立, 更具体的说, 最大的差值为 282。

5.1.3 因数最多数字与其因数数量

N 的量级	1	2	3	4	5	6	7
因数最多数字的因数数量	4	25	32	64	128	240	448
因数最多的数字	-	-	-	7560, 9240	83160, 98280	720720, 831600, 942480, 982800, 997920	-

5.2 欧拉筛 (线性筛)

时间复杂度为 $\mathcal{O}(N \log \log N)$ 。

```
1 vector<int> prime; // 这里储存筛出来的全部质数
2 auto euler_Prime = [&](int n) -> void {
3     vector<int> v(n + 1);
4     for (int i = 2; i <= n; ++i) {
5         if (!v[i]) {
6             v[i] = i;
7             prime.push_back(i);
8         }
9         for (int j = 0; j < prime.size(); ++j) {
10             if (prime[j] > v[i] || prime[j] > n / i) break;
11             v[i * prime[j]] = prime[j];
12         }
13     }
```

```
14 };
```

5.3 防爆模乘

5.3.1 借助浮点数实现

以 $\mathcal{O}(1)$ 计算 $a \cdot b \bmod p$ ，由于不取模，常数比 int128 法小很多。其中 $1 \leq n, k, p \leq 10^{18}$ 。

```
1 int mul(int a, int b, int m) {
2     int r = a * b - m * (int)(1.L / m * a * b);
3     return r - m * (r >= m) + m * (r < 0);
4 }
```

5.3.2 借助 int128 实现

```
1 int mul(int a, int b, int m) {
2     return (__int128)a * b % m;
3 }
```

5.4 裴蜀定理

$ax + by = c$ ($x \in \mathbb{Z}^*, y \in \mathbb{Z}^*$) 成立的充要条件是 $\gcd(a, b) \mid c$ ($\mathbb{Z}^*$ 表示正整数集)。

例题：给定一个序列 a ，找到一个序列 x ，使得 $\sum_{i=1}^n a_i x_i$ 最小。

```
1 LL n, a, ans;
2 LL gcd(LL a, LL b){
3     return b ? gcd(b, a % b) : a;
4 }
5 int main(){
6     cin >> n;
7     for (int i = 0; i < n; i++){
8         cin >> a;
9         if (a < 0) a = -a;
10        ans = gcd(ans, a);
11    }
12    cout << ans << "\n";
13    return 0;
14 }
```

5.5 逆元

5.5.1 费马小定理理解（借助快速幂）

单次计算的复杂度即为快速幂的复杂度 $\mathcal{O}(\log X)$ 。限制： MOD 必须是质数，且需要满足 x 与 MOD 互质。

```
1 LL inv(LL x) { return mypow(x, mod - 2, mod); }
```

5.5.2 扩展欧几里得解

此方法的 MOD 没有限制，复杂度为 $\mathcal{O}(\log X)$ ，但是比快速幂法常数大一些。

```
1 int x, y;
2 int exgcd(int a, int b, int &x, int &y) { //扩展欧几里得算法
3     if (b == 0) {
4         x = 1, y = 0;
```

```

5         return a; //到达递归边界开始向上一层返回
6     }
7     int r = exgcd(b, a % b, x, y);
8     int temp = y; //把x y变成上一层的
9     y = x - (a / b) * y;
10    x = temp;
11    return r; //得到a b的最大公因数
12 }
13 LL getInv(int a, int mod) { //求a在mod下的逆元, 不存在逆元返回-1
14     LL x, y, d = exgcd(a, mod, x, y);
15     return d == 1 ? (x % mod + mod) % mod : -1;
16 }

```

5.5.3 离线求解：线性递推解

以 $O(N)$ 的复杂度完成 $1 - N$ 中全部逆元的计算。

```

1 inv[1] = 1;
2 for (int i = 2; i <= n; i++)
3     inv[i] = (p - p / i) * inv[p % i] % p;

```

5.6 扩展欧几里得 exgcd

可以用于求解形如 $A \cdot x + B \cdot y = \gcd(a, b)$ 的不定方程特解。

```

1 LL exgcd(LL a, LL b, LL &x, LL &y){
2     if (!b){
3         x = 1, y = 0;
4         return a;
5     }
6     LL d = exgcd(b, a % b, y, x);
7     y -= a / b * x;
8     return d;
9 }

```

例题：求解二元一次不定方程 $A \cdot x + B \cdot y = C$ 。如果其无整数解，输出 -1 ；如果其有正整数解，输出解的数量、最小的 x, y 、最大的 x, y ；如果其解不为正整数，输出最小的正整数 x, y （注意，是最大最小的值，而不是最大最小的一组解）。

```

1 template <class T> T sign(const T &a) {
2     return a == 0 ? 0 : (a < 0 ? -1 : 1);
3 }
4 template <class T> T ceil(const T &a, const T &b) {
5     T A = abs(a), B = abs(b);
6     assert(b != 0);
7     return sign(a) * sign(b) > 0 ? (A + B - 1) / B : -A / B;
8 }
9 int exgcd(int a, int b, int &x, int &y) {
10     if (!b) {
11         x = 1, y = 0;
12         return a;
13     }
14     int d = exgcd(b, a % b, y, x);
15     y -= a / b * x;
16     return d;
17 }
18
19 auto clac = [&](int a, int b, int c) -> void {
20     int x, y, d = exgcd(a, b, x, y);

```

```

21     if (c % d != 0) {
22         cout << -1 << endl;
23         return;
24     }
25     x *= c / d, y *= c / d;
26     int p = b / d, q = a / d, k;
27     if (x < 0) {
28         k = ceil(1 - x, p);
29         x += p * k;
30         y -= q * k;
31     }
32     else if (x >= 0) { //将x提高到最小正整数
33         k = (x - 1) / p;
34         x -= p * k; //将x降低到最小正整数
35         y += q * k;
36     }
37     if (y > 0) { //有正整数解
38         cout << (y - 1) / q + 1 << endl; //将y减到1的方案数即为解的个数
39         cout << x << endl; //当前的x即为最小正整数x
40         cout << (y - 1) % q + 1 << endl; //将y取到最小正整数
41         cout << x + (y - 1) / q * p << endl; //将x提升到最大
42         cout << y << endl; //特解即为y最大值
43     } else { //无整数解
44         cout << x << endl; //当前的x即为最小的正整数x
45         cout << y + q * ceil(1 - y, q) << endl; //将y提高到正整数
46     }
47 };

```

5.7 离散对数 bsgs 与 exbsgs

以 $O(\sqrt{P})$ 的复杂度求解 $a^x \equiv b \pmod{P}$ 。其中标准 BSGS 算法不能计算 a 与 MOD 互质的情况，而 exbsgs 则可以。

```

1  namespace BSGS {
2  LL a, b, p;
3  map<LL, LL> f;
4  inline LL gcd(LL a, LL b) { return b > 0 ? gcd(b, a % b) : a; }
5  inline LL ps(LL n, LL k, int p) {
6      LL r = 1;
7      for (; k >>= 1) {
8          if (k & 1) r = r * n % p;
9          n = n * n % p;
10     }
11     return r;
12 }
13 void exgcd(LL a, LL b, LL &x, LL &y) {
14     if (!b)
15         x = 1, y = 0;
16     else {
17         exgcd(b, a % b, x, y);
18         LL t = x;
19         x = y;
20         y = t - a / b * y;
21     }
22 }
23 LL inv(LL a, LL b) {
24     LL x, y;
25     exgcd(a, b, x, y);
26     return (x % b + b) % b;
27 }
28 LL bsgs(LL a, LL b, LL p) {
29     f.clear();

```

```

30     int m = ceil(sqrt(p));
31     b %= p;
32     for (int i = 1; i <= m; i++) {
33         b = b * a % p;
34         f[b] = i;
35     }
36     LL tmp = ps(a, m, p);
37     b = 1;
38     for (int i = 1; i <= m; i++) {
39         b = b * tmp % p;
40         if (f.count(b) > 0) return (i * m - f[b] + p) % p;
41     }
42     return -1;
43 }
44 LL exbsgs(LL a, LL b, LL p) {
45     if (b == 1 || p == 1) return 0;
46     LL g = gcd(a, p), k = 0, na = 1;
47     while (g > 1) {
48         if (b % g != 0) return -1;
49         k++;
50         b /= g;
51         p /= g;
52         na = na * (a / g) % p;
53         if (na == b) return k;
54         g = gcd(a, p);
55     }
56     LL f = bsgs(a, b * inv(na, p) % p, p);
57     if (f == -1) return -1;
58     return f + k;
59 }
60 } // namespace BSGS
61
62 using namespace BSGS;
63
64 int main() {
65     IOS;
66     cin >> p >> a >> b;
67     a %= p, b %= p;
68     LL ans = exbsgs(a, b, p);
69     if (ans == -1) cout << "no solution\n";
70     else cout << ans << "\n";
71     return 0;
72 }

```

5.8 欧拉函数

5.8.1 直接求解单个数的欧拉函数

1 到 N 中与 N 互质数的个数称为欧拉函数，记作 $\varphi(N)$ 。求解欧拉函数的过程即为分解质因数的过程，复杂度 $\mathcal{O}(\sqrt{n})$ 。

```

1  int phi(int n) { //求解 phi(n)
2      int ans = n;
3      for(int i = 2; i <= n / i; i++) { //注意, 这里要写 n / i , 以防止 int 型溢出风险和
sqrt  超时风险
4          if(n % i == 0) {
5              ans = ans / i * (i - 1);
6              while(n % i == 0) n /= i;
7          }
8      }
9      if(n > 1) ans = ans / n * (n - 1); //特判 n 为质数的情况
10     return ans;
11 }

```

5.8.2 求解 1 到 N 所有数的欧拉函数

利用上述第四条性质, 我们可以快速递推出 $2 - N$ 中每个数的欧拉函数, 复杂度 $\mathcal{O}(N)$, 而该算法即是线性筛的算法。

```

1  const int N = 1e5 + 7;
2  int v[N], prime[N], phi[N];
3  void euler(int n) {
4      ms(v, 0); //最小质因子
5      int m = 0; //质数数量
6      for (int i = 2; i <= n; ++i) {
7          if (v[i] == 0) { // i 是质数
8              v[i] = i, prime[++m] = i;
9              phi[i] = i - 1;
10         }
11         //为当前的数 i 乘上一个质因子
12         for (int j = 1; j <= m; ++j) {
13             //如 i 有比 prime[j] 更小的质因子, 或超出 n , 停止
14             if(prime[j] > v[i] || prime[j] > n / i) break;
15             // prime[j] 是合数 i * prime[j] 的最小质因子
16             v[i * prime[j]] = prime[j];
17             phi[i * prime[j]] = phi[i] * (i % prime[j] ? prime[j] - 1 : prime[j]);
18         }
19     }
20 }
21 int main() {
22     int n; cin >> n; euler(n);
23     for (int i = 1; i <= n; ++i) cout << phi[i] << endl;
24     return 0;
25 }

```

5.8.3 使用莫比乌斯反演求解欧拉函数

```

1  int phi[N];
2  vector<int> fac[N];
3  void get_eulers() {
4      for (int i = 1; i <= N - 10; i++) {
5          for (int j = i; j <= N - 10; j += i) {
6              fac[j].push_back(i);
7          }
8      }
9      phi[1] = 1;
10     for (int i = 2; i <= N - 10; i++) {
11         phi[i] = i;
12         for (auto j : fac[i]) {
13             if (j == i) continue;
14             phi[i] -= phi[j];

```

```

15     }
16 }
17 }

```

5.9 组合数

5.9.1 debug

提供一组测试数据： $\binom{132}{66} = 377'389'666'165'540'953'244'592'352'291'892'721'700$ ，模数为 998244353 时为 241'200'029； $10^9 + 7$ 时为 598375978。

5.9.2 逆元+卢卡斯定理（质数取模）

$O(N)$ ，模数必须为质数。

```

1  struct Comb {
2      int n;
3      vector<Z> _fac, _inv;
4
5      Comb() : _fac{1}, _inv{0} {}
6      Comb(int n) : Comb() {
7          init(n);
8      }
9      void init(int m) {
10         if (m <= n) return;
11         _fac.resize(m + 1);
12         _inv.resize(m + 1);
13         for (int i = n + 1; i <= m; i++) {
14             _fac[i] = _fac[i - 1] * i;
15         }
16         _inv[m] = _fac[m].inv();
17         for (int i = m; i > n; i--) {
18             _inv[i - 1] = _inv[i] * i;
19         }
20         n = m;
21     }
22     Z fac(int x) {
23         if (x > n) init(x);
24         return _fac[x];
25     }
26     Z inv(int x) {
27         if (x > n) init(x);
28         return _inv[x];
29     }
30     Z C(int x, int y) {
31         if (x < 0 || y < 0 || x < y) return 0;
32         return fac(x) * inv(y) * inv(x - y);
33     }
34     Z P(int x, int y) {
35         if (x < 0 || y < 0 || x < y) return 0;
36         return fac(x) * inv(x - y);
37     }
38 } comb(1 << 21);

```

5.9.3 质因数分解

此法适用于： $1 < n, m, MOD < 10^7$ 的情况。

```

1  int n,m,p,b[10000005],prime[1000005],t,min_prime[10000005];
2  void euler_Prime(int n){//用欧拉筛求出1~n中每个数的最小质因数的编号是多少，保存在
    min_prime中

```

```

3     for(int i=2;i<=n;i++){
4         if(b[i]==0){
5             prime[++t]=i;
6             min_prime[i]=t;
7         }
8         for(int j=1;j<=t&&i*prime[j]<=n;j++){
9             b[prime[j]*i]=1;
10            min_prime[prime[j]*i]=j;
11            if(i%prime[j]==0) break;
12        }
13    }
14 }
15 long long c(int n,int m,int p){//计算C(n,m)%p的值
16     euler_Prime(n);
17     int a[t+5]; //t代表1~n中质数的个数，a[i]代表编号为i的质数在答案中出现的次数
18     for(int i=1;i<=t;i++) a[i]=0; //注意清0，一开始是随机数
19     for(int i=n;i>=n-m+1;i--){ //处理分子
20         int x=i;
21         while (x!=1){
22             a[min_prime[x]]++; //注意min_prime中保存的是这个数的最小质因数的编号 (1~t)
23             x/=prime[min_prime[x]];
24         }
25     }
26     for(int i=1;i<=m;i++){ //处理分母
27         int x=i;
28         while (x!=1){
29             a[min_prime[x]]--;
30             x/=prime[min_prime[x]];
31         }
32     }
33     long long ans=1;
34     for(int i=1;i<=t;i++){ //枚举质数的编号，看它出现了几次
35         while(a[i]>0){
36             ans=ans*prime[i]%p;
37             a[i]--;
38         }
39     }
40     return ans;
41 }
42 int main(){
43     cin>>n>>m;
44     m=min(m,n-m); //小优化
45     cout<<c(n,m,MOD);
46 }

```

5.9.4 杨辉三角（精确计算）

60 以内 `long long` 可解，130 以内 `__int128` 可解。

```

1 vector C(n + 1, vector<int>(n + 1));
2 C[0][0] = 1;
3 for (int i = 1; i <= n; i++) {
4     C[i][0] = 1;
5     for (int j = 1; j <= n; j++) {
6         C[i][j] = C[i - 1][j] + C[i - 1][j - 1];
7     }
8 }
9 cout << C[n][m] << endl;

```

5.10 求解连续数字的正约数集合—倍数法

使用规律递推优化，时间复杂度为 $\mathcal{O}(N \log N)$ ，如果不需要详细的输出集合，则直接将 `vector` 换为普通数组即可（时间更快）。

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  const int N = 1e5 + 7;
4  vector<int> f[N];
5
6  void divide(int n) {
7      for (int i = 1; i <= n; ++ i)
8          for (int j = 1; j <= n / i; ++ j)
9              f[i * j].push_back(i);
10     for (int i = 1; i <= n; ++ i) {
11         for (auto it : f[i]) cout << it << " ";
12         cout << endl;
13     }
14 }
15 int main() {
16     int x; cin >> x; divide(x);
17     return 0;
18 }
```

5.11 试除法判是否是质数

5.11.1 标准解

$\mathcal{O}(\sqrt{N})$ 。

```
1  bool is_prime(int n) {
2      if (n < 2) return false;
3      for (int i = 2; i <= x / i; i++) {
4          if (n % i == 0) return false;
5      }
6      return true;
7  }
```

5.11.2 常数优化法

常数优化，达到 $\mathcal{O}(\frac{\sqrt{N}}{3})$ 。

```
1  bool is_prime(int n) {
2      if (n < 2) return false;
3      if (n == 2 || n == 3) return true;
4      if (n % 6 != 1 && n % 6 != 5) return false;
5      for (int i = 5, j = n / i; i <= j; i += 6) {
6          if (n % i == 0 || n % (i + 2) == 0) {
7              return false;
8          }
9      }
10     return true;
11 }
```

5.12 容斥原理

定义:

$$|S_1 \cup S_2 \cup S_3 \cup \dots \cup S_n| = \sum_{i=1}^N |S_i| - \sum_{i,j=1}^N |S_i \cap S_j| + \sum_{i,j,k=1}^N |S_i \cap S_j \cap S_k| - \dots$$

例题: 给定一个整数 n 和 m 个不同的质数 $p_1, p_2, \dots, p_m$, 请你求出 $1 \sim n$ 中能被 $p_1, p_2, \dots, p_m$ 中的至少一个数整除的整数有多少个。

5.12.1 二进制枚举解

```
1  int main(){
2      ios::sync_with_stdio(false);cin.tie(0);
3      LL n, m;
4      cin >> n >> m;
5      vector <LL> p(m);
6      for (int i = 0; i < m; i ++ )
7          cin >> p[i];
8      LL ans = 0;
9      for (int i = 1; i < (1 << m); i ++ ){
10         LL t = 1, cnt = 0;
11         for (int j = 0; j < m; j ++ ){
12             if (i >> j & 1){
13                 cnt ++ ;
14                 t *= p[j];
15                 if (t > n){
16                     t = -1;
17                     break;
18                 }
19             }
20         }
21         if (t != -1){
22             if (cnt & 1) ans += n / t;
23             else ans -= n / t;
24         }
25     }
26     cout << ans << "\n";
27     return 0;
28 }
```

5.12.2 dfs 解

```
1  int main(){
2      ios::sync_with_stdio(false);cin.tie(0);
3      LL n, m;
4      cin >> n >> m;
5      vector <LL> p(m);
6      for (int i = 0; i < m; i ++ )
7          cin >> p[i];
8      LL ans = 0;
9      function<void(LL, LL, LL)> dfs = [&](LL x, LL s, LL odd){
10         if (x == m){
11             if (s == 1) return;
12             ans += odd * (n / s);
13             return;
14         }
15         dfs(x + 1, s, odd);
16         if (s <= n / p[x]) dfs(x + 1, s * p[x], -odd);
17     };
18     dfs(0, 1, -1);
19     cout << ans << "\n";
20     return 0;
21 }
```

5.13 同余方程组、拓展中国剩余定理 excrt

公式: $x \equiv b_i \pmod{a_i}$, 即 $(x - b_i) \mid a_i$ 。

```

1  int n; LL ai[maxn], bi[maxn];
2  inline int mypow(int n, int k, int p) {
3      int r = 1;
4      for (; k >>= 1; n = n * n % p)
5          if (k & 1) r = r * n % p;
6      return r;
7  }
8  LL exgcd(LL a, LL b, LL &x, LL &y) {
9      if (b == 0) { x = 1, y = 0; return a; }
10     LL gcd = exgcd(b, a % b, x, y), tp = x;
11     x = y, y = tp - a / b * y;
12     return gcd;
13 }
14 LL excrt() {
15     LL x, y, k;
16     LL M = bi[1], ans = ai[1];
17     for (int i = 2; i <= n; ++i) {
18         LL a = M, b = bi[i], c = (ai[i] - ans % b + b) % b;
19         LL gcd = exgcd(a, b, x, y), bg = b / gcd;
20         if (c % gcd != 0) return -1;
21         x = mul(x, c / gcd, bg);
22         ans += x * M;
23         M *= bg;
24         ans = (ans % M + M) % M;
25     }
26     return (ans % M + M) % M;
27 }
28 int main() {
29     cin >> n;
30     for (int i = 1; i <= n; ++i) cin >> bi[i] >> ai[i];
31     cout << excrt() << endl;
32     return 0;
33 }

```

5.14 求解连续按位异或

以 $\mathcal{O}(1)$ 复杂度计算 $0 \oplus 1 \oplus \cdots \oplus n$ 。

```

1  unsigned xor_n(unsigned n) {
2      unsigned t = n & 3;
3      if (t & 1) return t / 2u ^ 1;
4      return t / 2u ^ n;
5  }

```

```

1  i64 xor_n(i64 n) {
2      if (n % 4 == 1) return 1;
3      else if (n % 4 == 2) return n + 1;
4      else if (n % 4 == 3) return 0;
5      else return n;
6  }

```

5.15 高斯消元求解线性方程组

题目大意：输入一个包含 N 个方程 N 个未知数的线性方程组，系数与常数均为实数（两位小数）。求解这个方程组。如果存在唯一解，则输出所有 N 个未知数的解，结果保留两位小数。如果无数解，则输出 X ，如果无解，则输出 N 。

```
1  const int N = 110;
2  const double eps = 1e-8;
3  LL n;
4  double a[N][N];
5  LL gauss(){
6      LL c, r;
7      for (c = 0, r = 0; c < n; c ++ ){
8          LL t = r;
9          for (int i = r; i < n; i ++ )    //找到绝对值最大的行
10             if (fabs(a[i][c]) > fabs(a[t][c]))
11                 t = i;
12             if (fabs(a[t][c]) < eps) continue;
13             for (int j = c; j < n + 1; j ++ ) swap(a[t][j], a[r][j]);    //将绝对值最大的
一行换到最顶端
14             for (int j = n; j >= c; j -- ) a[r][j] /= a[r][c];    //将当前行首位变成 1
15             for (int i = r + 1; i < n; i ++ )    //将下面列消成 0
16                 if (fabs(a[i][c]) > eps)
17                     for (int j = n; j >= c; j -- )
18                         a[i][j] -= a[r][j] * a[i][c];
19             r ++ ;
20         }
21         if (r < n){
22             for (int i = r; i < n; i ++ )
23                 if (fabs(a[i][n]) > eps)
24                     return 2;
25             return 1;
26         }
27         for (int i = n - 1; i >= 0; i -- )
28             for (int j = i + 1; j < n; j ++ )
29                 a[i][n] -= a[i][j] * a[j][n];
30         return 0;
31     }
32 int main(){
33     cin >> n;
34     for (int i = 0; i < n; i ++ )
35         for (int j = 0; j < n + 1; j ++ )
36             cin >> a[i][j];
37     LL t = gauss();
38     if (t == 0){
39         for (int i = 0; i < n; i ++ ){
40             if (fabs(a[i][n]) < eps) a[i][n] = abs(a[i][n]);
41             printf("%.21f\n", a[i][n]);
42         }
43     }
44     else if (t == 1) cout << "Infinite group solutions\n";
45     else cout << "No solution\n";
46     return 0;
47 }
48
```

5.16 康拓展开

5.16.1 正向展开普通解法

将一个字典序排列转换成序号。例如：12345->1, 12354->2。

```
1  int f[20];
2  void jie_cheng(int n) { // 打出1-n的阶乘表
3      f[0] = f[1] = 1; // 0的阶乘为1
4      for (int i = 2; i <= n; i++) f[i] = f[i - 1] * i;
5  }
6  string str;
7  int kangtuo() {
8      int ans = 1; // 注意, 因为 12345 是算作0开始计算的, 最后结果要把12345看作是第一个
9      int len = str.length();
10     for (int i = 0; i < len; i++) {
11         int tmp = 0; // 用来计数的
12         // 计算str[i]是第几大的数, 或者说计算有几个比他小的数
13         for (int j = i + 1; j < len; j++)
14             if (str[i] > str[j]) tmp++;
15         ans += tmp * f[len - i - 1];
16     }
17     return ans;
18 }
19 int main() {
20     jie_cheng(10);
21     string str = "52413";
22     cout << kangtuo() << endl;
23 }
```

5.16.2 正向展开树状数组解

给定一个全排列, 求出它是 $1 \sim n$ 所有全排列的第几个, 答案对 998244353 取模。

答案就是 $\sum_{i=1}^n res_{a_i}(n-i)!$ 。 res_x 表示剩下的比 x 小的数字的数量, 通过树状数组处理。

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define LL long long
4  const int mod = 998244353, N = 1e6 + 10;
5  LL fact[N];
6  struct fwt{
7      LL n;
8      vector <LL> a;
9      fwt(LL n) : n(n), a(n + 1) {}
10     LL sum(LL x){
11         LL res = 0;
12         for (; x; x -= x & -x)
13             res += a[x];
14         return res;
15     }
16     void add(LL x, LL k){
17         for (; x <= n; x += x & -x)
18             a[x] += k;
19     }
20     LL query(LL x, LL y){
21         return sum(y) - sum(x - 1);
22     }
23 };
24 int main(){
25     ios::sync_with_stdio(false);cin.tie(0);
```

```

26     LL n;
27     cin >> n;
28     fwt a(n);
29     fact[0] = 1;
30     for (int i = 1; i <= n; i ++ ){
31         fact[i] = fact[i - 1] * i % mod;
32         a.add(i, 1);
33     }
34     LL ans = 0;
35     for (int i = 1; i <= n; i ++ ){
36         LL x;
37         cin >> x;
38         ans = (ans + a.query(1, x - 1) * fact[n - i] % mod ) % mod;
39         a.add(x, -1);
40     }
41     cout << (ans + 1) % mod << "\n";
42     return 0;
43 }

```

5.16.3 逆向还原

```

1  string str;
2  int kangtuo(){
3      int ans = 1; //注意, 因为 12345 是算作0开始计算的, 最后结果要把12345看作是第一个
4      int len = str.length();
5      for(int i = 0; i < len; i++){
6          int tmp = 0; //用来计数的
7          for(int j = i + 1; j < len; j++){
8              if(str[i] > str[j]) tmp++;
9              //计算str[i]是第几大的数, 或者说计算有几个比他小的数
10         }
11         ans += tmp * f[len - i - 1];
12     }
13     return ans;
14 }
15 int main(){
16     jie_cheng(10);
17     string str = "52413";
18     cout<<kangtuo()<<endl;
19 }

```

5.17 Min25 筛

求解 $1 - N$ 的质数和, 其中 $N \leq 10^{10}$ 。

```

1  namespace min25{
2      const int N = 1000000 + 10;
3      int prime[N], id1[N], id2[N], flag[N], ncnt, m;
4      LL g[N], sum[N], a[N], T;
5      LL n;
6      LL mod;
7      inline LL ps(LL n, LL k) {LL r=1;for(;k;k>>=1){if(k&1)r=r*n%mod;n=n*n%mod;}return r;}
8      void finit(){ // 最开始清0
9          memset(g, 0, sizeof(g));
10         memset(a, 0, sizeof(a));
11         memset(sum, 0, sizeof(sum));
12         memset(prime, 0, sizeof(prime));
13         memset(id1, 0, sizeof(id1));
14         memset(id2, 0, sizeof(id2));
15         memset(flag, 0, sizeof(flag));

```

```

16     ncnt = m = 0;
17 }
18 int ID(LL x) {
19     return x <= T ? id1[x] : id2[n / x];
20 }
21
22 LL calc(LL x) {
23     return x * (x + 1) / 2 - 1;
24 }
25
26 LL init(LL x) {
27     T = sqrt(x + 0.5);
28     for (int i = 2; i <= T; i++) {
29         if (!flag[i]) prime[++ncnt] = i, sum[ncnt] = sum[ncnt - 1] + i;
30         for (int j = 1; j <= ncnt && i * prime[j] <= T; j++) {
31             flag[i * prime[j]] = 1;
32             if (i % prime[j] == 0) break;
33         }
34     }
35     for (LL l = 1; l <= x; l = x / (x / l) + 1) {
36         a[++m] = x / l;
37         if (a[m] <= T) id1[a[m]] = m; else id2[x / a[m]] = m;
38         g[m] = calc(a[m]);
39     }
40     for (int i = 1; i <= ncnt; i++)
41         for (int j = 1; j <= m && (LL) prime[i] * prime[i] <= a[j]; j++)
42             g[j] = g[j] - (LL) prime[i] * (g[ID(a[j] / prime[i])] - sum[i - 1]);
43 }
44 LL solve(LL x) {
45     if (x <= 1) return x;
46     return n = x, init(n), g[ID(n)];
47 }
48 }
49
50 using namespace min25;
51
52 int main() {
53     // while (1) {
54     int tt;
55     scanf("%d", &tt);
56     while(tt--){
57         finit();
58         scanf("%lld%lld", &n, &mod);
59         LL ans = (n + 3) % mod * n % mod * ps(2, mod - 2) % mod + solve(n + 1) -
60 4;
61         // cout << solve(n) << endl;
62         // ans = (ans + mod) % mod;
63         ans = (ans + mod) % mod;
64         printf("%lld\n", ans);
65     }
66     // }
67 }

```

5.18 矩阵四则运算

[封装来自](#)。矩阵乘法复杂度 $\mathcal{O}(N^3)$ 。

```

1 const int SIZE = 2;
2 struct Matrix {
3     ll M[SIZE + 5][SIZE + 5];
4     void clear() { memset(M, 0, sizeof(M)); }

```

```

5 void reset() { //初始化
6     clear();
7     for (int i = 1; i <= SIZE; ++i) M[i][i] = 1;
8 }
9 Matrix friend operator*(const Matrix &A, const Matrix &B) {
10     Matrix Ans;
11     Ans.clear();
12     for (int i = 1; i <= SIZE; ++i)
13         for (int j = 1; j <= SIZE; ++j)
14             for (int k = 1; k <= SIZE; ++k)
15                 Ans.M[i][j] = (Ans.M[i][j] + A.M[i][k] * B.M[k][j]) % mod;
16     return Ans;
17 }
18 Matrix friend operator+(const Matrix &A, const Matrix &B) {
19     Matrix Ans;
20     Ans.clear();
21     for (int i = 1; i <= SIZE; ++i)
22         for (int j = 1; j <= SIZE; ++j)
23             Ans.M[i][j] = (A.M[i][j] + B.M[i][j]) % mod;
24     return Ans;
25 }
26 };
27
28 inline int mypow(LL n, LL k, int p = MOD) {
29     LL r = 1;
30     for (; k >= 1; n = n * n % p) {
31         if (k & 1) r = r * n % p;
32     }
33     return r;
34 }
35 bool ok = 1;
36 Matrix getinv(Matrix a) { //矩阵求逆
37     int n = SIZE, m = SIZE * 2;
38     for (int i = 1; i <= n; i++) a.M[i][i + n] = 1;
39     for (int i = 1; i <= n; i++) {
40         int pos = i;
41         for (int j = i + 1; j <= n; j++)
42             if (abs(a.M[j][i]) > abs(a.M[pos][i])) pos = j;
43         if (i != pos) swap(a.M[i], a.M[pos]);
44         if (!a.M[i][i]) {
45             puts("No Solution");
46             ok = 0;
47         }
48         ll inv = q_pow(a.M[i][i], mod - 2);
49         for (int j = 1; j <= n; j++)
50             if (j != i) {
51                 ll mul = a.M[j][i] * inv % mod;
52                 for (int k = i; k <= m; k++)
53                     a.M[j][k] = ((a.M[j][k] - a.M[i][k] * mul) % mod + mod) % mod;
54             }
55         for (int j = 1; j <= m; j++) a.M[i][j] = a.M[i][j] * inv % mod;
56     }
57     Matrix res;
58     res.clear();
59     for (int i = 1; i <= n; i++)
60         for (int j = 1; j <= m; j++)
61             res.M[i][j] = a.M[i][n + j];
62     return res;
63 }

```

5.19 矩阵快速幂

以 $O(N^3 \log M)$ 的复杂度计算。

```
1  const int N = 110, mod = 1e9 + 7;
2  LL n, k, a[N][N], b[N][N], t[N][N];
3  void matrixQp(LL y){
4      while (y){
5          if (y & 1){
6              memset(t, 0, sizeof t);
7              for (int i = 1; i <= n; i ++ )
8                  for (int j = 1; j <= n; j ++ )
9                      for (int k = 1; k <= n; k ++ )
10                         t[i][j] = ( t[i][j] + (a[i][k] * b[k][j]) % mod ) % mod;
11              memcpy(b, t, sizeof t);
12          }
13          y >>= 1;
14          memset(t, 0, sizeof t);
15          for (int i = 1; i <= n; i ++ )
16              for (int j = 1; j <= n; j ++ )
17                  for (int k = 1; k <= n; k ++ )
18                     t[i][j] = ( t[i][j] + (a[i][k] * a[k][j]) % mod ) % mod;
19          memcpy(a, t, sizeof t);
20      }
21 }
22 int main(){
23     cin >> n >> k;
24     for (int i = 1; i <= n; i ++ )
25         for (int j = 1; j <= n; j ++ ){
26             cin >> b[i][j];
27             a[i][j] = b[i][j];
28         }
29     matrixQp(k - 1);
30     for (int i = 1; i <= n; i ++ )
31         for (int j = 1; j <= n; j ++ )
32             cout << b[i][j] << " \n"[j == n];
33     return 0;
34 }
```

5.20 矩阵加速

```
1  const int mod = 1e9 + 7;
2  LL T, n, t[5][5], a[5][5], b[5][5];
3  void matrixQp(LL y){
4      while (y){
5          if (y & 1){
6              memset(t, 0, sizeof t);
7              for (int i = 1; i <= 3; i ++ )
8                  for (int j = 1; j <= 1; j ++ )
9                      for (int k = 1; k <= 3; k ++ )
10                         t[i][j] = ( t[i][j] + (a[i][k] * b[k][j]) % mod ) % mod;
11              memcpy(b, t, sizeof t);
12          }
13          y >>= 1;
14          memset(t, 0, sizeof t);
15          for (int i = 1; i <= 3; i ++ )
16              for (int j = 1; j <= 3; j ++ )
17                  for (int k = 1; k <= 3; k ++ )
18                     t[i][j] = ( t[i][j] + (a[i][k] * a[k][j]) % mod ) % mod;
19          memcpy(a, t, sizeof t);
20      }
```

```

21 }
22 void init(){
23     b[1][1] = b[2][1] = b[3][1] = 1;
24     memset(a, 0, sizeof a);
25     a[1][1] = a[2][1] = a[1][3] = a[3][2] = 1;
26 }
27 void solve(){
28     cin >> n;
29     if (n <= 3) cout << "1\n";
30     else{
31         init();
32         matrixQp(n - 3);
33         cout << b[1][1] << "\n";
34     }
35 }
36 int main(){
37     cin >> T;
38     while (T -- )
39         solve();
40     return 0;
41 }
42

```

5.21 莫比乌斯函数/反演

莫比乌斯函数定义： $\mu(n) = \begin{cases} 1 & n = 1 \\ (-1)^k & n = \prod_{i=1}^k p_i \text{ 且 } p_i \text{ 互质} \\ 0 & \text{else} \end{cases}$ 。

莫比乌斯函数性质：对于任意正整数 n 满足 $\sum_{d|n} \mu(d) = \begin{cases} 1 & n = 1 \\ 0 & n \neq 1 \end{cases}$ ； $\sum_{d|n} \frac{\mu(d)}{d} = \frac{\varphi(n)}{n}$ 。

莫比乌斯反演定义：定义： $F(n)$ 和 $f(n)$ 是定义在非负整数集合上的两个函数，并且满足 $F(n) = \sum_{d|n} f(d)$ ，可得 $f(n) = \sum_{d|n} \mu(d) F\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$ 。

```

1  const int N = 5e4 + 10;
2  bool st[N];
3  int mu[N], prime[N], cnt, sum[N];
4  void getMu() {
5      mu[1] = 1;
6      for (int i = 2; i <= N - 10; i++) {
7          if (!st[i]) {
8              prime[++cnt] = i;
9              mu[i] = -1;
10         }
11         for (int j = 1; j <= cnt && i * prime[j] <= N - 10; j++) {
12             st[i * prime[j]] = true;
13             if (i % prime[j] == 0) {
14                 mu[i * prime[j]] = 0;
15                 break;
16             }
17             mu[i * prime[j]] = -mu[i];
18         }
19     }
20     for (int i = 1; i <= N - 10; i++) {
21         sum[i] = sum[i - 1] + mu[i];
22     }
23 }
24 void solve() {

```

```

25     int n, m, k; cin >> n >> m >> k;
26     n = n / k, m = m / k;
27     if (n < m) swap(n, m);
28     LL ans = 0;
29     for (int i = 1, j = 0; i <= m; i = j + 1) {
30         j = min(n / (n / i), m / (m / i));
31         ans += (LL)(sum[j] - sum[i - 1]) * (n / i) * (m / i);
32     }
33     cout << ans << "\n";
34 }
35 int main() {
36     getMu();
37     int T; cin >> T;
38     while (T--) solve();
39 }

```

5.22 整除（数论）分块

$\left\lfloor \frac{n}{l} \right\rfloor = \left\lfloor \frac{n}{l+1} \right\rfloor = \dots = \left\lfloor \frac{n}{r} \right\rfloor \iff \left\lfloor \frac{n}{l} \right\rfloor \leq \frac{n}{r} < \left\lfloor \frac{n}{l} \right\rfloor + 1$ ，根据不等式左侧，得到
 $r \leq \left\lfloor \frac{n}{\left\lfloor \frac{n}{l} \right\rfloor} \right\rfloor$ 。

```

1 void solve() {
2     LL n; cin >> n;
3     LL ans = 0;
4     for (LL i = 1, j; i <= n; i = j + 1) {
5         j = n / (n / i);
6         ans += (LL)(j - i + 1) * (n / i);
7     }
8     cout << ans << "\n";
9 }
10 int main() {
11     int T; cin >> T;
12     while (T--) solve();
13 }

```

5.23 Miller - Rabin 素数测试

以平均 $\mathcal{O}(4 \cdot \log^3 X)$ 的复杂度判定数字 X 是否是素数，这里记录的版本复杂度非常优秀，基本可以看作是 $\mathcal{O}(1)$ 。

```

1 int mul(int a, int b, int m) {
2     int r = a * b - m * (int)(1.L / m * a * b);
3     return r - m * (r >= m) + m * (r < 0);
4 }
5 int mypow(int a, int b, int m) {
6     int res = 1 % m;
7     for (; b; b >>= 1, a = mul(a, a, m)) {
8         if (b & 1) {
9             res = mul(res, a, m);
10        }
11    }
12    return res;
13 }
14
15 int B[] = {2, 3, 5, 7, 11, 13, 17, 19, 23};
16 bool MR(int n) {
17     if (n <= 1) return 0;
18     for (int p : B) {

```

```

19     if (n == p) return 1;
20     if (n % p == 0) return 0;
21 }
22 int m = (n - 1) >> __builtin_ctz(n - 1);
23 for (int p : B) {
24     int t = m, a = mypow(p, m, n);
25     while (t != n - 1 && a != 1 && a != n - 1) {
26         a = mul(a, a, n);
27         t *= 2;
28     }
29     if (a != n - 1 && t % 2 == 0) return 0;
30 }
31 return 1;
32 }

```

5.24 Pollard - Rho 因式分解

以单个因子 $\mathcal{O}(\log X)$ 的复杂度输出数字 X 的全部质因数，由于需要结合素数测试，总复杂度会略高一些。如果遇到超时的情况，可能需要考虑进一步优化，例如检查题目是否强制要求枚举全部质因数等等。此外，还有一个[较长的模板](#)可供参考，比这里记录的版本常数小约五倍。

```

1  int PR(int n) {
2      for (int p : B) {
3          if (n % p == 0) return p;
4      }
5      auto f = [&](int x) -> int {
6          x = mul(x, x, n) + 1;
7          return x >= n ? x - n : x;
8      };
9      int x = 0, y = 0, tot = 0, p = 1, q, g;
10     for (int i = 0; (i & 255) || (g = gcd(p, n)) == 1; i++, x = f(x), y = f(f(y))) {
11         if (x == y) {
12             x = tot++;
13             y = f(x);
14         }
15         q = mul(p, abs(x - y), n);
16         if (q) p = q;
17     }
18     return g;
19 }
20 vector<int> fac(int n) {
21     #define pb emplace_back
22     if (n == 1) return {};
23     if (MR(n)) return {n};
24     int d = PR(n);
25     auto v1 = fac(d), v2 = fac(n / d);
26     auto i1 = v1.begin(), i2 = v2.begin();
27     vector<int> ans;
28     while (i1 != v1.end() || i2 != v2.end()) {
29         if (i1 == v1.end()) {
30             ans.pb(*i2++);
31         } else if (i2 == v2.end()) {
32             ans.pb(*i1++);
33         } else {
34             if (*i1 < *i2) {
35                 ans.pb(*i1++);
36             } else {
37                 ans.pb(*i2++);
38             }
39         }
40     }
41     return ans;

```

5.25 常见结论

5.25.1 球盒模型 (八种)

[参考链接](#)。给定 n 个小球 m 个盒子。

- 球同，盒不同、不能空

隔板法： N 个小球即一共 $N - 1$ 个空，分成 M 堆即 $M - 1$ 个隔板，答案为 $\binom{n-1}{m-1}$ 。

- 球同，盒不同、能空

隔板法：多出 $M - 1$ 个虚空球，答案为 $\binom{m-1+n}{n}$ 。

- 球同，盒同、能空

$\frac{1}{(1-x)(1-x^2)\dots(1-x^m)}$ 的 x^n 项的系数。动态规划，答案为

$$dp[i][j] = \begin{cases} dp[i][j-1] + dp[i-j][j] & i \geq j \\ dp[i][j-1] & i < j \\ 1 & j == 1 \text{ 且 } i \leq 1 \end{cases}$$

```

1 int dp[15][15];
2 void choose(int n) {
3     for(int i = 0; i < n; i++) {
4         for(int j = 1; j < n; j++) {
5             if(i <= 1 || j == 1) dp[i][j] = 1;
6             else if (i < j) dp[i][j] = dp[i][j-1];
7             else dp[i][j] = dp[i][j-1] + dp[i-j][j];
8         }
9     }
10 }
```

- 球同，盒同、不能空

$\frac{x^m}{(1-x)(1-x^2)\dots(1-x^m)}$ 的 x^n 项的系数。动态规划，答案为

$$dp[n][m] = \begin{cases} dp[n-m][m] & n \geq m \\ 0 & n < m \end{cases}$$

- 球不同，盒同、不能空

第二类斯特林数 $Stirling2(n, m)$ ，答案为

$$dp[n][m] = \begin{cases} m * dp[n-1][m] + dp[n-1][m-1] & 1 \leq m < n \\ 1 & 0 \leq n == m \\ 0 & m == 0 \text{ 且 } 1 \leq n \end{cases}$$

- 球不同，盒同、能空

第二类斯特林数之和 $\sum_{i=1}^m Stirling2(n, m)$ ，答案为 $\sum_{i=0}^m dp[n][i]$ 。

- 球不同，盒不同、不能空

第二类斯特林数乘上 m 的阶乘 $m! \cdot \text{Stirling2}(n, m)$ ，答案为 $dp[n][m] * m!$ 。

- 球不同，盒不同、能空

答案为 m^n 。

5.25.2 麦乐鸡定理

给定两个互质的数 n, m ，定义 $x = a * n + b * m$ ($a \geq 0, b \geq 0$)，当 $x > n * m - n - m$ 时，该式子恒成立。

5.25.3 抽屉原理（鸽巢原理）

将 $n + 1$ 个物体，划分为 n 组，那么有至少一组有两个（或以上）的物体。

5.25.4 哥德巴赫猜想

任何一个大于 5 的整数都可写成三个质数之和；任何一个大于 2 的偶数都可写成两个素数之和。

5.25.5 除法、取模运算的本质

有公式： $x \div i = \left\lfloor \frac{x}{i} \right\rfloor + x - i \cdot \left\lfloor \frac{x}{i} \right\rfloor$ ， $x \bmod i = x - i \cdot \left\lfloor \frac{x}{i} \right\rfloor$ 。

5.25.6 与、或、异或

运算	运算符、数学符号表示	解释
与	&、and	同1出1
或	、or	有1出1
异或	^、⊕、xor	不同出1

一些结论：

对于给定的 X 和序列 $[a_1, a_2, \dots, a_n]$ ，有： $X = (X \& a_1) \text{or} (X \& a_2) \text{or} \dots \text{or} (X \& a_n)$ 。
原理是 and 意味着取交集， or 意味着取子集。来源 - 牛客小白月赛49C

5.25.7 调和级数近似公式

```
1 | log(n) + 0.5772156649 + 1.0 / (2 * n)
```

5.25.8 欧拉函数常见性质

- $1 - n$ 中与 n 互质的数之和为 $n * \varphi(n) / 2$ 。
- 若 a, b 互质，则 $\varphi(a * b) = \varphi(a) * \varphi(b)$ 。实际上，所有满足这一条件的函数统称为积性函数。
- 若 f 是积性函数，且有 $n = \prod_{i=1}^m p_i^{c_i}$ ，那么 $f(n) = \prod_{i=1}^m f(p_i^{c_i})$ 。
- 若 p 为质数，且满足 $p \mid n$ ，
 - $p^2 \mid n$ ，那么 $\varphi(n) = \varphi(n/p) * p$ 。
 - $p^2 \nmid n$ ，那么 $\varphi(n) = \varphi(n/p) * (p - 1)$ 。
- $\sum_{d \mid n} \varphi(d) = n$ 。

- 如 $n = 10$ ，则 $d = 10/5/2/1$ ，那么 $10 = \varphi(10) + \varphi(5) + \varphi(2) + \varphi(1)$ 。
- $\sum_{i=1}^n \gcd(i, n) = \sum_{d|n} \left\lfloor \frac{n}{d} \right\rfloor \varphi(d)$ (欧拉反演)。

5.25.9 组合数学常见性质

- $k * C_n^k = n * C_{n-1}^{k-1}$;
- $C_k^n * C_m^k = C_m^n * C_{m-n}^{m-k}$;
- $C_n^k + C_n^{k+1} = C_{n+1}^{k+1}$;
- $\sum_{i=0}^n C_n^i = 2^n$;
- $\sum_{k=0}^n (-1)^k * C_n^k = 0$ 。
- 二项式反演:
$$\begin{cases} f_n = \sum_{i=0}^n \binom{n}{i} g_i \Leftrightarrow g_n = \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} f_i \\ f_k = \sum_{i=k}^n \binom{i}{k} g_i \Leftrightarrow g_k = \sum_{i=k}^n (-1)^{i-k} \binom{i}{k} f_i \end{cases}$$
 ;
- $\sum_{i=1}^n i \binom{n}{i} = n * 2^{n-1}$;
- $\sum_{i=1}^n i^2 \binom{n}{i} = n * (n+1) * 2^{n-2}$;
- $\sum_{i=1}^n \frac{1}{i} \binom{n}{i} = \sum_{i=1}^n \frac{1}{i}$;
- $\sum_{i=0}^n \binom{n}{i}^2 = \binom{2n}{n}$;
- 拉格朗日恒等式: $\sum_{i=1}^n \sum_{j=i+1}^n (a_i b_j - a_j b_i)^2 = (\sum_{i=1}^n a_i)^2 (\sum_{i=1}^n b_i)^2 - (\sum_{i=1}^n a_i b_i)^2$ 。

5.25.10 范德蒙德卷积公式

在数量为 $n + m$ 的堆中选 k 个元素，和分别在数量为 n 、 m 的堆中选 i 、 $k - i$ 个元素的方案数是相同的，即 $\sum_{i=0}^k \binom{n}{i} \binom{m}{k-i} = \binom{n+m}{k}$;

变体:

- $\sum_{i=0}^k C_{i+n}^i = C_{k+n+1}^k$;
- $\sum_{i=0}^k C_n^i * C_m^i = \sum_{i=0}^k C_n^i * C_m^{m-i} = C_{n+m}^n$ 。

5.25.11 卡特兰数

是一类奇特的组合数，前几项为 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862。如遇到以下问题，则直接套用即可。

- 【括号匹配问题】 n 个左括号和 n 个右括号组成的合法括号序列的数量，为 Cat_n 。
- 【进出栈问题】 $1, 2, \dots, n$ 经过一个栈，形成的合法出栈序列的数量，为 Cat_n 。
- 【二叉树生成问题】 n 个节点构成的不同二叉树的数量，为 Cat_n 。

- 【路径数量问题】在平面直角坐标系上，每一步只能**向上**或**向右**走，从 $(0,0)$ 走到 (n,n) ，并且除两个端点外不接触直线 $y=x$ 的路线数量，为 $2Cat_{n-1}$ 。

$$\text{计算公式: } Cat_n = \frac{C_{2n}^n}{n+1}, \quad C_n = \frac{C_{n-1} * (4n-2)}{n+1}.$$

5.25.12 狄利克雷卷积

$$\sum_{d|n} \varphi(d) = n, \quad \sum_{d|n} \mu(d) \frac{n}{d} = \varphi(n).$$

5.25.13 斐波那契数列

$$\text{通项公式: } F_n = \frac{1}{\sqrt{5}} * \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right].$$

直接结论：

- 卡西尼性质： $F_{n-1} * F_{n+1} - F_n^2 = (-1)^n$ ；
- $F_n^2 + F_{n+1}^2 = F_{2n+1}$ ；
- $F_{n+1}^2 - F_{n-1}^2 = F_{2n}$ （由上一条写两遍相减得到）；
- 若存在序列 $a_0 = 1, a_n = a_{n-1} + a_{n-3} + a_{n-5} + \dots (n \geq 1)$ 则 $a_n = F_n (n \geq 1)$ ；
- 齐肯多夫定理：任何正整数都可以表示成若干个不连续的斐波那契数（ F_2 开始）可以用贪心实现。

求和公式结论：

- 奇数项求和： $F_1 + F_3 + F_5 + \dots + F_{2n-1} = F_{2n}$ ；
- 偶数项求和： $F_2 + F_4 + F_6 + \dots + F_{2n} = F_{2n+1} - 1$ ；
- 平方和： $F_1^2 + F_2^2 + F_3^2 + \dots + F_n^2 = F_n * F_{n+1}$ ；
- $F_1 + 2F_2 + 3F_3 + \dots + nF_n = nF_{n+2} - F_{n+3} + 2$ ；
- $-F_1 + F_2 - F_3 + \dots + (-1)^n F_n = (-1)^n (F_{n+1} - F_n) + 1$ ；
- $F_{2n-2m-2}(F_{2n} + F_{2n+2}) = F_{2m+2} + F_{4n-2m}$ 。

数论结论：

- $F_a \mid F_b \Leftrightarrow a \mid b$ ；
- $\gcd(F_a, F_b) = F_{\gcd(a,b)}$ ；
- 当 p 为 $5k \pm 1$ 型素数时，
$$\begin{cases} F_{p-1} \equiv 0 \pmod{p} \\ F_p \equiv 1 \pmod{p} \\ F_{p+1} \equiv 1 \pmod{p} \end{cases} ;$$
- 当 p 为 $5k \pm 2$ 型素数时，
$$\begin{cases} F_{p-1} \equiv 1 \pmod{p} \\ F_p \equiv -1 \pmod{p} \\ F_{p+1} \equiv 0 \pmod{p} \end{cases} ;$$
- $F(n) \% m$ 的周期 $\leq 6m$ （ $m = 2 \times 5^k$ 时取到等号）；
- 既是斐波那契数又是平方数的有且仅有 1, 144。

5.25.14 杂

- 负数取模得到的是负数，如果要用 0/1 判断的话请取绝对值；
- 辗转相除法原式为 $\gcd(x, y) = \gcd(x, y - x)$ ，推广到 N 项为 $\gcd(a_1, a_2, \dots, a_N) = \gcd(a_1, a_2 - a_1, \dots, a_N - a_{N-1})$ ，
 - 该推论在“四则运算后 gcd”这类题中有特殊意义，如求解 $\gcd(a_1 + X, a_2 + X, \dots, a_N + X)$ 时[See](#)；
- 以下式子成立： $\gcd(a, m) = \gcd(a + x, m) \Leftrightarrow \gcd(a, m) = \gcd(x, m)$ 。求解上式满足条件的 x 的数量即为求比 $\frac{m}{\gcd(a, m)}$ 小且与其互质的数的个数，即用欧拉函数求解 $\varphi\left(\frac{m}{\gcd(a, m)}\right)$ 。
- 已知序列 a ，定义集合 $S = \{a_i \cdot a_j \mid i < j\}$ ，现在要求解 $\gcd(S)$ ，即为求解 $\gcd(a_j, \gcd(a_i \mid i < j))$ ，换句话说，即为求解后缀 gcd。
- 连续四个数互质的情况如下，当 n 为奇数时， $n, n-1, n-2$ 一定互质；而当 n 为偶数时，
 - $\begin{cases} n, n-1, n-3 \text{ 互质} & \gcd(n, n-3) = 1 \text{ 时} \\ n-1, n-2, n-3 \text{ 互质} & \gcd(n, n-3) \neq 1 \text{ 时} \end{cases}$ [See](#)；
- 由 $a \bmod b = (b + a) \bmod b = (2 \cdot b + a) \bmod b = \dots = (K \cdot b + a) \bmod b$ 可以推广得到 $(a \bmod b) \bmod c = ((K \cdot bc + a) \bmod b) \bmod c$ ，由此可以得到一个 bc 的答案周期[See](#)；
- 对于长度为 $2 \cdot N$ 的数列 a ，将其任意均分为两个长度为 N 的数列 p, q ，随后对 p 非递减排序、对 q 非递增排序，定义 $f(p, q) = \sum_{i=1}^n |p_i - q_i|$ ，那么答案为 a 数列前 N 大的数之和减去前 N 小的数之和[See](#)。
- 令 $\begin{cases} X = a + b \\ Y = a \oplus b \end{cases}$ ，如果该式子有解，那么存在前提条件 $\begin{cases} X \geq Y \\ X, Y \text{ 同奇偶} \end{cases}$ ；进一步，此时最小的 a 的取值为 $\frac{X - Y}{2}$ [See](#)。

然而，上方方程并不总是有解的，只有当变量增加到三个时，才**一定有解**，即：**在保证上方前提条件成立的情况下**，求解 $\begin{cases} X = a + b + c \\ Y = a \oplus b \oplus c \end{cases}$ ，则一定存在一组解 $\{\frac{X - Y}{2}, \frac{X - Y}{2}, Y\}$ [See](#)。
- 已知序列 p 是由序列 a_1 、序列 a_2 、.....、序列 a_n 合并而成，且合并过程中各序列内元素相对顺序不变，记 $T(p)$ 是 p 序列的最大前缀和，则 $T(p) = \sum_{i=1}^n T(a_i)$ [See](#)。
- $x + y = x|y + x \& y$ ，对于两个数字 x 和 y ，如果将 x 变为 $x|y$ ，同时将 y 变为 $x \& y$ ，那么在本质上即将 x 二进制模式下的全部 1 移动到了 y 的对应的位置上 [See](#)。
- 一个正整数 x 异或、加上另一个正整数 y 后奇偶性不发生变化： $a + b \equiv a \oplus b \pmod{2}$ [See](#)。

5.26 常见例题

题意：将 1 至 N 的每个数字分组，使得每一组的数字之和均为质数。输出每一个数字所在的组别，且要求分出的组数最少 [See](#)。

考察哥德巴赫猜想，记全部数字之和为 S ，分类讨论如下：

- 为 S 质数时，只需要分入同一组；
- 当 S 为偶数时，由猜想可知一定能分成两个质数，可以证明其中较小的那个一定小于 N ，暴力枚举分组；

- 当 $S - 2$ 为质数时，特殊判断出答案；
- 其余情况一定能被分成三组，其中 3 单独成组， $S - 3$ 后成为偶数，重复讨论二的过程即可。

题意：给定一个长度为 n 的数组，定义这个数组是 *BAD* 的，当且仅当可以把数组分成两个子序列，这两个子序列的元素之和相等。现在你需要删除**最少**的元素，使得删除后的数组不是 *BAD* 的。

最少删除一个元素——如果原数组存在奇数，则直接删除这个奇数即可；反之，我们发现，对数列同除以一个数不影响计算，故我们只需要找到最大的满足 $2^k \mid a_i$ 成立的 2^k ，随后将全部的 a_i 变为 $\frac{a_i}{2^k}$ ，此时一定有一个奇数（换句话说，我们可以对原数列的每一个元素不断的除以 2 直到出现奇数为止），删除这个奇数即可 [See](#)。

题意：设当前有一个数字为 x ，减去、加上最少的数字使得其能被 k 整除。

最少减去 $x \bmod k$ 这个很好想；最少加上 $\left(\left\lceil \frac{x}{k} \right\rceil * k\right) \bmod k$ 也比较好想，但是更简便的方法为加上 $k - x \bmod k$ ，这个式子等价于前面这一坨。

题意：给定一个整数 n ，用恰好 k 个 2 的幂次数之和表示它。例如： $n = 9, k = 4$ ，答案为 $1 + 2 + 2 + 4$ 。

结论1： k 合法当且仅当 `__builtin_popcountll(n) <= k && k <= n`，显然。

结论2： $2^{k+1} = 2 \cdot 2^k$ ，所以我们可以将二进制位看作是数组，然后从高位向低位推，一个高位等于两个低位，直到数组之和恰好等于 k ，随后依次输出即可。举例说明， $\{1, 0, 0, 1\} \rightarrow \{0, 2, 0, 1\} \rightarrow \{0, 1, 2, 1\}$ ，即答案为 0 个 2^3 、1 个 2^2 、.....。

```
1 signed main() {
2     int n, k;
3     cin >> n >> k;
4
5     int cnt = __builtin_popcountll(n);
6
7     if (k < cnt || n < k) {
8         cout << "NO\n";
9         return 0;
10    }
11    cout << "YES\n";
12
13    vector<int> num;
14    while (n) {
15        num.push_back(n % 2);
16        n /= 2;
17    }
18
19    for (int i = num.size() - 1; i > 0; i--) {
20        int p = min(k - cnt, num[i]);
21        num[i] -= p;
22        num[i - 1] += 2 * p;
23        cnt += p;
24    }
25
26    for (int i = 0; i < num.size(); i++) {
27        for (int j = 1; j <= num[i]; j++) {
28            cout << (1LL << i) << " ";
29        }
30    }
31 }
```

题意： n 个取值在 $[0, k)$ 之间的数之和为 m 的方案数

答案为 $\sum_{i=0}^n -1^i \cdot \binom{n}{i} \cdot \binom{m - i \cdot k + n - 1}{n - 1}$ [See1](#) [See2](#)。

```
1  Z clac(int n, int k, int m) {  
2      Z ans = 0;  
3      ans += C(n, i) * C(m - i * k + n - 1, n - 1) * pow(-1, i);  
4  }  
5      return ans;  
6  }
```

¹ 先考虑没有 k 的限制，那么即球盒模型： m 个球放入 n 个盒子，球同、盒子不同、能空。使用隔板法得到公式： $C(m + n - 1, n - 1)$ ；² 下面加上取值范围后进一步考虑：假设现在 n 个数之和为 $m - k$ ，运用上述隔板法可得公式： $C(m - k + n - 1, n - 1)$ ；³ 随后，选择任意一个数字，将其加上 k ，这样，这个数字一定不满足条件，选法为： $C(n, 1)$ ；⁴ 此时，至少有一个数字是不满足条件的，按照一般流程，到这里， $C(m + n - 1, n - 1) - C(n, 1) * C(m - k + n - 1, n - 1)$ 即是答案；但是，这样的操作会导致重复的部分，所以这里要使用容斥原理将重复部分去除（关于为什么会重复，试比较概率论中的加法公式）。

/END/

6 几何

6.1 库实数类实现（双精度）

```
1 using Real = int;
2 using Point = complex<Real>;
3
4 Real cross(const Point &a, const Point &b) {
5     return (conj(a) * b).imag();
6 }
7 Real dot(const Point &a, const Point &b) {
8     return (conj(a) * b).real();
9 }
```

6.2 平面几何必要初始化

6.2.1 字符串读入浮点数

```
1 const int Knum = 4;
2 int read(int k = Knum) {
3     string s;
4     cin >> s;
5
6     int num = 0;
7     int it = s.find('.');
8     if (it != -1) { // 存在小数点
9         num = s.size() - it - 1; // 计算小数位数
10        s.erase(s.begin() + it); // 删除小数点
11    }
12    for (int i = 1; i <= k - num; i++) { // 补全小数位数
13        s += '0';
14    }
15    return stoi(s);
16 }
```

6.2.2 预置函数

```
1 using ld = long double;
2 const ld PI = acos(-1);
3 const ld EPS = 1e-7;
4 const ld INF = numeric_limits<ld>::max();
5 #define cc(x) cout << fixed << setprecision(x);
6
7 ld fgcd(ld x, ld y) { // 实数域gcd
8     return abs(y) < EPS ? abs(x) : fgcd(y, fmod(x, y));
9 }
10 template<class T, class S> bool equal(T x, S y) {
11     return -EPS < x - y && x - y < EPS;
12 }
13 template<class T> int sign(T x) {
14     if (-EPS < x && x < EPS) return 0;
15     return x < 0 ? -1 : 1;
16 }
```

6.2.3 点线封装

```
1  template<class T> struct Point { // 在C++17下使用 emplace_back 绑定可能会导致CE!
2      T x, y;
3      Point(T x_ = 0, T y_ = 0) : x(x_), y(y_) {} // 初始化
4      template<class U> operator Point<U>() { // 自动类型匹配
5          return Point<U>(U(x), U(y));
6      }
7      Point &operator+=(Point p) & { return x += p.x, y += p.y, *this; }
8      Point &operator+=(T t) & { return x += t, y += t, *this; }
9      Point &operator-=(Point p) & { return x -= p.x, y -= p.y, *this; }
10     Point &operator-=(T t) & { return x -= t, y -= t, *this; }
11     Point &operator*=(T t) & { return x *= t, y *= t, *this; }
12     Point &operator/=(T t) & { return x /= t, y /= t, *this; }
13     Point operator-() const { return Point(-x, -y); }
14     friend Point operator+(Point a, Point b) { return a += b; }
15     friend Point operator+(Point a, T b) { return a += b; }
16     friend Point operator-(Point a, Point b) { return a -= b; }
17     friend Point operator-(Point a, T b) { return a -= b; }
18     friend Point operator*(Point a, T b) { return a *= b; }
19     friend Point operator*(T a, Point b) { return b *= a; }
20     friend Point operator/(Point a, T b) { return a /= b; }
21     friend bool operator<(Point a, Point b) {
22         return equal(a.x, b.x) ? a.y < b.y - EPS : a.x < b.x - EPS;
23     }
24     friend bool operator>(Point a, Point b) { return b < a; }
25     friend bool operator==(Point a, Point b) { return !(a < b) && !(b < a); }
26     friend bool operator!=(Point a, Point b) { return a < b || b < a; }
27     friend auto &operator>>(istream &is, Point &p) {
28         return is >> p.x >> p.y;
29     }
30     friend auto &operator<<(ostream &os, Point p) {
31         return os << "(" << p.x << ", " << p.y << ")";
32     }
33 };
34 template<class T> struct Line {
35     Point<T> a, b;
36     Line(Point<T> a_ = Point<T>(), Point<T> b_ = Point<T>()) : a(a_), b(b_) {}
37     template<class U> operator Line<U>() { // 自动类型匹配
38         return Line<U>(Point<U>(a), Point<U>(b));
39     }
40     friend auto &operator<<(ostream &os, Line l) {
41         return os << "<" << l.a << ", " << l.b << ">";
42     }
43 };
```

6.2.4 叉乘

定义公式 $a \times b = |a||b| \sin \theta$ 。

```
1  template<class T> T cross(Point<T> a, Point<T> b) { // 叉乘
2      return a.x * b.y - a.y * b.x;
3  }
4  template<class T> T cross(Point<T> p1, Point<T> p2, Point<T> p0) { // 叉乘 (p1 - p0)
5      x (p2 - p0);
6      return cross(p1 - p0, p2 - p0);
7  }
```

6.2.5 点乘

定义公式 $a \times b = |a||b| \cos \theta$ 。

```
1 template<class T> T dot(Point<T> a, Point<T> b) { // 点乘
2     return a.x * b.x + a.y * b.y;
3 }
4 template<class T> T dot(Point<T> p1, Point<T> p2, Point<T> p0) { // 点乘 (p1 - p0) *
    (p2 - p0);
5     return dot(p1 - p0, p2 - p0);
6 }
```

6.2.6 欧几里得距离公式

最常用的距离公式。**需要注意**，开根号会丢失精度，如无强制要求，先不要开根号，留到最后一步一起开。

```
1 template <class T> ld dis(T x1, T y1, T x2, T y2) {
2     ld val = (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2);
3     return sqrt(val);
4 }
5 template <class T> ld dis(Point<T> a, Point<T> b) {
6     return dis(a.x, a.y, b.x, b.y);
7 }
```

6.2.7 曼哈顿距离公式

```
1 template <class T> T dis1(Point<T> p1, Point<T> p2) { // 曼哈顿距离公式
2     return abs(p1.x - p2.x) + abs(p1.y - p2.y);
3 }
```

6.2.8 将向量转换为单位向量

```
1 Point<ld> standardize(Point<ld> vec) { // 转换为单位向量
2     return vec / sqrt(vec.x * vec.x + vec.y * vec.y);
3 }
```

6.2.9 向量旋转

将当前向量移动至原点后顺时针旋转 90° ，即获取垂直于当前向量的、起点为原点的向量。在计算垂线时非常有用。例如，要想获取点 a 绕点 o 顺时针旋转 90° 后的点，可以这样书写代码：`auto ans = o + rotate(o, a);`；如果是逆时针旋转，那么只需更改符号即可：`auto ans = o - rotate(o, a);`。

```
1 template<class T> Point<T> rotate(Point<T> p1, Point<T> p2) { // 旋转
2     Point<T> vec = p1 - p2;
3     return {-vec.y, vec.x};
4 }
```

6.3 平面角度与弧度

6.3.1 弧度角度相互转换

```
1 ld toDeg(ld x) { // 弧度转角度
2     return x * 180 / PI;
3 }
4 ld toArc(ld x) { // 角度转弧度
5     return PI / 180 * x;
6 }
```

6.3.2 正弦定理

$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$, 其中 R 为三角形外接圆半径;

6.3.3 余弦定理 (已知三角形三边, 求角)

$\cos C = \frac{a^2 + b^2 - c^2}{2ab}$, $\cos B = \frac{a^2 + c^2 - b^2}{2ac}$, $\cos A = \frac{b^2 + c^2 - a^2}{2bc}$ 。可以借此推导出三角形面积公式 $S_{\triangle ABC} = \frac{ab \cdot \sin C}{2} = \frac{bc \cdot \sin A}{2} = \frac{ac \cdot \sin B}{2}$ 。

注意, 计算格式是: 由 b, c, a 三边求 $\angle A$; 由 a, c, b 三边求 $\angle B$; 由 a, b, c 三边求 $\angle C$ 。

```
1 ld angle(ld a, ld b, ld c) { // 余弦定理
2     ld val = acos((a * a + b * b - c * c) / (2.0 * a * b)); // 计算弧度
3     return val;
4 }
```

6.3.4 求两向量的夹角

能够计算 $[0^\circ, 180^\circ]$ 区间的角度。

```
1 ld angle(Point<ld> a, Point<ld> b) {
2     ld val = abs(cross(a, b));
3     return abs(atan2(val, a.x * b.x + a.y * b.y));
4 }
```

6.3.5 向量旋转任意角度

逆时针旋转, 转换公式:
$$\begin{cases} x' = x \cos \theta - y \sin \theta \\ y' = x \sin \theta + y \cos \theta \end{cases}$$

```
1 Point<ld> rotate(Point<ld> p, ld rad) {
2     return {p.x * cos(rad) - p.y * sin(rad), p.x * sin(rad) + p.y * cos(rad)};
3 }
```

6.3.6 点绕点旋转任意角度

逆时针旋转, 转换公式:
$$\begin{cases} x' = (x_0 - x_1) \cos \theta + (y_0 - y_1) \sin \theta + x_1 \\ y' = (x_1 - x_0) \sin \theta + (y_0 - y_1) \cos \theta + y_1 \end{cases}$$

```

1 Point<ld> rotate(Point<ld> a, Point<ld> b, ld rad) {
2     ld x = (a.x - b.x) * cos(rad) + (a.y - b.y) * sin(rad) + b.x;
3     ld y = (b.x - a.x) * sin(rad) + (a.y - b.y) * cos(rad) + b.y;
4     return {x, y};
5 }

```

6.4 平面点线相关

6.4.1 点是否在直线上（三点是否共线）

```

1 template<class T> bool onLine(Point<T> a, Point<T> b, Point<T> c) {
2     return sign(cross(b, a, c)) == 0;
3 }
4 template<class T> bool onLine(Point<T> p, Line<T> l) {
5     return onLine(p, l.a, l.b);
6 }

```

6.4.2 点是否在向量（直线）左侧

需要注意，向量的方向会影响答案；点在向量上时不视为在左侧。

```

1 template<class T> bool pointOnLineLeft(Pt p, Lt l) {
2     return cross(l.b, p, l.a) > 0;
3 }

```

6.4.3 两点是否在直线同侧/异侧

```

1 template<class T> bool pointOnLineSide(Pt p1, Pt p2, Lt vec) {
2     T val = cross(p1, vec.a, vec.b) * cross(p2, vec.a, vec.b);
3     return sign(val) == 1;
4 }
5 template<class T> bool pointNotOnLineSide(Pt p1, Pt p2, Lt vec) {
6     T val = cross(p1, vec.a, vec.b) * cross(p2, vec.a, vec.b);
7     return sign(val) == -1;
8 }

```

6.4.4 两直线相交交点

在使用前需要先判断直线是否平行。

```

1 Pd lineIntersection(Ld l1, Ld l2) {
2     ld val = cross(l2.b - l2.a, l1.a - l2.a) / cross(l2.b - l2.a, l1.a - l1.b);
3     return l1.a + (l1.b - l1.a) * val;
4 }

```

6.4.5 两直线是否平行/垂直/相同

```
1 template<class T> bool lineParallel(Lt p1, Lt p2) {
2     return sign(cross(p1.a - p1.b, p2.a - p2.b)) == 0;
3 }
4 template<class T> bool lineVertical(Lt p1, Lt p2) {
5     return sign(dot(p1.a - p1.b, p2.a - p2.b)) == 0;
6 }
7 template<class T> bool same(Line<T> l1, Line<T> l2) {
8     return lineParallel(Line{l1.a, l2.b}, {l1.b, l2.a}) &&
9         lineParallel(Line{l1.a, l2.a}, {l1.b, l2.b}) && lineParallel(l1, l2);
10 }
```

6.4.6 点到直线的最近距离与最近点

```
1 pair<Pd, Ld> pointToLine(Pd p, Ld l) {
2     Pd ans = lineIntersection({p, p + rotate(l.a, l.b)}, l);
3     return {ans, dis(p, ans)};
4 }
```

如果只需要计算最近距离，下方的写法可以减少书写的代码量，效果一致。

```
1 template<class T> Ld disPointToLine(Pt p, Lt l) {
2     Ld ans = cross(p, l.a, l.b);
3     return abs(ans) / dis(l.a, l.b); // 面积除以底边长
4 }
```

6.4.7 点是否在线段上

```
1 template<class T> bool pointOnSegment(Pt p, Lt l) { // 端点也算作在直线上
2     return sign(cross(p, l.a, l.b)) == 0 && min(l.a.x, l.b.x) <= p.x && p.x <=
max(l.a.x, l.b.x) &&
3         min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y);
4 }
5 template<class T> bool pointOnSegment(Pt p, Lt l) { // 端点不算
6     return pointOnSegment(p, l) && min(l.a.x, l.b.x) < p.x && p.x < max(l.a.x, l.b.x)
&&
7         min(l.a.y, l.b.y) < p.y && p.y < max(l.a.y, l.b.y);
8 }
```

6.4.8 点到线段的最近距离与最近点

```
1 pair<Pd, Ld> pointToSegment(Pd p, Ld l) {
2     if (sign(dot(p, l.b, l.a)) == -1) { // 特判到两端点的距离
3         return {l.a, dis(p, l.a)};
4     } else if (sign(dot(p, l.a, l.b)) == -1) {
5         return {l.b, dis(p, l.b)};
6     }
7     return pointToLine(p, l);
8 }
```

6.4.9 点在直线上的投影点（垂足）

```
1 Pd project(Pd p, Ld l) { // 投影
2     Pd vec = l.b - l.a;
3     Ld r = dot(vec, p - l.a) / (vec.x * vec.x + vec.y * vec.y);
4     return l.a + vec * r;
5 }
```

6.4.10 线段的中垂线

```
1 template<class T> Lt midSegment(Lt l) {
2     Pt mid = (l.a + l.b) / 2; // 线段中点
3     return {mid, mid + rotate(l.a, l.b)};
4 }
```

6.4.11 两线段是否相交及交点

该扩展版可以同时返回相交状态和交点，分为四种情况：0 代表不相交；1 代表普通相交；2 代表重叠（交于两个点）；3 代表相交于端点。**需要注意**，部分运算可能会使用到直线求交点，此时务必保证变量类型为浮点数！

```
1 template<class T> tuple<int, Pt, Pt> segmentIntersection(Lt l1, Lt l2) {
2     auto [s1, e1] = l1;
3     auto [s2, e2] = l2;
4     auto A = max(s1.x, e1.x), AA = min(s1.x, e1.x);
5     auto B = max(s1.y, e1.y), BB = min(s1.y, e1.y);
6     auto C = max(s2.x, e2.x), CC = min(s2.x, e2.x);
7     auto D = max(s2.y, e2.y), DD = min(s2.y, e2.y);
8     if (A < CC || C < AA || B < DD || D < BB) {
9         return {0, {}, {}};
10    }
11    if (sign(cross(e1 - s1, e2 - s2)) == 0) {
12        if (sign(cross(s2, e1, s1)) != 0) {
13            return {0, {}, {}};
14        }
15        Pt p1(max(AA, CC), max(BB, DD));
16        Pt p2(min(A, C), min(B, D));
17        if (!pointOnSegment(p1, l1)) {
18            swap(p1.y, p2.y);
19        }
20        if (p1 == p2) {
21            return {3, p1, p2};
22        } else {
23            return {2, p1, p2};
24        }
25    }
26    auto cp1 = cross(s2 - s1, e2 - s1);
27    auto cp2 = cross(s2 - e1, e2 - e1);
28    auto cp3 = cross(s1 - s2, e1 - s2);
29    auto cp4 = cross(s1 - e2, e1 - e2);
30    if (sign(cp1 * cp2) == 1 || sign(cp3 * cp4) == 1) {
31        return {0, {}, {}};
32    }
33    // 使用下方函数时请使用浮点数
34    Pd p = lineIntersection(l1, l2);
35    if (sign(cp1) != 0 && sign(cp2) != 0 && sign(cp3) != 0 && sign(cp4) != 0) {
36        return {1, p, p};
37    } else {
38        return {3, p, p};
39    }
```

如果不需要求交点，那么使用快速排斥+跨立实验即可，其中重叠、相交于端点均视为相交。

```

1  template<class T> bool segmentIntersection(Lt l1, Lt l2) {
2      auto [s1, e1] = l1;
3      auto [s2, e2] = l2;
4      auto A = max(s1.x, e1.x), AA = min(s1.x, e1.x);
5      auto B = max(s1.y, e1.y), BB = min(s1.y, e1.y);
6      auto C = max(s2.x, e2.x), CC = min(s2.x, e2.x);
7      auto D = max(s2.y, e2.y), DD = min(s2.y, e2.y);
8      return A >= CC && B >= DD && C >= AA && D >= BB &&
9              sign(cross(s1, s2, e1) * cross(s1, e1, e2)) == 1 &&
10             sign(cross(s2, s1, e2) * cross(s2, e2, e1)) == 1;
11 }

```

6.5 平面圆相关（浮点数处理）

6.5.1 点到圆的最近点

同时返回最近点与最近距离。**需要注意**，当点为圆心时，这样的点有无数个，此时我们视作输入错误，直接返回圆心。

```

1  pair<Pd, ld> pointToCircle(Pd p, Pd o, ld r) {
2      Pd U = o, V = o;
3      ld d = dis(p, o);
4      if (sign(d) == 0) { // p 为圆心时返回圆心本身
5          return {o, 0};
6      }
7      ld val1 = r * abs(o.x - p.x) / d;
8      ld val2 = r * abs(o.y - p.y) / d * ((o.x - p.x) * (o.y - p.y) < 0 ? -1 : 1);
9      U.x += val1, U.y += val2;
10     V.x -= val1, V.y -= val2;
11     if (dis(U, p) < dis(V, p)) {
12         return {U, dis(U, p)};
13     } else {
14         return {V, dis(V, p)};
15     }
16 }

```

6.5.2 根据圆心角获取圆上某点

将圆上最右侧的点以圆心为旋转中心，逆时针旋转 `rad` 度。

```

1  Point<ld> getPoint(Point<ld> p, ld r, ld rad) {
2      return {p.x + cos(rad) * r, p.y + sin(rad) * r};
3  }

```

6.5.3 直线是否与圆相交及交点

0 代表不相交；1 代表相切；2 代表相交。

```

1 tuple<int, Pd, Pd> lineCircleCross(Ld l, Pd o, ld r) {
2     Pd P = project(o, l);
3     ld d = dis(P, o), tmp = r * r - d * d;
4     if (sign(tmp) == -1) {
5         return {0, {}, {}};
6     } else if (sign(tmp) == 0) {
7         return {1, P, {}};
8     }
9     Pd vec = standardize(l.b - l.a) * sqrt(tmp);
10    return {2, P + vec, P - vec};
11 }

```

6.5.4 线段是否与圆相交及交点

0 代表不相交；1 代表相切；2 代表相交于一个点；3 代表相交于两个点。

```

1 tuple<int, Pd, Pd> segmentCircleCross(Ld l, Pd o, ld r) {
2     auto [type, U, V] = lineCircleCross(l, o, r);
3     bool f1 = pointOnSegment(U, l), f2 = pointOnSegment(V, l);
4     if (type == 1 && f1) {
5         return {1, U, {}};
6     } else if (type == 2 && f1 && f2) {
7         return {3, U, V};
8     } else if (type == 2 && f1) {
9         return {2, U, {}};
10    } else if (type == 2 && f2) {
11        return {2, V, {}};
12    } else {
13        return {0, {}, {}};
14    }
15 }

```

6.5.5 两圆是否相交及交点

0 代表内含；1 代表相离；2 代表相切；3 代表相交。

```

1 tuple<int, Pd, Pd> circleIntersection(Pd p1, ld r1, Pd p2, ld r2) {
2     ld x1 = p1.x, x2 = p2.x, y1 = p1.y, y2 = p2.y, d = dis(p1, p2);
3     if (sign(abs(r1 - r2) - d) == 1) {
4         return {0, {}, {}};
5     } else if (sign(r1 + r2 - d) == -1) {
6         return {1, {}, {}};
7     }
8     ld a = r1 * (x1 - x2) * 2, b = r1 * (y1 - y2) * 2, c = r2 * r2 - r1 * r1 - d *
d;
9     ld p = a * a + b * b, q = -a * c * 2, r = c * c - b * b;
10    ld cosa, sina, cosb, sinb;
11    if (sign(d - (r1 + r2)) == 0 || sign(d - abs(r1 - r2)) == 0) {
12        cosa = -q / p / 2;
13        sina = sqrt(1 - cosa * cosa);
14        Point<ld> p0 = {x1 + r1 * cosa, y1 + r1 * sina};
15        if (sign(dis(p0, p2) - r2)) {
16            p0.y = y1 - r1 * sina;
17        }
18        return {2, p0, p0};
19    } else {
20        ld delta = sqrt(q * q - p * r * 4);
21        cosa = (delta - q) / p / 2;
22        cosb = (-delta - q) / p / 2;
23        sina = sqrt(1 - cosa * cosa);

```

```

24     sinb = sqrt(1 - cosb * cosb);
25     Pd ans1 = {x1 + r1 * cosa, y1 + r1 * sina};
26     Pd ans2 = {x1 + r1 * cosb, y1 + r1 * sinb};
27     if (sign(dis(ans1, p1) - r2)) ans1.y = y1 - r1 * sina;
28     if (sign(dis(ans2, p2) - r2)) ans2.y = y1 - r1 * sinb;
29     if (ans1 == ans2) ans1.y = y1 - r1 * sina;
30     return {3, ans1, ans2};
31 }
32 }

```

6.5.6 两圆相交面积

上述所言四种相交情况均可计算，之所以不使用三角形面积计算公式是因为在计算过程中会出现“负数”面积（扇形面积与三角形面积的符号关系会随圆的位置关系发生变化），故公式全部重新推导，这里采用的是扇形面积减去扇形内部的那个三角形的面积。

```

1  ld circleIntersectionArea(Pd p1, ld r1, Pd p2, ld r2) {
2      ld x1 = p1.x, x2 = p2.x, y1 = p1.y, y2 = p2.y, d = dis(p1, p2);
3      if (sign(abs(r1 - r2) - d) >= 0) {
4          return PI * min(r1 * r1, r2 * r2);
5      } else if (sign(r1 + r2 - d) == -1) {
6          return 0;
7      }
8      ld theta1 = angle(r1, dis(p1, p2), r2);
9      ld area1 = r1 * r1 * (theta1 - sin(theta1 * 2) / 2);
10     ld theta2 = angle(r2, dis(p1, p2), r1);
11     ld area2 = r2 * r2 * (theta2 - sin(theta2 * 2) / 2);
12     return area1 + area2;
13 }

```

6.5.7 三点确定一圆

```

1  tuple<int, Pd, ld> getCircle(Pd A, Pd B, Pd C) {
2      if (onLine(A, B, C)) { // 特判三点共线
3          return {0, {}, 0};
4      }
5      Ld l1 = midSegment(Line{A, B});
6      Ld l2 = midSegment(Line{A, C});
7      Pd O = lineIntersection(l1, l2);
8      return {1, O, dis(A, O)};
9  }

```

6.5.8 求解点到圆的切线数量与切点

```

1  pair<int, vector<Point<ld>>> tangent(Point<ld> p, Point<ld> A, ld r) {
2      vector<Point<ld>> ans; // 储存切点
3      Point<ld> u = A - p;
4      ld d = sqrt(dot(u, u));
5      if (d < r) {
6          return {0, {}};
7      } else if (sign(d - r) == 0) { // 点在圆上
8          ans.push_back(u);
9          return {1, ans};
10     } else {
11         ld ang = asin(r / d);
12         ans.push_back(getPoint(A, r, -ang));
13         ans.push_back(getPoint(A, r, ang));
14         return {2, ans};
15     }

```

6.5.9 求解两圆的内公、外公切线数量与切点

同时返回公切线数量以及每个圆的切点。

```

1  tuple<int, vector<Point<ld>>, vector<Point<ld>>> tangent(Point<ld> A, ld Ar,
   Point<ld> B, ld Br) {
2      vector<Point<ld>> a, b; // 储存切点
3      if (Ar < Br) {
4          swap(Ar, Br);
5          swap(A, B);
6          swap(a, b);
7      }
8      int d = disEx(A, B), dif = Ar - Br, sum = Ar + Br;
9      if (d < dif * dif) { // 内含, 无
10         return {0, {}, {}};
11     }
12     ld base = atan2(B.y - A.y, B.x - A.x);
13     if (d == 0 && Ar == Br) { // 完全重合, 无数条外公切线
14         return {-1, {}, {}};
15     }
16     if (d == dif * dif) { // 内切, 1条外公切线
17         a.push_back(getPoint(A, Ar, base));
18         b.push_back(getPoint(B, Br, base));
19         return {1, a, b};
20     }
21     ld ang = acos(dif / sqrt(d));
22     a.push_back(getPoint(A, Ar, base + ang)); // 保底2条外公切线
23     a.push_back(getPoint(A, Ar, base - ang));
24     b.push_back(getPoint(B, Br, base + ang));
25     b.push_back(getPoint(B, Br, base - ang));
26     if (d == sum * sum) { // 外切, 多1条内公切线
27         a.push_back(getPoint(A, Ar, base));
28         b.push_back(getPoint(B, Br, base + PI));
29     } else if (d > sum * sum) { // 相离, 多2条内公切线
30         ang = acos(sum / sqrt(d));
31         a.push_back(getPoint(A, Ar, base + ang));
32         a.push_back(getPoint(A, Ar, base - ang));
33         b.push_back(getPoint(B, Br, base + ang + PI));
34         b.push_back(getPoint(B, Br, base - ang + PI));
35     }
36     return {a.size(), a, b};
37 }

```

6.6 平面三角形相关 (浮点数处理)

6.6.1 三角形面积

```

1  ld area(Point<ld> a, Point<ld> b, Point<ld> c) {
2      return abs(cross(b, c, a)) / 2;
3  }

```

6.6.2 三角形外心

三角形外接圆的圆心，即三角形三边垂直平分线的交点。

```

1 template<class T> Pt center1(Pt p1, Pt p2, Pt p3) { // 外心
2     return lineIntersection(midSegment({p1, p2}), midSegment({p2, p3}));
3 }

```

6.6.3 三角形内心

三角形内切圆的圆心，也是三角形三个内角的角平分线的交点。其到三角形三边的距离相等。

```

1 Pd center2(Pd p1, Pd p2, Pd p3) { // 内心
2     #define atan2(p) atan2(p.y, p.x) // 注意先后顺序
3     Line<ld> U = {p1, {}}, V = {p2, {}};
4     ld m, n, alpha;
5     m = atan2((p2 - p1));
6     n = atan2((p3 - p1));
7     alpha = (m + n) / 2;
8     U.b = {p1.x + cos(alpha), p1.y + sin(alpha)};
9     m = atan2((p1 - p2));
10    n = atan2((p3 - p2));
11    alpha = (m + n) / 2;
12    V.b = {p2.x + cos(alpha), p2.y + sin(alpha)};
13    return lineIntersection(U, V);
14 }

```

6.6.4 三角形垂心

三角形的三条高线所在直线的交点。锐角三角形的垂心在三角形内；直角三角形的垂心在直角顶点上；钝角三角形的垂心在三角形外。

```

1 Pd center3(Pd p1, Pd p2, Pd p3) { // 垂心
2     Ld U = {p1, p1 + rotate(p2, p3)}; // 垂线
3     Ld V = {p2, p2 + rotate(p1, p3)};
4     return lineIntersection(U, V);
5 }

```

6.7 平面直线方程转换

6.7.1 浮点数计算直线的斜率

一般很少使用到这个函数，因为斜率的取值不可控（例如接近平行于 x, y 轴时）。**需要注意**，当直线平行于 y 轴时斜率为 `inf`。

```

1 template <class T> ld slope(Pt p1, Pt p2) { // 斜率, 注意 inf 的情况
2     return (p1.y - p2.y) / (p1.x - p2.x);
3 }
4 template <class T> ld slope(Lt l) {
5     return slope(l.a, l.b);
6 }

```

6.7.2 分数精确计算直线的斜率

调用分数四则运算精确计算斜率，返回最简分数，只适用于整数计算。

```

1 template<class T> Frac<T> slopeEx(Pt p1, Pt p2) {
2     Frac<T> U = p1.y - p2.y;
3     Frac<T> V = p1.x - p2.x;
4     return U / V; // 调用分数精确计算
5 }

```

6.7.3 两点式转一般式

返回由三个整数构成的方程，在输入较大时可能找不到较小的满足题意的一组整数解。可以处理平行于 x, y 轴、两点共点的情况。

```
1  template<class T> tuple<int, int, int> getfun(Lt p) {
2      T A = p.a.y - p.b.y, B = p.b.x - p.a.x, C = p.a.x * A + p.a.y * B;
3      if (A < 0) { // 符号调整
4          A = -A, B = -B, C = -C;
5      } else if (A == 0) {
6          if (B < 0) {
7              B = -B, C = -C;
8          } else if (B == 0 && C < 0) {
9              C = -C;
10         }
11     }
12     if (A == 0) { // 数值计算
13         if (B == 0) {
14             C = 0; // 共点特判
15         } else {
16             T g = fgcd(abs(B), abs(C));
17             B /= g, C /= g;
18         }
19     } else if (B == 0) {
20         T g = fgcd(abs(A), abs(C));
21         A /= g, C /= g;
22     } else {
23         T g = fgcd(fgcd(abs(A), abs(B)), abs(C));
24         A /= g, B /= g, C /= g;
25     }
26     return tuple{A, B, C}; // Ax + By = C
27 }
```

6.7.4 一般式转两点式

由于整数点可能很大或者不存在，故直接采用浮点数；如果与 x, y 轴有交点则取交点。可以处理平行于 x, y 轴的情况。

```
1  Line<ld> getfun(int A, int B, int C) { // Ax + By = C
2      ld x1 = 0, y1 = 0, x2 = 0, y2 = 0;
3      if (A && B) { // 正常
4          if (C) {
5              x1 = 0, y1 = 1. * C / B;
6              y2 = 0, x2 = 1. * C / A;
7          } else { // 过原点
8              x1 = 1, y1 = 1. * -A / B;
9              x2 = 0, y2 = 0;
10         }
11     } else if (A && !B) { // 垂直
12         if (C) {
13             y1 = 0, x1 = 1. * C / A;
14             y2 = 1, x2 = x1;
15         } else {
16             x1 = 0, y1 = 1;
17             x2 = 0, y2 = 0;
18         }
19     } else if (!A && B) { // 水平
20         if (C) {
21             x1 = 0, y1 = 1. * C / B;
22             x2 = 1, y2 = y1;
23         } else {
```

```

24         x1 = 1, y1 = 0;
25         x2 = 0, y2 = 0;
26     }
27     } else { // 不合法, 请特判
28         assert(false);
29     }
30     return {{x1, y1}, {x2, y2}};
31 }

```

6.7.5 抛物线与 x 轴是否相交及交点

0 代表没有交点; 1 代表相切; 2 代表有两个交点。

```

1  tuple<int, ld, ld> getAns(ld a, ld b, ld c) {
2      ld delta = b * b - a * c * 4;
3      if (delta < 0.) {
4          return {0, 0, 0};
5      }
6      delta = sqrt(delta);
7      ld ans1 = -(delta + b) / 2 / a;
8      ld ans2 = (delta - b) / 2 / a;
9      if (ans1 > ans2) {
10         swap(ans1, ans2);
11     }
12     if (sign(delta) == 0) {
13         return {1, ans2, 0};
14     }
15     return {2, ans1, ans2};
16 }

```

6.8 平面多边形

6.8.1 两向量构成的平面四边形有向面积

```

1  template<class T> T areaEx(Point<T> p1, Point<T> p2, Point<T> p3) {
2      return cross(b, c, a);
3  }

```

6.8.2 判断四个点能否组成矩形/正方形

可以处理浮点数、共点的情况。返回分为三种情况：2 代表构成正方形；1 代表构成矩形；0 代表其他情况。

```

1  template<class T> int isSquare(vector<Pt> x) {
2      sort(x.begin(), x.end());
3      if (equal(dis(x[0], x[1]), dis(x[2], x[3])) && sign(dis(x[0], x[1])) &&
4          equal(dis(x[0], x[2]), dis(x[1], x[3])) && sign(dis(x[0], x[2])) &&
5          lineParallel(Lt{x[0], x[1]}, Lt{x[2], x[3]}) &&
6          lineParallel(Lt{x[0], x[2]}, Lt{x[1], x[3]}) &&
7          lineVertical(Lt{x[0], x[1]}, Lt{x[0], x[2]})) {
8          return equal(dis(x[0], x[1]), dis(x[0], x[2])) ? 2 : 1;
9      }
10     return 0;
11 }

```

6.8.3 点是否在任意多边形内

射线法判定, t 为穿越次数, 当其为奇数时即代表点在多边形内部; 返回 2 代表点在多边形边界上。

```
1 template<class T> int pointInPolygon(Point<T> a, vector<Point<T>> p) {
2     int n = p.size();
3     for (int i = 0; i < n; i++) {
4         if (pointOnSegment(a, Line{p[i], p[(i + 1) % n]})) {
5             return 2;
6         }
7     }
8     int t = 0;
9     for (int i = 0; i < n; i++) {
10        auto u = p[i], v = p[(i + 1) % n];
11        if (u.x < a.x && v.x >= a.x && pointOnLineLeft(a, Line{v, u})) {
12            t ^= 1;
13        }
14        if (u.x >= a.x && v.x < a.x && pointOnLineLeft(a, Line{u, v})) {
15            t ^= 1;
16        }
17    }
18    return t == 1;
19 }
```

6.8.4 线段是否在任意多边形内部

```
1 template<class T>
2 bool segmentInPolygon(Line<T> l, vector<Point<T>> p) {
3     // 线段与多边形边界不相交且两端点都在多边形内部
4     #define L(x, y) pointOnLineLeft(x, y)
5     int n = p.size();
6     if (!pointInPolygon(l.a, p)) return false;
7     if (!pointInPolygon(l.b, p)) return false;
8     for (int i = 0; i < n; i++) {
9         auto u = p[i];
10        auto v = p[(i + 1) % n];
11        auto w = p[(i + 2) % n];
12        auto [t, p1, p2] = segmentIntersection(l, Line(u, v));
13        if (t == 1) return false;
14        if (t == 0) continue;
15        if (t == 2) {
16            if (pointOnSegment(v, l) && v != l.a && v != l.b) {
17                if (cross(v - u, w - v) > 0) {
18                    return false;
19                }
20            }
21        } else {
22            if (p1 != u && p1 != v) {
23                if (L(l.a, Line(v, u)) || L(l.b, Line(v, u))) {
24                    return false;
25                }
26            } else if (p1 == v) {
27                if (l.a == v) {
28                    if (L(u, l)) {
29                        if (L(w, l) && L(w, Line(u, v))) {
30                            return false;
31                        }
32                    } else {
33                        if (L(w, l) || L(w, Line(u, v))) {
34                            return false;
35                        }
36                    }
37            }
38        }
39    }
40 }
```

```

37         } else if (l.b == v) {
38             if (L(u, Line(l.b, l.a))) {
39                 if (L(w, Line(l.b, l.a)) && L(w, Line(u, v))) {
40                     return false;
41                 }
42             } else {
43                 if (L(w, Line(l.b, l.a)) || L(w, Line(u, v))) {
44                     return false;
45                 }
46             }
47         } else {
48             if (L(u, l)) {
49                 if (L(w, Line(l.b, l.a)) || L(w, Line(u, v))) {
50                     return false;
51                 }
52             } else {
53                 if (L(w, l) || L(w, Line(u, v))) {
54                     return false;
55                 }
56             }
57         }
58     }
59 }
60 }
61 return true;
62 }

```

6.8.5 任意多边形的面积

```

1 template<class T> ld area(vector<Point<T>> P) {
2     int n = P.size();
3     ld ans = 0;
4     for (int i = 0; i < n; i++) {
5         ans += cross(P[i], P[(i + 1) % n]);
6     }
7     return ans / 2.0;
8 }

```

6.8.6 皮克定理

绘制在方格纸上的多边形面积公式可以表示为 $S = n + \frac{s}{2} - 1$ ，其中 n 表示多边形内部的点数、 s 表示多边形边界上的点数。一条线段上的点数为 $\gcd(|x_1 - x_2|, |y_1 - y_2|) + 1$ 。

6.8.7 任意多边形上/内的网格点个数（仅能处理整数）

皮克定理用。

```

1 int onPolygonGrid(vector<Point<int>> p) { // 多边形上
2     int n = p.size(), ans = 0;
3     for (int i = 0; i < n; i++) {
4         auto a = p[i], b = p[(i + 1) % n];
5         ans += gcd(abs(a.x - b.x), abs(a.y - b.y));
6     }
7     return ans;
8 }
9 int inPolygonGrid(vector<Point<int>> p) { // 多边形内
10    int n = p.size(), ans = 0;
11    for (int i = 0; i < n; i++) {
12        auto a = p[i], b = p[(i + 1) % n], c = p[(i + 2) % n];
13        ans += b.y * (a.x - c.x);

```

```

14     }
15     ans = abs(ans);
16     return (ans - onPolygonGrid(p)) / 2 + 1;
17 }

```

6.9 二维凸包

6.9.1 获取二维静态凸包 (Andrew算法)

`flag` 用于判定凸包边上的点、重复的顶点是否要加入到凸包中，为 0 时代表加入凸包（不严格）；为 1 时不加入凸包（严格）。时间复杂度为 $O(N \log N)$ 。

```

1  template<class T> vector<Point<T>> staticConvexHull(vector<Point<T>> A, int flag =
2  1) {
3      int n = A.size();
4      if (n <= 2) { // 特判
5          return A;
6      }
7      vector<Point<T>> ans(n * 2);
8      sort(A.begin(), A.end());
9      int now = -1;
10     for (int i = 0; i < n; i++) { // 维护下凸包
11         while (now > 0 && cross(A[i], ans[now], ans[now - 1]) < flag) {
12             now--;
13         }
14         ans[++now] = A[i];
15     }
16     int pre = now;
17     for (int i = n - 2; i >= 0; i--) { // 维护上凸包
18         while (now > pre && cross(A[i], ans[now], ans[now - 1]) < flag) {
19             now--;
20         }
21         ans[++now] = A[i];
22     }
23     ans.resize(now);
24     return ans;
25 }

```

6.9.2 二维动态凸包

固定为 `int` 型，需要重新书写 `Line` 函数，`cmp` 用于判定边界情况。可以处理如下两个要求：

- 动态插入点 (x, y) 到当前凸包中；
- 判断点 (x, y) 是否在凸包上或是在内部（包括边界）。

```

1  template<class T> bool turnRight(Pt a, Pt b) {
2      return cross(a, b) < 0 || (cross(a, b) == 0 && dot(a, b) < 0);
3  }
4  struct Line {
5      static int cmp;
6      mutable Point<int> a, b;
7      friend bool operator<(Line x, Line y) {
8          return cmp ? x.a < y.a : turnRight(x.b, y.b);
9      }
10     friend auto &operator<<(ostream &os, Line l) {
11         return os << "<" << l.a << ", " << l.b << ">";
12     }
13 };
14
15 int Line::cmp = 1;

```

```

16 struct UpperConvexHull : set<Line> {
17     bool contains(const Point<int> &p) const {
18         auto it = lower_bound({p, 0});
19         if (it != end() && it->a == p) return true;
20         if (it != begin() && it != end() && cross(prev(it)->b, p - prev(it)->a) <=
21             0) {
22             return true;
23         }
24         return false;
25     }
26     void add(const Point<int> &p) {
27         if (contains(p)) return;
28         auto it = lower_bound({p, 0});
29         for (; it != end(); it = erase(it)) {
30             if (turnRight(it->a - p, it->b)) {
31                 break;
32             }
33         }
34         for (; it != begin() && prev(it) != begin(); erase(prev(it))) {
35             if (turnRight(prev(prev(it))->b, p - prev(prev(it))->a)) {
36                 break;
37             }
38         }
39         if (it != begin()) {
40             prev(it)->b = p - prev(it)->a;
41         }
42         if (it == end()) {
43             insert({p, {0, -1}});
44         } else {
45             insert({p, it->a - p});
46         }
47     };
48 struct ConvexHull {
49     UpperConvexHull up, low;
50     bool empty() const {
51         return up.empty();
52     }
53     bool contains(const Point<int> &p) const {
54         Line::cmp = 1;
55         return up.contains(p) && low.contains(-p);
56     }
57     void add(const Point<int> &p) {
58         Line::cmp = 1;
59         up.add(p);
60         low.add(-p);
61     }
62     bool isIntersect(int A, int B, int C) const {
63         Line::cmp = 0;
64         if (empty()) return false;
65         Point<int> k = {-B, A};
66         if (k.x < 0) k = -k;
67         if (k.x == 0 && k.y < 0) k.y = -k.y;
68         Point<int> P = up.upper_bound({{0, 0}, k})->a;
69         Point<int> Q = -low.upper_bound({{0, 0}, k})->a;
70         return sign(A * P.x + B * P.y - C) * sign(A * Q.x + B * Q.y - C) > 0;
71     }
72     friend ostream &operator<<(ostream &out, const ConvexHull &ch) {
73         for (const auto &line : ch.up) out << "(" << line.a.x << "," << line.a.y <<
74             ")";
75         cout << "/";
76         for (const auto &line : ch.low) out << "(" << -line.a.x << "," << -line.a.y
77             << ")";
78         return out;

```

```

77     }
78 };

```

6.9.3 点与凸包的位置关系

0 代表点在凸包外面；1 代表在凸壳上；2 代表在凸包内部。

```

1  template<class T> int contains(Point<T> p, vector<Point<T>> A) {
2      int n = A.size();
3      bool in = false;
4      for (int i = 0; i < n; i++) {
5          Point<T> a = A[i] - p, b = A[(i + 1) % n] - p;
6          if (a.y > b.y) {
7              swap(a, b);
8          }
9          if (a.y <= 0 && 0 < b.y && cross(a, b) < 0) {
10             in = !in;
11         }
12         if (cross(a, b) == 0 && dot(a, b) <= 0) {
13             return 1;
14         }
15     }
16     return in ? 2 : 0;
17 }

```

6.9.4 闵可夫斯基和

计算两个凸包合成的大凸包。

```

1  template<class T> vector<Point<T>> mincowski(vector<Point<T>> P1, vector<Point<T>>
2  P2) {
3      int n = P1.size(), m = P2.size();
4      vector<Point<T>> V1(n), V2(m);
5      for (int i = 0; i < n; i++) {
6          V1[i] = P1[(i + 1) % n] - P1[i];
7      }
8      for (int i = 0; i < m; i++) {
9          V2[i] = P2[(i + 1) % m] - P2[i];
10     }
11     vector<Point<T>> ans = {P1.front() + P2.front()};
12     int t = 0, i = 0, j = 0;
13     while (i < n && j < m) {
14         Point<T> val = sign(cross(V1[i], V2[j])) > 0 ? V1[i++] : V2[j++];
15         ans.push_back(ans.back() + val);
16     }
17     while (i < n) ans.push_back(ans.back() + V1[i++]);
18     while (j < m) ans.push_back(ans.back() + V2[j++]);
19     return ans;
20 }

```

6.9.5 半平面交

计算多条直线左边平面部分的交集。

```

1  template<class T> vector<Point<T>> halfcut(vector<Line<T>> lines) {
2      sort(lines.begin(), lines.end(), [&](auto l1, auto l2) {
3          auto d1 = l1.b - l1.a;
4          auto d2 = l2.b - l2.a;
5          if (sign(d1) != sign(d2)) {
6              return sign(d1) == 1;

```

```

7         }
8         return cross(d1, d2) > 0;
9     });
10    deque<Line<T>> ls;
11    deque<Point<T>> ps;
12    for (auto l : lines) {
13        if (ls.empty()) {
14            ls.push_back(l);
15            continue;
16        }
17        while (!ps.empty() && !pointOnLineLeft(ps.back(), l)) {
18            ps.pop_back();
19            ls.pop_back();
20        }
21        while (!ps.empty() && !pointOnLineLeft(ps[0], l)) {
22            ps.pop_front();
23            ls.pop_front();
24        }
25        if (cross(l.b - l.a, ls.back().b - ls.back().a) == 0) {
26            if (dot(l.b - l.a, ls.back().b - ls.back().a) > 0) {
27                if (!pointOnLineLeft(ls.back().a, l)) {
28                    assert(ls.size() == 1);
29                    ls[0] = l;
30                }
31                continue;
32            }
33            return {};
34        }
35        ps.push_back(lineIntersection(ls.back(), l));
36        ls.push_back(l);
37    }
38    while (!ps.empty() && !pointOnLineLeft(ps.back(), ls[0])) {
39        ps.pop_back();
40        ls.pop_back();
41    }
42    if (ls.size() <= 2) {
43        return {};
44    }
45    ps.push_back(lineIntersection(ls[0], ls.back()));
46    return vector(ps.begin(), ps.end());
47 }

```

6.10 三维几何必要初始化

6.10.1 点线面封装

```

1 struct Point3 {
2     ld x, y, z;
3     Point3(ld x_ = 0, ld y_ = 0, ld z_ = 0) : x(x_), y(y_), z(z_) {}
4     Point3 &operator+=(Point3 p) & {
5         return x += p.x, y += p.y, z += p.z, *this;
6     }
7     Point3 &operator-=(Point3 p) & {
8         return x -= p.x, y -= p.y, z -= p.z, *this;
9     }
10    Point3 &operator*=(Point3 p) & {
11        return x *= p.x, y *= p.y, z *= p.z, *this;
12    }
13    Point3 &operator*=(ld t) & {
14        return x *= t, y *= t, z *= t, *this;
15    }
16    Point3 &operator/=(ld t) & {

```

```

17         return x /= t, y /= t, z /= t, *this;
18     }
19     friend Point3 operator+(Point3 a, Point3 b) { return a += b; }
20     friend Point3 operator-(Point3 a, Point3 b) { return a -= b; }
21     friend Point3 operator*(Point3 a, Point3 b) { return a *= b; }
22     friend Point3 operator*(Point3 a, ld b) { return a *= b; }
23     friend Point3 operator*(ld a, Point3 b) { return b *= a; }
24     friend Point3 operator/(Point3 a, ld b) { return a /= b; }
25     friend auto &operator>>(istream &is, Point3 &p) {
26         return is >> p.x >> p.y >> p.z;
27     }
28     friend auto &operator<<(ostream &os, Point3 p) {
29         return os << "(" << p.x << ", " << p.y << ", " << p.z << ")";
30     }
31 };
32 struct Line3 {
33     Point3 a, b;
34 };
35 struct Plane {
36     Point3 u, v, w;
37 };

```

6.10.2 其他函数

```

1 ld len(P3 p) { // 原点到当前点的距离计算
2     return sqrt(p.x * p.x + p.y * p.y + p.z * p.z);
3 }
4 P3 crossEx(P3 a, P3 b) { // 叉乘
5     P3 ans;
6     ans.x = a.y * b.z - a.z * b.y;
7     ans.y = a.z * b.x - a.x * b.z;
8     ans.z = a.x * b.y - a.y * b.x;
9     return ans;
10 }
11 ld cross(P3 a, P3 b) {
12     return len(crossEx(a, b));
13 }
14 ld dot(P3 a, P3 b) { // 点乘
15     return a.x * b.x + a.y * b.y + a.z * b.z;
16 }
17 P3 getVec(Plane s) { // 获取平面法向量
18     return crossEx(s.u - s.v, s.v - s.w);
19 }
20 ld dis(P3 a, P3 b) { // 三维欧几里得距离公式
21     ld val = (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y) + (a.z - b.z) *
(a.z - b.z);
22     return sqrt(val);
23 }
24 P3 standardize(P3 vec) { // 将三维向量转换为单位向量
25     return vec / len(vec);
26 }

```

6.11 三维点线面相关

6.11.1 空间三点是否共线

其中第二个函数是专门用来判断给定的三个点能否构成平面的，因为不共线的三点才能构成平面。

```

1 bool onLine(P3 p1, P3 p2, P3 p3) { // 三点是否共线
2     return sign(cross(p1 - p2, p3 - p2)) == 0;
3 }
4 bool onLine(Plane s) {
5     return onLine(s.u, s.v, s.w);
6 }

```

6.11.2 四点是否共面

```

1 bool onPlane(P3 p1, P3 p2, P3 p3, P3 p4) { // 四点是否共面
2     ld val = dot(getVec({p1, p2, p3}), p4 - p1);
3     return sign(val) == 0;
4 }

```

6.11.3 空间点是否在线段上

```

1 bool pointOnSegment(P3 p, L3 l) {
2     return sign(cross(p - l.a, p - l.b)) == 0 && min(l.a.x, l.b.x) <= p.x &&
3         p.x <= max(l.a.x, l.b.x) && min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y,
4         l.b.y) &&
5         min(l.a.z, l.b.z) <= p.z && p.z <= max(l.a.z, l.b.z);
6 }
7 bool pointOnSegmentEx(P3 p, L3 l) { // pointOnSegment去除端点版
8     return sign(cross(p - l.a, p - l.b)) == 0 && min(l.a.x, l.b.x) < p.x &&
9         p.x < max(l.a.x, l.b.x) && min(l.a.y, l.b.y) < p.y && p.y < max(l.a.y,
10        l.b.y) &&
11        min(l.a.z, l.b.z) < p.z && p.z < max(l.a.z, l.b.z);
12 }

```

6.11.4 空间两点是否在线段同侧

当给定的两点与线段不共面、点在线段上时返回 *false*。

```

1 bool pointOnSegmentSide(P3 p1, P3 p2, L3 l) {
2     if (!onPlane(p1, p2, l.a, l.b)) { // 特判不共面
3         return 0;
4     }
5     ld val = dot(crossEx(l.a - l.b, p1 - l.b), crossEx(l.a - l.b, p2 - l.b));
6     return sign(val) == 1;
7 }

```

6.11.5 两点是否在平面同侧

点在平面上时返回 *false*。

```

1 bool pointOnPlaneSide(P3 p1, P3 p2, Plane s) {
2     ld val = dot(getVec(s), p1 - s.u) * dot(getVec(s), p2 - s.u);
3     return sign(val) == 1;
4 }

```

6.11.6 空间两直线是否平行/垂直

```
1 bool lineParallel(L3 l1, L3 l2) {
2     return sign(cross(l1.a - l1.b, l2.a - l2.b)) == 0;
3 }
4 bool lineVertical(L3 l1, L3 l2) {
5     return sign(dot(l1.a - l1.b, l2.a - l2.b)) == 0;
6 }
```

6.11.7 两平面是否平行/垂直

```
1 bool planeParallel(Plane s1, Plane s2) {
2     ld val = cross(getVec(s1), getVec(s2));
3     return sign(val) == 0;
4 }
5 bool planeVertical(Plane s1, Plane s2) {
6     ld val = dot(getVec(s1), getVec(s2));
7     return sign(val) == 0;
8 }
```

6.11.8 空间两直线是否是同一条

```
1 bool same(L3 l1, L3 l2) {
2     return lineParallel(l1, l2) && lineParallel({l1.a, l2.b}, {l1.b, l2.a});
3 }
```

6.11.9 两平面是否是同一个

```
1 bool same(Plane s1, Plane s2) {
2     return onPlane(s1.u, s2.u, s2.v, s2.w) && onPlane(s1.v, s2.u, s2.v, s2.w) &&
3         onPlane(s1.w, s2.u, s2.v, s2.w);
4 }
```

6.11.10 直线是否与平面平行

```
1 bool linePlaneParallel(L3 l, Plane s) {
2     ld val = dot(l.a - l.b, getVec(s));
3     return sign(val) == 0;
4 }
```

6.11.11 空间两线段是否相交

```
1 bool segmentIntersection(L3 l1, L3 l2) { // 重叠、相交于端点均视为相交
2     if (!onPlane(l1.a, l1.b, l2.a, l2.b)) { // 特判不共面
3         return 0;
4     }
5     if (!onLine(l1.a, l1.b, l2.a) || !onLine(l1.a, l1.b, l2.b)) {
6         return !pointOnSegmentSide(l1.a, l1.b, l2) && !pointOnSegmentSide(l2.a,
7             l2.b, l1);
8     }
9     return pointOnSegment(l1.a, l2) || pointOnSegment(l1.b, l2) ||
10        pointOnSegment(l2.a, l1) ||
11        pointOnSegment(l2.b, l2);
12 }
13 bool segmentIntersection1(L3 l1, L3 l2) { // 重叠、相交于端点不视为相交
14     return onPlane(l1.a, l1.b, l2.a, l2.b) && !pointOnSegmentSide(l1.a, l1.b, l2) &&
15        !pointOnSegmentSide(l2.a, l2.b, l1);
16 }
```

6.11.12 空间两直线是否相交及交点

当两直线不共面、两直线平行时返回 *false*。

```

1 pair<bool, P3> lineIntersection(L3 l1, L3 l2) {
2     if (!onPlane(l1.a, l1.b, l2.a, l2.b) || lineParallel(l1, l2)) {
3         return {0, {}};
4     }
5     auto [s1, e1] = l1;
6     auto [s2, e2] = l2;
7     ld val = 0;
8     if (!onPlane(l1.a, l1.b, {0, 0, 0}, {0, 0, 1})) {
9         val = ((s1.x - s2.x) * (s2.y - e2.y) - (s1.y - s2.y) * (s2.x - e2.x)) /
10              ((s1.x - e1.x) * (s2.y - e2.y) - (s1.y - e1.y) * (s2.x - e2.x));
11     } else if (!onPlane(l1.a, l1.b, {0, 0, 0}, {0, 1, 0})) {
12         val = ((s1.x - s2.x) * (s2.z - e2.z) - (s1.z - s2.z) * (s2.x - e2.x)) /
13              ((s1.x - e1.x) * (s2.z - e2.z) - (s1.z - e1.z) * (s2.x - e2.x));
14     } else {
15         val = ((s1.y - s2.y) * (s2.z - e2.z) - (s1.z - s2.z) * (s2.y - e2.y)) /
16              ((s1.y - e1.y) * (s2.z - e2.z) - (s1.z - e1.z) * (s2.y - e2.y));
17     }
18     return {1, s1 + (e1 - s1) * val};
19 }

```

6.11.13 直线与平面是否相交及交点

当直线与平面平行、给定的点构不成平面时返回 *false*。

```

1 pair<bool, P3> linePlaneCross(L3 l, Plane s) {
2     if (linePlaneParallel(l, s)) {
3         return {0, {}};
4     }
5     P3 vec = getVec(s);
6     P3 U = vec * (s.u - l.a), V = vec * (l.b - l.a);
7     ld val = (U.x + U.y + U.z) / (V.x + V.y + V.z);
8     return {1, l.a + (l.b - l.a) * val};
9 }

```

6.11.14 两平面是否相交及交线

当两平面平行、两平面为同一个时返回 *false*。

```

1 pair<bool, L3> planeIntersection(Plane s1, Plane s2) {
2     if (planeParallel(s1, s2) || same(s1, s2)) {
3         return {0, {}};
4     }
5     P3 U = linePlaneParallel({s2.u, s2.v}, s1) ? linePlaneCross({s2.v, s2.w},
6 s1).second
7 : linePlaneCross({s2.u, s2.v},
8 s1).second;
9     P3 V = linePlaneParallel({s2.w, s2.u}, s1) ? linePlaneCross({s2.v, s2.w},
10 s1).second
11 : linePlaneCross({s2.w, s2.u},
12 s1).second;
13     return {1, {U, V}};
14 }

```

6.11.15 点到直线的最近点与最近距离

```
1 pair<ld, P3> pointToLine(P3 p, L3 l) {
2     ld val = cross(p - l.a, l.a - l.b) / dis(l.a, l.b); // 面积除以底边长
3     ld val1 = dot(p - l.a, l.a - l.b) / dis(l.a, l.b);
4     return {val, l.a + val1 * standardize(l.a - l.b)};
5 }
```

6.11.16 点到平面的最近点与最近距离

```
1 pair<ld, P3> pointToPlane(P3 p, Plane s) {
2     P3 vec = getVec(s);
3     ld val = dot(vec, p - s.u);
4     val = abs(val) / len(vec); // 面积除以底边长
5     return {val, p - val * standardize(vec)};
6 }
```

6.11.17 空间两直线的最近距离与最近点对

```
1 tuple<ld, P3, P3> lineToLine(L3 l1, L3 l2) {
2     P3 vec = crossEx(l1.a - l1.b, l2.a - l2.b); // 计算同时垂直于两直线的向量
3     ld val = abs(dot(l1.a - l2.a, vec)) / len(vec);
4     P3 U = l1.b - l1.a, V = l2.b - l2.a;
5     vec = crossEx(U, V);
6     ld p = dot(vec, vec);
7     ld t1 = dot(crossEx(l2.a - l1.a, V), vec) / p;
8     ld t2 = dot(crossEx(l2.a - l1.a, U), vec) / p;
9     return {val, l1.a + (l1.b - l1.a) * t1, l2.a + (l2.b - l2.a) * t2};
10 }
```

6.12 三维角度与弧度

6.12.1 空间两直线夹角的 cos 值

任意位置的空间两直线。

```
1 ld lineCos(L3 l1, L3 l2) {
2     return dot(l1.a - l1.b, l2.a - l2.b) / len(l1.a - l1.b) / len(l2.a - l2.b);
3 }
```

6.12.2 空间两平面夹角的 cos 值

```
1 ld planeCos(Plane s1, Plane s2) {
2     P3 U = getVec(s1), V = getVec(s2);
3     return dot(U, V) / len(U) / len(V);
4 }
```

6.12.3 直线与平面夹角的 sin 值

```
1 ld linePlaneSin(L3 l, Plane s) {
2     P3 vec = getVec(s);
3     return dot(l.a - l.b, vec) / len(l.a - l.b) / len(vec);
4 }
```

6.13 空间多边形

6.13.1 正N棱锥体积公式

棱锥通用体积公式 $V = \frac{1}{3}Sh$ ，当其恰好是棱长为 l 的正 n 棱锥时，有公式

$$V = \frac{l^3 \cdot n}{12 \tan \frac{\pi}{n}} \cdot \sqrt{1 - \frac{1}{4 \cdot \sin^2 \frac{\pi}{n}}}。$$

```
1 ld V(ld l, int n) { // 正n棱锥体积公式
2     return l * l * l * n / (12 * tan(PI / n)) * sqrt(1 - 1 / (4 * sin(PI / n) *
3     sin(PI / n)));
}
```

6.13.2 四面体体积

```
1 ld V(P3 a, P3 b, P3 c, P3 d) {
2     return abs(dot(d - a, crossEx(b - a, c - a))) / 6;
3 }
```

6.13.3 点是否在空间三角形上

点位于边界上时返回 *false*。

```
1 bool pointOnTriangle(P3 p, P3 p1, P3 p2, P3 p3) {
2     return pointOnSegmentSide(p, p1, {p2, p3}) && pointOnSegmentSide(p, p2, {p1, p3})
3     &&
4     pointOnSegmentSide(p, p3, {p1, p2});
}
```

6.13.4 线段是否与空间三角形相交及交点

只有交点在空间三角形内部时才视作相交。

```
1 pair<bool, P3> segmentOnTriangle(P3 l, P3 r, P3 p1, P3 p2, P3 p3) {
2     P3 x = crossEx(p2 - p1, p3 - p1);
3     if (sign(dot(x, r - l)) == 0) {
4         return {0, {}};
5     }
6     ld t = dot(x, p1 - l) / dot(x, r - l);
7     if (t < 0 || t - 1 > 0) { // 不在线段上
8         return {0, {}};
9     }
10    bool type = pointOnTriangle(l + (r - l) * t, p1, p2, p3);
11    if (type) {
12        return {1, l + (r - l) * t};
13    } else {
14        return {0, {}};
15    }
16 }
```

6.13.5 空间三角形是否相交

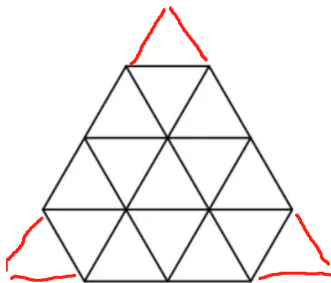
相交线段在空间三角形内部时才视作相交。

```
1 bool triangleIntersection(vector<P3> a, vector<P3> b) {
2     for (int i = 0; i < 3; i++) {
3         if (segmentOnTriangle(b[i], b[(i + 1) % 3], a[0], a[1], a[2]).first) {
4             return 1;
5         }
6         if (segmentOnTriangle(a[i], a[(i + 1) % 3], b[0], b[1], b[2]).first) {
7             return 1;
8         }
9     }
10    return 0;
11 }
```

6.14 常用结论

6.14.1 平面几何结论归档

- `hypot` 函数可以直接计算直角三角形的斜边长;
- **边心距**是指正多边形的外接圆圆心到正多边形某一边的距离, 边长为 s 的正 n 角形的边心距公式为 $a = \frac{t}{2 \cdot \tan \frac{\pi}{n}}$, 外接圆半径为 R 的正 n 角形的边心距公式为 $a = R \cdot \cos \frac{\pi}{n}$;
- **三角形外接圆半径**为 $\frac{a}{2 \sin A} = \frac{abc}{4S}$, 其中 S 为三角形面积, 内切圆半径为 $\frac{2S}{a+b+c}$;
- 由小正三角形拼成的大正三角形, 耗费的小三角形数量即为构成一条边的小三角形数量的平方。如下图, 总数量即为 4^2 [See](#)。



- 正 n 边形圆心角为 $\frac{360^\circ}{n}$, 圆周角为 $\frac{180^\circ}{n}$ 。定义正 n 边形上的三个顶点 A, B 和 C (可以不相邻), 使得 $\angle ABC = \theta$, 当 $n \leq 360$ 时, θ 可以取 1° 到 179° 间的任何一个整数 [See](#)。
- 某一点 B 到直线 AC 的距离公式为 $\frac{|\vec{BA} \times \vec{BC}|}{|AC|}$, 等价于 $\frac{|aX + bY + c|}{\sqrt{a^2 + b^2}}$ 。
- `atan(y / x)` 函数仅用于计算第一、四象限的值, 而 `atan2(y, x)` 则允许计算所有四个象限的正反切, 在使用这个函数时, 需要尽量保证 x 和 y 的类型为整数型, 如果使用浮点数, 实测会慢十倍。
- 在平面上有奇数个点 $A_0, A_1, \dots, A_n$ 以及一个点 X_0 , 构造 X_1 使得 X_0, X_1 关于 A_0 对称、构造 X_2 使得 X_1, X_2 关于 A_1 对称、.....、构造 X_j 使得 X_{j-1}, X_j 关于 $A_{(j-1) \bmod n}$ 对称。那么周期为 $2n$, 即 A_0 与 A_{2n} 共点、 A_1 与 A_{2n+1} 共点 [See](#)。
- 已知 $A(x_A, y_A)$ 和 $X(x_X, y_X)$ 两点及这两点的坐标, 构造 Y 使得 X, Y 关于 A 对称, 那么 Y 的坐标为 $(2 \cdot x_A - x_X, 2 \cdot y_A - y_X)$ 。

- **海伦公式**：已知三角形三边长 a, b 和 c ，定义 $p = \frac{a+b+c}{2}$ ，则 $S_{\triangle} = \sqrt{p(p-a)(p-b)(p-c)}$ ，在使用时需要注意越界问题，本质是铅锤定理，一般多使用叉乘计算三角形面积而不使用该公式。
- 棱台体积 $V = \frac{1}{3}(S_1 + S_2 + \sqrt{S_1 S_2}) \cdot h$ ，其中 S_1, S_2 为上下底面积。
- 正棱台侧面积 $\frac{1}{2}(C_1 + C_2) \cdot L$ ，其中 C_1, C_2 为上下底周长， L 为斜高（上下底对应的平行边的距离）。
- 球面积 $4\pi r^2$ ，体积 $\frac{4}{3}\pi r^3$ 。
- 正三角形面积 $\frac{\sqrt{3}a^2}{4}$ ，正四面体面积 $\frac{\sqrt{2}a^3}{12}$ 。
- 设扇形对应的圆心角弧度为 θ ，则面积为 $S = \frac{\theta}{2} \cdot R^2$ 。

6.14.2 立体几何结论归档

- 已知向量 $\vec{r} = \{x, y, z\}$ ，则该向量的三个方向余弦为 $\cos \alpha = \frac{x}{|\vec{r}|} = \frac{x}{\sqrt{x^2 + y^2 + z^2}}$ ； $\cos \beta = \frac{y}{|\vec{r}|}$ ； $\cos \gamma = \frac{z}{|\vec{r}|}$ 。其中 $\alpha, \beta, \gamma \in [0, \pi]$ ， $\cos^2 \alpha + \cos^2 \beta + \cos^2 \gamma = 1$ 。

6.15 常用例题

6.15.1 将平面某点旋转任意角度

题意：给定平面上一点 (a, b) ，输出将其逆时针旋转 d 度之后的坐标。

```
1 signed main() {
2     int a, b, d;
3     cin >> a >> b >> d;
4
5     ld l = hypot(a, b); // 库函数，求直角三角形的斜边
6     ld alpha = atan2(b, a) + toArc(d);
7
8     cout << l * cos(alpha) << " " << l * sin(alpha) << endl;
9 }
```

6.15.2 平面最近点对 (set解)

借助 `set`，在严格 $\mathcal{O}(N \log N)$ 复杂度内求解，比常见的分治法稍快。

```
1 template<class T> T sqr(T x) {
2     return x * x;
3 }
4
5 using V = Point<int>;
6 signed main() {
7     int n;
8     cin >> n;
9
10    vector<V> in(n);
11    for (auto &it : in) {
12        cin >> it;
13    }
14
15    int dis = disEx(in[0], in[1]); // 设定阈值
16    sort(in.begin(), in.end());
17 }
```

```

18     set<V> S;
19     for (int i = 0, h = 0; i < n; i++) {
20         V now = {in[i].y, in[i].x};
21         while (dis && dis <= sqr(in[i].x - in[h].x)) { // 删除超过阈值的点
22             S.erase({in[h].y, in[h].x});
23             h++;
24         }
25         auto it = S.lower_bound(now);
26         for (auto k = it; k != S.end() && sqr(k->x - now.x) < dis; k++) {
27             dis = min(dis, disEx(*k, now));
28         }
29         if (it != S.begin()) {
30             for (auto k = prev(it); sqr(k->x - now.x) < dis; k--) {
31                 dis = min(dis, disEx(*k, now));
32                 if (k == S.begin()) break;
33             }
34         }
35         S.insert(now);
36     }
37     cout << sqrt(dis) << endl;
38 }

```

6.15.3 平面若干点能构成的最大四边形的面积（简单版，暴力枚举）

题意：平面上存在若干个点，保证没有两点重合、没有三点共线，你需要从中选出四个点，使得它们构成的四边形面积是最大的，注意这里能组成的四边形可以不是凸四边形。

暴力枚举其中一条对角线后枚举剩余两个点， $\mathcal{O}(N^3)$ 。

```

1  signed main() {
2      int n;
3      cin >> n;
4      vector<Pi> in(n);
5      for (auto &it : in) {
6          cin >> it;
7      }
8      ld ans = 0;
9      for (int i = 0; i < n; i++) {
10         for (int j = i + 1; j < n; j++) { // 枚举对角线
11             ld l = 0, r = 0;
12             for (int k = 0; k < n; k++) { // 枚举第三点
13                 if (k == i || k == j) continue;
14                 if (pointOnLineLeft(in[k], {in[i], in[j]})) {
15                     l = max(l, triangleS(in[k], in[j], in[i]));
16                 } else {
17                     r = max(r, triangleS(in[k], in[j], in[i]));
18                 }
19             }
20             if (l * r != 0) { // 确保构成的是四边形
21                 ans = max(ans, l + r);
22             }
23         }
24     }
25     cout << ans << endl;
26 }

```

6.15.4 平面若干点能构成的最大四边形的面积（困难版，分类讨论+旋转卡壳）

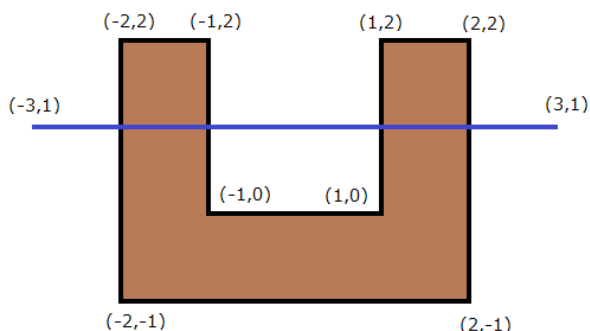
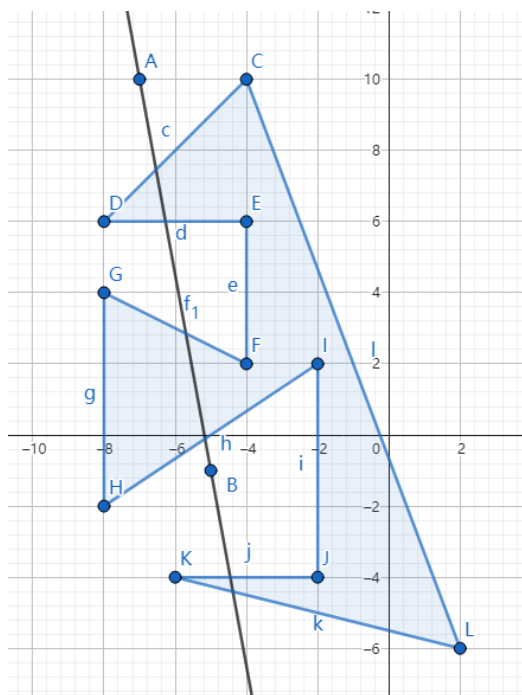
题意：平面上存在若干个点，可能存在多点重合、共线的情况，你需要从中选出四个点，使得它们构成的四边形面积是最大的，注意这里能组成的四边形可以不是凸四边形、可以是退化的四边形。

当凸包大小 ≤ 2 时，说明是退化的四边形，答案直接为 0；大小恰好为 3 时，说明是凹四边形，我们枚举不在凸包上的那一点，将两个三角形面积相减即可得到答案；大小恰好为 4 时，说明是凸四边形，使用旋转卡壳求解。

```
1  using V = Point<int>;
2  signed main() {
3      int Task = 1;
4      for (cin >> Task; Task; Task--) {
5          int n;
6          cin >> n;
7
8          vector<V> in_(n);
9          for (auto &it : in_) {
10             cin >> it;
11         }
12         auto in = staticConvexHull(in_, 0);
13         n = in.size();
14
15         int ans = 0;
16         if (n > 3) {
17             ans = rotatingCalipers(in);
18         } else if (n == 3) {
19             int area = triangleAreaEx(in[0], in[1], in[2]);
20             for (auto it : in_) {
21                 if (it == in[0] || it == in[1] || it == in[2]) continue;
22                 int Min = min({triangleAreaEx(it, in[0], in[1]), triangleAreaEx(it,
in[0], in[2]), triangleAreaEx(it, in[1], in[2])});
23                 ans = max(ans, area - Min);
24             }
25         }
26
27         cout << ans / 2;
28         if (ans % 2) {
29             cout << ".5";
30         }
31         cout << endl;
32     }
33 }
```

6.15.5 线段将多边形切割为几个部分

题意：给定平面上一线段与一个任意多边形，求解线段将多边形切割为几个部分；保证线段的端点不在多边形内、多边形边上，多边形顶点不位于线段上，多边形的边不与线段重叠；多边形端点按逆时针顺序给出。下方的几个样例均合法，答案均为 3。



当线段切割多边形时，本质是与多边形的边交于两个点、或者说是与多边形的两条边相交，设交点数目为 x ，那么答案即为 $\frac{x}{2} + 1$ 。于是，我们只需要计算交点数量即可，先判断某一条边是否与线段相交，再判断边的两个端点是否位于线段两侧。

```
1 signed main() {
2     Pi s, e;
3     cin >> s >> e; // 读入线段
4
5     int n;
6     cin >> n;
7     vector<Pi> in(n);
8     for (auto &it : in) {
9         cin >> it; // 读入多边形端点
10    }
11
12    int cnt = 0;
13    for (int i = 0; i < n; i++) {
14        Pi x = in[i], y = in[(i + 1) % n];
15        cnt += (pointNotOnLineSide(x, y, {s, e}) && segmentIntersection(Line{x, y},
16    {s, e}));
17    }
18    cout << cnt / 2 + 1 << endl;
19 }
```

6.15.6 平面若干点能否构成凸包（暴力枚举）

题意：给定平面上若干个点，判断其是否构成凸包 [See](#)。

可以直接使用凸包模板，但是代码较长；在这里我们使用暴力枚举试点，也能以 $\mathcal{O}(N)$ 的复杂度通过。当两个向量的叉乘 ≤ 0 时说明其夹角大于等于 180° ，使用这一点即可判定。

```
1 signed main() {
2     int n;
3     cin >> n;
4
5     vector<Point<ld>> in(n);
6     for (auto &it : in) {
7         cin >> it;
8     }
9 }
```

```

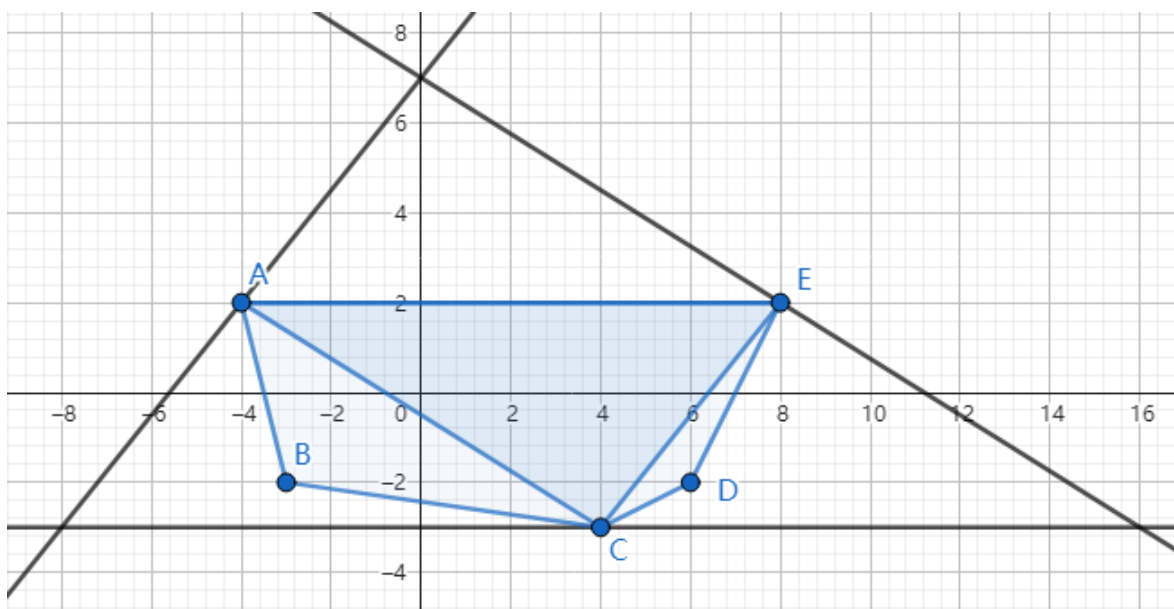
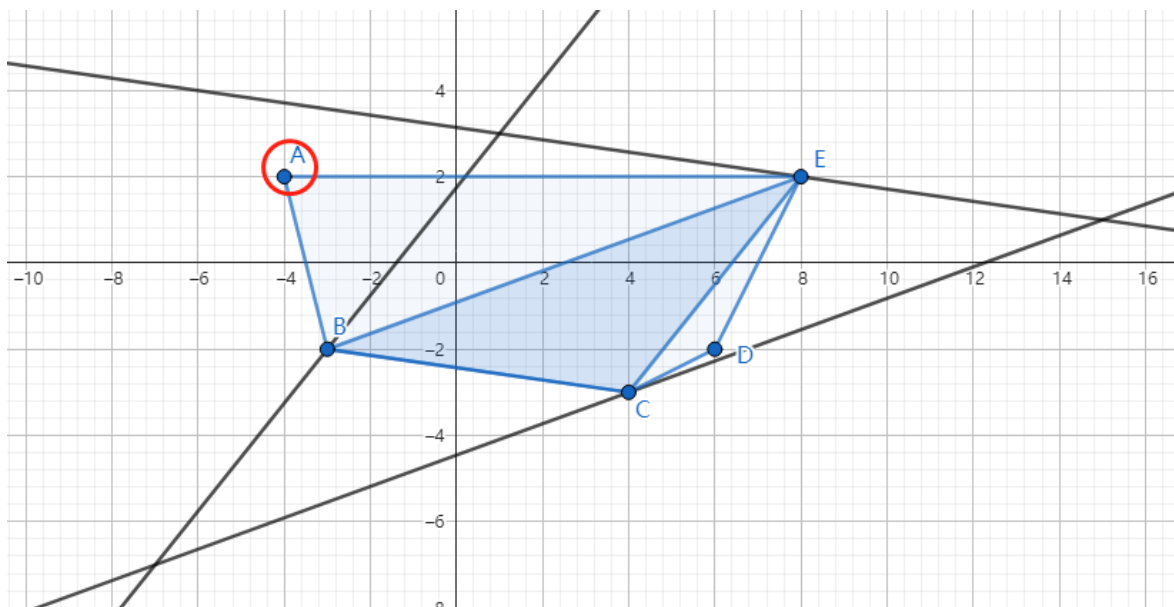
8     }
9
10    for (int i = 0; i < n; i++) {
11        auto A = in[(i - 1 + n) % n];
12        auto B = in[i];
13        auto C = in[(i + 1) % n];
14        if (cross(A - B, C - B) > 0) {
15            cout << "No\n";
16            return 0;
17        }
18    }
19    cout << "Yes\n";
20 }

```

6.15.7 凸包上的点能构成的最大三角形（暴力枚举）

可以直接使用凸包模板，但是代码较长；在这里我们使用暴力枚举试点，也能以 $\mathcal{O}(N)$ 的复杂度通过。

另外补充一点性质：所求三角形的反互补三角形一定包含了凸包上的所有点（可以在边界）。通俗的说，构成的三角形是这个反互补三角形的中点三角形。如下图所示，点 A 不在 $\triangle BCE$ 的反互补三角形内部，故 $\triangle BCE$ 不是最大三角形； $\triangle ACE$ 才是。



```

1 signed main() {
2     int n;
3     cin >> n;
4
5     vector<Point<int>> in(n);
6     for (auto &it : in) {
7         cin >> it;
8     }
9
10    #define S(x, y, z) triangleAreaEx(in[x], in[y], in[z])
11
12    int i = 0, j = 1, k = 2;
13    while (true) {
14        int val = S(i, j, k);
15        if (S((i + 1) % n, j, k) > val) {
16            i = (i + 1) % n;
17        } else if (S((i - 1 + n) % n, j, k) > val) {
18            i = (i - 1 + n) % n;
19        } else if (S(i, (j + 1) % n, k) > val) {
20            j = (j + 1) % n;
21        } else if (S(i, (j - 1 + n) % n, k) > val) {
22            j = (j - 1 + n) % n;
23        } else if (S(i, j, (k + 1) % n) > val) {
24            k = (k + 1) % n;
25        } else if (S(i, j, (k - 1 + n) % n) > val) {
26            k = (k - 1 + n) % n;
27        } else {
28            break;
29        }
30    }
31    cout << i + 1 << " " << j + 1 << " " << k + 1 << endl;
32 }

```

6.15.8 凸包上的点能构成的最大四边形的面积（旋转卡壳）

由于是凸包上的点，所以保证了四边形一定是凸四边形，时间复杂度 $\mathcal{O}(N^2)$ 。

```

1 template<class T> T rotatingCalipers(vector<Point<T>> &p) {
2     #define S(x, y, z) triangleAreaEx(p[x], p[y], p[z])
3     int n = p.size();
4     T ans = 0;
5     auto nxt = [&](int i) -> int {
6         return i == n - 1 ? 0 : i + 1;
7     };
8     for (int i = 0; i < n; i++) {
9         int p1 = nxt(i), p2 = nxt(nxt(nxt(i)));
10        for (int j = nxt(nxt(i)); nxt(j) != i; j = nxt(j)) {
11            while (nxt(p1) != j && S(i, j, nxt(p1)) > S(i, j, p1)) {
12                p1 = nxt(p1);
13            }
14            if (p2 == j) {
15                p2 = nxt(p2);
16            }
17            while (nxt(p2) != i && S(i, j, nxt(p2)) > S(i, j, p2)) {
18                p2 = nxt(p2);
19            }
20            ans = max(ans, S(i, j, p1) + S(i, j, p2));
21        }
22    }
23    return ans;
24    #undef S

```

6.15.9 判断一个凸包是否完全在另一个凸包内

题意：给定一个凸多边形 A 和一个凸多边形 B ，询问 B 是否被 A 包含，分别判断严格/不严格包含。[例题](#)。

考虑严格包含，使用 A 点集计算出凸包 T_1 ，使用 A, B 两个点集计算出不严格凸包 T_2 ，如果包含，那么 T_1 应该与 T_2 完全相等；考虑不严格包含，在计算凸包 T_2 时严格即可。最终以 $O(N)$ 复杂度求解，且代码不算很长。

/END/

7 多项式

7.1 多项式封装

```
1  template<int P = 998244353> struct Poly : public vector<MInt<P>> {
2      using Value = MInt<P>;
3
4      Poly() : vector<Value>() {}
5      explicit constexpr Poly(int n) : vector<Value>(n) {}
6
7      explicit constexpr Poly(const vector<Value> &a) : vector<Value>(a) {}
8      constexpr Poly(const initializer_list<Value> &a) : vector<Value>(a) {}
9
10     template<class InputIt, class = _RequireInputIter<InputIt>>
11     explicit constexpr Poly(InputIt first, InputIt last) : vector<Value>(first,
12 last) {}
13
14     template<class F> explicit constexpr Poly(int n, F f) : vector<Value>(n) {
15         for (int i = 0; i < n; i++) {
16             (*this)[i] = f(i);
17         }
18     }
19
20     constexpr Poly shift(int k) const {
21         if (k >= 0) {
22             auto b = *this;
23             b.insert(b.begin(), k, 0);
24             return b;
25         } else if (this->size() <= -k) {
26             return Poly();
27         } else {
28             return Poly(this->begin() + (-k), this->end());
29         }
30     }
31     constexpr Poly trunc(int k) const {
32         Poly f = *this;
33         f.resize(k);
34         return f;
35     }
36     constexpr friend Poly operator+(const Poly &a, const Poly &b) {
37         Poly res(max(a.size(), b.size()));
38         for (int i = 0; i < a.size(); i++) {
39             res[i] += a[i];
40         }
41         for (int i = 0; i < b.size(); i++) {
42             res[i] += b[i];
43         }
44         return res;
45     }
46     constexpr friend Poly operator-(const Poly &a, const Poly &b) {
47         Poly res(max(a.size(), b.size()));
48         for (int i = 0; i < a.size(); i++) {
49             res[i] += a[i];
50         }
51         for (int i = 0; i < b.size(); i++) {
52             res[i] -= b[i];
53         }
54         return res;
55     }
56     constexpr friend Poly operator-(const Poly &a) {
57         vector<Value> res(a.size());
58         for (int i = 0; i < int(res.size()); i++) {
```

```

58         res[i] = -a[i];
59     }
60     return Poly(res);
61 }
62 constexpr friend Poly operator*(Poly a, Poly b) {
63     if (a.size() == 0 || b.size() == 0) {
64         return Poly();
65     }
66     if (a.size() < b.size()) {
67         swap(a, b);
68     }
69     int n = 1, tot = a.size() + b.size() - 1;
70     while (n < tot) {
71         n *= 2;
72     }
73     if (((P - 1) & (n - 1)) != 0 || b.size() < 128) {
74         Poly c(a.size() + b.size() - 1);
75         for (int i = 0; i < a.size(); i++) {
76             for (int j = 0; j < b.size(); j++) {
77                 c[i + j] += a[i] * b[j];
78             }
79         }
80         return c;
81     }
82     a.resize(n);
83     b.resize(n);
84     dft(a);
85     dft(b);
86     for (int i = 0; i < n; ++i) {
87         a[i] *= b[i];
88     }
89     idft(a);
90     a.resize(tot);
91     return a;
92 }
93 constexpr friend Poly operator*(Value a, Poly b) {
94     for (int i = 0; i < int(b.size()); i++) {
95         b[i] *= a;
96     }
97     return b;
98 }
99 constexpr friend Poly operator*(Poly a, Value b) {
100     for (int i = 0; i < int(a.size()); i++) {
101         a[i] *= b;
102     }
103     return a;
104 }
105 constexpr friend Poly operator/(Poly a, Value b) {
106     for (int i = 0; i < int(a.size()); i++) {
107         a[i] /= b;
108     }
109     return a;
110 }
111 constexpr Poly &operator+=(Poly b) {
112     return (*this) = (*this) + b;
113 }
114 constexpr Poly &operator-=(Poly b) {
115     return (*this) = (*this) - b;
116 }
117 constexpr Poly &operator*=(Poly b) {
118     return (*this) = (*this) * b;
119 }
120 constexpr Poly &operator*=(Value b) {
121     return (*this) = (*this) * b;

```

```

122     }
123     constexpr Poly &operator/=(Value b) {
124         return (*this) = (*this) / b;
125     }
126     constexpr Poly deriv() const {
127         if (this->empty()) {
128             return Poly();
129         }
130         Poly res(this->size() - 1);
131         for (int i = 0; i < this->size() - 1; ++i) {
132             res[i] = (i + 1) * (*this)[i + 1];
133         }
134         return res;
135     }
136     constexpr Poly integr() const {
137         Poly res(this->size() + 1);
138         for (int i = 0; i < this->size(); ++i) {
139             res[i + 1] = (*this)[i] / (i + 1);
140         }
141         return res;
142     }
143     constexpr Poly inv(int m) const {
144         Poly x((*this)[0].inv());
145         int k = 1;
146         while (k < m) {
147             k *= 2;
148             x = (x * (Poly{2} - trunc(k) * x)).trunc(k);
149         }
150         return x.trunc(m);
151     }
152     constexpr Poly log(int m) const {
153         return (deriv() * inv(m)).integr().trunc(m);
154     }
155     constexpr Poly exp(int m) const {
156         Poly x{1};
157         int k = 1;
158         while (k < m) {
159             k *= 2;
160             x = (x * (Poly{1} - x.log(k) + trunc(k))).trunc(k);
161         }
162         return x.trunc(m);
163     }
164     constexpr Poly pow(int k, int m) const {
165         int i = 0;
166         while (i < this->size() && (*this)[i] == 0) {
167             i++;
168         }
169         if (i == this->size() || 1LL * i * k >= m) {
170             return Poly(m);
171         }
172         Value v = (*this)[i];
173         auto f = shift(-i) * v.inv();
174         return (f.log(m - i * k) * k).exp(m - i * k).shift(i * k) * power(v, k);
175     }
176     constexpr Poly sqrt(int m) const {
177         Poly x{1};
178         int k = 1;
179         while (k < m) {
180             k *= 2;
181             x = (x + (trunc(k) * x.inv(k)).trunc(k)) * CInv<2, P>;
182         }
183         return x.trunc(m);
184     }
185     constexpr Poly mulT(Poly b) const {

```

```

186         if (b.size() == 0) {
187             return Poly();
188         }
189         int n = b.size();
190         reverse(b.begin(), b.end());
191         return ((*this) * b).shift(-(n - 1));
192     }
193     constexpr vector<Value> eval(vector<Value> x) const {
194         if (this->size() == 0) {
195             return vector<Value>(x.size(), 0);
196         }
197         const int n = max(x.size(), this->size());
198         vector<Poly> q(4 * n);
199         vector<Value> ans(x.size());
200         x.resize(n);
201         function<void(int, int, int)> build = [&](int p, int l, int r) {
202             if (r - l == 1) {
203                 q[p] = Poly{1, -x[l]};
204             } else {
205                 int m = (l + r) / 2;
206                 build(2 * p, l, m);
207                 build(2 * p + 1, m, r);
208                 q[p] = q[2 * p] * q[2 * p + 1];
209             }
210         };
211         build(1, 0, n);
212         function<void(int, int, int, const Poly &)> work = [&](int p, int l, int r,
213             const Poly
214             &num) {
215             if (r - l == 1) {
216                 if (l < int(ans.size())) {
217                     ans[l] = num[0];
218                 }
219             } else {
220                 int m = (l + r) / 2;
221                 work(2 * p, l, m, num.mulT(q[2 * p + 1]).resize(m - 1));
222                 work(2 * p + 1, m, r, num.mulT(q[2 * p]).resize(r - m));
223             }
224         };
225         work(1, 0, n, mulT(q[1].inv(n)));
226         return ans;
227     };

```

7.2 离散傅里叶变换 dft 与其逆变换 idft

```

1  vector<int> rev;
2  template<int P> vector<MInt<P>> roots{0, 1};
3
4  template<int P> constexpr MInt<P> findPrimitiveRoot() {
5      MInt<P> i = 2;
6      int k = __builtin_ctz(P - 1);
7      while (true) {
8          if (power(i, (P - 1) / 2) != 1) {
9              break;
10         }
11         i += 1;
12     }
13     return power(i, (P - 1) >> k);
14 }
15
16 template<int P> constexpr MInt<P> primitiveRoot = findPrimitiveRoot<P>();

```

```

17 template<> constexpr MInt<998244353> primitiveRoot<998244353>{31};
18
19 template<int P> constexpr void dft(vector<MInt<P>> &a) { // 离散傅里叶变换
20     int n = a.size();
21
22     if (int(rev.size()) != n) {
23         int k = __builtin_ctz(n) - 1;
24         rev.resize(n);
25         for (int i = 0; i < n; i++) {
26             rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
27         }
28     }
29
30     for (int i = 0; i < n; i++) {
31         if (rev[i] < i) {
32             swap(a[i], a[rev[i]]);
33         }
34     }
35     if (roots<P>.size() < n) {
36         int k = __builtin_ctz(roots<P>.size());
37         roots<P>.resize(n);
38         while ((1 << k) < n) {
39             auto e = power(primitiveRoot<P>, 1 << (__builtin_ctz(P - 1) - k - 1));
40             for (int i = 1 << (k - 1); i < (1 << k); i++) {
41                 roots<P>[2 * i] = roots<P>[i];
42                 roots<P>[2 * i + 1] = roots<P>[i] * e;
43             }
44             k++;
45         }
46     }
47     for (int k = 1; k < n; k *= 2) {
48         for (int i = 0; i < n; i += 2 * k) {
49             for (int j = 0; j < k; j++) {
50                 MInt<P> u = a[i + j];
51                 MInt<P> v = a[i + j + k] * roots<P>[k + j];
52                 a[i + j] = u + v;
53                 a[i + j + k] = u - v;
54             }
55         }
56     }
57 }
58 template<int P> constexpr void idft(vector<MInt<P>> &a) { // 逆变换
59     int n = a.size();
60     reverse(a.begin() + 1, a.end());
61     dft(a);
62     MInt<P> inv = (1 - P) / n;
63     for (int i = 0; i < n; i++) {
64         a[i] *= inv;
65     }
66 }

```

7.3 Berlekamp-Massey 算法 (杜教筛)

求解数列的最短线性递推式，最坏复杂度为 $\mathcal{O}(NM)$ ，其中 N 为数列长度， M 为它的最短递推式的阶数。

```

1 template<int P = 998244353> Poly<P> berlekampMassey(const Poly<P> &s) {
2     Poly<P> c;
3     Poly<P> oldC;
4     int f = -1;
5     for (int i = 0; i < s.size(); i++) {
6         auto delta = s[i];

```

```

7     for (int j = 1; j <= c.size(); j++) {
8         delta -= c[j - 1] * s[i - j];
9     }
10    if (delta == 0) {
11        continue;
12    }
13    if (f == -1) {
14        c.resize(i + 1);
15        f = i;
16    } else {
17        auto d = oldC;
18        d *= -1;
19        d.insert(d.begin(), 1);
20        MInt<P> df1 = 0;
21        for (int j = 1; j <= d.size(); j++) {
22            df1 += d[j - 1] * s[f + 1 - j];
23        }
24        assert(df1 != 0);
25        auto coef = delta / df1;
26        d *= coef;
27        Poly<P> zeros(i - f - 1);
28        zeros.insert(zeros.end(), d.begin(), d.end());
29        d = zeros;
30        auto temp = c;
31        c += d;
32        if (i - temp.size() > f - oldC.size()) {
33            oldC = temp;
34            f = i;
35        }
36    }
37 }
38 c *= -1;
39 c.insert(c.begin(), 1);
40 return c;
41 }

```

7.4 Linear-Recurrence 算法

```

1  template<int P = 998244353> MInt<P> linearRecurrence(Poly<P> p, Poly<P> q, i64 n) {
2      int m = q.size() - 1;
3      while (n > 0) {
4          auto newq = q;
5          for (int i = 1; i <= m; i += 2) {
6              newq[i] *= -1;
7          }
8          auto newp = p * newq;
9          newq = q * newq;
10         for (int i = 0; i < m; i++) {
11             p[i] = newp[i * 2 + n % 2];
12         }
13         for (int i = 0; i <= m; i++) {
14             q[i] = newq[i * 2];
15         }
16         n /= 2;
17     }
18     return p[0] / q[0];
19 }

```

7.5 快速傅里叶变换 FFT

$\mathcal{O}(N \log N)$ 。

```
1 struct Polynomial {
2     constexpr static double PI = acos(-1);
3     struct Complex {
4         double x, y;
5         Complex(double _x = 0.0, double _y = 0.0) {
6             x = _x;
7             y = _y;
8         }
9         Complex operator-(const Complex &rhs) const {
10             return Complex(x - rhs.x, y - rhs.y);
11         }
12         Complex operator+(const Complex &rhs) const {
13             return Complex(x + rhs.x, y + rhs.y);
14         }
15         Complex operator*(const Complex &rhs) const {
16             return Complex(x * rhs.x - y * rhs.y, x * rhs.y + y * rhs.x);
17         }
18     };
19     vector<Complex> c;
20     Polynomial(vector<int> &a) {
21         int n = a.size();
22         c.resize(n);
23         for (int i = 0; i < n; i++) {
24             c[i] = Complex(a[i], 0);
25         }
26         fft(c, n, 1);
27     }
28     void change(vector<Complex> &a, int n) {
29         for (int i = 1, j = n / 2; i < n - 1; i++) {
30             if (i < j) swap(a[i], a[j]);
31             int k = n / 2;
32             while (j >= k) {
33                 j -= k;
34                 k /= 2;
35             }
36             if (j < k) j += k;
37         }
38     }
39     void fft(vector<Complex> &a, int n, int opt) {
40         change(a, n);
41         for (int h = 2; h <= n; h *= 2) {
42             Complex wn(cos(2 * PI / h), sin(opt * 2 * PI / h));
43             for (int j = 0; j < n; j += h) {
44                 Complex w(1, 0);
45                 for (int k = j; k < j + h / 2; k++) {
46                     Complex u = a[k];
47                     Complex t = w * a[k + h / 2];
48                     a[k] = u + t;
49                     a[k + h / 2] = u - t;
50                     w = w * wn;
51                 }
52             }
53         }
54         if (opt == -1) {
55             for (int i = 0; i < n; i++) {
56                 a[i].x /= n;
57             }
58         }
59     }
60 }
```

```
60 };
```

7.6 快速数论变换 NTT

$\mathcal{O}(N \log N)$ 。

```
1 struct Polynomial {
2     vector<Z> z;
3     vector<int> r;
4     Polynomial(vector<int> &a) {
5         int n = a.size();
6         z.resize(n);
7         r.resize(n);
8         for (int i = 0; i < n; i++) {
9             z[i] = a[i];
10            r[i] = (i & 1) * (n / 2) + r[i / 2] / 2;
11        }
12        ntt(z, n, 1);
13    }
14    LL power(LL a, int b) {
15        LL res = 1;
16        for (; b; b /= 2, a = a * a % mod) {
17            if (b % 2) {
18                res = res * a % mod;
19            }
20        }
21        return res;
22    }
23    void ntt(vector<Z> &a, int n, int opt) {
24        for (int i = 0; i < n; i++) {
25            if (r[i] < i) {
26                swap(a[i], a[r[i]]);
27            }
28        }
29        for (int k = 2; k <= n; k *= 2) {
30            Z gn = power(3, (mod - 1) / k);
31            for (int i = 0; i < n; i += k) {
32                Z g = 1;
33                for (int j = 0; j < k / 2; j++, g *= gn) {
34                    Z t = a[i + j + k / 2] * g;
35                    a[i + j + k / 2] = a[i + j] - t;
36                    a[i + j] = a[i + j] + t;
37                }
38            }
39        }
40        if (opt == -1) {
41            reverse(a.begin() + 1, a.end());
42            Z inv = power(n, mod - 2);
43            for (int i = 0; i < n; i++) {
44                a[i] *= inv;
45            }
46        }
47    }
48 };
```

7.7 拉格朗日插值

$n + 1$ 个点可以唯一确定一个最高为 n 次的多项式。普通情况: $f(k) = \sum_{i=1}^{n+1} y_i \prod_{i \neq j} \frac{k - x[j]}{x[i] - x[j]}$ 。

```
1 struct Lagrange {
2     int n;
3     vector<Z> x, y, fac, invfac;
4     Lagrange(int n) {
5         this->n = n;
6         x.resize(n + 3);
7         y.resize(n + 3);
8         fac.resize(n + 3);
9         invfac.resize(n + 3);
10        init(n);
11    }
12    void init(int n) {
13        iota(x.begin(), x.end(), 0);
14        for (int i = 1; i <= n + 2; i++) {
15            Z t;
16            y[i] = y[i - 1] + t.power(i, n);
17        }
18        fac[0] = 1;
19        for (int i = 1; i <= n + 2; i++) {
20            fac[i] = fac[i - 1] * i;
21        }
22        invfac[n + 2] = fac[n + 2].inv();
23        for (int i = n + 1; i >= 0; i--) {
24            invfac[i] = invfac[i + 1] * (i + 1);
25        }
26    }
27    Z solve(LL k) {
28        if (k <= n + 2) {
29            return y[k];
30        }
31        vector<Z> sub(n + 3);
32        for (int i = 1; i <= n + 2; i++) {
33            sub[i] = k - x[i];
34        }
35        vector<Z> mul(n + 3);
36        mul[0] = 1;
37        for (int i = 1; i <= n + 2; i++) {
38            mul[i] = mul[i - 1] * sub[i];
39        }
40        Z ans = 0;
41        for (int i = 1; i <= n + 2; i++) {
42            ans = ans + y[i] * mul[n + 2] * sub[i].inv() * pow(-1, n + 2 - i) *
43            invfac[i - 1] *
44                invfac[n + 2 - i];
45        }
46        return ans;
47    };
48 }
```

7.8 常用结论

7.8.1 杂

- 求 $B_i = \sum_{k=i}^n C_k^i A_k$, 即 $B_i = \frac{1}{i!} \sum_{k=i}^n \frac{1}{(k-i)!} \cdot k! A_k$, 反转后卷积。
- NTT中, $\omega_n = \text{qpow}(G, (\text{mod}-1)/n)$ 。
- 遇到 $\sum_{i=0}^n [i \% k = 0] f(i)$ 可以转换为 $\sum_{i=0}^n \frac{1}{k} \sum_{j=0}^{k-1} (\omega_k^i)^j f(i)$ 。(单位根卷积)
- 广义二项式定理 $(1+x)^\alpha = \sum_{i=0}^{\infty} \binom{n}{\alpha} x^i$ 。

7.8.2 普通生成函数 / OGF

- 普通生成函数: $A(x) = a_0 + a_1x + a_2x^2 + \dots = \langle a_0, a_1, a_2, \dots \rangle$;
- $1 + x^k + x^{2k} + \dots = \frac{1}{1 - x^k}$;
- 取对数后 $= -\ln(1 - x^k) = \sum_{i=1}^{\infty} \frac{1}{i} x^{ki}$ 即 $\sum_{i=1}^{\infty} \frac{1}{i} x^i \otimes x^k$ (polymul_special) ;
- $x + \frac{x^2}{2} + \frac{x^3}{3} + \dots = -\ln(1 - x)$;
- $1 + x + x^2 + \dots + x^{m-1} = \frac{1 - x^m}{1 - x}$;
- $1 + 2x + 3x^2 + \dots = \frac{1}{(1 - x)^2}$ (借用导数, $nx^{n-1} = (x^n)'$) ;
- $C_m^0 + C_m^1x + C_m^2x^2 + \dots + C_m^mx^m = (1 + x)^m$ (二项式定理) ;
- $C_m^0 + C_{m+1}^1x + C_{m+2}^2x^2 + \dots = \frac{1}{(1 - x)^{m+1}}$ (归纳法证明) ;
- $\sum_{n=0}^{\infty} F_n x^n = \frac{(F_1 - F_0)x + F_0}{1 - x - x^2}$ (F 为斐波那契数列, 列方程
 $G(x) = xG(x) + x^2G(x) + (F_1 - F_0)x + F_0$) ;
- $\sum_{n=0}^{\infty} H_n x^n = \frac{1 - \sqrt{1 - 4x}}{2x}$ (H 为卡特兰数) ;
- 前缀和 $\sum_{n=0}^{\infty} s_n x^n = \frac{1}{1 - x} f(x)$;
- 五边形数定理: $\prod_{i=1}^{\infty} (1 - x^i) = \sum_{k=0}^{\infty} (-1)^k x^{\frac{1}{2}k(3k+1)}$ 。

7.8.3 指数生成函数 / EGF

- 指数生成函数: $A(x) = a_0 + a_1x + a_2 \frac{x^2}{2!} + a_3 \frac{x^3}{3!} + \dots = \langle a_0, a_1, a_2, a_3, \dots \rangle$;
- 普通生成函数转换为指数生成函数: 系数乘以 $n!$;
- $1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots = \exp x$;

- 长度为 n 的循环置换数为 $P(x) = -\ln(1-x)$, 长度为 n 的置换数为 $\exp P(x) = \frac{1}{1-x}$
(注意是**指数**生成函数)

- n 个点的生成树个数是 $P(x) = \sum_{n=1}^{\infty} n^{n-2} \frac{x^n}{n!}$, n 个点的生成森林个数是 $\exp P(x)$;
- n 个点的无向连通图个数是 $P(x)$, n 个点的无向图个数是 $\exp P(x) = \sum_{n=0}^{\infty} 2^{\frac{1}{2}n(n-1)} \frac{x^n}{n!}$;
- 长度为 $n(n \geq 2)$ 的循环置换数是 $P(x) = -\ln(1-x) - x$, 长度为 n 的错排数是 $\exp P(x)$ 。

/END/

8 数据结构

8.1 并查集（全功能）

```
1 struct DSU {
2     vector<int> fa, p, e, f;
3
4     DSU(int n) {
5         fa.resize(n + 1);
6         iota(fa.begin(), fa.end(), 0);
7         p.resize(n + 1, 1);
8         e.resize(n + 1);
9         f.resize(n + 1);
10    }
11    int get(int x) {
12        while (x != fa[x]) {
13            x = fa[x] = fa[fa[x]];
14        }
15        return x;
16    }
17    bool merge(int x, int y) { // 设x是y的祖先
18        if (x == y) f[get(x)] = 1;
19        x = get(x), y = get(y);
20        e[x]++;
21        if (x == y) return false;
22        if (x < y) swap(x, y); // 将编号小的合并到大的上
23        fa[y] = x;
24        f[x] |= f[y], p[x] += p[y], e[x] += e[y];
25        return true;
26    }
27    bool same(int x, int y) {
28        return get(x) == get(y);
29    }
30    bool F(int x) { // 判断连通块内是否存在自环
31        return f[get(x)];
32    }
33    int size(int x) { // 输出连通块中点的数量
34        return p[get(x)];
35    }
36    int E(int x) { // 输出连通块中边的数量
37        return e[get(x)];
38    }
39 };
40
```

8.2 树状数组

8.2.1 常规（单点增减+区间和查询）

```
1 template<class T> struct BIT {
2     int n;
3     vector<T> w;
4     BIT(int n, auto &in) : n(n), w(n + 1) { // 预处理填值
5         for (int i = 1; i <= n; i++) {
6             add(i, in[i]);
7         }
8     }
9     void add(int x, T v) {
10        for (; x <= n; x += x & -x) {
11            w[x] += v;
12        }
13    }
14 }
```

```

12     }
13 }
14 T ask(int x) { // 前缀和查询
15     T ans = 0;
16     for (; x; x -= x & -x) {
17         ans += w[x];
18     }
19     return ans;
20 }
21 T ask(int l, int r) { // 差分实现区间和查询
22     return ask(r) - ask(l - 1);
23 }
24 };

```

8.2.2 逆序对扩展

```

1 struct BIT {
2     int n;
3     vector<int> w, chk; // chk 为传入的待处理数组
4     BIT(int n, auto &in) : n(n), w(n + 1), chk(in) {}
5     /* 需要全部常规封装 */
6     int get() {
7         vector<array<int, 2>> alls;
8         for (int i = 1; i <= n; i++) {
9             alls.push_back({chk[i], i});
10        }
11        sort(alls.begin(), alls.end());
12        int ans = 0;
13        for (auto [val, idx] : alls) {
14            ans += ask(idx + 1, n);
15            add(idx, 1);
16        }
17        return ans;
18    }
19 };

```

8.2.3 前驱后继扩展 (常规+第 k 小值查询+元素排名查询+元素前驱后继查询)

注意，被查询的值都应该小于等于 N ，否则会越界；如果离散化不可使用，则需要使用平衡树替代。

```

1 struct BIT {
2     int n;
3     vector<int> w;
4     BIT(int n) : n(n), w(n + 1) {}
5     void add(int x, int v) {
6         for (; x <= n; x += x & -x) {
7             w[x] += v;
8         }
9     }
10    int kth(int x) { // 查找第 k 小的值
11        int ans = 0;
12        for (int i = __lg(n); i >= 0; i--) {
13            int val = ans + (1 << i);
14            if (val < n && w[val] < x) {
15                x -= w[val];
16                ans = val;
17            }
18        }
19        return ans + 1;

```

```

20     }
21     int get(int x) { // 查找 x 的排名
22         int ans = 1;
23         for (x--; x; x -= x & -x) {
24             ans += w[x];
25         }
26         return ans;
27     }
28     int pre(int x) { return kth(get(x) - 1); } // 查找 x 的前驱
29     int suf(int x) { return kth(get(x) + 1); } // 查找 x 的后继
30 };
31 const int N = 1000000; // 可以用于在线处理平衡二叉树的全部要求
32 signed main() {
33     BIT bit(N + 1); // 在线处理不能够离散化，一定要开到比最大值更大
34     int n;
35     cin >> n;
36     for (int i = 1; i <= n; i++) {
37         int op, x;
38         cin >> op >> x;
39         if (op == 1) bit.add(x, 1); // 插入 x
40         else if (op == 2) bit.add(x, -1); // 删除任意一个 x
41         else if (op == 3) cout << bit.get(x) << "\n"; // 查询 x 的排名
42         else if (op == 4) cout << bit.kth(x) << "\n"; // 查询排名为 x 的数
43         else if (op == 5) cout << bit.pre(x) << "\n"; // 求小于 x 的最大值 (前驱)
44         else if (op == 6) cout << bit.suf(x) << "\n"; // 求大于 x 的最小值 (后继)
45     }
46 }

```

8.2.4 最值查询扩展 (常规+区间最值查询+单点修改)

以 $\mathcal{O}(\log \log N)$ 的复杂度运行，但是即便如此依然略优于线段树（后者常数较大）。

```

1  template<class T> struct BIT {
2      int n;
3      vector<T> base, w;
4      inline int LB(int x) { return x & -x; }
5      BIT(auto &in) : n(in.size() - 1), w(in), base(in) {
6          for (int i = 1; i <= n; i++) {
7              update(i, in[i]);
8          }
9      }
10     /* 可以增加并使用常规封装中的几个函数 */
11     void update(int x, int v) { // 单点修改
12         base[x] = v;
13         for (; x <= n; x += LB(x)) {
14             w[x] = base[x];
15             for (int i = 1; i < LB(x); i *= 2) {
16                 w[x] = max(w[x], w[x - i]);
17             }
18         }
19     }
20     T getMax(int l, int r) { // 最值查询
21         T ans = T();
22         while (r >= 1) {
23             ans = max(base[r], ans);
24             for (r--; r - LB(r) >= 1; r -= LB(r)) {
25                 ans = max(w[r], ans);
26             }
27         }
28         return ans;
29     }
30 };

```

8.3 二维树状数组

封装一：该版本不能同时进行区间修改+区间查询。无离散化版本的空间占用为 $\mathcal{O}(NM)$ 、建树复杂度为 $\mathcal{O}(NM)$ 、单次查询复杂度为 $\mathcal{O}(\log N \cdot \log M)$ 。

```
1 struct BIT_2D {
2     int n, m;
3     vector<vector<int>>> w;
4
5     BIT_2D(int n, int m) : n(n), m(m) {
6         w.resize(n + 1, vector<int>(m + 1));
7     }
8     void add(int x, int y, int k) {
9         for (int i = x; i <= n; i += i & -i) {
10             for (int j = y; j <= m; j += j & -j) {
11                 w[i][j] += k;
12             }
13         }
14     }
15     void add(int x, int y, int X, int Y, int k) { // 区块修改: 二维差分
16         X++, Y++;
17         add(x, y, k), add(X, y, -k);
18         add(X, Y, k), add(x, Y, -k);
19     }
20     int ask(int x, int y) { // 单点查询
21         int ans = 0;
22         for (int i = x; i; i -= i & -i) {
23             for (int j = y; j; j -= j & -j) {
24                 ans += w[i][j];
25             }
26         }
27         return ans;
28     }
29     int ask(int x, int y, int X, int Y) { // 区块查询: 二维前缀和
30         x--, y--;
31         return ask(X, Y) - ask(x, Y) - ask(X, y) + ask(x, y);
32     }
33 };
```

封装二：该版本支持全部操作。但是时空复杂度均比上一个版本多 4 倍。

```
1 struct BIT_2D {
2     int n, m;
3     vector<vector<int>>> b1, b2, b3, b4;
4
5     BIT_2D(int n, int m) : n(n), m(m) {
6         b1.resize(n + 1, vector<int>(m + 1));
7         b2.resize(n + 1, vector<int>(m + 1));
8         b3.resize(n + 1, vector<int>(m + 1));
9         b4.resize(n + 1, vector<int>(m + 1));
10    }
11    void add(auto &w, int x, int y, int k) { // 单点修改
12        for (int i = x; i <= n; i += i & -i) {
13            for (int j = y; j <= m; j += j & -j) {
14                w[i][j] += k;
15            }
16        }
17    }
18    void add(int x, int y, int k) { // 多了一步计算
19        add(b1, x, y, k);
20        add(b2, x, y, k * (x - 1));
```

```

21     add(b3, x, y, k * (y - 1));
22     add(b4, x, y, k * (x - 1) * (y - 1));
23 }
24 void add(int x, int y, int X, int Y, int k) { // 区块修改: 二维差分
25     X++, Y++;
26     add(x, y, k), add(X, y, -k);
27     add(X, Y, k), add(x, Y, -k);
28 }
29 int ask(auto &w, int x, int y) { // 单点查询
30     int ans = 0;
31     for (int i = x; i; i -= i & -i) {
32         for (int j = y; j; j -= j & -j) {
33             ans += w[i][j];
34         }
35     }
36     return ans;
37 }
38 int ask(int x, int y) { // 多了一步计算
39     int ans = 0;
40     ans += x * y * ask(b1, x, y);
41     ans -= y * ask(b2, x, y);
42     ans -= x * ask(b3, x, y);
43     ans += ask(b4, x, y);
44     return ans;
45 }
46 int ask(int x, int y, int X, int Y) { // 区块查询: 二维前缀和
47     x--, y--;
48     return ask(X, Y) - ask(x, Y) - ask(X, y) + ask(x, y);
49 }
50 };

```

8.4 线段树

8.4.1 完整封装

```

1  template<class Info, class Tag> struct LazySegmentTree {
2      int n;
3      vector<Info> info;
4      vector<Tag> tag;
5      LazySegmentTree(vector<Info> init) {
6          n = init.size();
7          info.assign(4 * n, Info());
8          tag.assign(4 * n, Tag());
9          function<void(int, int, int)> build = [&](int p, int l, int r) {
10             if (l == r) {
11                 info[p] = init[l - 1];
12                 return;
13             }
14             int m = (l + r) / 2;
15             build(2 * p, l, m);
16             build(2 * p + 1, m + 1, r);
17             pull(p);
18         };
19         build(1, 1, n);
20     }
21     void pull(int p) {
22         info[p] = info[2 * p] + info[2 * p + 1];
23     }
24     void apply(int p, const Tag &v) {
25         info[p].apply(v);
26         tag[p].apply(v);
27     }

```

```

28 void push(int p) {
29     apply(2 * p, tag[p]);
30     apply(2 * p + 1, tag[p]);
31     tag[p] = Tag();
32 }
33 void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
34     if (l > y || r < x) {
35         return;
36     }
37     if (l >= x && r <= y) {
38         apply(p, v);
39         return;
40     }
41     push(p);
42     int m = (l + r) / 2;
43     rangeApply(2 * p, l, m, x, y, v);
44     rangeApply(2 * p + 1, m + 1, r, x, y, v);
45     pull(p);
46 }
47 void rangeApply(int l, int r, const Tag &v) {
48     return rangeApply(1, 1, n, l, r, v);
49 }
50 Info rangeQuery(int p, int l, int r, int x, int y) {
51     if (l > y || r < x) {
52         return Info();
53     }
54     if (l >= x && r <= y) {
55         return info[p];
56     }
57     push(p);
58     int m = (l + r) / 2;
59     return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m + 1, r, x,
60 y);
61 }
62 Info rangeQuery(int l, int r) {
63     return rangeQuery(1, 1, n, l, r);
64 }
65 };
66 struct Tag {
67     LL x;
68     Tag(LL x = 0) {
69         this->x = x;
70     }
71     void apply(const Tag &t) {
72         x += t.x;
73     }
74 };
75 struct Info {
76     LL x, l, r;
77     Info(LL x = 0, LL l = 0, LL r = 0) {
78         this->x = x;
79         this->l = l;
80         this->r = r;
81     }
82     void apply(const Tag &t) {
83         if (t.x) {
84             x += t.x * (r - l + 1);
85         }
86     }
87 };
88 Info operator+(const Info &a, const Info &b) {
89     return Info(a.x + b.x, a.l, b.r);
90 }

```

8.4.2 快速线段树 (单点修改+区间最值)

```
1 struct Segt {
2     vector<int> w;
3     int n;
4     Segt(int n) : w(2 * n, (int)-2E9), n(n) {}
5
6     void modify(int pos, int val) {
7         for (w[pos += n] = val; pos > 1; pos /= 2) {
8             w[pos / 2] = max(w[pos], w[pos ^ 1]);
9         }
10    }
11
12    int ask(int l, int r) {
13        int res = -2E9;
14        for (l += n, r += n; l < r; l /= 2, r /= 2) {
15            if (l % 2) res = max(res, w[l++]);
16            if (r % 2) res = max(res, w[--r]);
17        }
18        return res;
19    }
20};
```

8.4.3 区间加法修改、区间最小值查询、区间合并

```
1 template<class T> struct Segt_ {
2     struct node {
3         int l, r;
4         T w, rmq, lazy;
5     };
6     vector<T> w;
7     vector<node> t;
8
9     Segt_() {}
10    Segt_(int n) { init(n); }
11    Segt_(vector<int> in) {
12        int n = in.size() - 1;
13        w.resize(n + 1);
14        for (int i = 1; i <= n; i++) {
15            w[i] = in[i];
16        }
17        init(in.size() - 1);
18    }
19
20    #define GL (k << 1)
21    #define GR (k << 1 | 1)
22
23    void init(int n) {
24        w.resize(n + 1);
25        t.resize(n * 4 + 1);
26        auto build = [&](auto self, int l, int r, int k = 1) {
27            if (l == r) {
28                t[k] = {l, r, w[l], w[l], -1}; // 如果有赋值为 0 的操作, 则懒标记必须要
-1
29                return;
30            }
31            t[k] = {l, r};
32            int mid = (l + r) / 2;
33            self(self, l, mid, GL);
34            self(self, mid + 1, r, GR);
35            pushup(k);
36        };
37    }
```

```

37     build(build, 1, n);
38 }
39 void pushdown(node &p, T lazy) { /* 【在此更新下递函数】 */
40     p.w += (p.r - p.l + 1) * lazy;
41     p.Min += lazy;
42     p.lazy += lazy;
43 }
44 void pushdown(int k) {
45     if (t[k].lazy == -1) return;
46     pushdown(t[GL], t[k].lazy);
47     pushdown(t[GR], t[k].lazy);
48     t[k].lazy = -1;
49 }
50 void pushup(int k) {
51     auto pushup = [&](node &p, node &l, node &r) { /* 【在此更新上传函数】 */
52         p.w = l.w + r.w;
53         p.rmql = min(l.rmql, r.rmql); // RMQ -> min/max
54     };
55     pushup(t[k], t[GL], t[GR]);
56 }
57 void modify(int l, int r, T val, int k = 1) { // 区间修改
58     if (l <= t[k].l && t[k].r <= r) {
59         pushdown(t[k], val);
60         return;
61     }
62     pushdown(k);
63     int mid = (t[k].l + t[k].r) / 2;
64     if (l <= mid) modify(l, r, val, GL);
65     if (mid < r) modify(l, r, val, GR);
66     pushup(k);
67 }
68 T rmql(int l, int r, int k = 1) { // 区间询问最小值
69     if (l <= t[k].l && t[k].r <= r) {
70         return t[k].rmql;
71     }
72     pushdown(k);
73     int mid = (t[k].l + t[k].r) / 2;
74     T ans = numeric_limits<T>::lowest(); // RMQ -> 为 max 时需要修改为 ::max()
75     if (l <= mid) ans = min(ans, rmql(l, r, GL)); // RMQ -> min/max
76     if (mid < r) ans = min(ans, rmql(l, r, GR)); // RMQ -> min/max
77     return ans;
78 }
79 T ask(int l, int r, int k = 1) { // 区间询问, 不合并
80     if (l <= t[k].l && t[k].r <= r) {
81         return t[k].w;
82     }
83     pushdown(k);
84     int mid = (t[k].l + t[k].r) / 2;
85     T ans = 0;
86     if (l <= mid) ans += ask(l, r, GL);
87     if (mid < r) ans += ask(l, r, GR);
88     return ans;
89 }
90 void debug(int k = 1) {
91     cout << "[" << t[k].l << ", " << t[k].r << "]: ";
92     cout << "w = " << t[k].w << ", ";
93     cout << "Min = " << t[k].Min << ", ";
94     cout << "lazy = " << t[k].lazy << ", ";
95     cout << endl;
96     if (t[k].l == t[k].r) return;
97     debug(GL), debug(GR);
98 }
99 };

```

8.4.4 同时需要处理区间加法与乘法修改

```
1  template <class T> struct Segt_ {
2      struct node {
3          int l, r;
4          T w, add, mul = 1; // 注意初始赋值
5      };
6      vector<T> w;
7      vector<node> t;
8
9      Segt_(int n) {
10         w.resize(n + 1);
11         t.resize((n << 2) + 1);
12         build(1, n);
13     }
14     Segt_(vector<int> in) {
15         int n = in.size() - 1;
16         w.resize(n + 1);
17         for (int i = 1; i <= n; i++) {
18             w[i] = in[i];
19         }
20         t.resize((n << 2) + 1);
21         build(1, n);
22     }
23     void pushdown(node &p, T add, T mul) { // 在此更新下递函数
24         p.w = p.w * mul + (p.r - p.l + 1) * add;
25         p.add = p.add * mul + add;
26         p.mul *= mul;
27     }
28     void pushup(node &p, node &l, node &r) { // 在此更新上传函数
29         p.w = l.w + r.w;
30     }
31 #define GL (k << 1)
32 #define GR (k << 1 | 1)
33     void pushdown(int k) { // 不需要动
34         pushdown(t[GL], t[k].add, t[k].mul);
35         pushdown(t[GR], t[k].add, t[k].mul);
36         t[k].add = 0, t[k].mul = 1;
37     }
38     void pushup(int k) { // 不需要动
39         pushup(t[k], t[GL], t[GR]);
40     }
41     void build(int l, int r, int k = 1) {
42         if (l == r) {
43             t[k] = {l, r, w[l]};
44             return;
45         }
46         t[k] = {l, r};
47         int mid = (l + r) / 2;
48         build(l, mid, GL);
49         build(mid + 1, r, GR);
50         pushup(k);
51     }
52     void modify(int l, int r, T val, int k = 1) { // 区间修改
53         if (l <= t[k].l && t[k].r <= r) {
54             t[k].w += (t[k].r - t[k].l + 1) * val;
55             t[k].add += val;
56             return;
57         }
58         pushdown(k);
59         int mid = (t[k].l + t[k].r) / 2;
60         if (l <= mid) modify(l, r, val, GL);
61         if (mid < r) modify(l, r, val, GR);
```

```

62     pushup(k);
63 }
64 void modify2(int l, int r, T val, int k = 1) { // 区间修改
65     if (l <= t[k].l && t[k].r <= r) {
66         t[k].w *= val;
67         t[k].add *= val;
68         t[k].mul *= val;
69         return;
70     }
71     pushdown(k);
72     int mid = (t[k].l + t[k].r) / 2;
73     if (l <= mid) modify2(l, r, val, GL);
74     if (mid < r) modify2(l, r, val, GR);
75     pushup(k);
76 }
77 T ask(int l, int r, int k = 1) { // 区间询问, 不合并
78     if (l <= t[k].l && t[k].r <= r) {
79         return t[k].w;
80     }
81     pushdown(k);
82     int mid = (t[k].l + t[k].r) / 2;
83     T ans = 0;
84     if (l <= mid) ans += ask(l, r, GL);
85     if (mid < r) ans += ask(l, r, GR);
86     return ans;
87 }
88 void debug(int k = 1) {
89     cout << "[" << t[k].l << ", " << t[k].r << "]: ";
90     cout << "w = " << t[k].w << ", ";
91     cout << "add = " << t[k].add << ", ";
92     cout << "mul = " << t[k].mul << ", ";
93     cout << endl;
94     if (t[k].l == t[k].r) return;
95     debug(GL), debug(GR);
96 }
97 };

```

8.4.5 区间赋值/推平

如果存在推平为 0 的操作, 那么需要将 lazy 初始赋值为 -1 。

```

1 void pushdown(node &p, T lazy) { /* 【在此更新下递函数】 */
2     p.w = (p.r - p.l + 1) * lazy;
3     p.lazy = lazy;
4 }
5 void modify(int l, int r, T val, int k = 1) {
6     if (l <= t[k].l && t[k].r <= r) {
7         t[k].w = val;
8         t[k].lazy = val;
9         return;
10    }
11    // 剩余部分不变
12 }

```

8.4.6 区间取模

原题需要进行“单点赋值+区间取模+区间求和”[See](#)。该操作不需要懒标记。

需要额外维护一个区间最大值, 当模数大于区间最大值时剪枝, 否则进行单点取模。由于单点 $\text{MOD} < x$ 时 $x \bmod \text{MOD} < \frac{x}{2}$, 故单点取模至 0 最劣只需要 $\log x$ 次。

```

1 void modifyMod(int l, int r, T val, int k = 1) {
2     if (l <= t[k].l && t[k].r <= r) {
3         if (t[k].rmq < val) return; // 重要剪枝
4     }
5     if (t[k].l == t[k].r) {
6         t[k].w %= val;
7         t[k].rmq %= val;
8         return;
9     }
10    int mid = (t[k].l + t[k].r) / 2;
11    if (l <= mid) modifyMod(l, r, val, GL);
12    if (mid < r) modifyMod(l, r, val, GR);
13    pushup(k);
14 }

```

8.4.7 区间异或修改

原题需要维护“区间异或修改+区间求和” [See](#)。

```

1 struct Segt { // #define GL (k << 1) // #define GR (k << 1 | 1)
2     struct node {
3         int l, r;
4         int w[N], lazy; // 注意这里为了方便计算, w 只需要存位
5     };
6     vector<int> base;
7     vector<node> t;
8
9     Segt(vector<int> in) : base(in) {
10        int n = in.size() - 1;
11        t.resize(n * 4 + 1);
12        auto build = [&](auto self, int l, int r, int k = 1) {
13            t[k] = {l, r}; // 前置赋值
14            if (l == r) {
15                for (int i = 0; i < N; i++) {
16                    t[k].w[i] = base[l] >> i & 1;
17                }
18                return;
19            }
20            int mid = (l + r) / 2;
21            self(self, l, mid, GL);
22            self(self, mid + 1, r, GR);
23            pushup(k);
24        };
25        build(build, 1, n);
26    }
27    void pushdown(node &p, int lazy) { /* 【在此更新下递函数】 */
28        int len = p.r - p.l + 1;
29        for (int i = 0; i < N; i++) {
30            if (lazy >> i & 1) { // 即 p.w = (p.r - p.l + 1) - p.w;
31                p.w[i] = len - p.w[i];
32            }
33        }
34        p.lazy ^= lazy;
35    }
36    void pushdown(int k) { // 【不需要动】
37        if (t[k].lazy == 0) return;
38        pushdown(t[GL], t[k].lazy);
39        pushdown(t[GR], t[k].lazy);
40        t[k].lazy = 0;
41    }
42    void pushup(int k) {
43        auto pushup = [&](node &p, node &l, node &r) { /* 【在此更新上传函数】 */

```

```

44         for (int i = 0; i < N; i++) {
45             p.w[i] = l.w[i] + r.w[i]; // 即 p.w = l.w + r.w;
46         }
47     };
48     pushup(t[k], t[GL], t[GR]);
49 }
50 void modify(int l, int r, int val, int k = 1) { // 区间修改
51     if (l <= t[k].l && t[k].r <= r) {
52         pushdown(t[k], val);
53         return;
54     }
55     pushdown(k);
56     int mid = (t[k].l + t[k].r) / 2;
57     if (l <= mid) modify(l, r, val, GL);
58     if (mid < r) modify(l, r, val, GR);
59     pushup(k);
60 }
61 i64 ask(int l, int r, int k = 1) { // 区间求和
62     if (l <= t[k].l && t[k].r <= r) {
63         i64 ans = 0;
64         for (int i = 0; i < N; i++) {
65             ans += t[k].w[i] * (1LL << i);
66         }
67         return ans;
68     }
69     pushdown(k);
70     int mid = (t[k].l + t[k].r) / 2;
71     i64 ans = 0;
72     if (l <= mid) ans += ask(l, r, GL);
73     if (mid < r) ans += ask(l, r, GR);
74     return ans;
75 }
76 };

```

8.4.8 拆位运算

原题同上。使用若干棵线段树维护每一位的值，区间异或转变为区间翻转。

```

1  template<class T> struct Segt_ { // GL 为 (k << 1), GR 为 (k << 1 | 1)
2      struct node {
3          int l, r;
4          T w;
5          bool lazy; // 注意懒标记用布尔型足以
6      };
7      vector<T> w;
8      vector<node> t;
9
10     Segt_() {}
11     void init(vector<int> in) {
12         int n = in.size() - 1;
13         w.resize(n * 4 + 1);
14         for (int i = 0; i <= n; i++) { w[i] = in[i]; }
15         t.resize(n * 4 + 1);
16         build(1, n);
17     }
18     void pushdown(node &p, bool lazy = 1) { // 【在此更新下递函数】
19         p.w = (p.r - p.l + 1) - p.w;
20         p.lazy ^= lazy;
21     }
22     void pushup(node &p, node &l, node &r) { // 【在此更新上传函数】
23         p.w = l.w + r.w;
24     }

```

```

25 void pushdown(int k) { // 【不需要动】
26     if (t[k].lazy == 0) return;
27     pushdown(t[GL]), pushdown(t[GR]); // 注意这里不再需要传入第二个参数
28     t[k].lazy = 0;
29 }
30 void pushup(int k) { pushup(t[k], t[GL], t[GR]); } // 【不需要动】
31 void build(int l, int r, int k = 1) {
32     if (l == r) {
33         t[k] = {l, r, w[l], 0}; // 注意懒标记初始为 0
34         return;
35     }
36     t[k] = {l, r};
37     int mid = (l + r) / 2;
38     build(l, mid, GL);
39     build(mid + 1, r, GR);
40     pushup(k);
41 }
42 void reverse(int l, int r, int k = 1) { // 区间翻转
43     if (l <= t[k].l && t[k].r <= r) {
44         pushdown(t[k], 1);
45         return;
46     }
47     pushdown(k);
48     int mid = (t[k].l + t[k].r) / 2;
49     if (l <= mid) reverse(l, r, GL);
50     if (mid < r) reverse(l, r, GR);
51     pushup(k);
52 }
53 T ask(int l, int r, int k = 1) { // 区间求和
54     if (l <= t[k].l && t[k].r <= r) {
55         return t[k].w;
56     }
57     pushdown(k);
58     int mid = (t[k].l + t[k].r) / 2;
59     T ans = 0;
60     if (l <= mid) ans += ask(l, r, GL);
61     if (mid < r) ans += ask(l, r, GR);
62     return ans;
63 }
64 };
65 signed main() {
66     int n; cin >> n;
67     vector in(20, vector<int>(n + 1));
68     Segt_<i64> segt[20]; // 拆位建线段树
69     for (int i = 1, x; i <= n; i++) { cin >> x;
70         for (int bit = 0; bit < 20; bit++) {
71             in[bit][i] = x >> bit & 1;
72         }
73     }
74     for (int i = 0; i < 20; i++) {
75         segt[i].init(in[i]);
76     }
77
78     int m, op;
79     for (cin >> m; m; m--) { cin >> op;
80         if (op == 1) {
81             int l, r; i64 ans = 0; cin >> l >> r;
82             for (int i = 0; i < 20; i++) {
83                 ans += segt[i].ask(l, r) * (1LL << i);
84             }
85             cout << ans << "\n";
86         } else {
87             int l, r, val; cin >> l >> r >> val;
88             for (int i = 0; i < 20; i++) {

```

```

89         if (val >> i & 1) { segt[i].reverse(l, r); }
90     }
91 }
92 }
93 }

```

8.5 坐标压缩与离散化

8.5.1 简单版本

```

1 sort(alls.begin(), alls.end());
2 alls.erase(unique(alls.begin(), alls.end()), alls.end());
3 auto get = [&](int x) {
4     return lower_bound(alls.begin(), alls.end(), x) - alls.begin();
5 };

```

8.5.2 封装

```

1 template <typename T> struct Compress_ {
2     int n, shift = 0; // shift 用于标记下标偏移量
3     vector<T> alls;
4
5     Compress_() {}
6     Compress_(auto in) : alls(in) {
7         init();
8     }
9     void add(T x) {
10         alls.emplace_back(x);
11     }
12     template <typename... Args> void add(T x, Args... args) {
13         add(x, add(args...));
14     }
15     void init() {
16         alls.emplace_back(numeric_limits<T>::max());
17         sort(alls.begin(), alls.end());
18         alls.erase(unique(alls.begin(), alls.end()), alls.end());
19         this->n = alls.size();
20     }
21     int size() {
22         return n;
23     }
24     int operator[](T x) { // 返回 x 元素的新下标
25         return upper_bound(alls.begin(), alls.end(), x) - alls.begin() + shift;
26     }
27     T Get(int x) { // 根据新下标返回原来元素
28         assert(x - shift < n);
29         return x - shift < n ? alls[x - shift] : -1;
30     }
31     bool count(T x) { // 查找元素 x 是否存在
32         return binary_search(alls.begin(), alls.end(), x);
33     }
34     friend auto &operator<< (ostream &o, const auto &j) {
35         cout << "{";
36         for (auto it : j.alls) {
37             o << it << " ";
38         }
39         return o << "}";
40     }
41 };
42 using Compress = Compress_<int>;

```

8.6 轻重链剖分/树链剖分

将线段树处理的部分分离，方便修改。支持链上查询/修改、子树查询/修改，建树时间复杂度 $\mathcal{O}(N \log N)$ ，单次查询时间复杂度 $\mathcal{O}(\log^2 N)$ 。

```
1 struct Segt {
2     struct node {
3         int l, r, w, lazy;
4     };
5     vector<int> w;
6     vector<node> t;
7
8     Segt() {}
9     #define GL (k << 1)
10    #define GR (k << 1 | 1)
11
12    void init(vector<int> in) {
13        int n = in.size() - 1;
14        w.resize(n + 1);
15        for (int i = 1; i <= n; i++) {
16            w[i] = in[i];
17        }
18        t.resize(n * 4 + 1);
19        auto build = [&](auto self, int l, int r, int k = 1) {
20            if (l == r) {
21                t[k] = {l, r, w[l], 0}; // 如果有赋值为 0 的操作，则懒标记必须要 -1
22                return;
23            }
24            t[k] = {l, r};
25            int mid = (l + r) / 2;
26            self(self, l, mid, GL);
27            self(self, mid + 1, r, GR);
28            pushup(k);
29        };
30        build(build, 1, n);
31    }
32    void pushdown(node &p, int lazy) { /* 【在此更新下递函数】 */
33        p.w += (p.r - p.l + 1) * lazy;
34        p.lazy += lazy;
35    }
36    void pushdown(int k) { // 不需要动
37        if (t[k].lazy == 0) return;
38        pushdown(t[GL], t[k].lazy);
39        pushdown(t[GR], t[k].lazy);
40        t[k].lazy = 0;
41    }
42    void pushup(int k) { // 不需要动
43        auto pushup = [&](node &p, node &l, node &r) { /* 【在此更新上传函数】 */
44            p.w = l.w + r.w;
45        };
46        pushup(t[k], t[GL], t[GR]);
47    }
48    void modify(int l, int r, int val, int k = 1) {
49        if (l <= t[k].l && t[k].r <= r) {
50            pushdown(t[k], val);
51            return;
52        }
53        pushdown(k);
54        int mid = (t[k].l + t[k].r) / 2;
55        if (l <= mid) modify(l, r, val, GL);
56        if (mid < r) modify(l, r, val, GR);
57        pushup(k);
```

```

58     }
59     int ask(int l, int r, int k = 1) {
60         if (l <= t[k].l && t[k].r <= r) {
61             return t[k].w;
62         }
63         pushdown(k);
64         int mid = (t[k].l + t[k].r) / 2;
65         int ans = 0;
66         if (l <= mid) ans += ask(l, r, GL);
67         if (mid < r) ans += ask(l, r, GR);
68         return ans;
69     }
70 };
71
72 struct HLD {
73     int n, idx;
74     vector<vector<int>> ver;
75     vector<int> siz, dep;
76     vector<int> top, son, parent;
77     vector<int> in, id, val;
78     Segt segt;
79
80     HLD(int n) {
81         this->n = n;
82         ver.resize(n + 1);
83         siz.resize(n + 1);
84         dep.resize(n + 1);
85
86         top.resize(n + 1);
87         son.resize(n + 1);
88         parent.resize(n + 1);
89
90         idx = 0;
91         in.resize(n + 1);
92         id.resize(n + 1);
93         val.resize(n + 1);
94     }
95     void add(int x, int y) { // 建立双向边
96         ver[x].push_back(y);
97         ver[y].push_back(x);
98     }
99     void dfs1(int x) {
100         siz[x] = 1;
101         dep[x] = dep[parent[x]] + 1;
102         for (auto y : ver[x]) {
103             if (y == parent[x]) continue;
104             parent[y] = x;
105             dfs1(y);
106             siz[x] += siz[y];
107             if (siz[y] > siz[son[x]]) {
108                 son[x] = y;
109             }
110         }
111     }
112     void dfs2(int x, int up) {
113         id[x] = ++idx;
114         val[idx] = in[x]; // 建立编号
115         top[x] = up;
116         if (son[x]) dfs2(son[x], up);
117         for (auto y : ver[x]) {
118             if (y == parent[x] || y == son[x]) continue;
119             dfs2(y, y);
120         }
121     }

```

```

122 void modify(int l, int r, int val) { // 链上修改
123     while (top[l] != top[r]) {
124         if (dep[top[l]] < dep[top[r]]) {
125             swap(l, r);
126         }
127         segt.modify(id[top[l]], id[l], val);
128         l = parent[top[l]];
129     }
130     if (dep[l] > dep[r]) {
131         swap(l, r);
132     }
133     segt.modify(id[l], id[r], val);
134 }
135 void modify(int root, int val) { // 子树修改
136     segt.modify(id[root], id[root] + siz[root] - 1, val);
137 }
138 int ask(int l, int r) { // 链上查询
139     int ans = 0;
140     while (top[l] != top[r]) {
141         if (dep[top[l]] < dep[top[r]]) {
142             swap(l, r);
143         }
144         ans += segt.ask(id[top[l]], id[l]);
145         l = parent[top[l]];
146     }
147     if (dep[l] > dep[r]) {
148         swap(l, r);
149     }
150     return ans + segt.ask(id[l], id[r]);
151 }
152 int ask(int root) { // 子树查询
153     return segt.ask(id[root], id[root] + siz[root] - 1);
154 }
155 void work(auto in, int root = 1) { // 在此初始化
156     assert(in.size() == n + 1);
157     this->in = in;
158     dfs1(root);
159     dfs2(root, root);
160     segt.init(val); // 建立线段树
161 }
162 void work(int root = 1) { // 在此初始化
163     dfs1(root);
164     dfs2(root, root);
165     segt.init(val); // 建立线段树
166 }
167 };

```

8.7 普通莫队

以 $\mathcal{O}(N\sqrt{N})$ 的复杂度完成 Q 次询问的离线查询，其中每个分块的大小取 $\sqrt{N} = \sqrt{10^5} = 317$ ，也可以使用 `n / min<int>(n, sqrt(q))`、`ceil((double)n / (int)sqrt(n))` 或者 `sqrt(n)` 划分。

```

1
2 signed main() {
3     int n;
4     cin >> n;
5     vector<int> w(n + 1);
6     for (int i = 1; i <= n; i++) {
7         cin >> w[i];
8     }
9
10    int q;

```

```

11     cin >> q;
12     vector<array<int, 3>> query(q + 1);
13     for (int i = 1; i <= q; i++) {
14         int l, r;
15         cin >> l >> r;
16         query[i] = {l, r, i};
17     }
18
19     int Knum = n / min<int>(n, sqrt(q)); // 计算块长
20     vector<int> K(n + 1);
21     for (int i = 1; i <= n; i++) { // 固定块长
22         K[i] = (i - 1) / Knum + 1;
23     }
24     sort(query.begin() + 1, query.end(), [&](auto x, auto y) {
25         if (K[x[0]] != K[y[0]]) return x[0] < y[0];
26         if (K[x[0]] & 1) return x[1] < y[1];
27         return x[1] > y[1];
28     });
29
30     int l = 1, r = 0, val = 0;
31     vector<int> ans(q + 1);
32     for (int i = 1; i <= q; i++) {
33         auto [ql, qr, id] = query[i];
34         auto add = [&](int x) -> void {};
35         auto del = [&](int x) -> void {};
36         while (l > ql) add(w[--l]);
37         while (r < qr) add(w[++r]);
38         while (l < ql) del(w[l++]);
39         while (r > qr) del(w[r--]);
40         ans[id] = val;
41     }
42     for (int i = 1; i <= q; i++) {
43         cout << ans[i] << endl;
44     }
45 }

```

需要注意的是，在普通莫队中，`K` 数组的作用是根据左边界值进行排序，当询问次数很少时（ $q \ll n$ ），可以直接合并到 `query` 数组中。

```

1 vector<array<int, 4>> query(q);
2 for (int i = 1; i <= q; i++) {
3     int l, r;
4     cin >> l >> r;
5     query[i] = {l, r, i, (l - 1) / Knum + 1}; // 合并
6 }
7 sort(query.begin() + 1, query.end(), [&](auto x, auto y) {
8     if (x[3] != y[3]) return x[3] < y[3];
9     if (x[3] & 1) return x[1] < y[1];
10    return x[1] > y[1];
11 });

```

8.8 带修改的莫队（带时间维度的莫队）

以 $O(N^{\frac{5}{3}})$ 的复杂度完成 Q 次询问的离线查询，其中每个分块的大小取 $N^{\frac{2}{3}} = \sqrt[3]{100000^2} = 2154$ （直接取会略快），也可以使用 `pow(n, 0.6666)` 划分。

```

1 signed main() {
2     int n, q;
3     cin >> n >> q;
4     vector<int> w(n + 1);

```

```

5   for (int i = 1; i <= n; i++) {
6       cin >> w[i];
7   }
8
9   vector<array<int, 4>> query = {{}}; // {左区间, 右区间, 累计修改次数, 下标}
10  vector<array<int, 2>> modify = {{}}; // {修改的值, 修改的元素下标}
11  for (int i = 1; i <= q; i++) {
12      char op;
13      cin >> op;
14      if (op == 'Q') {
15          int l, r;
16          cin >> l >> r;
17          query.push_back({l, r, (int)modify.size() - 1, (int)query.size()});
18      } else {
19          int idx, w;
20          cin >> idx >> w;
21          modify.push_back({w, idx});
22      }
23  }
24
25  int Knum = 2154; // 计算块长
26  vector<int> K(n + 1);
27  for (int i = 1; i <= n; i++) { // 固定块长
28      K[i] = (i - 1) / Knum + 1;
29  }
30  sort(query.begin() + 1, query.end(), [&](auto x, auto y) {
31      if (K[x[0]] != K[y[0]]) return x[0] < y[0];
32      if (K[x[1]] != K[y[1]]) return x[1] < y[1];
33      return x[3] < y[3];
34  });
35
36  int l = 1, r = 0, val = 0;
37  int t = 0; // 累计修改次数
38  vector<int> ans(query.size());
39  for (int i = 1; i < query.size(); i++) {
40      auto [ql, qr, qt, id] = query[i];
41      auto add = [&](int x) -> void {};
42      auto del = [&](int x) -> void {};
43      auto time = [&](int x, int l, int r) -> void {};
44      while (l > ql) add(w[--l]);
45      while (r < qr) add(w[++r]);
46      while (l < ql) del(w[l++]);
47      while (r > qr) del(w[r--]);
48      while (t < qt) time(++t, ql, qr);
49      while (t > qt) time(t--, ql, qr);
50      ans[id] = val;
51  }
52  for (int i = 1; i < ans.size(); i++) {
53      cout << ans[i] << endl;
54  }
55 }

```

8.9 分数运算类

定义了分数的四则运算，如果需要处理浮点数，那么需要将函数中的 `gcd` 运算替换为 `fgcd`。

```

1  template<class T> struct Frac {
2      T x, y;
3      Frac() : Frac(0, 1) {}
4      Frac(T x_) : Frac(x_, 1) {}
5      Frac(T x_, T y_) : x(x_), y(y_) {
6          if (y < 0) {

```

```

7         y = -y;
8         x = -x;
9     }
10 }
11
12 constexpr double val() const {
13     return 1. * x / y;
14 }
15 constexpr Frac norm() const { // 调整符号、转化为最简形式
16     T p = gcd(x, y);
17     return {x / p, y / p};
18 }
19 friend constexpr auto &operator<<(ostream &o, const Frac &j) {
20     T p = gcd(j.x, j.y);
21     if (j.y == p) {
22         return o << j.x / p;
23     } else {
24         return o << j.x / p << "/" << j.y / p;
25     }
26 }
27 constexpr Frac &operator/=(const Frac &i) {
28     x *= i.y;
29     y *= i.x;
30     if (y < 0) {
31         x = -x;
32         y = -y;
33     }
34     return *this;
35 }
36 constexpr Frac &operator+=(const Frac &i) { return x = x * i.y + y * i.x, y *=
i.y, *this; }
37 constexpr Frac &operator-=(const Frac &i) { return x = x * i.y - y * i.x, y *=
i.y, *this; }
38 constexpr Frac &operator*=(const Frac &i) { return x *= i.x, y *= i.y, *this; }
39 friend constexpr Frac operator+(const Frac i, const Frac j) { return i += j; }
40 friend constexpr Frac operator-(const Frac i, const Frac j) { return i -= j; }
41 friend constexpr Frac operator*(const Frac i, const Frac j) { return i *= j; }
42 friend constexpr Frac operator/(const Frac i, const Frac j) { return i /= j; }
43 friend constexpr Frac operator-(const Frac i) { return Frac(-i.x, i.y); }
44 friend constexpr bool operator<(const Frac i, const Frac j) { return i.x * j.y <
i.y * j.x; }
45 friend constexpr bool operator>(const Frac i, const Frac j) { return i.x * j.y >
i.y * j.x; }
46 friend constexpr bool operator==(const Frac i, const Frac j) { return i.x * j.y
== i.y * j.x; }
47 friend constexpr bool operator!=(const Frac i, const Frac j) { return i.x * j.y
!= i.y * j.x; }
48 };

```

8.10 主席树（可持久化线段树）

以 $O(N \log N)$ 的时间复杂度建树、查询、修改。

```

1 struct PresidentTree {
2     static constexpr int N = 2e5 + 10;
3     int cntNodes, root[N];
4     struct node {
5         int l, r;
6         int cnt;
7     } tr[4 * N + 17 * N];
8     void modify(int &u, int v, int l, int r, int x) {
9         u = ++cntNodes;

```

```

10     tr[u] = tr[v];
11     tr[u].cnt++;
12     if (l == r) return;
13     int mid = (l + r) / 2;
14     if (x <= mid)
15         modify(tr[u].l, tr[v].l, l, mid, x);
16     else
17         modify(tr[u].r, tr[v].r, mid + 1, r, x);
18 }
19 int kth(int u, int v, int l, int r, int k) {
20     if (l == r) return l;
21     int res = tr[tr[v].l].cnt - tr[tr[u].l].cnt;
22     int mid = (l + r) / 2;
23     if (k <= res)
24         return kth(tr[u].l, tr[v].l, l, mid, k);
25     else
26         return kth(tr[u].r, tr[v].r, mid + 1, r, k - res);
27 }
28 };

```

8.11 KD Tree

在第 k 维上的单次查询复杂度最坏为 $\mathcal{O}(n^{1-k^{-1}})$ 。

```

1  struct KDT {
2      constexpr static int N = 1e5 + 10, K = 2;
3      double alpha = 0.725;
4      struct node {
5          int info[K];
6          int mn[K], mx[K];
7      } tr[N];
8      int ls[N], rs[N], siz[N], id[N], d[N];
9      int idx, rt, cur;
10     int ans;
11     KDT() {
12         rt = 0;
13         cur = 0;
14         memset(ls, 0, sizeof ls);
15         memset(rs, 0, sizeof rs);
16         memset(d, 0, sizeof d);
17     }
18     void apply(int p, int son) {
19         if (son) {
20             for (int i = 0; i < K; i++) {
21                 tr[p].mn[i] = min(tr[p].mn[i], tr[son].mn[i]);
22                 tr[p].mx[i] = max(tr[p].mx[i], tr[son].mx[i]);
23             }
24             siz[p] += siz[son];
25         }
26     }
27     void maintain(int p) {
28         for (int i = 0; i < K; i++) {
29             tr[p].mn[i] = tr[p].info[i];
30             tr[p].mx[i] = tr[p].info[i];
31         }
32         siz[p] = 1;
33         apply(p, ls[p]);
34         apply(p, rs[p]);
35     }
36     int build(int l, int r) {
37         if (l > r) return 0;
38         vector<double> avg(K);

```

```

39     for (int i = 0; i < K; i++) {
40         for (int j = l; j <= r; j++) {
41             avg[i] += tr[id[j]].info[i];
42         }
43         avg[i] /= (r - l + 1);
44     }
45     vector<double> var(K);
46     for (int i = 0; i < K; i++) {
47         for (int j = l; j <= r; j++) {
48             var[i] += (tr[id[j]].info[i] - avg[i]) * (tr[id[j]].info[i] -
avg[i]);
49         }
50     }
51     int mid = (l + r) / 2;
52     int x = max_element(var.begin(), var.end()) - var.begin();
53     nth_element(id + l, id + mid, id + r + 1, [&](int a, int b) {
54         return tr[a].info[x] < tr[b].info[x];
55     });
56     d[id[mid]] = x;
57     ls[id[mid]] = build(l, mid - 1);
58     rs[id[mid]] = build(mid + 1, r);
59     maintain(id[mid]);
60     return id[mid];
61 }
62 void print(int p) {
63     if (!p) return;
64     print(ls[p]);
65     id[++idx] = p;
66     print(rs[p]);
67 }
68 void rebuild(int &p) {
69     idx = 0;
70     print(p);
71     p = build(1, idx);
72 }
73 bool bad(int p) {
74     return alpha * siz[p] <= max(siz[ls[p]], siz[rs[p]]);
75 }
76 void insert(int &p, int cur) {
77     if (!p) {
78         p = cur;
79         maintain(p);
80         return;
81     }
82     if (tr[p].info[d[p]] > tr[cur].info[d[p]]) insert(ls[p], cur);
83     else insert(rs[p], cur);
84     maintain(p);
85     if (bad(p)) rebuild(p);
86 }
87 void insert(vector<int> &a) {
88     cur++;
89     for (int i = 0; i < K; i++) {
90         tr[cur].info[i] = a[i];
91     }
92     insert(rt, cur);
93 }
94 bool out(int p, vector<int> &a) {
95     for (int i = 0; i < K; i++) {
96         if (a[i] < tr[p].mn[i]) {
97             return true;
98         }
99     }
100     return false;
101 }

```

```

102     bool in(int p, vector<int> &a) {
103         for (int i = 0; i < K; i++) {
104             if (a[i] < tr[p].info[i]) {
105                 return false;
106             }
107         }
108         return true;
109     }
110     bool all(int p, vector<int> &a) {
111         for (int i = 0; i < K; i++) {
112             if (a[i] < tr[p].mx[i]) {
113                 return false;
114             }
115         }
116         return true;
117     }
118     void query(int p, vector<int> &a) {
119         if (!p) return;
120         if (out(p, a)) return;
121         if (all(p, a)) {
122             ans += siz[p];
123             return;
124         }
125         if (in(p, a)) ans++;
126         query(ls[p], a);
127         query(rs[p], a);
128     }
129     int query(vector<int> &a) {
130         ans = 0;
131         query(rt, a);
132         return ans;
133     }
134 };

```

8.12 ST 表

用于解决区间可重复贡献问题，需要满足 x 运算符 $x = x$ （如区间最大值： $\max(x, x) = x$ 、区间 gcd： $\gcd(x, x) = x$ 等），但是不支持修改操作。 $\mathcal{O}(N \log N)$ 预处理， $\mathcal{O}(1)$ 查询。

```

1  struct ST {
2      const int n, k;
3      vector<int> in1, in2;
4      vector<vector<int>> Max, Min;
5      ST(int n) : n(n), in1(n + 1), in2(n + 1), k(__lg(n)) {
6          Max.resize(k + 1, vector<int>(n + 1));
7          Min.resize(k + 1, vector<int>(n + 1));
8      }
9      void init() {
10         for (int i = 1; i <= n; i++) {
11             Max[0][i] = in1[i];
12             Min[0][i] = in2[i];
13         }
14         for (int i = 0, t = 1; i < k; i++, t <= 1) {
15             const int T = n - (t < 1) + 1;
16             for (int j = 1; j <= T; j++) {
17                 Max[i + 1][j] = max(Max[i][j], Max[i][j + t]);
18                 Min[i + 1][j] = min(Min[i][j], Min[i][j + t]);
19             }
20         }
21     }
22     int getMax(int l, int r) {
23         if (l > r) {

```

```

24         swap(l, r);
25     }
26     int k = __lg(r - l + 1);
27     return max(Max[k][l], Max[k][r - (1 << k) + 1]);
28 }
29 int getMin(int l, int r) {
30     if (l > r) {
31         swap(l, r);
32     }
33     int k = __lg(r - l + 1);
34     return min(Min[k][l], Min[k][r - (1 << k) + 1]);
35 }
36 };

```

8.13 基于状压的线性 RMQ 算法

严格 $\mathcal{O}(N)$ 预处理, $\mathcal{O}(1)$ 查询。

```

1  template<class T, class Cmp = less<T>> struct RMQ {
2      const Cmp cmp = Cmp();
3      static constexpr unsigned B = 64;
4      using u64 = unsigned long long;
5      int n;
6      vector<vector<T>> a;
7      vector<T> pre, suf, ini;
8      vector<u64> stk;
9      RMQ() {}
10     RMQ(const vector<T> &v) {
11         init(v);
12     }
13     void init(const vector<T> &v) {
14         n = v.size();
15         pre = suf = ini = v;
16         stk.resize(n);
17         if (!n) {
18             return;
19         }
20         const int M = (n - 1) / B + 1;
21         const int lg = __lg(M);
22         a.assign(lg + 1, vector<T>(M));
23         for (int i = 0; i < M; i++) {
24             a[0][i] = v[i * B];
25             for (int j = 1; j < B && i * B + j < n; j++) {
26                 a[0][i] = min(a[0][i], v[i * B + j], cmp);
27             }
28         }
29         for (int i = 1; i < n; i++) {
30             if (i % B) {
31                 pre[i] = min(pre[i], pre[i - 1], cmp);
32             }
33         }
34         for (int i = n - 2; i >= 0; i--) {
35             if (i % B != B - 1) {
36                 suf[i] = min(suf[i], suf[i + 1], cmp);
37             }
38         }
39         for (int j = 0; j < lg; j++) {
40             for (int i = 0; i + (2 << j) <= M; i++) {
41                 a[j + 1][i] = min(a[j][i], a[j][i + (1 << j)], cmp);
42             }
43         }
44         for (int i = 0; i < M; i++) {

```

```

45         const int l = i * B;
46         const int r = min(1U * n, l + B);
47         u64 s = 0;
48         for (int j = 1; j < r; j++) {
49             while (s && cmp(v[j], v[__lg(s) + 1])) {
50                 s ^= 1ULL << __lg(s);
51             }
52             s |= 1ULL << (j - 1);
53             stk[j] = s;
54         }
55     }
56 }
57 T operator()(int l, int r) {
58     if (l / B != (r - 1) / B) {
59         T ans = min(suf[l], pre[r - 1], cmp);
60         l = l / B + 1;
61         r = r / B;
62         if (l < r) {
63             int k = __lg(r - l);
64             ans = min({ans, a[k][l], a[k][r - (1 << k)]}, cmp);
65         }
66         return ans;
67     } else {
68         int x = B * (l / B);
69         return ini[__builtin_ctzll(stk[r - 1] >> (1 - x)) + 1];
70     }
71 }
72 };

```

8.14 pbds 扩展库实现平衡二叉树

记得加上相应的头文件，同时需要注意定义时的参数，一般只需要修改第三个参数：即定义的是大根堆还是小根堆。

附常见成员函数：

```

1  empty() / size()
2  insert(x) // 插入元素x
3  erase(x) // 删除元素/迭代器x
4  order_of_key(x) // 返回元素x的排名
5  find_by_order(x) // 返回排名为x的元素迭代器
6  lower_bound(x) / upper_bound(x) // 返回迭代器
7  join(Tree) // 将Tree树的全部元素并入当前的树
8  split(x, Tree) // 将大于x的元素放入Tree树

```

```

1  #include <ext/pb_ds/assoc_container.hpp>
2  using namespace __gnu_pbds;
3  using V = pair<int, int>;
4  tree<V, null_type, less<V>, rb_tree_tag, tree_order_statistics_node_update> ver;
5  map<int, int> dic;
6
7  int n; cin >> n;
8  for (int i = 1, op, x; i <= n; i++) {
9      cin >> op >> x;
10     if (op == 1) { // 插入一个元素x, 允许重复
11         ver.insert({x, ++dic[x]});
12     } else if (op == 2) { // 删除元素x, 若有重复, 则任意删除一个
13         ver.erase({x, dic[x]--});
14     } else if (op == 3) { // 查询元素x的排名 (排名定义为比当前数小的数的个数+1)
15         cout << ver.order_of_key({x, 1}) + 1 << endl;
16     } else if (op == 4) { // 查询排名为x的元素

```

```

17     cout << ver.find_by_order(--x)->first << endl;
18 } else if (op == 5) { // 查询元素x的前驱
19     int idx = ver.order_of_key({x, 1}) - 1; // 无论x存不存在, idx都代表x的位置, 需要-1
20     cout << ver.find_by_order(idx)->first << endl;
21 } else if (op == 6) { // 查询元素x的后继
22     int idx = ver.order_of_key({x, dic[x]}); // 如果x不存在, 那么idx就是x的后继
23     if (ver.find({x, 1}) != ver.end()) idx++; // 如果x存在, 那么idx是x的位置, 需要+1
24     cout << ver.find_by_order(idx)->first << endl;
25 }
26 }

```

8.15 vector 模拟实现平衡二叉树

```

1  #define ALL(x) x.begin(), x.end()
2  #define pre lower_bound
3  #define suf upper_bound
4  int n; cin >> n;
5  vector<int> ver;
6  for (int i = 1, op, x; i <= n; i++) {
7      cin >> op >> x;
8      if (op == 1) ver.insert(pre(ALL(ver), x), x);
9      if (op == 2) ver.erase(pre(ALL(ver), x));
10     if (op == 3) cout << pre(ALL(ver), x) - ver.begin() + 1 << endl;
11     if (op == 4) cout << ver[x - 1] << endl;
12     if (op == 5) cout << ver[pre(ALL(ver), x) - ver.begin() - 1] << endl;
13     if (op == 6) cout << ver[suf(ALL(ver), x) - ver.begin()] << endl;
14 }

```

8.16 线性基 (高斯消元法)

设向量长度为 N (一般取 63), 总数为 M , 时间复杂度为 $\mathcal{O}(NM)$ 。

```

1  struct LB { // Linear Basis
2      using i64 = long long;
3      const int BASE = 63;
4      vector<i64> d, p;
5      int cnt, flag;
6
7      LB() {
8          d.resize(BASE + 1);
9          p.resize(BASE + 1);
10         cnt = flag = 0;
11     }
12     bool insert(i64 val) {
13         for (int i = BASE - 1; i >= 0; i--) {
14             if (val & (1ll << i)) {
15                 if (!d[i]) {
16                     d[i] = val;
17                     return true;
18                 }
19                 val ^= d[i];
20             }
21         }
22         flag = 1; //可以异或出0
23         return false;
24     }
25     bool check(i64 val) { // 判断 val 是否能被异或得到
26         for (int i = BASE - 1; i >= 0; i--) {
27             if (val & (1ll << i)) {

```

```

28         if (!d[i]) {
29             return false;
30         }
31         val ^= d[i];
32     }
33 }
34 return true;
35 }
36 i64 ask_max() {
37     i64 res = 0;
38     for (int i = BASE - 1; i >= 0; i--) {
39         if ((res ^ d[i]) > res) res ^= d[i];
40     }
41     return res;
42 }
43 i64 ask_min() {
44     if (flag) return 0; // 特判 0
45     for (int i = 0; i <= BASE - 1; i++) {
46         if (d[i]) return d[i];
47     }
48 }
49 void rebuild() { // 第k小值独立预处理
50     for (int i = BASE - 1; i >= 0; i--) {
51         for (int j = i - 1; j >= 0; j--) {
52             if (d[i] & (1ll << j)) d[i] ^= d[j];
53         }
54     }
55     for (int i = 0; i <= BASE - 1; i++) {
56         if (d[i]) p[cnt++] = d[i];
57     }
58 }
59 i64 kthquery(i64 k) { // 查询能被异或得到的第 k 小值，如不存在则返回 -1
60     if (flag) k--; // 特判 0，如果不需要 0，直接删去
61     if (!k) return 0;
62     i64 res = 0;
63     if (k >= (1ll << cnt)) return -1;
64     for (int i = BASE - 1; i >= 0; i--) {
65         if (k & (1ll << i)) res ^= p[i];
66     }
67     return res;
68 }
69 void Merge(const LB &b) { // 合并两个线性基
70     for (int i = BASE - 1; i >= 0; i--) {
71         if (b.d[i]) {
72             insert(b.d[i]);
73         }
74     }
75 }
76 };

```

8.17 珂朵莉树 (OD Tree)

区间赋值的数据结构都可以骗分，在数据随机的情况下，复杂度可以保证，时间复杂度： $\mathcal{O}(N \log \log N)$ 。

```

1 struct ODT {
2     struct node {
3         int l, r;
4         mutable LL v;
5         node(int l, int r = -1, LL v = 0) : l(l), r(r), v(v) {}
6         bool operator<(const node &o) const {
7             return l < o.l;

```

```

8     }
9 };
10 set<node> s;
11 ODT() {
12     s.clear();
13 }
14 auto split(int pos) {
15     auto it = s.lower_bound(node(pos));
16     if (it != s.end() && it->l == pos) return it;
17     it--;
18     int l = it->l, r = it->r;
19     LL v = it->v;
20     s.erase(it);
21     s.insert(node(l, pos - 1, v));
22     return s.insert(node(pos, r, v)).first;
23 }
24 void assign(int l, int r, LL x) {
25     auto itr = split(r + 1), itl = split(l);
26     s.erase(itl, itr);
27     s.insert(node(l, r, x));
28 }
29 void add(int l, int r, LL x) {
30     auto itr = split(r + 1), itl = split(l);
31     for (auto it = itl; it != itr; it++) {
32         it->v += x;
33     }
34 }
35 LL kth(int l, int r, int k) {
36     vector<pair<LL, int>> a;
37     auto itr = split(r + 1), itl = split(l);
38     for (auto it = itl; it != itr; it++) {
39         a.push_back(pair<LL, int>(it->v, it->r - it->l + 1));
40     }
41     sort(a.begin(), a.end());
42     for (auto [val, len] : a) {
43         k -= len;
44         if (k <= 0) return val;
45     }
46 }
47 LL power(LL a, int b, int mod) {
48     a %= mod;
49     LL res = 1;
50     for (; b; b /= 2, a = a * a % mod) {
51         if (b % 2) {
52             res = res * a % mod;
53         }
54     }
55     return res;
56 }
57 LL powersum(int l, int r, int x, int mod) {
58     auto itr = split(r + 1), itl = split(l);
59     LL ans = 0;
60     for (auto it = itl; it != itr; it++) {
61         ans = (ans + power(it->v, x, mod) * (it->r - it->l + 1) % mod) % mod;
62     }
63     return ans;
64 }
65 };

```

8.18 取模类

集成了常见的取模四则运算，运算速度与手动取模相差无几，效率极高。

```
1  template<class T> T mypow(T n, int k, T r = 1) {
2      for (; k >>= 1, n *= n) {
3          if (k & 1)
4              r *= n;
5      }
6      return r;
7  }
8
9  #undef int long long
10 template<int MOD> struct Zmod {
11     int x;
12     Zmod(int x = 0) : x(norm(x % MOD)) {}
13     int val() const { return x; }
14     int norm(int x) const { return (x + MOD) % MOD; }
15     Zmod operator-() const {
16         int val = norm(MOD - x);
17         return Zmod(val);
18     }
19     Zmod &operator++() { return x = norm(x + 1), *this; }
20     Zmod operator++(int) {
21         Zmod res = *this;
22         ++*this;
23         return res;
24     }
25     Zmod &operator--() { return x = norm(x - 1), *this; }
26     Zmod operator--(int) {
27         Zmod res = *this;
28         --*this;
29         return res;
30     }
31     Zmod inv() const {
32         assert(x != 0);
33         return mypow(*this, MOD - 2);
34     }
35     Zmod &operator*=(const Zmod &i) { return x = x * i.x % MOD, *this; }
36     Zmod &operator+=(const Zmod &i) { return x = norm(x + i.x), *this; }
37     Zmod &operator-=(const Zmod &i) { return x = norm(x - i.x), *this; }
38     Zmod &operator/=(const Zmod &i) { return *this *= i.inv(); }
39     Zmod &operator%=(const int &i) { return x %= i, *this; }
40     friend Zmod operator*(const Zmod &i, const Zmod &j) {
41         Zmod res = i;
42         res *= j;
43         return res;
44     }
45     friend Zmod operator+(const Zmod &i, const Zmod &j) {
46         Zmod res = i;
47         res += j;
48         return res;
49     }
50     friend Zmod operator-(const Zmod &i, const Zmod &j) {
51         Zmod res = i;
52         res -= j;
53         return res;
54     }
55     friend Zmod operator/(const Zmod &i, const Zmod &j) {
56         Zmod res = i;
57         res /= j;
58         return res;
59     }
```

```

60     friend Zmod operator%(const Zmod &i, const int &j) {
61         Zmod res = i;
62         res %= j;
63         return res;
64     }
65     friend auto &operator>>(istream &it, Zmod &j) {
66         int v;
67         it >> v;
68         j = Zmod(v);
69         return it;
70     }
71     friend auto &operator<<(ostream &o, const Zmod &j) { return o << j.x; }
72     bool operator<(const Zmod &i) const { return x < i.x; }
73     bool operator>(const Zmod &i) const { return x > i.x; }
74     bool operator==(const Zmod &i) const { return x == i.x; }
75     bool operator!=(const Zmod &i) const { return x != i.x; }
76 };
77 #define int long long
78
79 using Z = Zmod<998244353>;

```

8.19 大整数类（高精度计算）

```

1  const int base = 1000000000;
2  const int base_digits = 9; // 分解为九个数位一个数字
3  struct bigint {
4      vector<int> a;
5      int sign;
6
7      bigint() : sign(1) {}
8      bigint operator-() const {
9          bigint res = *this;
10         res.sign = -sign;
11         return res;
12     }
13     bigint(long long v) {
14         *this = v;
15     }
16     bigint(const string &s) {
17         read(s);
18     }
19     void operator=(const bigint &v) {
20         sign = v.sign;
21         a = v.a;
22     }
23     void operator=(long long v) {
24         a.clear();
25         sign = 1;
26         if (v < 0) sign = -1, v = -v;
27         for (; v > 0; v = v / base) {
28             a.push_back(v % base);
29         }
30     }
31
32     // 基础加减乘除
33     bigint operator+(const bigint &v) const {
34         if (sign == v.sign) {
35             bigint res = v;
36             for (int i = 0, carry = 0; i < (int)max(a.size(), v.a.size()) || carry;
37 ++i) {
38                 if (i == (int)res.a.size()) {
39                     res.a.push_back(0);

```

```

39         }
40         res.a[i] += carry + (i < (int)a.size() ? a[i] : 0);
41         carry = res.a[i] >= base;
42         if (carry) {
43             res.a[i] -= base;
44         }
45     }
46     return res;
47 }
48 return *this - (-v);
49 }
50 bigint operator-(const bigint &v) const {
51     if (sign == v.sign) {
52         if (abs() >= v.abs()) {
53             bigint res = *this;
54             for (int i = 0, carry = 0; i < (int)v.a.size() || carry; ++i) {
55                 res.a[i] -= carry + (i < (int)v.a.size() ? v.a[i] : 0);
56                 carry = res.a[i] < 0;
57                 if (carry) {
58                     res.a[i] += base;
59                 }
60             }
61             res.trim();
62             return res;
63         }
64         return -(v - *this);
65     }
66     return *this + (-v);
67 }
68 void operator*=(int v) {
69     check(v);
70     for (int i = 0, carry = 0; i < (int)a.size() || carry; ++i) {
71         if (i == (int)a.size()) {
72             a.push_back(0);
73         }
74         long long cur = a[i] * (long long)v + carry;
75         carry = (int)(cur / base);
76         a[i] = (int)(cur % base);
77     }
78     trim();
79 }
80 void operator/=(int v) {
81     check(v);
82     for (int i = (int)a.size() - 1, rem = 0; i >= 0; --i) {
83         long long cur = a[i] + rem * (long long)base;
84         a[i] = (int)(cur / v);
85         rem = (int)(cur % v);
86     }
87     trim();
88 }
89 int operator%(int v) const {
90     if (v < 0) {
91         v = -v;
92     }
93     int m = 0;
94     for (int i = a.size() - 1; i >= 0; --i) {
95         m = (a[i] + m * (long long)base) % v;
96     }
97     return m * sign;
98 }
99
100 void operator+=(const bigint &v) {
101     *this = *this + v;
102 }

```

```

103 void operator--(const bigint &v) {
104     *this = *this - v;
105 }
106 bigint operator*(int v) const {
107     bigint res = *this;
108     res *= v;
109     return res;
110 }
111 bigint operator/(int v) const {
112     bigint res = *this;
113     res /= v;
114     return res;
115 }
116 void operator%=(const int &v) {
117     *this = *this % v;
118 }
119
120 bool operator<(const bigint &v) const {
121     if (sign != v.sign) return sign < v.sign;
122     if (a.size() != v.a.size()) return a.size() * sign < v.a.size() * v.sign;
123     for (int i = a.size() - 1; i >= 0; i--)
124         if (a[i] != v.a[i]) return a[i] * sign < v.a[i] * sign;
125     return false;
126 }
127 bool operator>(const bigint &v) const {
128     return v < *this;
129 }
130 bool operator<=(const bigint &v) const {
131     return !(v < *this);
132 }
133 bool operator>=(const bigint &v) const {
134     return !(*this < v);
135 }
136 bool operator==(const bigint &v) const {
137     return !(*this < v) && !(v < *this);
138 }
139 bool operator!=(const bigint &v) const {
140     return *this < v || v < *this;
141 }
142
143 bigint abs() const {
144     bigint res = *this;
145     res.sign *= res.sign;
146     return res;
147 }
148 void check(int v) { // 检查输入的是否为负数
149     if (v < 0) {
150         sign = -sign;
151         v = -v;
152     }
153 }
154 void trim() { // 去除前导零
155     while (!a.empty() && !a.back()) a.pop_back();
156     if (a.empty()) sign = 1;
157 }
158 bool isZero() const { // 判断是否等于零
159     return a.empty() || (a.size() == 1 && !a[0]);
160 }
161 friend bigint gcd(const bigint &a, const bigint &b) {
162     return b.isZero() ? a : gcd(b, a % b);
163 }
164 friend bigint lcm(const bigint &a, const bigint &b) {
165     return a / gcd(a, b) * b;
166 }

```

```

167 void read(const string &s) {
168     sign = 1;
169     a.clear();
170     int pos = 0;
171     while (pos < (int)s.size() && (s[pos] == '-' || s[pos] == '+')) {
172         if (s[pos] == '-') sign = -sign;
173         ++pos;
174     }
175     for (int i = s.size() - 1; i >= pos; i -= base_digits) {
176         int x = 0;
177         for (int j = max(pos, i - base_digits + 1); j <= i; j++) x = x * 10 +
s[j] - '0';
178         a.push_back(x);
179     }
180     trim();
181 }
182 friend istream &operator>>(istream &stream, bigint &v) {
183     string s;
184     stream >> s;
185     v.read(s);
186     return stream;
187 }
188 friend ostream &operator<<(ostream &stream, const bigint &v) {
189     if (v.sign == -1) stream << '-';
190     stream << (v.a.empty() ? 0 : v.a.back());
191     for (int i = (int)v.a.size() - 2; i >= 0; --i)
192         stream << setw(base_digits) << setfill('0') << v.a[i];
193     return stream;
194 }
195
196 /* 大整数乘除大整数部分 */
197 typedef vector<long long> vll;
198 bigint operator*(const bigint &v) const { // 大整数乘大整数
199     vector<int> a6 = convert_base(this->a, base_digits, 6);
200     vector<int> b6 = convert_base(v.a, base_digits, 6);
201     vll a(a6.begin(), a6.end());
202     vll b(b6.begin(), b6.end());
203     while (a.size() < b.size()) a.push_back(0);
204     while (b.size() < a.size()) b.push_back(0);
205     while (a.size() & (a.size() - 1)) a.push_back(0), b.push_back(0);
206     vll c = karatsubaMultiply(a, b);
207     bigint res;
208     res.sign = sign * v.sign;
209     for (int i = 0, carry = 0; i < (int)c.size(); i++) {
210         long long cur = c[i] + carry;
211         res.a.push_back((int)(cur % 1000000));
212         carry = (int)(cur / 1000000);
213     }
214     res.a = convert_base(res.a, 6, base_digits);
215     res.trim();
216     return res;
217 }
218 friend pair<bigint, bigint> divmod(const bigint &a1,
219                                 const bigint &b1) { // 大整数除大整数, 同时返

```

答案与余数

```

220     int norm = base / (b1.a.back() + 1);
221     bigint a = a1.abs() * norm;
222     bigint b = b1.abs() * norm;
223     bigint q, r;
224     q.a.resize(a.a.size());
225     for (int i = a.a.size() - 1; i >= 0; i--) {
226         r *= base;
227         r += a.a[i];
228         int s1 = r.a.size() <= b.a.size() ? 0 : r.a[b.a.size()];

```

```

229         int s2 = r.a.size() <= b.a.size() - 1 ? 0 : r.a[b.a.size() - 1];
230         int d = ((long long)base * s1 + s2) / b.a.back();
231         r -= b * d;
232         while (r < 0) r += b, --d;
233         q.a[i] = d;
234     }
235     q.sign = a1.sign * b1.sign;
236     r.sign = a1.sign;
237     q.trim();
238     r.trim();
239     return make_pair(q, r / norm);
240 }
241 static vector<int> convert_base(const vector<int> &a, int old_digits, int
new_digits) {
242     vector<long long> p(max(old_digits, new_digits) + 1);
243     p[0] = 1;
244     for (int i = 1; i < (int)p.size(); i++) p[i] = p[i - 1] * 10;
245     vector<int> res;
246     long long cur = 0;
247     int cur_digits = 0;
248     for (int i = 0; i < (int)a.size(); i++) {
249         cur += a[i] * p[cur_digits];
250         cur_digits += old_digits;
251         while (cur_digits >= new_digits) {
252             res.push_back((int)(cur % p[new_digits]));
253             cur /= p[new_digits];
254             cur_digits -= new_digits;
255         }
256     }
257     res.push_back((int)cur);
258     while (!res.empty() && !res.back()) res.pop_back();
259     return res;
260 }
261 static vll karatsubaMultiply(const vll &a, const vll &b) {
262     int n = a.size();
263     vll res(n + n);
264     if (n <= 32) {
265         for (int i = 0; i < n; i++) {
266             for (int j = 0; j < n; j++) {
267                 res[i + j] += a[i] * b[j];
268             }
269         }
270         return res;
271     }
272
273     int k = n >> 1;
274     vll a1(a.begin(), a.begin() + k);
275     vll a2(a.begin() + k, a.end());
276     vll b1(b.begin(), b.begin() + k);
277     vll b2(b.begin() + k, b.end());
278
279     vll a1b1 = karatsubaMultiply(a1, b1);
280     vll a2b2 = karatsubaMultiply(a2, b2);
281
282     for (int i = 0; i < k; i++) a2[i] += a1[i];
283     for (int i = 0; i < k; i++) b2[i] += b1[i];
284
285     vll r = karatsubaMultiply(a2, b2);
286     for (int i = 0; i < (int)a1b1.size(); i++) r[i] -= a1b1[i];
287     for (int i = 0; i < (int)a2b2.size(); i++) r[i] -= a2b2[i];
288
289     for (int i = 0; i < (int)r.size(); i++) res[i + k] += r[i];
290     for (int i = 0; i < (int)a1b1.size(); i++) res[i] += a1b1[i];
291     for (int i = 0; i < (int)a2b2.size(); i++) res[i + n] += a2b2[i];

```

```

292         return res;
293     }
294
295     void operator*=(const bigint &v) {
296         *this = *this * v;
297     }
298     bigint operator/(const bigint &v) const {
299         return divmod(*this, v).first;
300     }
301     void operator/=(const bigint &v) {
302         *this = *this / v;
303     }
304     bigint operator%(const bigint &v) const {
305         return divmod(*this, v).second;
306     }
307     void operator%=(const bigint &v) {
308         *this = *this % v;
309     }
310 };

```

8.20 常见结论

题意：（区间移位问题）要求将整个序列左移/右移若干个位置，例如，原序列为 $A = (a_1, a_2, \dots, a_n)$ ，右移 x 位后变为 $A = (a_{x+1}, a_{x+2}, \dots, a_n, a_1, a_2, \dots, a_x)$ 。

区间的端点只是一个数字，即使被改变了，通过一定的转换也能够还原，所以我们可以 $\mathcal{O}(1)$ 解决这一问题。为了方便计算，我们规定下标从 0 开始，即整个线段的区间为 $[0, n)$ ，随后，使用一个偏移量 `shift` 记录。使用 `shift = (shift + x) % n;` 更新偏移量；此后的区间查询/修改前，再将坐标偏移回去即可，下方代码使用区间修改作为示例。

```

1  cin >> l >> r >> x;
2  l--; // 坐标修改为 0 开始
3  r--;
4  l = (l + shift) % n; // 偏移
5  r = (r + shift) % n;
6  if (l > r) { // 区间分离则分别操作
7      segt.modify(l, n - 1, x);
8      segt.modify(0, r, x);
9  } else {
10     segt.modify(l, r, x);
11 }

```

8.21 常见例题

题意：（带修莫队 - 维护队列）要求能够处理以下操作：

- `'Q' l r`：询问区间 $[l, r]$ 有几个颜色；
- `'R' idx w`：将下标 `idx` 的颜色修改为 `w`。

输入格式为：第一行 n 和 q ($1 \leq n, q \leq 133333$) 分别代表区间长度和操作数量；第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^6$) 代表初始颜色；随后 q 行为具体操作。

```

1  const int N = 1e6 + 7;
2  signed main() {
3      int n, q;
4      cin >> n >> q;
5      vector<int> w(n + 1);
6      for (int i = 1; i <= n; i++) {
7          cin >> w[i];

```

```

8     }
9
10    vector<array<int, 4>> query = {{}}; // {左区间, 右区间, 累计修改次数, 下标}
11    vector<array<int, 2>> modify = {{}}; // {修改的值, 修改的元素下标}
12    for (int i = 1; i <= q; i++) {
13        char op;
14        cin >> op;
15        if (op == 'Q') {
16            int l, r;
17            cin >> l >> r;
18            query.push_back({l, r, (int)modify.size() - 1, (int)query.size()});
19        } else {
20            int idx, w;
21            cin >> idx >> w;
22            modify.push_back({w, idx});
23        }
24    }
25
26    int Knum = 2154; // 计算块长
27    vector<int> K(n + 1);
28    for (int i = 1; i <= n; i++) { // 固定块长
29        K[i] = (i - 1) / Knum + 1;
30    }
31    sort(query.begin() + 1, query.end(), [&](auto x, auto y) {
32        if (K[x[0]] != K[y[0]]) return x[0] < y[0];
33        if (K[x[1]] != K[y[1]]) return x[1] < y[1];
34        return x[3] < y[3];
35    });
36
37    int l = 1, r = 0, val = 0;
38    int t = 0; // 累计修改次数
39    vector<int> ans(query.size(), cnt(N));
40    for (int i = 1; i < query.size(); i++) {
41        auto [ql, qr, qt, id] = query[i];
42        auto add = [&](int x) -> void {
43            if (cnt[x] == 0) ++ val;
44            ++ cnt[x];
45        };
46        auto del = [&](int x) -> void {
47            -- cnt[x];
48            if (cnt[x] == 0) -- val;
49        };
50        auto time = [&](int x, int l, int r) -> void {
51            if (l <= modify[x][1] && modify[x][1] <= r) { //当修改的位置在询问期间内部
时才会改变num的值
52                del(w[modify[x][1]]);
53                add(modify[x][0]);
54            }
55            swap(w[modify[x][1]], modify[x][0]); //直接交换修改数组的值与原始值, 减少额
外的数组开销, 且方便复原
56        };
57        while (l > ql) add(w[--l]);
58        while (r < qr) add(w[++r]);
59        while (l < ql) del(w[l++]);
60        while (r > qr) del(w[r--]);
61        while (t < qt) time(++t, ql, qr);
62        while (t > qt) time(t--, ql, qr);
63        ans[id] = val;
64    }
65    for (int i = 1; i < ans.size(); i++) {
66        cout << ans[i] << endl;
67    }
68 }

```

/END/

9 动态规划

9.1 01背包

有 n 件物品和一个容量为 W 的背包，第 i 件物品的体积为 $w[i]$ ，价值为 $v[i]$ ，求解将哪些物品装入背包中使总价值最大。

思路：

当放入一个体积为 $w[i]$ 的物品后，价值增加了 $v[i]$ ，于是我们可以构建一个二维的 $dp[i][j]$ 数组，装入第 i 件物品时，背包容量为 j 能实现的 **最大价值**，可以得到 **转移方程**
 $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w[i]] + v[i])$ 。

```
1 for (int i = 1; i <= n; i++)
2     for (int j = 0; j <= W; j++){
3         dp[i][j] = dp[i-1][j];
4         if (j >= w[i])
5             dp[i][j] = max(dp[i][j], dp[i-1][j-w[i]] + v[i]);
6     }
```

我们可以发现，第 i 个物品的状态是由第 $i-1$ 个物品转移过来的，每次的 j 转移过来后，第 $i-1$ 个方程的 j 已经没用了，于是我们想到可以把二维方程压缩成 **一维** 的，用以 **优化空间复杂度**。

```
1 for (int i = 1; i <= n; i++) //当前装第 i 件物品
2     for (int j = W; j >= w[i]; j--) //背包容量为 j
3         dp[j] = max(dp[j], dp[j-w[i]] + v[i]); //判断背包容量为 j 的情况下能是实现总
    价值最大是多少
```

9.2 完全背包

有 n 件物品和一个容量为 W 的背包，第 i 件物品的体积为 $w[i]$ ，价值为 $v[i]$ ，每件物品有**无限个**，求解将哪些物品装入背包中使总价值最大。

思路：

思路 and 01背包差不多，但是每一件物品有**无限个**，其实就是从每**种**物品中取 $0, 1, 2, \dots$ 件物品加入背包中

```
1 for (int i = 1; i <= n; i++)
2     for (int j = 0; j <= W; j++)
3         for (int k = 0; k * w[i] <= j; k++) //选取几个物品
4             dp[i][j] = max(dp[i][j], dp[i-1][j-k*w[i]] + k*v[i]);
```

实际上，我们可以发现，取 k 件物品可以从取 $k-1$ 件转移过来，那么我们就可以将 k 的循环优化掉

```
1 for (int i = 1; i <= n; i++)
2     for (int j = 0; j <= W; j++){
3         dp[i][j] = dp[i-1][j];
4         if (j >= w[i])
5             dp[i][j] = max(dp[i][j], dp[i][j-w[i]] + v[i]);
6     }
```

和 01 背包 类似地压缩成一维

```

1 for (int i = 1; i <= n; i++)
2     for (int j = w[i]; j <= W; j++)
3         dp[j] = max(dp[j], dp[j - w[i]] + v[i]);

```

9.3 多重背包

有 n 种物品和一个容量为 W 的背包，第 i 种物品的体积为 $w[i]$ ，价值为 $v[i]$ ，数量为 $s[i]$ ，求解将哪些物品装入背包中使总价值最大。

思路：

对于每一种物品，都有 $s[i]$ 种取法，我们可以将其转化为**01背包**问题

```

1 for (int i = 1; i <= n; i++){
2     for (int j = W; j >= 0; j--){
3         for (int k = 0; k <= s[i]; k++){
4             if (j - k * w[i] < 0) break;
5             dp[j] = max(dp[j], dp[j - k * w[i]] + k * v[i]);
6         }
7     }
8 }

```

上述方法的时间复杂度为 $O(n * m * s)$ 。

```

1 for (int i = 1; i <= n; i++){
2     scanf("%lld%lld%lld", &x, &y, &s); //x 为体积， y 为价值， s 为数量
3     t = 1;
4     while (s >= t){
5         w[++num] = x * t;
6         v[num] = y * t;
7         s -= t;
8         t *= 2;
9     }
10    w[++num] = x * s;
11    v[num] = y * s;
12 }
13 for (int i = 1; i <= num; i++)
14     for (int j = W; j >= w[i]; j--)
15         dp[j] = max(dp[j], dp[j - w[i]] + v[i]);

```

尽管采用了**二进制优化**，时间复杂度还是太高，采用**单调队列优化**，将时间复杂度优化至 $O(n * m)$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 2e5 + 10;
4 int n, W, w, v, s, f[N], g[N], q[N];
5 int main(){
6     ios::sync_with_stdio(false); cin.tie(0);
7     cin >> n >> W;
8     for (int i = 0; i < n; i++){
9         memcpy ( g, f, sizeof f);
10        cin >> w >> v >> s;
11        for (int j = 0; j < w; j++){
12            int head = 0, tail = -1;
13            for (int k = j; k <= W; k += w){
14                if ( head <= tail && k - s * w > q[head] ) head ++ ; //保证队列长度 <=
15                while ( head <= tail && g[q[tail]] - (q[tail] - j) / w * v <= g[k] -
16                (k - j) / w * v ) tail -- ; //保证队列单调递减
17                q[ ++ tail] = k;
18            }
19        }
20    }
21 }

```

```

17         f[k] = g[q[head]] + (k - q[head]) / w * v;
18     }
19 }
20 }
21 cout << f[W] << "\n";
22 return 0;
23 }

```

9.4 混合背包

放入背包的物品可能只有 **1** 件 (01背包)，也可能有**无限件** (完全背包)，也可能只有**可数的几件** (多重背包)。

思路：

分类讨论即可，哪一类就用哪种方法去 dp 。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int n, W, w, v, s;
4  int main(){
5      cin >> n >> W;
6      vector <int> f(W + 1);
7      for (int i = 0; i < n; i ++ ){
8          cin >> w >> v >> s;
9          if (s == -1){
10             for (int j = W; j >= w; j -- )
11                 f[j] = max(f[j], f[j - w] + v);
12         }
13         else if (s == 0){
14             for (int j = w; j <= W; j ++ )
15                 f[j] = max(f[j], f[j - w] + v);
16         }
17         else {
18             int t = 1, cnt = 0;
19             vector <int> x(s + 1), y(s + 1);
20             while (s >= t){
21                 x[++cnt] = w * t;
22                 y[cnt] = v * t;
23                 s -= t;
24                 t *= 2;
25             }
26             x[++cnt] = w * s;
27             y[cnt] = v * s;
28             for (int i = 1; i <= cnt; i ++ )
29                 for (int j = W; j >= x[i]; j -- )
30                     f[j] = max(f[j], f[j - x[i]] + y[i]);
31         }
32     }
33     cout << f[W] << "\n";
34     return 0;
35 }

```

9.5 二维费用的背包

有 n 件物品和一个容量为 W 的背包，背包能承受的最大重量为 M ，每件物品只能用一次，第 i 件物品的体积是 $w[i]$ ，重量为 $m[i]$ ，价值为 $v[i]$ ，求解将哪些物品放入背包中使总体积不超过背包容量，总重量不超过背包最大容量，且总价值最大。

思路：

背包的限制条件由一个变成两个，那么我们的循环再多一维即可。

```
1 for (int i = 1; i <= n; i++)
2     for (int j = W; j >= w; j--) //容量限制
3         for (int k = M; k >= m; k--) //重量限制
4             dp[j][k] = max(dp[j][k], dp[j - w][k - m] + v);
```

9.6 分组背包

有 n 组物品，一个容量为 W 的背包，每组物品有若干，同一组的物品最多选一个，第 i 组第 j 件物品的体积为 $w[i][j]$ ，价值为 $v[i][j]$ ，求解将哪些物品装入背包，可使物品总体积不超过背包容量，且使总价值最大。

思路：

考虑每组中的某件物品选不选，可以选的话，去下一组选下一个，否则在这组继续寻找可以选的物品，当这组遍历完后，去下一组寻找。

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 110;
4 int n, W, s[N], w[N][N], v[N][N], dp[N];
5 int main(){
6     cin >> n >> W;
7     for (int i = 1; i <= n; i++){
8         scanf("%d", &s[i]);
9         for (int j = 1; j <= s[i]; j++)
10             scanf("%d %d", &w[i][j], &v[i][j]);
11     }
12     for (int i = 1; i <= n; i++)
13         for (int j = W; j >= 0; j--)
14             for (int k = 1; k <= s[i]; k++)
15                 if (j - w[i][k] >= 0)
16                     dp[j] = max(dp[j], dp[j - w[i][k]] + v[i][k]);
17     cout << dp[W] << "\n";
18     return 0;
19 }
```

9.7 有依赖的背包

有 n 个物品和一个容量为 W 的背包，物品之间有依赖关系，且之间的依赖关系组成一颗 **树** 的形状，如果选择一个物品，则必须选择它的 **父节点**，第 i 件物品的体积是 $w[i]$ ，价值为 $v[i]$ ，依赖的父节点的编号为 $p[i]$ ，若 $p[i]$ 等于 -1，则为 **根节点**。求将哪些物品装入背包中，使总体积不超过总容量，且总价值最大。

思路：

定义 $f[i][j]$ 为以第 i 个节点为根，容量为 j 的背包的最大价值。那么结果就是 $f[root][W]$ ，为了知道根节点的最大价值，得通过其子节点来更新。所以采用递归的方式。

对于每一个点，先将这个节点装入背包，然后找到剩余容量可以实现的最大价值，最后更新父节点的最大价值即可。

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 110;
4 int n, W, w[N], v[N], p, f[N][N], root;
5 vector <int> g[N];
6 void dfs(int u){
```

```

7   for (int i = w[u]; i <= W; i ++ )
8       f[u][i] = v[u];
9   for (auto v : g[u]){
10      dfs(v);
11      for (int j = W; j >= w[u]; j -- )
12          for (int k = 0; k <= j - w[u]; k ++ )
13              f[u][j] = max(f[u][j], f[u][j - k] + f[v][k]);
14  }
15  }
16  int main(){
17      cin >> n >> W;
18      for (int i = 1; i <= n; i ++ ){
19          cin >> w[i] >> v[i] >> p;
20          if (p == -1) root = i;
21          else g[p].push_back(i);
22      }
23      dfs(root);
24      cout << f[root][W] << "\n";
25      return 0;
26  }

```

9.8 背包问题求方案数

有 n 件物品和一个容量为 W 的背包，每件物品只能用一次，第 i 件物品的重量为 $w[i]$ ，价值为 $v[i]$ ，求解将哪些物品放入背包使总重量不超过背包容量，且总价值最大，输出 **最优选法的方案数**，答案可能很大，输出答案模 $10^9 + 7$ 的结果。

思路：

开一个储存方案数的数组 cnt ， $cnt[i]$ 表示容量为 i 时的 **方案数**，先将 cnt 的每一个值都初始化为 1，因为 **不装任何东西就是一种方案**，如果装入这件物品使总的价值 **更大**，那么装入后的方案数 **等于** 装之前的方案数，如果装入后总价值 **相等**，那么方案数就是 **二者之和**

```

1   #include <bits/stdc++.h>
2   using namespace std;
3   #define LL long long
4   const int mod = 1e9 + 7, N = 1010;
5   LL n, W, cnt[N], f[N], w, v;
6   int main(){
7       cin >> n >> W;
8       for (int i = 0; i <= W; i ++ )
9           cnt[i] = 1;
10      for (int i = 0; i < n; i ++ ){
11          cin >> w >> v;
12          for (int j = W; j >= w; j -- )
13              if (f[j] < f[j - w] + v){
14                  f[j] = f[j - w] + v;
15                  cnt[j] = cnt[j - w];
16              }
17              else if (f[j] == f[j - w] + v){
18                  cnt[j] = (cnt[j] + cnt[j - w]) % mod;
19              }
20      }
21      cout << cnt[W] << "\n";
22      return 0;
23  }

```

9.9 背包问题求具体方案

有 n 件物品和一个容量为 W 的背包，每件物品只能用一次，第 i 件物品的重量为 $w[i]$ ，价值为 $v[i]$ ，求解将哪些物品放入背包使总重量不超过背包容量，且总价值最大，输出 **字典序最小的方案**

思路：

01 背包求解最优方案中 **字典序最小的方案**，首先我们先求 **01背包**，因为这道题需要输出方案，所以我们 **不能压缩空间**，得保留每一步的方案。

又由于输出字典序最小的，所以我们应该反过来，从 n 到 1 求解最优解，那么 $dp[1][W]$ 就是最优的解。

```
1 for (int i = n; i >= 1; i--)
2     for (int j = 0; j <= W; j++){
3         dp[i][j] = dp[i + 1][j];
4         if (j >= w[i])
5             dp[i][j] = max(dp[i][j], dp[i + 1][j - w[i]] + v[i]);
6     }
```

接下来 就是输出的问题，如何判断这个物品**被选中**，如果 $dp[i][k] = dp[i + 1][k - w[i]] + v[i]$ ，说明选择了第 i 个物品是最优的选择方案。

```
1 for (int i = 1; i <= n; i++)
2     if (W - w[i] >= 0 && dp[i][W] == dp[i + 1][W - w[i]] + v[i]){
3         cout << i << " ";
4         W -= w[i];
5     }
```

9.10 数位 DP

```
1  /* pos 表示当前枚举到第几位
2  sum 表示 d 出现的次数
3  limit 为 1 表示枚举的数字有限制
4  zero 为 1 表示有前导 0
5  d 表示要计算出出现次数的数 */
6  const int N = 15;
7  LL dp[N][N];
8  int num[N];
9  LL dfs(int pos, LL sum, int limit, int zero, int d) {
10     if (pos == 0) return sum;
11     if (!limit && !zero && dp[pos][sum] != -1) return dp[pos][sum];
12     LL ans = 0;
13     int up = (limit ? num[pos] : 9);
14     for (int i = 0; i <= up; i++) {
15         ans += dfs(pos - 1, sum + ((!zero || i) && (i == d)), limit && (i ==
num[pos]),
16                 zero && (i == 0), d);
17     }
18     if (!limit && !zero) dp[pos][sum] = ans;
19     return ans;
20 }
21 LL solve(LL x, int d) {
22     memset(dp, -1, sizeof dp);
23     int len = 0;
24     while (x) {
25         num[++len] = x % 10;
26         x /= 10;
27     }
28     return dfs(len, 0, 1, 1, d);
```

9.11 状压 DP

题意：在 $n * n$ 的棋盘里面放 k 个国王，使他们互不攻击，共有多少种摆放方案。国王能攻击到它上下左右，以及左上左下右上右下八个方向上附近的各一个格子，共8个格子。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define LL long long
4  const int N = 15, M = 150, K = 1500;
5  LL n, k;
6  LL cnt[K];    //每个状态的二进制中 1 的数量
7  LL tot;       //合法状态的数量
8  LL st[K];     //合法的状态
9  LL dp[N][M][K]; //第 i 行, 放置了 j 个国王, 状态为 k 的方案数
10 int main(){
11     ios::sync_with_stdio(false); cin.tie(0);
12     cin >> n >> k;
13     for (int s = 0; s < (1 << n); s ++ ){ //找出合法状态
14         LL sum = 0, t = s;
15         while(t){ //计算 1 的数量
16             sum += (t & 1);
17             t >>= 1;
18         }
19         cnt[s] = sum;
20         if ( ((s << 1) | (s >> 1)) & s) == 0 ){ //判断合法性
21             st[ ++ tot] = s;
22         }
23     }
24     dp[0][0][0] = 1;
25     for (int i = 1; i <= n + 1; i ++ ){
26         for (int j1 = 1; j1 <= tot; j1 ++ ){ //当前的状态
27             LL s1 = st[j1];
28             for (int j2 = 1; j2 <= tot; j2 ++ ){ //上一行的状态
29                 LL s2 = st[j2];
30                 if ( ((s2 | (s2 << 1) | (s2 >> 1)) & s1) == 0 ){
31                     for (int j = 0; j <= k; j ++ ){
32                         if (j - cnt[s1] >= 0)
33                             dp[i][j][s1] += dp[i - 1][j - cnt[s1]][s2];
34                     }
35                 }
36             }
37         }
38     }
39     cout << dp[n + 1][k][0] << "\n";
40     return 0;
41 }

```

9.12 常用例题

题意：在一篇文章（包含大小写英文字母、数字、和空白字符（制表/空格/回车））中寻找 helloworld（任意一个字母的大小写都行）的子序列出现了多少次，输出结果对 $10^9 + 7$ 的余数。

字符串 DP，构建一个二维 DP 数组， $dp[i][j]$ 的 i 表示文章中的第几个字符， j 表示寻找的字符串的第几个字符，当字符串中的字符和文章中的字符相同时，即找到符合条件的字符， $dp[i][j] = dp[i - 1][j] + dp[i - 1][j - 1]$ ，因为字符串中的每个字符不会对后面的结果产生影响，所以 DP 方程可以优化成一维的，由于字符串中有重复的字符，所以比较时应该从后往前。

```

1  #include <bits/stdc++.h>

```

```

2 using namespace std;
3 #define LL long long
4 const int mod = 1e9 + 7;
5 char c, s[20] = "!helloworld";
6 LL dp[20];
7 int main(){
8     dp[0] = 1;
9     while ((c = getchar()) != EOF)
10         for (int i = 10; i >= 1; i--)
11             if (c == s[i] || c == s[i] - 32)
12                 dp[i] = (dp[i] + dp[i - 1]) % mod;
13     cout << dp[10] << "\n";
14     return 0;
15 }

```

题意：（最长括号匹配）给一个只包含‘(’, ‘)’, ‘[’, ‘]’的非空字符串，“()”和“[]”是匹配的，寻找字符串中最长的括号匹配的子串，若有两串长度相同，输出靠前的一串。

设给定的字符串为 s ，可以定义数组 $dp[i]$ ， $dp[i]$ 表示以 $s[i]$ 结尾的字符串里最长的括号匹配的字符。显然，从 $i - dp[i] + 1$ 到 i 的字符串是括号匹配的，当找到一个字符是‘)’或‘]’时，再去判断第 $i - 1 - dp[i - 1]$ 的字符和第 i 位的字符是否匹配，如果是，那么 $dp[i] = dp[i - 1] + 2 + dp[i - 2 - dp[i - 1]]$

。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int maxn = 1e6 + 10;
4 string s;
5 int len, dp[maxn], ans, id;
6 int main(){
7     cin >> s;
8     len = s.length();
9     for (int i = 1; i < len; i++){
10         if ((s[i] == '(' && s[i - 1 - dp[i - 1]] == ')') || (s[i] == ']' && s[i - 1 - dp[i - 1]] == '[')){
11             dp[i] = dp[i - 1] + 2 + dp[i - 2 - dp[i - 1]];
12             if (dp[i] > ans) {
13                 ans = dp[i]; //记录长度
14                 id = i; //记录位置
15             }
16         }
17     }
18     for (int i = id - ans + 1; i <= id; i++)
19         cout << s[i];
20     cout << "\n";
21     return 0;
22 }

```

题意：去掉区间内包含“4”和“62”的数字，输出剩余的数字个数

```

1 int T, n, m, len, a[20]; //a数组用于判断每一位能取到的最大值
2 ll l, r, dp[20][15];
3 ll dfs(int pos, int pre, int limit) { //记搜
4     //pos搜到的位置, pre前一位数
5     //limit判断是否有最高位限制
6     if (pos > len) return 1; //剪枝
7     if (dp[pos][pre] != -1 && !limit) return dp[pos][pre]; //记录当前值
8     ll ret = 0; //暂时记录当前方案数
9     int res = limit ? a[len - pos + 1] : 9; //res当前位能取到的最大值
10    for (int i = 0; i <= res; i++)

```

```

11         if(!(i==4 || (pre==6 && i==2)))
12             ret+=dfs(pos+1,i,i==res&&limit);
13         if(!limit) dp[pos][pre]=ret;//当前状态方案数记录
14         return ret;
15     }
16     ll part(ll x){//把数按位拆分
17         len=0;
18         while(x) a[++len]=x%10,x/=10;
19         memset(dp,-1,sizeof dp);//初始化-1 (因为有可能某些情况下的方案数是0)
20         return dfs(1,0,1);//进入记搜
21     }
22     int main(){
23         cin>>n;
24         while(n--){
25             cin>>l>>r;
26             if(l==0 && r==0)break;
27             if(l) printf("%lld\n",part(r)-part(l-1));//[1,r](l!=0)
28             else printf("%lld\n",part(r)-part(1));//从0开始要特判
29         }
30     }

```

/END/

10 串

10.1 子串与子序列

中文名称	常见英文名称	解释	常见数学描述
子串	substring	从母串中取出连续的部分，构成新串	$x = s[a \dots b] = s_{as_{a+1} \dots s_b} \ (1 \leq a \leq b \leq s)$
子序列	subsequence	从母串中随机取出部分，连接构成新串	从母串 s 中删除若干个（有可能是 0 个或全部）元素

10.2 字符串模式匹配算法 KMP

应用：

1. 在字符串中查找子串；
2. 最小周期：字符串长度-整个字符串的 border；
3. 最小循环节：区别于周期，当字符串长度 $n \bmod (n - \text{nxt}[n]) = 0$ 时，等于最小周期，否则为 n 。

10.2.1 全串匹配

```
1 auto KMP = [&](string s, string t) -> void {
2     int n = s.size(), m = t.size();
3     vector<int> nxt(m + 1);
4     s = '-' + s;
5     t = '-' + t;
6     for (int i = 2, j = 0; i <= m; i++) {
7         while (j && t[i] != t[j + 1]) j = nxt[j];
8         if (t[i] == t[j + 1]) j++;
9         nxt[i] = j;
10    }
11    for (int i = 1, j = 0; i <= n; i++) {
12        while (j && s[i] != t[j + 1]) j = nxt[j];
13        if (s[i] == t[j + 1]) j++;
14        if (j == m) {
15            cout << i - m + 1 << "\n"; // t 在 s 中出现的位置
16            j = nxt[j];
17        }
18    }
19 };
```

10.2.2 统计原串中某个子串重复出现的次数（例题）

给定一个字符串 S ，求解其某个子串 $S[l \dots r]$ 独立/非独立出现的次数。例如，对于原串 $S = \text{'aaaaa'}$ ，询问 $S[1 \dots 2]$ 独立出现的次数答案为 2，非独立答案为 4，需要注意原串需要添加前缀空白字符，下标从 1 开始，其本质是模板的带未知变量的扩展版本。

```
1 auto KMP = [&](int l, int r) -> int {
2     int ans = 0;
3     vector<int> nxt(r + 1);
4     nxt[1] = l - 1;
5     for (int i = l + 1, j = l - 1; i <= r; i++) {
6         while (j != l - 1 && s[i] != s[j + 1]) {
7             j = nxt[j];
8         }
9         if (s[i] == s[j + 1]) j++;
10    }
```

```

10     nxt[i] = j;
11 }
12 for (int i = 1, j = 1 - 1; i <= n; i++) {
13     while (j != 1 - 1 && s[i] != s[j + 1]) {
14         j = nxt[j];
15     }
16     if (s[i] == s[j + 1]) j++;
17     if (j == r) {
18         ans++;
19         j = 1 - 1; // 独立
20         j = nxt[j]; // 非独立
21     }
22 }
23 return ans;
24 };

```

10.3 Z函数 (扩展 KMP)

获取字符串 s 和 $s[i, n - 1]$ (即以 $s[i]$ 开头的后缀) 的最长公共前缀 (LCP) 的长度, 总复杂度 $\mathcal{O}(N)$ 。

```

1 vector<int> zFunction(string s) {
2     int n = s.size();
3     vector<int> z(n);
4     z[0] = n;
5     for (int i = 1, j = 1; i < n; i++) {
6         z[i] = max(0, min(j + z[j] - i, z[i - j]));
7         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
8             z[i]++;
9         }
10        if (i + z[i] > j + z[j]) {
11            j = i;
12        }
13    }
14    return z;
15 }

```

10.4 最长公共子序列 LCS

求解两个串的最长公共子序列的长度。

10.4.1 小数据解

针对 10^3 以内的数据。

```

1 const int N = 1e3 + 10;
2 char a[N], b[N];
3 int n, m, f[N][N];
4 void solve(){
5     cin >> n >> m >> a + 1 >> b + 1;
6     for (int i = 1; i <= n; i++)
7         for (int j = 1; j <= m; j++){
8             f[i][j] = max(f[i - 1][j], f[i][j - 1]);
9             if (a[i] == b[j]) f[i][j] = max(f[i][j], f[i - 1][j - 1] + 1);
10        }
11    cout << f[n][m] << "\n";
12 }
13 int main(){
14     solve();
15     return 0;

```

```
16 }
```

10.4.2 大数据解

针对 10^5 以内的数据。

```
1  const int INF = 0x7fffffff;
2  int n, a[maxn], b[maxn], f[maxn], p[maxn];
3  int main(){
4      cin >> n;
5      for (int i = 1; i <= n; i++){
6          scanf("%d", &a[i]);
7          p[a[i]] = i; //将第二个序列中的元素映射到第一个中
8      }
9      for (int i = 1; i <= n; i++){
10         scanf("%d", &b[i]);
11         f[i] = INF;
12     }
13     int len = 0;
14     f[0] = 0;
15     for (int i = 1; i <= n; i++){
16         if (p[b[i]] > f[len]) f[++len] = p[b[i]];
17         else {
18             int l = 0, r = len;
19             while (l < r){
20                 int mid = (l + r) >> 1;
21                 if (f[mid] > p[b[i]]) r = mid;
22                 else l = mid + 1;
23             }
24             f[l] = min(f[l], p[b[i]]);
25         }
26     }
27     cout << len << "\n";
28     return 0;
29 }
```

10.5 字符串哈希

10.5.1 双哈希封装

随机质数列表：1111111121、1211111123、1311111119。

```
1  const int N = 1 << 21;
2  static const int mod1 = 1E9 + 7, base1 = 127;
3  static const int mod2 = 1E9 + 9, base2 = 131;
4  using U = Zmod<mod1>;
5  using V = Zmod<mod2>;
6  vector<U> val1;
7  vector<V> val2;
8  void init(int n = N) {
9      val1.resize(n + 1), val2.resize(n + 2);
10     val1[0] = 1, val2[0] = 1;
11     for (int i = 1; i <= n; i++) {
12         val1[i] = val1[i - 1] * base1;
13         val2[i] = val2[i - 1] * base2;
14     }
15 }
16 struct String {
17     vector<U> hash1;
18     vector<V> hash2;
19     string s;
```

```

20
21 String(string s_) : s(s_), hash1{1}, hash2{1} {
22     for (auto it : s) {
23         hash1.push_back(hash1.back() * base1 + it);
24         hash2.push_back(hash2.back() * base2 + it);
25     }
26 }
27 pair<U, V> get() { // 输出整串的哈希值
28     return {hash1.back(), hash2.back()};
29 }
30 pair<U, V> substring(int l, int r) { // 输出子串的哈希值
31     if (l > r) swap(l, r);
32     U ans1 = hash1[r + 1] - hash1[l] * val1[r - l + 1];
33     V ans2 = hash2[r + 1] - hash2[l] * val2[r - l + 1];
34     return {ans1, ans2};
35 }
36 pair<U, V> modify(int idx, char x) { // 修改 idx 位为 x
37     int n = s.size() - 1;
38     U ans1 = hash1.back() + val1[n - idx] * (x - s[idx]);
39     V ans2 = hash2.back() + val2[n - idx] * (x - s[idx]);
40     return {ans1, ans2};
41 }
42 };

```

10.5.2 前后缀去重

sample please ease 去重后得到 samplease 。

```

1 string compress(vector<string> in) { // 前后缀压缩
2     vector<U> hash1{1};
3     vector<V> hash2{1};
4     string ans = "#";
5     for (auto s : in) {
6         s = "#" + s;
7         int st = 0;
8         U chk1 = 0;
9         V chk2 = 0;
10        for (int j = 1; j < s.size() && j < ans.size(); j++) {
11            chk1 = chk1 * base1 + s[j];
12            chk2 = chk2 * base2 + s[j];
13            if ((hash1.back() == hash1[ans.size() - 1 - j] * val1[j] + chk1) &&
14                (hash2.back() == hash2[ans.size() - 1 - j] * val2[j] + chk2)) {
15                st = j;
16            }
17        }
18        for (int j = st + 1; j < s.size(); j++) {
19            ans += s[j];
20            hash1.push_back(hash1.back() * base1 + s[j]);
21            hash2.push_back(hash2.back() * base2 + s[j]);
22        }
23    }
24    return ans.substr(1);
25 }

```

10.6 马拉车

$O(N)$ 时间求出字符串的最长回文子串。

```

1 string s;
2 cin >> s;
3 int n = s.length();

```

```

4  string t = "-#";
5  for (int i = 0; i < n; i++) {
6      t += s[i];
7      t += '#';
8  }
9  int m = t.length();
10 t += '+';
11 int mid = 0, r = 0;
12 vector<int> p(m);
13 for (int i = 1; i < m; i++) {
14     p[i] = i < r ? min(p[2 * mid - i], r - i) : 1;
15     while (t[i - p[i]] == t[i + p[i]]) p[i]++;
16     if (i + p[i] > r) {
17         r = i + p[i];
18         mid = i;
19     }
20 }

```

10.7 字典树 trie

10.7.1 基础封装

```

1  struct Trie {
2      int ch[N][63], cnt[N], idx = 0;
3      map<char, int> mp;
4      void init() {
5          LL id = 0;
6          for (char c = 'a'; c <= 'z'; c++) mp[c] = ++id;
7          for (char c = 'A'; c <= 'Z'; c++) mp[c] = ++id;
8          for (char c = '0'; c <= '9'; c++) mp[c] = ++id;
9      }
10     void insert(string s) {
11         int u = 0;
12         for (int i = 0; i < s.size(); i++) {
13             int v = mp[s[i]];
14             if (!ch[u][v]) ch[u][v] = ++idx;
15             u = ch[u][v];
16             cnt[u]++;
17         }
18     }
19     LL query(string s) {
20         int u = 0;
21         for (int i = 0; i < s.size(); i++) {
22             int v = mp[s[i]];
23             if (!ch[u][v]) return 0;
24             u = ch[u][v];
25         }
26         return cnt[u];
27     }
28     void Clear() {
29         for (int i = 0; i <= idx; i++) {
30             cnt[i] = 0;
31             for (int j = 0; j <= 62; j++) {
32                 ch[i][j] = 0;
33             }
34         }
35         idx = 0;
36     }
37 } trie;

```

10.7.2 01 字典树

```
1 struct Trie {
2     int n, idx;
3     vector<vector<int>> ch;
4     Trie(int n) {
5         this->n = n;
6         idx = 0;
7         ch.resize(30 * (n + 1), vector<int>(2));
8     }
9     void insert(int x) {
10         int u = 0;
11         for (int i = 30; ~i; i--) {
12             int &v = ch[u][x >> i & 1];
13             if (!v) v = ++idx;
14             u = v;
15         }
16     }
17     int query(int x) {
18         int u = 0, res = 0;
19         for (int i = 30; ~i; i--) {
20             int v = x >> i & 1;
21             if (ch[u][!v]) {
22                 res += (1 << i);
23                 u = ch[u][!v];
24             } else {
25                 u = ch[u][v];
26             }
27         }
28         return res;
29     }
30 };
```

10.8 后缀数组 SA

以 $O(N)$ 的复杂度求解。

```
1 struct SuffixArray {
2     int n;
3     vector<int> sa, rk, lc;
4     SuffixArray(const string &s) {
5         n = s.length();
6         sa.resize(n);
7         lc.resize(n - 1);
8         rk.resize(n);
9         iota(sa.begin(), sa.end(), 0);
10        sort(sa.begin(), sa.end(), [&](int a, int b) { return s[a] < s[b]; });
11        rk[sa[0]] = 0;
12        for (int i = 1; i < n; ++i) {
13            rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
14        }
15        int k = 1;
16        vector<int> tmp, cnt(n);
17        tmp.reserve(n);
18        while (rk[sa[n - 1]] < n - 1) {
19            tmp.clear();
20            for (int i = 0; i < k; ++i) {
21                tmp.push_back(n - k + i);
22            }
23            for (auto i : sa) {
24                if (i >= k) {
```

```

25         tmp.push_back(i - k);
26     }
27 }
28 fill(cnt.begin(), cnt.end(), 0);
29 for (int i = 0; i < n; ++i) {
30     ++cnt[rk[i]];
31 }
32 for (int i = 1; i < n; ++i) {
33     cnt[i] += cnt[i - 1];
34 }
35 for (int i = n - 1; i >= 0; --i) {
36     sa[--cnt[rk[tmp[i]]]] = tmp[i];
37 }
38 swap(rk, tmp);
39 rk[sa[0]] = 0;
40 for (int i = 1; i < n; ++i) {
41     rk[sa[i]] = rk[sa[i - 1]] + (tmp[sa[i - 1]] < tmp[sa[i]] || sa[i -
1] + k == n ||
42                                     tmp[sa[i - 1] + k] < tmp[sa[i] + k]);
43 }
44 k *= 2;
45 }
46 for (int i = 0, j = 0; i < n; ++i) {
47     if (rk[i] == 0) {
48         j = 0;
49         continue;
50     }
51     for (j -= j > 0;
52          i + j < n && sa[rk[i] - 1] + j < n && s[i + j] == s[sa[rk[i] - 1] +
j]); {
53         ++j;
54     }
55     lc[rk[i] - 1] = j;
56 }
57 }
58 };

```

10.9 AC 自动机

定义 $|s_i|$ 是模板串的长度, $|S|$ 是文本串的长度, $|\Sigma|$ 是字符集的大小 (常数, 一般为 26), 时间复杂度为 $O(\sum |s_i| + |S|)$ 。

```

1  // Trie+Kmp, 多模式串匹配
2  struct ACAutomaton {
3      static constexpr int N = 1e6 + 10;
4      int ch[N][26], fail[N], cntNodes;
5      int cnt[N];
6      ACAutomaton() {
7          cntNodes = 1;
8      }
9      void insert(string s) {
10         int u = 1;
11         for (auto c : s) {
12             int &v = ch[u][c - 'a'];
13             if (!v) v = ++cntNodes;
14             u = v;
15         }
16         cnt[u]++;
17     }
18     void build() {
19         fill(ch[0], ch[0] + 26, 1);
20         queue<int> q;

```

```

21     q.push(1);
22     while (!q.empty()) {
23         int u = q.front();
24         q.pop();
25         for (int i = 0; i < 26; i++) {
26             int &v = ch[u][i];
27             if (!v)
28                 v = ch[fail[u]][i];
29             else {
30                 fail[v] = ch[fail[u]][i];
31                 q.push(v);
32             }
33         }
34     }
35 }
36 LL query(string t) {
37     LL ans = 0;
38     int u = 1;
39     for (auto c : t) {
40         u = ch[u][c - 'a'];
41         for (int v = u; v && ~cnt[v]; v = fail[v]) {
42             ans += cnt[v];
43             cnt[v] = -1;
44         }
45     }
46     return ans;
47 }
48 };

```

10.10 回文自动机 PAM (回文树)

应用:

1. 本质不同的回文串个数: $idx - 2$;
2. 回文子串出现次数。

对于一个字符串 s , 它的本质不同回文子串个数最多只有 $|s|$ 个, 那么, 构造 s 的回文树的时间复杂度是 $\mathcal{O}(|s|)$ 。

```

1 struct PalindromeAutomaton {
2     constexpr static int N = 5e5 + 10;
3     int tr[N][26], fail[N], len[N];
4     int cntNodes, last;
5     int cnt[N];
6     string s;
7     PalindromeAutomaton(string s) {
8         memset(tr, 0, sizeof tr);
9         memset(fail, 0, sizeof fail);
10        len[0] = 0, fail[0] = 1;
11        len[1] = -1, fail[1] = 0;
12        cntNodes = 1;
13        last = 0;
14        this->s = s;
15    }
16    void insert(char c, int i) {
17        int u = get_fail(last, i);
18        if (!tr[u][c - 'a']) {
19            int v = ++cntNodes;
20            fail[v] = tr[get_fail(fail[u], i)][c - 'a'];
21            tr[u][c - 'a'] = v;
22            len[v] = len[u] + 2;

```

```

23         cnt[v] = cnt[fail[v]] + 1;
24     }
25     last = tr[u][c - 'a'];
26 }
27 int get_fail(int u, int i) {
28     while (i - len[u] - 1 <= -1 || s[i - len[u] - 1] != s[i]) {
29         u = fail[u];
30     }
31     return u;
32 }
33 };

```

10.11 后缀自动机 SAM

定义 $|\Sigma|$ 是字符集的大小，复杂度为 $\mathcal{O}(N \log |\Sigma|)$ 。

```

1 // 有向无环图
2 struct SuffixAutomaton {
3     static constexpr int N = 1e6;
4     struct node {
5         int len, link, nxt[26];
6         int siz;
7     } t[N << 1];
8     int cntNodes;
9     SuffixAutomaton() {
10         cntNodes = 1;
11         fill(t[0].nxt, t[0].nxt + 26, 1);
12         t[0].len = -1;
13     }
14     int extend(int p, int c) {
15         if (t[p].nxt[c]) {
16             int q = t[p].nxt[c];
17             if (t[q].len == t[p].len + 1) {
18                 return q;
19             }
20             int r = ++cntNodes;
21             t[r].siz = 0;
22             t[r].len = t[p].len + 1;
23             t[r].link = t[q].link;
24             copy(t[q].nxt, t[q].nxt + 26, t[r].nxt);
25             t[q].link = r;
26             while (t[p].nxt[c] == q) {
27                 t[p].nxt[c] = r;
28                 p = t[p].link;
29             }
30             return r;
31         }
32         int cur = ++cntNodes;
33         t[cur].len = t[p].len + 1;
34         t[cur].siz = 1;
35         while (!t[p].nxt[c]) {
36             t[p].nxt[c] = cur;
37             p = t[p].link;
38         }
39         t[cur].link = extend(p, c);
40         return cur;
41     }
42 };

```

/END/

11 博弈论

11.1 巴什博弈

有 N 个石子，两名玩家轮流行动，按以下规则取石子：

规定：每人每次可以取走 $X(1 \leq X \leq M)$ 个石子，拿到最后一颗石子的一方获胜。

双方均采用最优策略，询问谁会获胜。

两名玩家轮流报数。

规定：第一个报数的人可以报 $X(1 \leq X \leq M)$ ，后报数的人需要比前者所报数大 $Y(1 \leq Y \leq M)$ ，率先报到 N 的人获胜。

双方均采用最优策略，询问谁会获胜。

- $N = K \cdot (M + 1)$ (其中 $K \in \mathbb{N}^+$)，后手必胜 (后手可以控制每一回合结束时双方恰好取走 $M + 1$ 个，重复 K 轮后即胜利)；
- $N = K \cdot (M + 1) + R$ (其中 $K \in \mathbb{N}^+, 0 < R < M + 1$)，先手必胜 (先手先取走 R 个，之后控制每一回合结束时双方恰好取走 $M + 1$ 个，重复 K 轮后即胜利)。

11.2 扩展巴什博弈

有 N 颗石子，两名玩家轮流行动，按以下规则取石子：。

规定：每人每次可以取走 $X(a \leq X \leq b)$ 个石子，如果最后剩余物品的数量小于 a 个，则不能再取，拿到最后一颗石子的一方获胜。

双方均采用最优策略，询问谁会获胜。

- $N = K \cdot (a + b)$ 时，后手必胜；
- $N = K \cdot (a + b) + R_1$ (其中 $K \in \mathbb{N}^+, 0 < R_1 < a$) 时，后手必胜 (这些数量不够再取一次，先手无法逆转局面)；
- $N = K \cdot (a + b) + R_2$ (其中 $K \in \mathbb{N}^+, a \leq R_2 \leq b$) 时，先手必胜；
- $N = K \cdot (a + b) + R_3$ (其中 $K \in \mathbb{N}^+, b < R_3 < a + b$) 时，先手必胜 (这些数量不够再取一次，后手无法逆转局面)；

11.3 Nim 博弈

有 N 堆石子，给出每一堆的石子数量，两名玩家轮流行动，按以下规则取石子：

规定：每人每次任选一堆，取走正整数颗石子，拿到最后一颗石子的一方获胜 (注：几个特点是不能跨堆、不能不拿)。

双方均采用最优策略，询问谁会获胜。

记初始时各堆石子的数量 $(A_1, A_2, \dots, A_n)$ ，定义尼姆和 $Sum_N = A_1 \oplus A_2 \oplus \dots \oplus A_n$ 。

当 $Sum_N = 0$ 时先手必败，反之先手必胜。

11.4 Nim 游戏具体取法

先计算出尼姆和，再对每一堆石子计算 $A_i \oplus Sum_N$ ，记为 X_i 。

若得到的值 $X_i < A_i$ ， X_i 即为一个可行解，即剩下 X_i 颗石头，取走 $A_i - X_i$ 颗石头（这里取小于号是因为至少要取走 1 颗石子）。

11.5 Moore's Nim 游戏 (Nim-K 游戏)

有 N 堆石子，给出每一堆的石子数量，两名玩家轮流行动，按以下规则取石子：

规定：每人每次任选不超过 K 堆，对每堆都取走不同的正整数颗石子，拿到最后一颗石子的一方获胜。

双方均采用最优策略，询问谁会获胜。

把每一堆石子的石子数用二进制表示，定义 One_i 为二进制第 i 位上 1 的个数。

以下局面先手必胜：

对于每一位， $One_1, One_2, \dots, One_N$ 均不为 $K + 1$ 的倍数。

11.6 Anti-Nim 游戏 (反 Nim 游戏)

有 N 堆石子，给出每一堆的石子数量，两名玩家轮流行动，按以下规则取石子：

规定：每人每次任选一堆，取走正整数颗石子，拿到最后一颗石子的一方出局。

双方均采用最优策略，询问谁会获胜。

- 所有堆的石头数量均不超过 1，且 $Sum_N = 0$ （也可看作“且有偶数堆”）；
- 至少有一堆的石头数量大于 1，且 $Sum_N \neq 0$ 。

11.7 阶梯 - Nim 博弈

有 N 级台阶，每一级台阶上均有一定数量的石子，给出每一级石子的数量，两名玩家轮流行动，按以下规则操作石子：

规定：每人每次任选一级台阶，拿走正整数颗石子放到下一级台阶中，已经拿到地面上的石子不能再拿，拿到最后一颗石子的一方获胜。

双方均采用最优策略，询问谁会获胜。

对奇数台阶做传统 Nim 博弈，当 $Sum_N = 0$ 时先手必败，反之先手必胜。 **

11.8 SG 游戏 (有向图游戏)

我们使用以下几条规则来定义暴力求解的过程：

- 使用数字来表示输赢情况，0 代表局面必败，非 0 代表存在必胜可能，我们称这个数字为这个局面的 SG 值；
- 找到最终态，根据题意人为定义最终态的输赢情况；
- 对于非最终态的某个节点，其 SG 值为所有子节点的 SG 值取 mex；
- 单个游戏的输赢态即对应根节点的 SG 值是否为 0，为 0 代表先手必败，非 0 代表先手必胜；
- 多个游戏的总 SG 值为单个游戏 SG 值的异或和。

使用哈希表，以 $\mathcal{O}(N + M)$ 的复杂度计算。

```

1  int n, m, a[N], num[N];
2  int sg(int x) {
3      if (num[x] != -1) return num[x];
4
5      unordered_set<int> S;
6      for (int i = 1; i <= m; ++ i)
7          if(x >= a[i])
8              S.insert(sg(x - a[i]));
9
10     for (int i = 0; ; ++ i)
11         if (S.count(i) == 0)
12             return num[x] = i;
13 }
14 void Solve() {
15     cin >> m;
16     for (int i = 1; i <= m; ++ i) cin >> a[i];
17     cin >> n;
18
19     int ans = 0; memset(num, -1, sizeof num);
20     for (int i = 1; i <= n; ++ i) {
21         int x; cin >> x;
22         ans ^= sg(x);
23     }
24
25     if (ans == 0) no;
26     else yes;
27 }

```

11.9 Anti-SG 游戏 (反 SG 游戏)

SG 游戏中最先不能行动的一方获胜。

以下局面先手必胜：

- 单局游戏的SG值均不超过 1，且总SG值为 0；
- 至少有一局单局游戏的SG值大于 1，且总SG值不为 0。

在本质上，这与 Anti-Nim 游戏的结论一致。

11.10 Lasker's-Nim 游戏 (Multi-SG 游戏)

有 N 堆石子，给出每一堆的石子数量，两名玩家轮流行动，每人每次任选以下规定的一种操作石子：

- 任选一堆，取走正整数颗石子；
- 任选数量大于 2 的一堆，分成两堆非空石子。

拿到最后一颗石子的一方获胜。双方均采用最优策略，询问谁会获胜。

本题使用SG函数求解，SG值定义为：

$$SG(x) = \begin{cases} x-1 & , x \bmod 4 = 0 \\ x & , x \bmod 4 = 1 \\ x & , x \bmod 4 = 2 \\ x+1 & , x \bmod 4 = 3 \end{cases}$$

11.11 Every-SG 游戏

给出一个有向无环图，其中 K 个顶点上放置了石子，两名玩家轮流行动，按以下规则操作石子：

移动图上所有还能够移动的石子；

无法移动石子的一方出局。双方均采用最优策略，询问谁会获胜。

定义 $step$ 为某一局游戏至多需要经过的回合数。

以下局面先手必胜： $step$ 为奇数。

11.12 威佐夫博弈

有两堆石子，给出每一堆的石子数量，两名玩家轮流行动，每人每次任选以下规定的一种操作石子：

- 任选一堆，取走正整数颗石子；
- 从两队中同时取走正整数颗石子。

拿到最后一颗石子的一方获胜。双方均采用最优策略，询问谁会获胜。

以下局面先手必败：

$(1, 2), (3, 5), (4, 7), (6, 10), \dots$ 具体而言，每一对的第一个数为此前没出现过的最小整数，第二个数为第一个数加上 $1, 2, 3, 4, \dots$ 。

更一般地，对于第 k 对数，第一个数为 $First_k = \left\lfloor \frac{k*(1+\sqrt{5})}{2} \right\rfloor$ ，第二个数为 $Second_k = First_k + k$ 。

其中，在两堆石子的数量均大于 10^9 次时，由于需要使用高精度计算，我们需要人为定义 $\frac{1+\sqrt{5}}{2}$ 的取值为 $lorry = 1.618033988749894848204586834$ 。

```
1 const double lorry = (sqrt(5.0) + 1.0) / 2.0;
2 //const double lorry = 1.618033988749894848204586834;
3 void Solve() {
4     int n, m; cin >> n >> m;
5     if (n < m) swap(n, m);
6     double x = n - m;
7     if ((int)(lorry * x) == m) cout << "lose\n";
8     else cout << "win\n";
9 }
```

11.13 斐波那契博弈

有一堆石子，数量为 N ，两名玩家轮流行动，按以下规则取石子：

先手第1次可以取任意多颗，但不能全部取完，此后每人取的石子数不能超过上个人的两倍，拿到最后一颗石子的一方获胜。

双方均采用最优策略，询问谁会获胜。

当且仅当 N 为斐波那契数时先手必败。

```

1  int fib[100] = {1, 2};
2  map<int, bool> mp;
3  void Force() {
4      for (int i = 2; i <= 86; ++ i) fib[i] = fib[i - 1] + fib[i - 2];
5      for (int i = 0; i <= 86; ++ i) mp[fib[i]] = 1;
6  }
7  void Solve() {
8      int n; cin >> n;
9      if (mp[n] == 1) cout << "lose\n";
10     else cout << "win\n";
11 }

```

11.14 树上删边游戏

给出一棵 N 个节点的有根树，两名玩家轮流行动，按以下规则操作：

选择任意一棵子树并删除（即删去任意一条边，不与根相连的部分会同步被删去）；

删掉最后一棵子树的一方获胜（换句话说，删去根节点的一方失败）。双方均采用最优策略，询问谁会获胜。

结论：当根节点SG值非 1 时先手必胜。

相较于传统SG值的定义，本题的SG函数值定义为：

- 叶子节点的SG值为 0。
- 非叶子节点的SG值为其所有孩子节点SG值 +1 的异或和。

```

1  auto dfs = [&](auto self, int x, int fa) -> int {
2      int x = 0;
3      for (auto y : ver[x]) {
4          if (y == fa) continue;
5          x ^= self(self, y, x);
6      }
7      return x + 1;
8  };
9  cout << (dfs(dfs, 1, 0) == 1 ? "Bob\n" : "Alice\n");

```

11.15 无向图删边游戏 (Fusion Principle 定理)

给出一张 N 个节点的无向联通图，有一个点作为图的根，两名玩家轮流行动，按以下规则操作：

选择任意一条边删除，不与根相连的部分会同步被删去；

删掉最后一条边的一方获胜。双方均采用最优策略，询问谁会获胜。

- 对于奇环，我们将其缩成一个新点+一条新边；
- 对于偶环，我们将其缩成一个新点；
- 所有连接到原来环上的边全部与新点相连。

此时，本模型转化为“树上删边游戏”。

/END/

12 常用例题

12.1 逆序对（归并排序解）

性质：交换序列的任意两元素，序列的逆序数的奇偶性必定发生改变。

```
1 LL a[N], tmp[N], n, ans = 0;
2 void mergeSort(LL l, LL r){
3     if (l >= r) return;
4     LL mid = (l + r) >> 1, i = l, j = mid + 1, cnt = 0;
5     mergeSort(l, mid);
6     mergeSort(mid + 1, r);
7     while (i <= mid || j <= r)
8         if (j > r || (i <= mid && a[i] <= a[j]))
9             tmp[cnt++] = a[i++];
10        else
11            tmp[cnt++] = a[j++], ans += mid - i + 1;
12    for (LL k = 0; k < r - l + 1; k++)
13        a[l + k] = tmp[k];
14 }
15 int main(){
16     cin >> n;
17     for (int i = 1; i <= n; i++)
18         scanf("%lld", &a[i]);
19     mergeSort(1, n);
20     cout << ans << "\n";
21     return 0;
22 }
```

12.2 统计区间不同数字的数量（离线查询）

核心在于使用 `pre` 数组滚动维护每一个数字出现的最后位置，配以树状数组统计数量。由于滚动维护具有后效性，所以需要离线操作，从前往后更新。时间复杂度 $\mathcal{O}(N \log N)$ ，常数瓶颈在于 `map`，用手造哈希或者离散化可以优化到理想区间；同时也有莫队做法，复杂度稍劣。[例题链接](#)。

```
1 signed main() {
2     int n;
3     cin >> n;
4     vector<int> in(n + 1);
5     for (int i = 1; i <= n; i++) {
6         cin >> in[i];
7     }
8
9     int q;
10    cin >> q;
11    vector<array<int, 3>> query;
12    for (int i = 0; i < q; i++) {
13        int l, r;
14        cin >> l >> r;
15        query.push_back({r, l, i});
16    }
17    sort(query.begin(), query.end());
18
19    vector<pair<int, int>> ans;
20    map<int, int> pre;
21    int st = 1;
22    BIT bit(n);
23    for (auto [r, l, id] : query) {
24        for (int i = st; i <= r; i++, st++) {
25            if (pre.count(in[i])) { // 消除此前操作的影响
```

```

26         bit.add(pre[in[i]], -1);
27     }
28     bit.add(i, 1);
29     pre[in[i]] = i; // 更新操作
30 }
31 ans.push_back({id, bit.ask(r) - bit.ask(l - 1)});
32 }
33
34 sort(ans.begin(), ans.end());
35 for (auto [id, w] : ans) {
36     cout << w << endl;
37 }
38 }

```

12.3 选数 (DFS 解)

从 N 个整数中任选 K 个整数相加。使用 DFS 求解。

```

1  int n, k; cin >> n >> k;
2  vector<int> in(n), now(n);
3  for (auto &it : in) { cin >> it; }
4  auto dfs = [&](auto self, int k, int bit, int idx) -> void {
5      for (int i = idx; i < n; i++) {
6          now[bit] = in[i];
7          if (bit < k - 1) { self(self, k, bit + 1, i + 1); }
8          if (bit == k - 1) {
9              int add = 0;
10             for (int j = 0; j < k; j++) {
11                 add += now[j];
12             }
13             cout << add << endl;
14         }
15     }
16 };
17 dfs(dfs, k, 0, 0);

```

12.4 选数 (位运算状压)

```

1  int n, k; cin >> n >> k;
2  vector<int> in(n);
3  for (auto &it : in) { cin >> it; }
4  int comb = (1 << k) - 1, U = 1 << n;
5  while (comb < U) {
6      int add = 0;
7      for (int i = 0; i < n; i++) {
8          if (1 << i & comb) {
9              add += in[i];
10         }
11     }
12     cout << add << "\n";
13
14     int x = comb & -comb;
15     int y = comb + x;
16     int z = comb & ~y;
17     comb = (z / x >> 1) | y;
18 }

```

12.5 网格路径计数

从 $(0, 0)$ 走到 (a, b) , 规定每次只能从 (x, y) 走到左下或者右下, 方案数记为 $f(a, b)$ 。

- $f(a, b) = \binom{a}{\frac{a+b}{2}}$;
- 若路径和直线 $y = k, k \notin [0, b]$ 不能有交点, 则方案数为 $f(a, b) - f(a, 2k - b)$;
- 若路径和两条直线 $y = k_1, y = k_2$ ($k_1 < 0 \leq b < k_2$) 不能有交点, 方案数记为 $g(a, b, k_1, k_2)$, 可以使用 $\mathcal{O}(N)$ 递归求解;
- 若路径必须碰到 $y = k_1$ 但是不能碰到 $y = k_2$, 方案数记为 $h(a, b, k_1, k_2)$, 可以使用 $\mathcal{O}(N)$ 递归求解 (递归过程中两条直线距离会越来越大)。

从 $(0, 0)$ 走到 $(a, 0)$, 规定每次只能走到左下或者右下, 且必须有**恰好一次**传送 (向下 b 单位), 且不能走到 x 轴下方, 方案数为 $\binom{a+1}{\frac{a-b}{2} + k + 1}$ 。

12.6 德州扑克

读入牌型, 并且支持两副牌之间的大小比较。 [代码参考](#)

```
1 struct card {
2     int suit, rank;
3     friend bool operator < (const card &a, const card &b) {
4         return a.rank < b.rank;
5     }
6     friend bool operator == (const card &a, const card &b) {
7         return a.rank == b.rank;
8     }
9     friend bool operator != (const card &a, const card &b) {
10        return a.rank != b.rank;
11    }
12    friend auto &operator>> (istream &it, card &C) {
13        string S, T; it >> S;
14        T = "__23456789TJQKA"; //点数
15        FOR (i, 0, T.sz - 1) {
16            if (T[i] == S[0]) C.rank = i;
17        }
18        T = "_SHCD"; //花色
19        FOR (i, 0, T.sz - 1) {
20            if (T[i] == S[1]) C.suit = i;
21        }
22        return it;
23    }
24 };
25 struct game {
26     int level;
27     vector<card> peo;
28     int a, b, c, d, e;
29     int u, v, w, x, y;
30     bool Rk10() { //Rk10: Royal Flush, 五张牌同花色, 且点数为AKQJT (14,13,12,11,10)
31         sort(ALL(peo));
32         reverse(ALL(peo));
33         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
34         peo[4].rank;
35         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
36         peo[4].suit;
37         if (u != v || v != w || w != x || x != y) return 0;
38         if (a == 14 && b == 13 && c == 12 && d == 11 && e == 10) return 1;
```

```

38         return 0;
39     }
40     bool Dif(vector<card> &peo) { //专门用于检查A2345这种顺子的情况 (这是最小的顺子)
41         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
42         peo[4].rank;
43         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
44         peo[4].suit;
45         if (a != 14 || b != 5 || c != 4 || d != 3 || e != 2) return 0;
46         vector<card> peo2 = {peo[1], peo[2], peo[3], peo[4], peo[0]}; //重新排序
47         peo = peo2;
48         return 1;
49     }
50     bool Rk9() { //Rk9: Straight Flush, 五张牌同花色, 且顺连【r1 > r2 > r3 > r4 >
51     r5】
52         sort(ALL(peo));
53         reverse(ALL(peo));
54         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
55         peo[4].rank;
56         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
57         peo[4].suit;
58         if (u != v || v != w || w != x || x != y) return 0;
59         if (Dif(peo)) return 1; //特判: A2345
60         if (a == b + 1 && b == c + 1 && c == d + 1 && d == e + 1) return 1;
61         return 0;
62     }
63     bool Rk8() { //Rk8: Four of a Kind, 四张牌点数一样【r1 = r2 = r3 = r4】
64         sort(ALL(peo));
65         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
66         peo[4].rank;
67         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
68         peo[4].suit;
69         if (a == b && b == c && c == d) return 1;
70         if (b == c && c == d && d == e) {
71             reverse(ALL(peo));
72             return 1;
73         }
74         return 0;
75     }
76     bool Rk7() { //Rk7: Fullhouse, 三张牌点数一样, 另外两张点数也一样【r1 = r2 = r3, r4
77     = r5】
78         sort(ALL(peo));
79         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
80         peo[4].rank;
81         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
82         peo[4].suit;
83         if (a == b && b == c && d == e) return 1;
84         if (a == b && c == d && d == e) {
85             reverse(ALL(peo));
86             return 1;
87         }
88         return 0;
89     }
90     bool Rk6() { //Rk6: Flush, 五张牌同花色【r1 > r2 > r3 > r4 > r5】
91         sort(ALL(peo));
92         reverse(ALL(peo));
93         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
94         peo[4].rank;
95         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
96         peo[4].suit;

```

```

90         if (u != v || v != w || w != x || x != y) return 0;
91         return 1;
92     }
93     bool Rk5() { //Rk5: Straight, 五张牌顺连 【r1 > r2 > r3 > r4 > r5】
94         sort(ALL(peo));
95         reverse(ALL(peo));
96         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
97         peo[4].rank;
98         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
99         peo[4].suit;
100         if (Dif(peo)) return 1; //特判: A2345
101         if (a == b + 1 && b == c + 1 && c == d + 1 && d == e + 1) return 1;
102         return 0;
103     }
104     bool Rk4() { //Rk4: Three of a kind, 三张牌点数一样 【r1 = r2 = r3, r4 > r5】
105         sort(ALL(peo));
106         reverse(ALL(peo));
107         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
108         peo[4].rank;
109         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
110         peo[4].suit;
111         if (a == b && b == c) return 1;
112         if (b == c && c == d) {
113             swap(peo[3], peo[0]);
114             return 1;
115         }
116         if (c == d && d == e) {
117             swap(peo[3], peo[0]);
118             swap(peo[4], peo[1]);
119             return 1;
120         }
121         return 0;
122     }
123     bool Rk3() { //Rk3: Two Pairs, 两张牌点数一样, 另外有两张点数也一样 (两个对子) 【r1
124     = r2 > r3 = r4】
125         sort(ALL(peo));
126         reverse(ALL(peo));
127         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
128         peo[4].rank;
129         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
130         peo[4].suit;
131         if (a == b && c == d) return 1;
132         if (a == b && d == e) {
133             swap(peo[2], peo[4]);
134             return 1;
135         }
136         if (b == c && d == e) {
137             swap(peo[0], peo[2]);
138             swap(peo[2], peo[4]);
139             return 1;
140         }
141         return 0;
142     }
143     bool Rk2() { //Rk2: One Pairs, 两张牌点数一样 (一个对子) 【r1 = r2, r3 > r4 > r5】
144         sort(ALL(peo));
145         reverse(ALL(peo));
146         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e =
147         peo[4].rank;
148         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y =
149         peo[4].suit;

```

```

145     vector<card> peo2;
146     if (a == b) return 1;
147     if (b == c) {
148         peo2 = {peo[1], peo[2], peo[0], peo[3], peo[4]};
149         peo = peo2;
150         return 1;
151     }
152     if (c == d) {
153         peo2 = {peo[2], peo[3], peo[0], peo[1], peo[4]};
154         peo = peo2;
155         return 1;
156     }
157     if (d == e) {
158         peo2 = {peo[3], peo[4], peo[0], peo[1], peo[2]};
159         peo = peo2;
160         return 1;
161     }
162     return 0;
163 }
164 bool Rk1() { //Rk1: high card
165     sort(ALL(peo));
166     reverse(ALL(peo));
167     return 1;
168 }
169 game (vector<card> New_peo) {
170     peo = New_peo;
171     if (Rk10()) { level = 10; return; }
172     if (Rk9()) { level = 9; return; }
173     if (Rk8()) { level = 8; return; }
174     if (Rk7()) { level = 7; return; }
175     if (Rk6()) { level = 6; return; }
176     if (Rk5()) { level = 5; return; }
177     if (Rk4()) { level = 4; return; }
178     if (Rk3()) { level = 3; return; }
179     if (Rk2()) { level = 2; return; }
180     if (Rk1()) { level = 1; return; }
181 }
182 friend bool operator < (const game &a, const game &b) {
183     if (a.level != b.level) return a.level < b.level;
184     FOR (i, 0, 4) if (a.peo[i] != b.peo[i]) return a.peo[i] < b.peo[i];
185     return 0;
186 }
187 friend bool operator == (const game &a, const game &b) {
188     if (a.level != b.level) return 0;
189     FOR (i, 0, 4) if (a.peo[i] != b.peo[i]) return 0;
190     return 1;
191 }
192 };
193 void debug(vector<card> peo) {
194     for (auto it : peo) cout << it.rank << " " << it.suit << " ";
195     cout << "\n\n";
196 }
197 int clac(vector<card> Ali, vector<card> Bob) {
198     game atype(Ali), btype(Bob);
199     if (atype < btype) return -1;
200     else if (atype == btype) return 0;
201     return 1;
202 }

```

12.7 N*M 数独字典序最小方案

规则：每个宫大小为 $2^N * 2^M$ ，大图一共由 $M * N$ 个宫组成（总大小即 $2^N 2^M * 2^N 2^M$ ），要求每行、每列、每宫都要出现 1 到 $2^N * 2^M$ 的全部数字。输出字典序最小方案。

下例为 2, 1 和 1, 2 时数独字典序最小的示意。

	0	1	2	3	4	5	6	7			0	1	2	3	4	5	6	7	
0	1	2	3	4	5	6	7	8			0	1	2	3	4	5	6	7	8
1	3	4	1	2	7	8	5	6			1	5	6	7	8	1	2	3	4
2	5	6	7	8	1	2	3	4			2	2	1	4	3	6	5	8	7
3	7	8	5	6	3	4	1	2			3	6	5	8	7	2	1	4	3
4	2	1	4	3	6	5	8	7			4	3	4	1	2	7	8	5	6
5	4	3	2	1	8	7	6	5			5	7	8	5	6	3	4	1	2
6	6	5	8	7	2	1	4	3			6	4	3	2	1	8	7	6	5
7	8	7	6	5	4	3	2	1			7	8	7	6	5	4	3	2	1

公式：(i, j) 格所填的内容为 $(i \bmod 2^N \oplus \lfloor \frac{j}{2^M} \rfloor) \cdot 2^M + (\lfloor \frac{i}{2^N} \rfloor \oplus j \bmod 2^M) + 1$ ，注意 i, j 从 0 开始。

12.8 高精度进制转换

2 — 62 进制相互转换。输入格式："转换前进制 转换后进制 要转换的数据"。注释：进制排序为 0-9, A-Z, a-z。

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 map<char, int> mp; //将字符转化为数字
4 map<int, char> mp2; //将数字转化为字符
5 int main(){
6     for(int i = 0; i < 10; i++) mp[(char)i + 48] = i, mp2[i] = (char)i + 48;
7     for(int i = 10; i < 36; i++) mp[(char)i + 55] = i, mp2[i] = (char)i + 55;
8     for(int i = 36; i < 62; i++) mp[(char)i + 61] = i, mp2[i] = (char)i + 61;
9
10    int tt = 1, a, b; cin >> tt;
11    while(tt--){
12        string s, sh;
13        vector<int> nums, ans;
14        cin >> a >> b >> s;
15        for(auto c : s) nums.push_back(mp[c]);
16        reverse(nums.begin(), nums.end());
17        while(nums.size()){ //短除法，将整个大数一直除 b，取余数
18            int remainder = 0;
19            for(int i = nums.size() - 1; ~i; i--){
20                nums[i] += remainder * a;
21                remainder = nums[i] % b;
22                nums[i] /= b;
23            }
24            ans.push_back(remainder); //得到余数
25            while(nums.size() && nums.back() == 0) nums.pop_back(); //去掉前导 0
```

```

26     }
27     reverse(ans.begin(), ans.end());
28     for(int i : ans) sh += mp2[i];
29     cout << a << ' ' << s << endl;
30     cout << b << ' ' << sh << endl << endl;
31 }
32 return 0;
33 }

```

12.9 物品装箱

有 N 个物品，第 i 个物品为 $a[i]$ ，有无限个容量为 C 的空箱子。两种装箱方式，输出需要多少个箱子才能装完所有物品。

12.9.1 从前往后装（线段树解）

选择最前面的能放下物品的箱子放入物品。

```

1  const int N = 1e6 + 10;
2  int T, n, a[N], c, tr[N << 2];
3  void pushup(int u){
4      tr[u] = max(tr[u << 1], tr[u << 1 | 1]);
5  }
6  void build(int u, int l, int r){
7      if (l == r) tr[u] = c;
8      else {
9          int mid = l + r >> 1;
10         build(u << 1, l, mid);
11         build(u << 1 | 1, mid + 1, r);
12         pushup(u);
13     }
14 }
15 void update(int u, int l, int r, int p, int k){
16     if (l > p || r < p) return;
17     if (l == r) tr[u] -= k;
18     else {
19         int mid = l + r >> 1;
20         update(u << 1, l, mid, p, k);
21         update(u << 1 | 1, mid + 1, r, p, k);
22         pushup(u);
23     }
24 }
25 int query(int u, int l, int r, int k){
26     if (l == r){
27         if (tr[u] >= k) return l;
28         return n + 1;
29     }
30     int mid = l + r >> 1;
31     if (tr[u << 1] >= k) return query(u << 1, l, mid, k);
32     else return query(u << 1 | 1, mid + 1, r, k);
33 }
34 int main() {
35     cin >> n >> c;
36     for (int i = 1; i <= n; i++) cin >> a[i];
37     build(1, 1, n);
38     for (int i = 1; i <= n; i++)
39         update(1, 1, n, query(1, 1, n, a[i]), a[i]);
40     cout << query(1, 1, n, c) - 1 << " ";
41 }

```

12.9.2 选择最优的箱子装 (multiset 解)

选择能放下物品且剩余容量最小的箱子放物品

```
1 void solve(){
2     cin >> n >> c;
3     for (int i = 1; i <= n; i++) cin >> a[i];
4     multiset<int> s;
5     for (int i = 1; i <= n; i++){
6         auto it = s.lower_bound(a[i]);
7         if (it == s.end()) s.insert(c - a[i]);
8         else {
9             int x = *it;
10            // multiset 可以存放重复数据, 如果是删除某个值的话, 会去掉多个箱子
11            // 导致答案错误, 所以直接删除对应位置的元素
12            s.erase(it);
13            s.insert(x - a[i]);
14        }
15    }
16    cout << s.size() << "\n";
17 }
```

12.10 浮点数比较

比较下列浮点数的大小: $x^{yz}, x^{zy}, (x^y)^z, (x^z)^y, y^{xz}, y^{zx}, (y^x)^z, (y^z)^x, z^{xy}, z^{yx}, (z^x)^y$ 和 $(z^y)^x$ 。

```
1 vector<pair<ld, int>> val = {
2     {log(x) * pow(y, z), 0}, {log(x) * pow(z, y), 1}, {log(x) * y * z, 2},
3     {log(x) * z * y, 3}, {log(y) * pow(x, z), 4}, {log(y) * pow(z, x), 5},
4     {log(y) * x * z, 6}, {log(y) * z * x, 7}, {log(z) * pow(x, y), 8},
5     {log(z) * pow(y, x), 9}, {log(z) * x * y, 10}, {log(z) * y * x, 11};
6
7 sort(val.begin(), val.end(), [&](auto x, auto y) {
8     if (equal(x.first, y.first)) return x.second < y.second; // equal比较两个浮点数是
9     return x.first > y.first;
10 });
11 cout << ans[val.front().second] << endl;
```

/END/

13 STL 与库函数

13.1 pb_ds 库

其中 `gp_hash_table` 使用的最多，其等价于 `unordered_map`，内部是无序的。

```
1 #include <bits/extc++.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 template<class S, class T> using omap = __gnu_pbds::gp_hash_table<S, T, myhash>;
```

13.2 查找后继 lower_bound、upper_bound

`lower` 表示 $\geq$ ，`upper` 表示 $>$ 。使用前记得**先进行排序**。

```
1 //返回a数组[start,end)区间中第一个>=x的地址【地址!!!】
2 cout << lower_bound(a + start, a + end, x);
3
4 cout << lower_bound(a, a + n, x) - a; //在a数组中查找第一个>=x的元素下标
5 upper_bound(a, a + n, k) - lower_bound(a, a + n, k) //查找k在a中出现了几次
```

13.3 数组打乱 shuffle

```
1 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
2 shuffle(ver.begin(), ver.end(), rnd);
```

13.4 二分搜索 binary_search

用于查找某一元素是否在容器中，相当于 `find` 函数。在使用前需要**先进行排序**。

```
1 //在a数组[start,end)区间中查找x是否存在，返回bool型
2 cout << binary_search(a + start, a + end, x);
```

13.5 批量递增赋值函数 iota

对容器递增初始化。

```
1 //将a数组[start,end)区间复制成"x, x+1, x+2, ..."
2 iota(a + start, a + end, x);
```

13.6 数组去重函数 unique

在使用前需要**先进行排序**。

其作用是，对于区间 `[开始位置, 结束位置)`，不停的把后面不重复的元素移到前面来，也可以说是用不重复的元素占领重复元素的位置。并且返回去重后容器中不重复序列的最后一个元素的下一个元素。所以在进行操作后，数组、容器的大小并没有发生改变。

```
1 //将a数组[start,end)区间去重，返回迭代器
2 unique(a + start, a + end);
3
4 //与erase函数结合，达到去重+删除的目的
5 a.erase(unique(ALL(a)), a.end());
```

13.7 位运算函数 `__builtin_`

```
1 // 样例: x = 1即(1), x = 8即(1000), x = 15即(1111)
2
3 // 返回x二进制下含1的数量
4 cout << __builtin_popcount (x); // 例如x=15时答案为4
5
6 // 返回x二进制下最后一个1的位置 (从1开始计算)
7 cout << __builtin_ffs(x); // 例如x=1答案为1, x=8答案为4
8
9 // 返回x二进制下后导0的个数
10 cout << __builtin_ctz(x); // 例如x=1答案为0, x=8答案为3
11
12 // 返回x二进制下第一个1的位置 (原函数是求前导0的个数)
13 cout << 31 - __builtin_clz(x); // x = 9(1001) 返回 3
14 cout << 63 - __builtin_clzll(x);
```

注: 以上函数的 `long long` 版本只需要在函数后面加上 `ll` 即可 (例如 `__builtin_popcountll(x)`), `unsigned long long` 加上 `ull`。

13.8 数字转字符串函数

`itoa` 虽然能将整数转换成任意进制的字符串, 但是其不是标准的C函数, 且为Windows独有, 且不支持 `long long`, 建议手写。

```
1 // to_string函数会直接将你的各种类型的数字转换为字符串。
2 // string to_string(T val);
3 double val = 12.12;
4 cout << to_string(val);
```

```
1 // 【不建议使用】itoa允许你将整数转换成任意进制的字符串, 参数为待转换整数、目标字符数组、进制。
2 // char* itoa(int value, char* string, int radix);
3 char ans[10] = {};
4 itoa(12, ans, 2);
5 cout << ans << endl; /*1100*/
6
7 // 长整型函数名ltoa, 最高支持到int型上限2^31。ultoa同理。
```

13.9 字符串转数字

```
1 // stoi直接使用
2 cout << stoi("12") << endl;
3
4 // 【不建议使用】stoi转换进制, 参数为待转换字符串、起始位置、进制。
5 // int stoi(string value, int st, int radix);
6 cout << stoi("1010", 0, 2) << endl; /*10*/
7 cout << stoi("c", 0, 16) << endl; /*12*/
8 cout << stoi("0x3f3f3f3f", 0, 0) << endl; /*1061109567*/
9
10 // 长整型函数名stoll, 最高支持到long long型上限2^63。stoull、stod、stold同理。
```

```

1 // atoi直接使用，空字符返回0，允许正负符号，数字字符前有其他字符返回0，数字字符前有空白字符自动去除
2 cout << atoi("12") << endl;
3 cout << atoi(" 12") << endl; /*12*/
4 cout << atoi("-12abc") << endl; /*-12*/
5 cout << atoi("abc12") << endl; /*0*/
6
7 // 长整型函数名atoll，最高支持到long long型上限2^63。

```

13.10 全排列算法 next_permutation、prev_permutation

在提及这个函数时，我们先需要补充几点字典序相关的知识。

对于三个字符所组成的序列 {a,b,c}，其按照字典序的6种排列分别为：

{abc}，{acb}，{bac}，{bca}，{cab}，{cba}

其排序原理是：先固定 a (序列内最小元素)，再对之后的元素排列。而 $b < c$ ，所以 $abc < acb$ 。同理，先固定 b (序列内次小元素)，再对之后的元素排列。即可得出以上序列。

next_permutation 算法，即是按照字典序顺序输出的全排列；相对应的，prev_permutation 则是按照逆字典序顺序输出的全排列。可以是数字，亦可以是其他类型元素。其直接在序列上进行更新，故直接输出序列即可。

```

1 int n;
2 cin >> n;
3 vector<int> a(n);
4 // iota(a.begin(), a.end(), 1);
5 for (auto &it : a) cin >> it;
6 sort(a.begin(), a.end());
7
8 do {
9     for (auto it : a) cout << it << " ";
10    cout << endl;
11 } while (next_permutation(a.begin(), a.end()));

```

13.11 字符串转换为数值函数 stoi

可以快捷的将一串字符串转换为指定进制的数字。

使用方法

- stoi(字符串, 0, x进制)：将一串 x 进制的字符串转换为 int 型数字。

```
void Solve() {
    cout << stoi("1010", 0, 2) << endl;
    cout << stoi("c", 0, 16) << endl;
    cout << stoi("0x3f3f3f3f", 0, 0) << endl;
    cout << stoi("10", 0, 8) << endl;
    cout << stoll("aaaaaaaaaa", 0, 16) << endl;
}
```

C:\Users\26099\Desktop\万能头文件.exe

10
12
1061109567
8
11728124029610

- `stoll(字符串, 0, x进制)` : 将一串 `x` 进制的字符串转换为 `long long` 型数字。
- `stoull`, `stod`, `stold` 同理。

13.12 数值转换为字符串函数 `to_string`

允许将各种数值类型转换为字符串类型。

```
1 //将数值num转换为字符串s
2 string s = to_string(num);
```

13.13 判断非递减 `is_sorted`

```
1 //a数组[start,end)区间是否是非递减的, 返回bool型
2 cout << is_sorted(a + start, a + end);
```

13.14 累加 `accumulate`

```
1 //将a数组[start,end)区间的元素进行累加, 并输出累加和+x的值
2 cout << accumulate(a + start, a + end, x);
```

13.15 迭代器 `iterator`

```
1 //构建一个UUU容器的正向迭代器, 名字叫it
2 UUU::iterator it;
3
4 vector<int>::iterator it; //创建一个正向迭代器, ++ 操作时指向下一个
5 vector<int>::reverse_iterator it; //创建一个反向迭代器, ++ 操作时指向上一个
```

13.16 容器与成员函数

13.16.1 元组 tuple

```
1 //获取obj对象中的第index个元素—get<index>(obj)
2 //需要注意的是这里的index只能手动输入，使用for循环这样的自动输入是不可以的
3 tuple<string, int, int> Student = {"Wida", 23, 45000};
4 cout << get<0>(Student) << endl; //获取Student对象中的第一个元素，这里的输出结果应
   为"Wida"
```

13.16.2 数组 array

```
1 array<int, 3> x; // 建立一个包含三个元素的数组x
2
3 [] // 跟正常数组一样，可以使用随机访问
4 cout << x[0]; // 获取数组重的第一个元素
```

13.16.3 变长数组 vector

```
1 resize(n) // 重设容器大小，但是不改变已有元素的值
2 assign(n, 0) // 重设容器大小为n，且替换容器内的内容为0
3
4 // 尽量不要使用[]的形式声明多维变长数组，而是使用嵌套的方式替代
5 vector<int> ver[n + 1]; // 不好的声明方式
6 vector<vector<int>> ver(n + 1);
7
8 // 嵌套时只需要在最后一个注明变量类型
9 vector dis(n + 1, vector<int>(m + 1));
10 vector dis(m + 1, vector(n + 1, vector<int>(n + 1)));
```

13.16.4 栈 stack

栈顶入，栈顶出。先进后出。

```
1 //没有clear函数
2 size() / empty()
3 push(x) //向栈顶插入x
4 top() //获取栈顶元素
5 pop() //弹出栈顶元素
```

13.16.5 队列 queue

队尾进，队头出。先进先出。

```
1 //没有clear函数
2 size() / empty()
3 push(x) //向队尾插入x
4 front() / back() //获取队头、队尾元素
5 pop() //弹出队头元素
```

```
1 //没有clear函数，但是可以用重新构造替代
2 queue<int> q;
3 q = queue<int>();
```

13.16.6 双向队列 deque

```
1 size() / empty() / clear()
2 push_front(x) / push_back(x)
3 pop_front(x) / pop_back(x)
4 front() / back()
5 begin() / end()
6 []
```

13.16.7 优先队列 priority_queue

默认升序（大根堆），自定义排序需要重载 `<`。

```
1 //没有clear函数
2 priority_queue<int, vector<int>, greater<int> > p; //重定义为降序（小根堆）
3 push(x); //向栈顶插入x
4 top(); //获取栈顶元素
5 pop(); //弹出栈顶元素
```

```
1 //重载运算符【注意，符号相反！！】
2 struct Node {
3     int x; string s;
4     friend bool operator < (const Node &a, const Node &b) {
5         if (a.x != b.x) return a.x > b.x;
6         return a.s > b.s;
7     }
8 };
```

13.16.8 字符串 string

```
1 size() / empty() / clear()
```

```
1 //从字符串S的S[start]开始，取出长度为len的子串—S.substr(start, len)
2 //len省略时默认取到结尾，超过字符串长度时也默认取到结尾
3 cout << S.substr(1, 12);
4
5 find(x) / rfind(x); //顺序、逆序查找x，返回下标，没找到时返回一个极大值【！建议与 size() 比较，而不要和 -1 比较，后者可能出错】
6 //注意，没有count函数
```

13.16.9 有序、多重有序集合 set、multiset

默认升序（大根堆），set 去重，multiset 不去重， $\mathcal{O}(\log N)$ 。

```
1 set<int, greater<> > s; //重定义为降序（小根堆）
2 size() / empty() / clear()
3 begin() / end()
4 ++ / -- //返回前驱、后继
5
6 insert(x); //插入x
7 find(x) / rfind(x); //顺序、逆序查找x，返回迭代器【迭代器！！】，没找到时返回end()
8 count(x); //返回x的个数
9 lower_bound(x); //返回第一个>=x的迭代器【迭代器！！】
10 upper_bound(x); //返回第一个>x的迭代器【迭代器！！】
```

特殊函数 `next` 和 `prev` 详解：

```

1 auto it = s.find(x); // 建立一个迭代器
2 prev(it) / next(it); // 默认返回迭代器it的前/后一个迭代器
3 prev(it, 2) / next(it, 2); // 可选参数可以控制返回前/后任意个迭代器
4
5 /* 以下是一些应用 */
6 auto pre = prev(s.lower_bound(x)); // 返回第一个<x的迭代器
7 int ed = *prev(S.end(), 1); // 返回最后一个元素

```

erase(x); 有两种删除方式:

- 当x为某一元素时, 删除**所有**这个数, 复杂度为 $\mathcal{O}(num_x + \log N)$;
- 当x为迭代器时, 删除这个迭代器。

```

1 //连续头部删除
2 set<int> S = {0, 9, 98, 1087, 894, 34, 756};
3 auto it = S.begin();
4 int len = S.size();
5 for (int i = 0; i < len; ++ i) {
6     if (*it >= 500) continue;
7     it = S.erase(it); //删除所有小于500的元素
8 }
9 //错误用法如下【千万不能这样用!!!】
10 //for (auto it : S) {
11 //    if (it >= 500) continue;
12 //    S.erase(it); //删除所有小于500的元素
13 //}

```

13.16.10 map、multimap

默认升序 (大根堆) , map 去重, multimap 不去重, $\mathcal{O}(\log S)$, 其中 S 为元素数量。

```

1 map<int, int, greater<>> > mp; //重定义为降序 (小根堆)
2 size() / empty() / clear()
3 begin() / end()
4 ++ / -- //返回前驱、后继
5
6 insert({x, y}); //插入二元组
7 [] //随机访问, multimap不支持
8 count(x); //返回x为下标的个数
9 lower_bound(x); //返回第一个下标>=x的迭代器
10 upper_bound(x); //返回第一个下标>x的迭代器

```

erase(x); 有两种删除方式:

- 当x为某一元素时, 删除所有**以这个元素为下标的二元组**, 复杂度为 $\mathcal{O}(num_x + \log N)$;
- 当x为迭代器时, 删除这个迭代器。

慎用随机访问! ——当不确定某次查询是否存在于容器中时, 不要直接使用下标查询, 而是先使用 count() 或者 find() 方法检查key值, 防止不必要的零值二元组被构造。

```

1 int q = 0;
2 if (mp.count(i)) q = mp[i];

```

慎用自带的 pair、tuple 作为key值类型! 使用自定义结构体!

```

1 struct fff {
2     LL x, y;
3     friend bool operator < (const fff &a, const fff &b) {
4         if (a.x != b.x) return a.x < b.x;
5         return a.y < b.y;
6     }
7 };
8 map<fff, int> mp;

```

13.16.11 bitset

将数据转换为二进制，从高位到低位排序，以 0 为最低位。当位数相同时支持全部的位运算。

```

1 // 如果输入的是01字符串，可以直接使用">>"读入
2 bitset<10> s;
3 cin >> s;
4
5 //使用只含01的字符串构造—bitset<容器长度>B (字符串)
6 string S; cin >> S;
7 bitset<32> B (S);
8
9 //使用整数构造 (两种方式)
10 int x; cin >> x;
11 bitset<32> B1 (x);
12 bitset<32> B2 = x;
13
14 // 构造时，尖括号里的数字不能是变量
15 int x; cin >> x;
16 bitset<x> ans; // 错误构造
17
18 [] //随机访问
19 set(x) //将第x位置1, x省略时默认全部位置1
20 reset(x) //将第x位置0, x省略时默认全部位置0
21 flip(x) //将第x位取反, x省略时默认全部位取反
22 to_ulong() //重转换为ULL类型
23 to_string() //重转换为ULL类型
24 count() //返回1的个数
25 any() //判断是否至少有一个1
26 none() //判断是否全为0
27
28 bitset<23> B1("11101001"), B2("11101000");
29 cout << (B1 ^ B2) << "\n"; //按位异或
30 cout << (B1 | B2) << "\n"; //按位或
31 cout << (B1 & B2) << "\n"; //按位与
32 cout << (B1 == B2) << "\n"; //比较是否相等
33 cout << B1 << " " << B2 << "\n"; //你可以直接使用cout输出

```

13.16.12 哈希系列 unordered

通常指代 unordered_map、unordered_set、unordered_multimap、unordered_multiset，与原版相比不进行排序。

如果将不支持哈希的类型作为 `key` 值代入，编译器就无法正常运行，这时需要我们为其手写哈希函数。而我们写的这个哈希函数的正确性其实并不是特别重要（但是不可以没有），当发生冲突时编译器会调用 `key` 的 `operator ==` 函数进行进一步判断。[参考](#)

13.16.12.1 对 pair、tuple 定义哈希

```
1 struct hash_pair {
2     template <class T1, class T2>
3     size_t operator()(const pair<T1, T2> &p) const {
4         return hash<T1>()(p.fi) ^ hash<T2>()(p.se);
5     }
6 };
7 unordered_set<pair<int, int>, int, hash_pair> S;
8 unordered_map<tuple<int, int, int>, int, hash_pair> M;
```

13.16.12.2 对结构体定义哈希

需要两个条件，一个是在结构体中重载等于号（区别于非哈希容器需要重载小于号，如上所述，当冲突时编译器需要根据重载的等于号判断），第二是写一个哈希函数。注意 `hash<>()` 的尖括号中的类型匹配。

```
1 struct fff {
2     string x, y;
3     int z;
4     friend bool operator == (const fff &a, const fff &b) {
5         return a.x == b.x || a.y == b.y || a.z == b.z;
6     }
7 };
8 struct hash_fff {
9     size_t operator()(const fff &p) const {
10         return hash<string>()(p.x) ^ hash<string>()(p.y) ^ hash<int>()(p.z);
11     }
12 };
13 unordered_map<fff, int, hash_fff> mp;
```

13.16.12.3 对 vector 定义哈希

以下两个方法均可。注意 `hash<>()` 的尖括号中的类型匹配。

```
1 struct hash_vector {
2     size_t operator()(const vector<int> &p) const {
3         size_t seed = 0;
4         for (auto it : p) {
5             seed ^= hash<int>()(it);
6         }
7         return seed;
8     }
9 };
10 unordered_map<vector<int>, int, hash_vector> mp;
```

```
1 namespace std {
2     template<> struct hash<vector<int>> {
3         size_t operator()(const vector<int> &p) const {
4             size_t seed = 0;
5             for (int i : p) {
6                 seed ^= hash<int>()(i) + 0x9e3779b9 + (seed << 6) + (seed >> 2);
7             }
8             return seed;
9         }
10    };
11 }
12 unordered_set<vector<int> > S;
```

13.17 程序标准化

13.17.1 使用 Lambda 函数

- `function` 统一写法

需要注意的是，虽然 `function` 定义时已经声明了返回值类型了，但是有的时候会出错（例如，声明返回 `long long` 但是返回 `int`，原因没去了解），所以推荐在后面使用 `->` 再行声明一遍。

```
1 function<void(int, int)> clac = [&](int x, int y) -> void {
2 };
3 clac(1, 2);
4
5 function<bool(int)> dfs = [&](int x) -> bool {
6     return dfs(x + 1);
7 };
8 dfs(1);
```

- `auto` 非递归写法

不需要使用递归函数时，直接用 `auto` 替换 `function` 即可。

```
1 auto clac = [&](int x, int y) -> void {
2 };
```

- `auto` 递归写法

相较于 `function` 写法，需要额外引用一遍自身。

```
1 auto dfs = [&](auto self, int x) -> bool {
2     return self(self, x + 1);
3 };
4 dfs(dfs, 1);
```

13.17.2 使用构造函数

可以将一些必要的声明和预处理放在构造函数，在编译时，无论放置在程序的哪个位置，都会先于主函数进行。下方是我将输入流控制声明的过程。

```
1 int __FAST_IO__ = []() { // 函数名称可以随意修改
2     ios::sync_with_stdio(0), cin.tie(0);
3     cout.tie(0);
4     cout << fixed << setprecision(12);
5     freopen("out.txt", "r", stdin);
6     freopen("in.txt", "w", stdout);
7     return 0;
8 }();
```

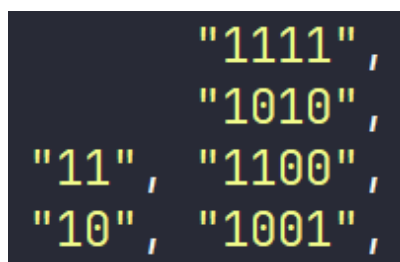
/END/

14 杂类

14.1 阿达马矩阵 (Hadamard matrix)

构造题用，其有一些性质：将 0 看作 -1 ；1 看作 $+1$ ，整个矩阵可以构成一个 2^k 维向量组，任意两个行、列向量的点积均为 0 [See](#)。例如，在 $k = 2$ 时行向量 $\vec{2}$ 和行向量 $\vec{3}$ 的点积为 $1 \cdot 1 + (-1) \cdot 1 + 1 \cdot (-1) + (-1) \cdot (-1) = 0$ 。

构造方式：
$$\begin{bmatrix} T & T \\ T & !T \end{bmatrix}$$

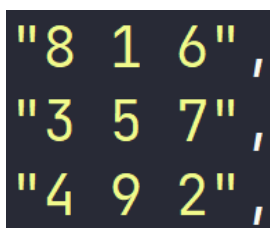


```
1  int n;
2  cin >> n;
3  int N = pow(2, n);
4  vector<vector<int>>(N);
5  ans[0][0] = 1;
6  for (int t = 0; t < n; t++) {
7      int m = pow(2, t);
8      for (int i = 0; i < m; i++) {
9          for (int j = m; j < 2 * m; j++) {
10             ans[i][j] = ans[i][j - m];
11         }
12     }
13     for (int i = m; i < 2 * m; i++) {
14         for (int j = 0; j < m; j++) {
15             ans[i][j] = ans[i - m][j];
16         }
17     }
18     for (int i = m; i < 2 * m; i++) {
19         for (int j = m; j < 2 * m; j++) {
20             ans[i][j] = 1 - ans[i - m][j - m];
21         }
22     }
23 }
```

14.2 幻方

构造题用，其有一些性质（保证 N 为奇数）：1 到 N^2 每个数字恰好使用一次，且每行、每列及两条对角线上的数字之和都相同，且为奇数 [See](#)。

构造方式：将 1 写在第一行的中间，随后不断向右上角位置填下一个数字，直到填满。



```

1  int n;
2  cin >> n;
3  int x = 1, y = (n + 1) / 2;
4  vector<int> ans(n + 1, vector<int>(n + 1));
5  for (int i = 1; i <= n * n; i++) {
6      ans[x][y] = i;
7      if (!ans[(x - 2 + n) % n + 1][y % n + 1]){
8          x = (x - 2 + n) % n + 1;
9          y = y % n + 1;
10     } else {
11         x = x % n + 1;
12     }
13 }

```

14.3 最长严格/非严格递增子序列 (LIS)

14.3.1 一维

注意子序列是不连续的。使用二分搜索，以 $\mathcal{O}(N \log N)$ 复杂度通过，另也有 $\mathcal{O}(N^2)$ 的 dp 解法。

```

1  vector<int> val; // 堆数
2  for (int i = 1, x; i <= n; i++) {
3      cin >> x;
4      int it = upper_bound(val.begin(), val.end(), x) - val.begin(); // low/upp: 严格/
非严格递增
5      if (it >= val.size()) { // 新增一堆
6          val.push_back(x);
7      } else { // 更新对应位置元素
8          val[it] = x;
9      }
10 }
11 cout << val.size() << endl;

```

14.3.2 二维+输出方案

```

1  vector<array<int, 3>> in(n + 1);
2  for (int i = 1; i <= n; i++) {
3      cin >> in[i][0] >> in[i][1];
4      in[i][2] = i;
5  }
6  sort(in.begin() + 1, in.end(), [&](auto x, auto y) {
7      if (x[0] != y[0]) return x[0] < y[0];
8      return x[1] > y[1];
9  });
10
11 vector<int> val{0}, idx{0}, pre(n + 1);
12 for (int i = 1; i <= n; i++) {
13     auto [x, y, z] = in[i];
14     int it = lower_bound(val.begin(), val.end(), y) - val.begin(); // low/upp: 严格/
非严格递增
15     if (it >= val.size()) { // 新增一堆
16         pre[z] = idx.back();
17         val.push_back(y);
18         idx.push_back(z);
19     } else { // 更新对应位置元素
20         pre[z] = idx[it - 1];
21         val[it] = y;
22         idx[it] = z;
23     }
24 }

```

```

25
26 vector<int> ans;
27 for (int i = idx.back(); i != 0; i = pre[i]) {
28     ans.push_back(i);
29 }
30 reverse(ans.begin(), ans.end());
31 cout << ans.size() << "\n";
32 for (auto it : ans) {
33     cout << it << " ";
34 }

```

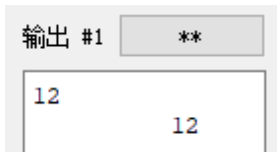
14.4 cout 输出流控制

设置字段宽度: `setw(x)` , 该函数可以使得补全 x 位输出, 默认用空格补全。

```

1 bool Solve() {
2     cout << 12 << endl;
3     cout << setw(12) << 12 << endl;
4     return 0;
5 }

```



输出 #1 **

```

12
12

```

设置填充字符: `setfill(x)` , 该函数可以设定补全类型, 注意这里的 x 只能为 `char` 类型。

```

1 bool Solve() {
2     cout << 12 << endl;
3     cout << setw(12) << setfill('*') << 12 << endl;
4     return 0;
5 }

```



输出 #1 **

```

12
*****12

```

14.5 读取一行数字, 个数未知

```

1 string s;
2 getline(cin, s);
3 stringstream ss;
4 ss << s;
5 while (ss >> s) {
6     auto res = stoi(s);
7     cout << res * 100 << endl;
8 }

```

14.6 约瑟夫问题

n 个人编号 $0, 1, 2, \dots, n-1$, 每次数到 k 出局, 求最后剩下的人的编号。

$\mathcal{O}(N)$ 。

```

1 int jos(int n,int k){
2     int res=0;
3     repeat(i,1,n+1)res=(res+k)%i;
4     return res; // res+1, 如果编号从1开始
5 }

```

$\mathcal{O}(K \log N)$, 适用于 K 较小的情况。

```

1 int jos(int n,int k){
2     if(n==1 || k==1)return n-1;
3     if(k>n)return (jos(n-1,k)+k)%n; // 线性算法
4     int res=jos(n-n/k,k)-n%k;
5     if(res<0)res+=n; // mod n
6     else res+=res/(k-1); // 还原位置
7     return res; // res+1, 如果编号从1开始
8 }

```

14.7 日期换算（基姆拉尔森公式）

已知年月日，求星期数。

```

1 int week(int y,int m,int d){
2     if(m<=2)m+=12,y--;
3     return (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)%7+1;
4 }

```

14.8 单调队列

查询区间 k 的最大最小值。

```

1 deque<int> D;
2 int n,k,x,a[MAX];
3 int main(){
4     IOS();
5     cin>>n>>k;
6     for(int i=1;i<=n;i++) cin>>a[i];
7     for(int i=1;i<=n;i++){
8         while(!D.empty() && a[D.back()]<=a[i]) D.pop_back();
9         D.emplace_back(i);
10        if(!D.empty()) if(i-D.front())>=k D.pop_front();
11        if(i>=k)cout<<a[D.front()]<<endl;
12    }
13    return 0;
14 }

```

14.9 高精度快速幂

求解 $n^k \bmod p$, 其中 $0 \leq n, k \leq 10^{1000000}$, $1 \leq p \leq 10^9$ 。容易发现 n 可以直接取模，瓶颈在于 k [See](#)。

14.9.1 魔改十进制快速幂（暴力计算）

该算法复杂度 $\mathcal{O}(\text{len}(k))$ 。

```

1 int mypow10(int n, vector<int> k, int p) {
2     int r = 1;
3     for (int i = k.size() - 1; i >= 0; i--) {

```

```

4         for (int j = 1; j <= k[i]; j++) {
5             r = r * n % p;
6         }
7         int v = 1;
8         for (int j = 0; j <= 9; j++) {
9             v = v * n % p;
10        }
11        n = v;
12    }
13    return r;
14 }
15 signed main() {
16     string n_, k_;
17     int p;
18     cin >> n_ >> k_ >> p;
19
20     int n = 0; // 转化并计算 n % p
21     for (auto it : n_) {
22         n = n * 10 + it - '0';
23         n %= p;
24     }
25     vector<int> k; // 转化 k
26     for (auto it : k_) {
27         k.push_back(it - '0');
28     }
29     cout << mypow10(n, k, p) << endl; // 暴力快速幂
30 }

```

14.9.2 扩展欧拉定理 (欧拉降幂公式)

$$n^k \equiv \begin{cases} n^{k \bmod \varphi(p)} & \gcd(n, p) = 1 \\ n^{k \bmod \varphi(p) + \varphi(p)} & \gcd(n, p) \neq 1 \wedge k \geq \varphi(p) \\ n^k & \gcd(n, p) \neq 1 \wedge k < \varphi(p) \end{cases}$$

最终我们可以将幂降到 $\varphi(p)$ 的级别, 使得能够直接使用快速幂解题, 复杂度瓶颈在求解欧拉函数 $\mathcal{O}(\sqrt{p})$ 。

```

1 int phi(int n) { //求解 phi(n)
2     int ans = n;
3     for (int i = 2; i <= n / i; i++) {
4         if (n % i == 0) {
5             ans = ans / i * (i - 1);
6             while (n % i == 0) {
7                 n /= i;
8             }
9         }
10    }
11    if (n > 1) { //特判 n 为质数的情况
12        ans = ans / n * (n - 1);
13    }
14    return ans;
15 }
16 signed main() {
17     string n_, k_;
18     int p;
19     cin >> n_ >> k_ >> p;
20
21     int n = 0; // 转化并计算 n % p
22     for (auto it : n_) {
23         n = n * 10 + it - '0';
24         n %= p;

```

```

25     }
26     int mul = phi(p), type = 0, k = 0; // 转化 k
27     for (auto it : k_) {
28         k = k * 10 + it - '0';
29         type |= (k >= mul);
30         k %= mul;
31     }
32     if (type) {
33         k += mul;
34     }
35     cout << mypow(n, k, p) << endl;
36 }

```

14.10 快读

注意读入到文件结尾才结束，直接运行会无输出。

```

1 namespace QuickRead { // 快读
2     char buf[1 << 21], *p1 = buf, *p2 = buf;
3     inline int getc() {
4         return p1 == p2 && (p2 = (p1 = buf) + fread(buf, 1, 1 << 21, stdin), p1 ==
5             p2) ? EOF : *p1++;
6     }
7     template <typename T> void Cin(T &a) {
8         T ans = 0;
9         bool f = 0;
10        char c = getc();
11        for (; c < '0' || c > '9'; c = getc()) {
12            if (c == '-') f = 1;
13        }
14        for (; c >= '0' && c <= '9'; c = getc()) {
15            ans = ans * 10 + c - '0';
16        }
17        a = f ? -ans : ans;
18    }
19    template <typename T, typename... Args> void Cin(T &a, Args &...args) {
20        Cin(a), Cin(args...);
21    }
22    template <typename T> void write(T x) { // 注意，这里输出不带换行
23        if (x < 0) putchar('-'), x = -x;
24        if (x > 9) write(x / 10);
25        putchar(x % 10 + '0');
26    }
27 } // namespace QuickRead

```

14.11 int128 输入输出流控制

int128 只在基于 Linux 系统的环境下可用，需要 C++20。38位精度，除输入输出外与普通数据类型无差别。该封装支持负数读入，需要注意 `write` 函数结尾不输出多余空格与换行。

```

1 namespace my128 { // 读入优化封装，支持__int128
2     using i64 = __int128_t;
3     i64 abs(const i64 &x) {
4         return x > 0 ? x : -x;
5     }
6     auto &operator>>(istream &it, i64 &j) {
7         string val;
8         it >> val;
9         reverse(val.begin(), val.end());
10        i64 ans = 0;
11        bool f = 0;

```

```

12     char c = val.back();
13     val.pop_back();
14     for (; c < '0' || c > '9'; c = val.back(), val.pop_back()) {
15         if (c == '-') {
16             f = 1;
17         }
18     }
19     for (; c >= '0' && c <= '9'; c = val.back(), val.pop_back()) {
20         ans = ans * 10 + c - '0';
21     }
22     j = f ? -ans : ans;
23     return it;
24 }
25 auto &operator<<(ostream &os, const i64 &j) {
26     string ans;
27     function<void(i64)> write = [&](i64 x) {
28         if (x < 0) ans += '-', x = -x;
29         if (x > 9) write(x / 10);
30         ans += x % 10 + '0';
31     };
32     write(j);
33     return os << ans;
34 }
35 } // namespace my128

```

14.12 对拍版子

- 文件控制

```

1 // BAD.cpp, 存放待寻找错误的代码
2 freopen("A.txt", "r", stdin);
3 freopen("BAD.out", "w", stdout);
4
5 // 1.cpp, 存放暴力或正确的代码
6 freopen("A.txt", "r", stdin);
7 freopen("1.out", "w", stdout);
8
9 // Ask.cpp
10 freopen("A.txt", "w", stdout);

```

- C++ 版 bat

```

1 int main() {
2     int T = 1E5;
3     while(T--) {
4         system("BAD.exe");
5         system("1.exe");
6         system("A.exe");
7         if (system("fc BAD.out 1.out")) {
8             puts("WA");
9             return 0;
10        }
11    }
12 }

```

14.13 随机数生成与样例构造

```
1 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
2 int r(int a, int b) {
3     return rnd() % (b - a + 1) + a;
4 }
5
6 void graph(int n, int root = -1, int m = -1) {
7     vector<pair<int, int>> t;
8     for (int i = 1; i < n; i++) { // 先建立一棵以0为根节点的树
9         t.emplace_back(i, r(0, i - 1));
10    }
11
12    vector<pair<int, int>> edge;
13    set<pair<int, int>> uni;
14    if (root == -1) root = r(0, n - 1); // 确定根节点
15    for (auto [x, y] : t) { // 偏移建树
16        x = (x + root) % n + 1;
17        y = (y + root) % n + 1;
18        edge.emplace_back(x, y);
19        uni.emplace(x, y);
20    }
21
22    if (m != -1) { // 如果是图, 则在树的基础上继续加边
23        for (int i = n; i <= m; i++) {
24            while (true) {
25                int x = r(1, n), y = r(1, n);
26                if (x == y) continue; // 拒绝自环
27                if (uni.count({x, y})) continue; // 拒绝重边
28                edge.emplace_back(x, y);
29                uni.emplace(x, y);
30            }
31        }
32    }
33
34    random_shuffle(edge.begin(), edge.end()); // 打乱节点
35    for (auto [x, y] : edge) {
36        cout << x << " " << y << endl;
37    }
38 }
```

14.14 手工哈希

```
1 struct myhash {
2     static uint64_t hash(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8     size_t operator()(uint64_t x) const {
9         static const uint64_t SEED =
10         chrono::steady_clock::now().time_since_epoch().count();
11         return hash(x + SEED);
12     }
13     size_t operator()(pair<uint64_t, uint64_t> x) const {
14         static const uint64_t SEED =
15         chrono::steady_clock::now().time_since_epoch().count();
16         return hash(x.first + SEED) ^ (hash(x.second + SEED) >> 1);
17     }
18 };
```

```
17 // unordered_map<int, int, myhash>
```

14.15 Python常用语法

14.15.1 读入与定义

- 读入多个变量并转换类型: `X, Y = map(int, input().split())`
- 读入列表: `X = eval(input())`
- 多维数组定义: `X = [[0 for j in range(0, 100)] for i in range(0, 200)]`

14.15.2 格式化输出

- 保留小数输出: `print("{:.12f}".format(X))` 指保留 12 位小数
- 对齐与宽度: `print("{:<12f}".format(X))` 指左对齐, 保留 12 个宽度

14.15.3 排序

- 倒序排序: 使用 `reverse` 实现倒序 `X.sort(reverse=True)`
- 自定义排序: 下方代码实现了先按第一关键字降序、再按第二关键字升序排序。

```
1 X.sort(key=lambda x: x[1])
2 X.sort(key=lambda x: x[0], reverse=True)
```

14.15.4 文件IO

- 打开要读取的文件: `r = open('X.txt', 'r', encoding='utf-8')`
- 打开要写入的文件: `w = open('Y.txt', 'w', encoding='utf-8')`
- 按行写入: `w.write(XX)`

14.15.5 增加输出流长度

```
1 import sys
2 sys.set_int_max_str_digits(200000)
```

14.15.6 自定义结构体

自定义结构体并且自定义排序

```
1 class node:
2     def __init__(self, A, B, C):
3         self.A = A
4         self.B = B
5         self.C = C
6
7 w = []
8 for i in range(1, 5):
9     a, b, c = input().split()
10    w.append(node(a, b, c))
11 w.sort(key=lambda x: x.C, reverse=True)
12 for i in w:
13     print(i.A, i.B, i.C)
14
```

14.15.7 数据结构

- 模拟于 C_{++}^{map} ，定义：`dic = dict()`
- 模拟栈与队列：使用常见的 `list` 即可完成，`list.insert(0, x)` 实现头部插入、`list.pop()` 实现尾部弹出、`list.pop(0)` 实现头部弹出

14.15.8 其他

- 获取ASCII码：`ord()` 函数
- 转换为ASCII字符：`chr()` 函数

14.16 OJ测试

对于一个未知属性的OJ，应当在正式赛前进行以下全部测试：

14.16.1 GNU C++ 版本测试

```
1  for (int i : {1, 2}) {} // GNU C++11 支持范围表达式
2
3  auto cc = [&](int x) { x++; }; // GNU C++11 支持 auto 与 lambda 表达式
4  cc(2);
5
6  tuple<string, int, int> V; // GNU C++11 引入
7  array<int, 3> C; // GNU C++11 引入
8
9  auto dfs = [&](auto self, int x) -> void { // GNU C++14 支持 auto 自递归
10     if (x > 10) return;
11     self(self, x + 1);
12 };
13 dfs(dfs, 1);
14
15 vector in(1, vector<int>(1)); // GNU C++17 支持 vector 模板类型缺失
16
17 map<int, int> dic;
18 for (auto [u, v] : dic) {} // GNU C++17 支持 auto 解绑
19 dic.contains(12); // GNU C++20 支持 contains 函数
```

14.16.2 编译器位数测试

```
1  using i64 = __int128; // 64 位 GNU C++11 支持
```

14.16.3 评测器环境测试

Windows 系统输出 `-1`；反之则为一个随机数。

```
1  #define int long long
2  map<int, int> dic;
3  int x = dic.size() - 1;
4  cout << x << endl;
```

14.16.4 运算速度测试

	本地-20(64)	CodeForces-20(64)	AtCoder-20(64)	牛客-17(64)	学院01	CodeForces-17(32)	马踏集
4E3量级-硬跑	2454	2886	874	4121	4807	2854	4986
4E3量级-手动加速	556	686	873	1716	1982	2246	2119

```
1 // #pragma GCC optimize("Ofast", "unroll-loops")
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 signed main() {
6     int n = 4E3;
7     bitset<30> ans;
8     for (int i = 1; i <= n; i++) {
9         for (int j = 1; j <= n; j += 2) {
10             for (int k = 1; k <= n; k += 4) {
11                 ans |= i | j | k;
12             }
13         }
14     }
15     cout << ans.to_ulong() << "\n";
16 }
```

14.17 编译器设置

```
1 g++ -O2 -std=c++20 -pipe
2 -Wall -Wextra -Wconversion /* 这部分是警告相关，可能用不到 */
3 -fstack-protector
4 -Wl,--stack=268435456
```

/END/

Created by *Wida*
Writer: *Wida & Hamine & ppafo*
Version: 8.4(2023.11.02)

ZJSU-AA Lab
2020.09.29 – 2023.11.02